



Guia do usuário da versão 1

# AWS Command Line Interface



# AWS Command Line Interface: Guia do usuário da versão 1

Copyright © 2024 Amazon Web Services, Inc. and/or its affiliates. All rights reserved.

As marcas comerciais e imagens comerciais da Amazon não podem ser usadas no contexto de nenhum produto ou serviço que não seja da Amazon, nem de qualquer maneira que possa gerar confusão entre clientes ou que deprecie ou desprestige a Amazon. Todas as outras marcas comerciais que não pertencem à Amazon pertencem a seus respectivos proprietários, que podem ou não ser afiliados, patrocinados pela Amazon ou ter conexão com ela.

---

# Table of Contents

|                                                                              |      |
|------------------------------------------------------------------------------|------|
| .....                                                                        | viii |
| Sobre o AWS CLI .....                                                        | 1    |
| Sobre a AWS CLI versão 1 .....                                               | 2    |
| Manutenção e suporte para as versões principais do SDK .....                 | 2    |
| Sobre a Amazon Web Services .....                                            | 2    |
| Sobre os exemplos .....                                                      | 3    |
| Documentação e recursos adicionais .....                                     | 4    |
| Documentação e recursos da AWS CLI .....                                     | 4    |
| Outros AWS SDKs .....                                                        | 4    |
| Instalar a AWS CLI .....                                                     | 6    |
| Requisitos da versão do Python .....                                         | 6    |
| Amazon Linux .....                                                           | 7    |
| Pré-requisitos .....                                                         | 7    |
| pip .....                                                                    | 8    |
| yum .....                                                                    | 9    |
| Solução de problemas de erros de instalação e desinstalação da AWS CLI ..... | 10   |
| Linux .....                                                                  | 10   |
| Pré-requisitos .....                                                         | 10   |
| Instalar e desinstalar usando o instalador empacotado .....                  | 11   |
| Instalar e desinstalar usando pip .....                                      | 17   |
| macOS .....                                                                  | 21   |
| Pré-requisitos .....                                                         | 22   |
| Instalar e desinstalar usando o instalador empacotado .....                  | 22   |
| Instalar e atualizar usando pip .....                                        | 28   |
| Solução de problemas de erros de instalação e desinstalação da AWS CLI ..... | 32   |
| Windows .....                                                                | 32   |
| Instalar, atualizar e desinstalar usando o instalador MSI .....              | 33   |
| Instalar, atualizar e desinstalar usando Python e pip .....                  | 34   |
| Adicionar o executável da AWS CLI ao caminho da linha de comando .....       | 36   |
| Solução de problemas de erros de instalação e desinstalação da AWS CLI ..... | 38   |
| Virtualenv .....                                                             | 38   |
| Pré-requisitos .....                                                         | 39   |
| Instalar e atualizar em um ambiente virtual .....                            | 39   |
| Solução de problemas de erros de instalação e desinstalação da AWS CLI ..... | 40   |

|                                                                                       |     |
|---------------------------------------------------------------------------------------|-----|
| Configurar o AWS CLI .....                                                            | 41  |
| Precedência de credenciais e configurações .....                                      | 41  |
| Tópicos adicionais nesta seção .....                                                  | 42  |
| Configurações do arquivo de configuração e credenciais .....                          | 42  |
| Formato dos arquivos de credenciais e configuração .....                              | 43  |
| Onde as definições de configuração ficam armazenadas? .....                           | 48  |
| Usar perfis nomeados .....                                                            | 49  |
| Definir e visualizar as configurações usando comandos .....                           | 50  |
| Definir novos exemplos de comandos de configuração e credenciais .....                | 52  |
| Configurações de arquivo config compatíveis .....                                     | 53  |
| Variáveis de ambiente .....                                                           | 71  |
| Como definir variáveis de ambiente .....                                              | 71  |
| Variáveis de ambiente compatíveis da AWS CLI .....                                    | 73  |
| Opções de linha de comando .....                                                      | 82  |
| Como usar as opções de linha de comando .....                                         | 83  |
| A AWS CLI comporta opções de linha de comando globais .....                           | 83  |
| Usos comuns das opções de linha de comando .....                                      | 87  |
| Conclusão de comando .....                                                            | 87  |
| Como funciona .....                                                                   | 88  |
| Configuração do preenchimento automático de comando no Linux ou no macOS .....        | 89  |
| Configuração do preenchimento de comandos no Windows .....                            | 92  |
| Repetições .....                                                                      | 94  |
| Modos de novas tentativas disponíveis .....                                           | 94  |
| Configuração um modo de nova tentativa .....                                          | 97  |
| Visualização de logs de novas tentativas .....                                        | 98  |
| Usar um proxy HTTP .....                                                              | 98  |
| Como usar os exemplos da .....                                                        | 99  |
| Autenticar para um proxy .....                                                        | 100 |
| Uso de proxy em instâncias do Amazon EC2 .....                                        | 101 |
| Solução de problemas .....                                                            | 101 |
| Endpoints .....                                                                       | 101 |
| Definir o endpoint para um único comando .....                                        | 102 |
| Definir um endpoint global para todos os Serviços da AWS .....                        | 102 |
| Definido para usar endpoints FIPs para todos os Serviços da AWS .....                 | 104 |
| Configurado para usar os endpoints de pilha dupla para todos os Serviços da AWS ..... | 105 |
| Definir endpoints específicos de serviço .....                                        | 106 |

|                                                                          |     |
|--------------------------------------------------------------------------|-----|
| Precedência de configurações e definições do endpoint .....              | 110 |
| Autenticação e credenciais de acesso .....                               | 111 |
| Precedência de credenciais e configurações .....                         | 111 |
| Tópicos adicionais nesta seção .....                                     | 112 |
| Credenciais de curto prazo .....                                         | 112 |
| Perfis do IAM .....                                                      | 114 |
| Pré-requisitos .....                                                     | 114 |
| Visão geral do uso de funções do IAM .....                               | 114 |
| Configurar e usar uma função .....                                       | 116 |
| Usar MFA .....                                                           | 118 |
| Funções entre contas e ID externo .....                                  | 119 |
| Especificar um nome de sessão de função para facilitar a auditoria ..... | 120 |
| Assumir a função com a identidade da web .....                           | 121 |
| Limpar as credenciais em cache .....                                     | 122 |
| IAM users .....                                                          | 123 |
| Etapa 1: criar o usuário do IAM .....                                    | 123 |
| Etapa 2: obter as chaves de acesso .....                                 | 123 |
| Configurar o AWS CLI .....                                               | 124 |
| Uso de credenciais para metadados de instância do Amazon EC2. ....       | 125 |
| Pré-requisitos .....                                                     | 125 |
| Configuração de um perfil para metadados do Amazon EC2 .....             | 126 |
| Credenciais externas .....                                               | 127 |
| Usar a AWS CLI .....                                                     | 130 |
| Obter ajuda .....                                                        | 130 |
| O comando de ajuda integrado da AWS CLI .....                            | 131 |
| Guia de referência da AWS CLI .....                                      | 136 |
| Documentação de API .....                                                | 136 |
| Solucionar de problemas de erros .....                                   | 137 |
| Ajuda adicional .....                                                    | 137 |
| Estrutura do comando .....                                               | 137 |
| Estrutura do comando .....                                               | 137 |
| Comandos de espera .....                                                 | 138 |
| Especificar valores de parâmetro .....                                   | 140 |
| Tipos comuns de parâmetros .....                                         | 141 |
| Aspas com strings .....                                                  | 145 |
| Parâmetros de arquivos .....                                             | 150 |

|                                                          |     |
|----------------------------------------------------------|-----|
| Gerar um modelo de esqueleto da CLI .....                | 153 |
| Sintaxe Simplificada .....                               | 160 |
| Controlar a saída do comando .....                       | 162 |
| Output Format .....                                      | 163 |
| Paginação .....                                          | 171 |
| Filtragem da saída .....                                 | 173 |
| Códigos de retorno .....                                 | 197 |
| Aliases .....                                            | 199 |
| Pré-requisitos .....                                     | 199 |
| Etapa 1: Criação do arquivo de alias .....               | 200 |
| Etapa 2: Criação de um alias .....                       | 201 |
| Passo 3: Como chamar um alias .....                      | 204 |
| Exemplos de repositório de alias .....                   | 206 |
| Recursos .....                                           | 207 |
| Exemplos de código .....                                 | 209 |
| Exemplos de comando guiado .....                         | 209 |
| DynamoDB .....                                           | 210 |
| Amazon EC2 .....                                         | 214 |
| S3 Glacier .....                                         | 233 |
| IAM .....                                                | 240 |
| Amazon S3 .....                                          | 245 |
| Amazon SNS .....                                         | 263 |
| Exemplos de script Bash .....                            | 266 |
| Ações e cenários .....                                   | 266 |
| Segurança .....                                          | 431 |
| Proteção de dados .....                                  | 432 |
| Criptografia de dados .....                              | 433 |
| Identity and Access Management .....                     | 433 |
| Público .....                                            | 434 |
| Autenticando com identidades .....                       | 434 |
| Gerenciamento do acesso usando políticas .....           | 438 |
| Como Serviços da AWS funcionam com o IAM .....           | 441 |
| Solução de problemas de identidade e acesso do AWS ..... | 441 |
| Compliance Validation .....                              | 443 |
| Resilience .....                                         | 444 |
| Infrastructure Security .....                            | 445 |

|                                                                                    |     |
|------------------------------------------------------------------------------------|-----|
| Aplicar uma versão mínima do TLS .....                                             | 446 |
| Solucionar erros .....                                                             | 450 |
| Solução geral de problemas para tentar primeiro .....                              | 450 |
| Verificar a formatação do comando AWS CLI .....                                    | 451 |
| Confirme se você está executando uma versão recente da AWS CLI .....               | 451 |
| Como usar a opção --debug .....                                                    | 452 |
| Habilitar e analisar logs de histórico de comandos da AWS CLI .....                | 457 |
| Confirmar se a AWS CLI está configurada .....                                      | 458 |
| Erros de comando não encontrado .....                                              | 458 |
| O comando "aws --version" retorna uma versão diferente da que você instalou .....  | 461 |
| O comando "aws --version" retorna uma versão após a desinstalação da AWS CLI ..... | 462 |
| A AWS CLI processou um comando com um nome de parâmetro incompleto .....           | 464 |
| Erros de acesso negado .....                                                       | 465 |
| Credenciais inválidas e erros de chave .....                                       | 466 |
| Assinatura não corresponde aos erros .....                                         | 468 |
| Erros de nenhum console do Windows encontrado .....                                | 469 |
| Erros de certificado SSL .....                                                     | 469 |
| Erros JSON inválidos .....                                                         | 470 |
| Recursos adicionais .....                                                          | 472 |
| Histórico do documento .....                                                       | 473 |

Esta documentação é somente para a versão 1 da AWS CLI. Para obter a documentação relacionada à AWS CLI versão 2, consulte o [Guia do usuário da versão 2](#).



# O que é o AWS Command Line Interface versão 1?

## Note

A versão 1 da AWS CLI não é a versão mais recente da AWS CLI. Alguns recursos apresentados na AWS CLI versão 2 não são compatíveis com a versão 1, e você deve fazer a atualização para acessá-los. Há algumas alterações "radicais" em relação à versão 1 que podem exigir alterações nos scripts. Para obter uma lista das alterações mais recentes na versão 2, consulte [Alterações mais recentes](#) no Guia do usuário da AWS CLI versão 2.

A AWS Command Line Interface (AWS CLI) é uma ferramenta de código aberto que permite interagir com os serviços da AWS usando comandos no shell da linha de comando. Com o mínimo de configuração, a AWS CLI permite começar a executar comandos que implementam uma funcionalidade equivalente àquela fornecida pelo AWS Management Console baseado em navegador do prompt de comando em seu programa de terminal:

- Shells do Linux: use programas comuns de shell, como [bash](#), [zsh](#) e [tcsh](#) para executar comandos no Linux ou macOS.
- Linha de comando do Windows: no Windows, execute comandos no prompt de comando do Windows ou no PowerShell.
- Remotamente: execute comandos em instâncias do Amazon Elastic Compute Cloud (Amazon EC2) por meio de um programa de terminal remoto, como PuTTY ou SSH, ou com o .AWS Systems Manager

Toda as funções administração, gerenciamento e acesso da AWS IaaS (Infraestrutura como um serviço) no AWS Management Console estão disponíveis na API da AWS e na AWS CLI. Os novos recursos e serviços da AWS IaaS fornecem funcionalidade completa do AWS Management Console por meio da API e da CLI no lançamento ou dentro de 180 dias após o lançamento.

A AWS CLI fornece acesso direto às APIs públicas de serviços da AWS. Você pode explorar os recursos de um serviço com a AWS CLI e desenvolver scripts de shell para gerenciar seus recursos. Além dos comandos de nível inferior equivalentes a API, vários serviços da AWS fornecem personalizações para a AWS CLI. As personalizações podem incluir comandos de nível mais elevado que simplificam o uso de um serviço com uma API complexa.

# Sobre a AWS CLI versão 1

A AWS CLI versão 1 é a AWS CLI original, e continuamos a oferecer suporte a ela. No entanto, é possível que a portabilidade dos principais recursos apresentados na AWS CLI versão 2 não seja feita para a AWS CLI versão 1. Para usar esses recursos, é necessário instalar a AWS CLI versão 2. A AWS CLI versão 1 é criada com o SDK for Python e, portanto, requer a instalação de uma versão compatível do Python.

Para instalar a AWS CLI versão 1, consulte [Instalar a AWS CLI](#).

Para verificar a versão atualmente instalada, use o seguinte comando:

```
$ aws --version  
aws-cli/1.27.69 Python/3.11.2 Linux/4.14.133-113.105.amzn2.x86_64 botocore/1.13
```

Para obter o histórico de versões, consulte o [Log de alterações da AWS CLI versão 1](#) no GitHub.

## Manutenção e suporte para as versões principais do SDK

Para obter informações sobre manutenção e suporte para versões principais do SDK e suas dependências subjacentes, consulte o seguinte no [Guia de referência de AWS SDKs e ferramentas](#):

- [Política de manutenção de ferramentas e SDKs da AWS](#)
- [Matriz de suporte a versões de ferramentas e SDKs da AWS](#)

## Sobre a Amazon Web Services

A Amazon Web Services (AWS) é um conjunto de serviços de infraestrutura digital que os desenvolvedores podem utilizar ao desenvolver suas aplicações. Os serviços incluem computação, armazenamento, banco de dados e sincronização de aplicações (sistema de mensagens e filas). A AWS usa um modelo de serviço de pagamento conforme o uso. Você será cobrado apenas pelos serviços que você ou suas aplicações usam. Além disso, para tornar AWS mais acessível como plataforma para criação de protótipos e experimentação, a AWS oferece um nível de uso gratuito. Neste nível, os serviços são gratuitos abaixo de um determinado nível de uso. Para obter mais informações sobre os custos e o nível gratuito da AWS, consulte [Nível gratuito da AWS](#). Para obter uma conta da AWS, abra a [página inicial da AWS](#) e selecione Criar uma conta da AWS.

# Sobre os exemplos da AWS CLI

Os exemplos da AWS Command Line Interface (AWS CLI) neste guia são formatados de acordo com seguintes convenções:

- Prompt: o prompt de comando usa o prompt do Linux e é exibido como (\$ ). Para comandos específicos do Windows, C:\> é usado como prompt. Não inclua prompt quando você digitar comandos.
- Diretório: quando comandos devem ser executados de um diretório específico, o nome do diretório é mostrado antes do símbolo do comando.
- Entrada do usuário o texto do comando inserido na linha de comando é formatado como **user input**.
- Texto substituível: o texto variável, incluindo nomes de recursos que você escolher ou IDs gerados pelos serviços da AWS, que você deve incluir em comandos é formatado como *texto substituível*. Em comandos ou comandos de várias linhas, em que é necessária uma entrada específica do teclado, os comandos do teclado também podem ser exibidos como texto substituível.
- Saída: a saída retornada pelos serviços da AWS é mostrada sob a entrada do usuário e é formatada como `computer output`.

O exemplo de comando **aws configure** a seguir inclui entradas do usuário, texto substituível e saída:

1. Insira **aws configure** na linha de comando e pressione Enter.
2. A AWS CLI gera linhas de texto, solicitando que você insira informações adicionais.
3. Insira cada uma de suas chaves de acesso e pressione Enter.
4. Depois, insira um nome de região da AWS no formato mostrado, pressione Enter e, depois, Enter uma última vez para ignorar o formato de saída.
5. O comando final Enter é mostrado como texto substituível porque não há entradas do usuário para essa linha.

```
$ aws configure
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY
Default region name [None]: us-west-2
```

Default output format [None]: **ENTER**

O exemplo a seguir mostra um comando simples com saída. Para usar este exemplo, insira o texto completo do comando (o texto destacado após o prompt) e pressione Enter. O nome do grupo de segurança, **my-sg**, pode ser substituído pelo nome do grupo de segurança desejado. O documento JSON, incluindo as chaves, é saída. Se configurar o CLI para resultar em texto ou formato de tabela, a saída será formatada de forma diferente. [JSON](#) é o formato de saída padrão.

```
$ aws ec2 create-security-group --group-name my-sg --description "My security group"
{
  "GroupId": "sg-903004f8"
}
```

## Documentação e recursos adicionais

### Documentação e recursos da AWS CLI

Além deste guia, os recursos a seguir são fontes online importantes para a AWS CLI.

- [AWS CLI Guia de referência da](#) versão 1
- [Repositório de exemplos de código da AWS CLI](#)
- [Repositório da AWS CLI no GitHub](#): você pode visualizar e bifurcar o código-fonte para a AWS CLI no GitHub. Faça parte da comunidade de usuários no GitHub para fornecer feedback, solicitar recursos e enviar suas próprias contribuições.
- [Repositório de exemplos de alias da AWS CLI](#): você pode visualizar e bifurcar exemplos de alias da AWS CLI no GitHub.
- [Log de alterações da AWS CLI versão 1](#)
- [Log de alterações da AWS CLI versão 2](#)

## Outros AWS SDKs

Dependendo do seu caso de uso, talvez você queira escolher um dos AWS SDKs ou o AWS Tools for PowerShell:

- [AWS Tools for PowerShell](#)

- [AWS SDK for Java](#)
- [AWS SDK for .NET](#)
- [AWS SDK for JavaScript](#)
- [AWS SDK for Ruby](#)
- [AWS SDK for Python \(Boto\)](#)
- [AWS SDK for PHP](#)
- [AWS SDK for Go](#)
- [AWS Mobile SDK for iOS](#)
- [AWS Mobile SDK for Android](#)

# Instalar, atualizar e desinstalar a AWS CLI

Este tópico fornece links para instalar, atualizar e desinstalar a versão original da AWS Command Line Interface (AWS CLI). A AWS CLI versão 1 atualmente é compatível, mas novos recursos adicionados à AWS CLI versão 2 não podem ser adicionados à AWS CLI versão 1. Para usar esses recursos, é necessário instalar a AWS CLI versão 2. Para obter mais informações sobre como instalar a versão 2, consulte [Instalar a AWS CLI versão 2](#).

Instruções para instalar, atualizar e desinstalar a AWS CLI:

- [Requisitos da versão do Python](#)
- [Instalar, atualizar e desinstalar a AWS CLI versão 1 no Amazon Linux](#)
- [Instalar, atualizar e desinstalar a AWS CLI versão 1 no Linux](#)
- [Instalar, atualizar e desinstalar a AWS CLI versão 1 no macOS](#)
- [Instalar, atualizar e desinstalar a AWS CLI versão 1 no Windows](#)
- [Instalar e atualizar a AWS CLI versão 1 em um ambiente virtual](#)

## Requisitos da versão do Python

A AWS CLI versão 1 é criada com o SDK for Python e, portanto, requer a instalação de uma versão compatível do Python.

Matriz de suporte a versões do Python

| Versão do AWS CLI | Versão do Python com suporte                     |
|-------------------|--------------------------------------------------|
| 1.32.0: atual     | Python 3.8 e posterior                           |
| 1.27.0–1.31.x     | Python 3.7 e posterior                           |
| 1.20.0–1.26.x     | Python 3.6                                       |
| 1.19.0: 1.19.x    | Python 2.7+, Python 3.6+                         |
| 1.17 – 1.18.x     | Python 2.7+, Python 3.4+                         |
| 1.0 – 1.16.x      | Python 2.6 e anteriores, Python 3.3 e anteriores |

Para obter informações sobre as versões mais recentes da AWS CLI, consulte o [Log de alterações da AWS CLI versão 2](#) no GitHub.

## Instalar, atualizar e desinstalar a AWS CLI versão 1 no Amazon Linux

A AWS CLI versão 1 está pré-instalada no Amazon Linux e no Amazon Linux 2. Verifique a versão atualmente instalada usando o comando a seguir.

```
$ aws --version
aws-cli/1.27.69 Python/3.11.2 Linux/4.14.133-113.105.amzn2.x86_64 botocore/1.13
```

Dependendo de quando você criou sua instância do Amazon Linux, a AWS CLI versão 1 é pré-instalada usando um dos seguintes gerenciadores de pacotes:

- [pip](#)
- [yum](#)

### Pré-requisitos

Você deve ter o Python 3.8 ou posterior instalado. Para obter instruções de instalação, consulte a página [Downloading Python](#) no Beginner's Guide do Python.

Matriz de suporte a versões do Python

| Versão do AWS CLI | Versão do Python com suporte                     |
|-------------------|--------------------------------------------------|
| 1.32.0: atual     | Python 3.8 e posterior                           |
| 1.27.0–1.31.x     | Python 3.7 e posterior                           |
| 1.20.0–1.26.x     | Python 3.6                                       |
| 1.19.0: 1.19.x    | Python 2.7+, Python 3.6+                         |
| 1.17 – 1.18.x     | Python 2.7+, Python 3.4+                         |
| 1.0 – 1.16.x      | Python 2.6 e anteriores, Python 3.3 e anteriores |

## Instalar, atualizar e desinstalar usando pip

A maioria das instâncias do Amazon Linux usa pip para pré-instalar a AWS CLI versão 1.

### Instalar ou atualizar a AWS CLI versão 1 no Amazon Linux usando pip

Para instalar a versão mais recente da AWS CLI versão 1 para o usuário atual, use as instruções a seguir.

1. Se você tiver o Python versão 3 ou posterior instalado, recomendamos usar o `pip3`. Use `pip3 install` para instalar ou atualizar para a versão mais recente da AWS CLI versão 1. Se você executar o comando em um [ambiente virtual Python \(venv\)](#), não será necessário usar a opção `--user`.

```
$ pip3 install --upgrade --user awscli
```

2. Verifique se a pasta que contém aws faz parte da variável PATH.
  - a. Encontre o script de perfil do shell no diretório de usuário. Se não tiver certeza de qual shell você tem, execute `echo $SHELL`.

```
$ ls -a ~  
.  ..  .bash_logout  .bash_profile  .bashrc  Desktop  Documents  Downloads
```

- Bash: `.bash_profile`, `.profile` ou `.bash_login`
  - Zsh: `.zshrc`
  - Tcsh: `.tcshrc`, `.cshrc` ou `.login`.
- b. Adicione um comando de exportação ao final do script de perfil que é semelhante ao exemplo a seguir.

```
export PATH=$HOME/.local/bin:$PATH
```

Esse comando insere o caminho `$HOME/.local/bin` neste exemplo na frente da variável `$PATH` existente.

- c. Recarregue o perfil em sua sessão atual para colocar essas alterações em vigor.

```
$ source ~/.bash_profile
```



3. Para verificar se você está executando a nova versão, use o comando `aws --version`.

```
$ aws --version
aws-cli/1.27.69 Python/3.11.2 Linux/4.14.133-113.105.amzn2.x86_64 botocore/1.13
```

## Desinstalar a AWS CLI versão 1 usando pip

Se precisar desinstalar a AWS CLI, use `pip uninstall`.

```
$ pip3 uninstall awscli
```

## Instalar, atualizar ou desinstalar usando yum

A maioria das instâncias do Amazon Linux usa yum para pré-instalar a AWS CLI versão 1.

### Instalar ou atualizar a AWS CLI versão 1 no Amazon Linux usando yum

Para instalar a versão mais recente da AWS CLI versão 1 disponível no Amazon Linux, execute o comando a seguir.

```
$ sudo yum install awscli
```

Para atualizar para versão mais recente da AWS CLI versão 1 disponível no Amazon Linux, execute o comando a seguir.

```
$ sudo yum update awscli
```

Para verificar se você está executando a versão mais recente, use o comando `aws --version`.

```
$ aws --version
aws-cli/1.27.69 Python/3.11.2 Linux/4.14.133-113.105.amzn2.x86_64 botocore/1.13
```

## Desinstalar a AWS CLI versão 1 usando yum

Para desinstalar a AWS CLI, use `yum remove`.

```
$ sudo yum remove awscli
```

# Solução de problemas de erros de instalação e desinstalação da AWS CLI

Se você encontrar problemas após instalar ou desinstalar a AWS CLI, consulte [Solucionar erros](#) para obter os passos para a solução de problemas. Para obter os passos mais relevantes para a solução de problemas, consulte [the section called “Erros de comando não encontrado”](#), [the section called “O comando “aws --version” retorna uma versão diferente da que você instalou”](#) e [the section called “O comando “aws --version” retorna uma versão após a desinstalação da AWS CLI”](#).

## Instalar, atualizar e desinstalar a AWS CLI versão 1 no Linux

É possível instalar a AWS Command Line Interface (AWS CLI) versão 1 e suas dependências na maioria das distribuições do Linux usando o gerenciador de pacotes `pip` ou o instalador empacotado.

Embora o pacote `awscli` esteja disponível em repositórios para outros gerenciadores de pacotes, como `apt` e `yum`, eles não são produzidos, gerenciados nem têm suporte da AWS. Recomendamos que você instale a AWS CLI apenas dos pontos de distribuição oficiais da AWS, conforme documentado neste guia.

### Seções

- [Pré-requisitos](#)
- [Instalar e desinstalar a AWS CLI versão 1 no Linux usando o instalador empacotado](#)
- [Instalar e desinstalar a AWS CLI versão 1 usando `pip`](#)

## Pré-requisitos

Você deve ter o Python 3.8 ou posterior instalado. Para obter instruções de instalação, consulte a página [Downloading Python](#) no Beginner's Guide do Python.

### Matriz de suporte a versões do Python

| Versão do AWS CLI | Versão do Python com suporte |
|-------------------|------------------------------|
| 1.32.0: atual     | Python 3.8 e posterior       |
| 1.27.0–1.31.x     | Python 3.7 e posterior       |
| 1.20.0–1.26.x     | Python 3.6                   |

| Versão do AWS CLI | Versão do Python com suporte                     |
|-------------------|--------------------------------------------------|
| 1.19.0: 1.19.x    | Python 2.7+, Python 3.6+                         |
| 1.17 – 1.18.x     | Python 2.7+, Python 3.4+                         |
| 1.0 – 1.16.x      | Python 2.6 e anteriores, Python 3.3 e anteriores |

## Instalar e desinstalar a AWS CLI versão 1 no Linux usando o instalador empacotado

No Linux ou macOS, é possível usar o instalador empacotado para instalar a versão 1 da AWS CLI. O pacote de instalador inclui todas as dependências do pacote e pode ser usado offline.

### Note

O pacote de instalador fornecido não é compatível com a instalação em caminhos com espaços.

### Tópicos

- [Instalar a AWS CLI versão 1 usando o instalador empacotado com sudo](#)
- [Instalar a AWS CLI versão 1 usando o instalador empacotado sem sudo](#)
- [Desinstalar o instalador empacotado da AWS CLI versão 1](#)

## Instalar a AWS CLI versão 1 usando o instalador empacotado com **sudo**

As etapas a seguir permitem que você instale a AWS CLI versão 1 via linha de comando em qualquer compilação do Linux ou do macOS.

Veja a seguir um resumo dos comandos de instalação explicados a seguir que você pode recortar e colar para executar como um único conjunto de comandos.

Para a versão mais recente da AWS CLI, use o seguinte bloco de comandos:

```
$ curl "https://s3.amazonaws.com/aws-cli/awscli-bundle.zip" -o "awscli-bundle.zip"
```

```
unzip awscli-bundle.zip
sudo ./awscli-bundle/install -i /usr/local/aws -b /usr/local/bin/aws
```

Para uma versão específica da AWS CLI, acrescente um hífen e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão **1.16.312** seria `awscli-bundle-1.16.312.zip`, o que resultaria no seguinte comando:

```
$ curl "https://s3.amazonaws.com/aws-cli/awscli-bundle-1.16.312.zip" -o "awscli-bundle.zip"
unzip awscli-bundle.zip
sudo ./awscli-bundle/install -i /usr/local/aws -b /usr/local/bin/aws
```

Siga estas etapas a partir da linha de comando para instalar a AWS CLI versão 1 usando o instalador empacotado.

Para instalar a AWS CLI versão 1 usando o instalador empacotado

1. Baixe o instalador empacotado da AWS CLI versão 1 usando um dos seguintes métodos.
  - Faça download usando o comando `curl`.

Para a versão mais recente da AWS CLI, use o seguinte bloco de comandos:

```
$ curl "https://s3.amazonaws.com/aws-cli/awscli-bundle.zip" -o "awscli-bundle.zip"
```

Para uma versão específica da AWS CLI, acrescente um hífen e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão **1.16.312** seria `awscli-bundle-1.16.312.zip`, o que resultaria no seguinte comando:

```
$ curl "https://s3.amazonaws.com/aws-cli/awscli-bundle-1.16.312.zip" -o "awscli-bundle.zip"
```

- Baixe usando o link direto.

Para a versão mais recente da AWS CLI: <https://s3.amazonaws.com/aws-cli/awscli-bundle.zip>

Para uma versão específica da AWS CLI, acrescente um hífen e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão **1.16.312** seria `awscli-`

`bundle-1.16.312.zip`, o que resultaria no seguinte URL: <https://s3.amazonaws.com/aws-cli/awscli-bundle-1.16.312.zip>

2. Extraia os arquivos do pacote. Se você não tiver `unzip` para extrair os arquivos, use a distribuição do Linux incluída no gerenciador de pacote para fazer a instalação.

```
$ unzip awscli-bundle.zip
```

3. Execute o programa de instalação. O instalador instala a AWS CLI no `/usr/local/aws` e cria o symlink `aws` no diretório `/usr/local/bin`. Usar a opção `-b` para criar um symlink elimina a necessidade de especificar o diretório de instalação na variável `$PATH` do usuário. Isso permite que todos os usuários chamem a AWS CLI ao inserir `aws` de qualquer diretório.

```
$ sudo ./awscli-bundle/install -i /usr/local/aws -b /usr/local/bin/aws
```

Por padrão, o script de instalação é executado sob a versão padrão do sistema do Python. Se uma versão diferente do Python estiver instalada e você quiser usar essa versão para instalar a AWS CLI, execute o script de instalação com essa versão por caminho absoluto para o Python executável, conforme indicado a seguir.

```
$ sudo /usr/local/bin/python3.7 awscli-bundle/install -i /usr/local/aws -b /usr/local/bin/aws
```

4. Verifique se o AWS CLI está instalado corretamente.

```
$ aws --version
aws-cli/1.27.69 Python/3.11.2 Linux/4.14.133-113.105.amzn2.x86_64 botocore/1.13
```

Se houver um erro, consulte [Solucionar erros da AWS CLI](#).

## Instalar a AWS CLI versão 1 usando o instalador empacotado sem **sudo**

Caso não tenha permissões `sudo` ou deseje instalar a AWS CLI apenas para o usuário atual, você pode usar uma versão modificada dos comandos anteriores. Os dois primeiros comandos são os mesmos.

Para a versão mais recente da AWS CLI, use o seguinte bloco de comandos:

```
$ curl "https://s3.amazonaws.com/aws-cli/awscli-bundle.zip" -o "awscli-bundle.zip"
```

```
unzip awscli-bundle.zip
./awscli-bundle/install -b ~/bin/aws
```

Para uma versão específica da AWS CLI, acrescente um hífen e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão **1.16.312** seria `awscli-bundle-1.16.312.zip`, o que resultaria no seguinte comando:

```
$ curl "https://s3.amazonaws.com/aws-cli/awscli-bundle-1.16.312.zip" -o "awscli-bundle.zip"
unzip awscli-bundle.zip
./awscli-bundle/install -b ~/bin/aws
```

Para instalar a AWS CLI versão 1 para o usuário atual

1. Baixe o instalador empacotado da AWS CLI versão 1 de uma das seguintes maneiras.

- Faça download usando o comando `curl`.

Para a versão mais recente da AWS CLI, use o seguinte bloco de comandos:

```
$ curl "https://s3.amazonaws.com/aws-cli/awscli-bundle.zip" -o "awscli-bundle.zip"
```

Para uma versão específica da AWS CLI, acrescente um hífen e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão **1.16.312** seria `awscli-bundle-1.16.312.zip`, o que resultaria no seguinte comando:

```
$ curl "https://s3.amazonaws.com/aws-cli/awscli-bundle-1.16.312.zip" -o "awscli-bundle.zip"
```

- Baixe usando o link direto.

Para a versão mais recente da AWS CLI: <https://s3.amazonaws.com/aws-cli/awscli-bundle.zip>

Para uma versão específica da AWS CLI, acrescente um hífen e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão **1.16.312** seria `awscli-bundle-1.16.312.zip`, o que resultaria no seguinte URL: <https://s3.amazonaws.com/aws-cli/awscli-bundle-1.16.312.zip>

2. Extraia os arquivos do pacote usando `unzip`. Se você não tiver `unzip`, use a distribuição do Linux incluída no gerenciador de pacote para fazer a instalação.

```
$ unzip awscli-bundle.zip
```

3. Execute o programa de instalação. O instalador instala a AWS CLI no `/usr/local/aws` e cria o symlink `aws` no diretório `/usr/local/bin`. O comando usa o parâmetro `-b` para especificar o diretório no qual o instalador coloca o arquivo symlink `aws`. É necessário ter permissões de gravação na pasta especificada.

```
$ ./awscli-bundle/install -b ~/bin/aws
```

Isso instalará a AWS CLI no local padrão (`~/.local/lib/aws`) e criará um link simbólico (symlink) em `~/bin/aws`. Verifique se o `~/bin` está na sua variável de ambiente `PATH` para que o symlink funcione.

```
$ echo $PATH | grep ~/bin      // See if $PATH contains ~/bin (output will be empty if it doesn't)
$ export PATH=~/bin:$PATH      // Add ~/bin to $PATH if necessary
```

4. Verifique se o diretório da AWS CLI versão 1 faz parte da variável `PATH`.
  - a. Encontre o script de perfil do shell em sua pasta de usuário. Se não tiver certeza de qual shell você tem, execute `echo $SHELL`.

```
$ ls -a ~
.  ..  .bash_logout  .bash_profile  .bashrc  Desktop  Documents  Downloads
```

- Bash: `.bash_profile`, `.profile` ou `.bash_login`
- Zsh: `.zshrc`
- Tcsh: `.tcshrc`, `.cshrc` ou `.login`.

- b. Adicione um comando de exportação ao final do script de perfil que é semelhante ao exemplo a seguir.

```
export PATH=~/local/bin:$PATH
```

Esse comando insere o caminho `~/local/bin` neste exemplo na frente da variável `PATH` existente.

- c. Recarregue o perfil em sua sessão atual para colocar essas alterações em vigor.

```
$ source ~/.bash_profile
```

5. Verifique se o AWS CLI está instalado corretamente.

```
$ aws --version
aws-cli/1.27.69 Python/3.11.2 Linux/4.14.133-113.105.amzn2.x86_64 botocore/1.13
```

Se houver um erro, consulte [Solucionar erros da AWS CLI](#).

## Desinstalar o instalador empacotado da AWS CLI versão 1

1. Se você instalou a AWS CLI usando o instalador empacotado, siga estas instruções. O pacote de instalador não retira nada do diretório de instalação, exceto o symlink opcional. Portanto, a desinstalação é feita da mesma maneira destes dois itens.

```
$ sudo rm -rf /usr/local/aws
$ sudo rm /usr/local/bin/aws
```

2. (Opcional) Remova as informações compartilhadas do AWS SDK e das configurações da AWS CLI na pasta `.aws`.

### Warning

Essas configurações de configuração e credenciais são compartilhadas em todos os AWS SDKs e na AWS CLI. Se você remover essa pasta, elas não poderão ser acessadas por nenhum AWS SDK que ainda estiver em seu sistema.

O local padrão da pasta `.aws` difere entre plataformas. Por padrão, a pasta está localizada em `~/.aws/`. Se o seu usuário tiver permissão de gravação nesse diretório, não será necessário usar `sudo`.

```
$ sudo rm -r ~/.aws/
```



# Instalar e desinstalar a AWS CLI versão 1 usando pip

## Tópicos

- [Instalar o pip](#)
- [Instalar e atualizar a AWS CLI versão 1 usando pip](#)
- [Adicionar o executável da AWS CLI versão 1 ao caminho da linha de comando](#)
- [Desinstalar a AWS CLI usando pip](#)
- [Solução de problemas de erros de instalação e desinstalação da AWS CLI](#)

## Instalar o pip

Se você ainda não tem o pip instalado, você pode instalá-lo usando o script que o Python Packaging Authority fornece. Execute o `pip --version` para ver se a sua versão do Linux já inclui o Python e o pip. Se o Python versão 3 ou posterior estiver instalado, recomendamos que você use o comando `pip3`.

1. Use o comando `curl` a seguir para baixar o script de instalação. O comando a seguir usa o parâmetro `-O` (letra “O” maiúscula) para especificar que o arquivo baixado deve ser armazenado no diretório atual usando o mesmo nome que ele tem no host remoto.

```
$ curl -O https://bootstrap.pypa.io/get-pip.py
```

2. Execute o script com o comando `python` ou `python3` para baixar e instalar a versão mais recente do pip e de outros pacotes de suporte necessários. Quando você inclui a chave `--user`, o script instala o pip no caminho `~/.local/bin`.

```
$ python3 get-pip.py --user
```

3. Verifique se o diretório que contém o pip faz parte da variável `PATH`.
  - a. Encontre o script de perfil do shell em sua pasta de usuário. Se não tiver certeza de qual shell você tem, execute `echo $SHELL`.

```
$ ls -a ~  
.  ..  .bash_logout  .bash_profile  .bashrc  Desktop  Documents  Downloads
```

- Bash: `.bash_profile`, `.profile` ou `.bash_login`

- Zsh: `.zshrc`
  - Tcsh: `.tcshrc`, `.cshrc` ou `.login`.
- b. Adicione um comando de exportação ao final do script de perfil que é semelhante ao exemplo a seguir.

```
export PATH=~/.local/bin:$PATH
```

Esse comando insere o caminho `~/.local/bin` neste exemplo na frente da variável `PATH` existente.

- c. Recarregue o perfil em sua sessão atual para colocar essas alterações em vigor.

```
$ source ~/.bash_profile
```

4. Para verificar se `pip` ou `pip3` está instalado corretamente, execute o comando a seguir.

```
$ pip3 --version
pip 19.2.3 from ~/.local/lib/python3.7/site-packages (python 3.7)
```

## Instalar e atualizar a AWS CLI versão 1 usando pip

1. Use o comando `pip` ou `pip3` para instalar ou atualizar a AWS CLI. Se você usar o Python versão 3 ou posterior, recomendamos usar o comando `pip3`. O switch `--user`, `pip` instala a AWS CLI em `~/.local/bin`.

Para a versão mais recente da AWS CLI, use o seguinte bloco de comandos:

```
$ pip3 install awscli --upgrade --user
```

Para uma versão específica da AWS CLI, acrescente um hífen e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão `1.16.312` seria `==1.16.312`, o que resultaria no seguinte comando:

```
$ pip3 install awscli==1.16.312 --upgrade --user
```

 Note

Use as regras de aspas apropriadas para o terminal. Para usar o caractere =, você pode precisar usar aspas simples ou duplas para realizar o escape corretamente. O escape de exemplo a seguir usa aspas simples:

```
$ pip3 install 'awscli==1.16.312' --upgrade --user
```

2. Verifique se o AWS CLI está instalado corretamente.

```
$ aws --version  
aws-cli/1.27.69 Python/3.11.2 Linux/4.14.133-113.105.amzn2.x86_64 botocore/1.13
```

Se houver um erro, consulte [Solucionar erros da AWS CLI](#).

## Adicionar o executável da AWS CLI versão 1 ao caminho da linha de comando

Após a instalação com pip, talvez seja necessário adicionar o executável aws à variável de ambiente PATH de seu sistema operacional.

É possível verificar em qual pasta o pip instalou a AWS CLI executando o comando a seguir.

```
$ which aws  
/home/username/.local/bin/aws
```

Você pode fazer referência a isso como ~/.local/bin/ porque /home/username corresponde a ~ no Linux.

Se você omitiu a chave --user e, portanto, não a instalou no modo usuário, o arquivo executável poderá estar na pasta bin de sua instalação do Python. Se você não souber onde o Python está instalado, execute este comando.

```
$ which python  
/usr/local/bin/python
```

A saída pode ser o caminho para um symlink, e não para o arquivo executável real. Execute ls -al para saber para onde ele aponta.

```
$ ls -al /usr/local/bin/python
/usr/local/bin/python -> ~/.local/Python/3.6/bin/python3.6
```

O pip instala programas na mesma pasta que contém o aplicativo Python. Adicione esta pasta à variável PATH.

Como modificar a variável **PATH**

1. Encontre o script de perfil do shell no diretório de usuário. Se não tiver certeza de qual shell você tem, execute `echo $SHELL`.

```
$ ls -a ~
.  ..  .bash_logout  .bash_profile  .bashrc  Desktop  Documents  Downloads
```

- Bash – `.bash_profile`, `.profile`, ou `.bash_login`
- Zsh – `.zshrc`
- Tcsh – `.tcshrc`, `.cshrc`, ou `.login`

2. Adicione um comando de exportação ao script de perfil.

```
export PATH=~/.local/bin:$PATH
```

Este comando adiciona um caminho, `~/.local/bin` neste exemplo, para a variável PATH atual.

3. Carregue o perfil atualizado em sua sessão atual.

```
$ source ~/.bash_profile
```

## Desinstalar a AWS CLI usando pip

1. Se você instalou a AWS CLI versão 1 usando pip, também será necessário desinstalá-la usando pip.

```
$ pip uninstall awscli
```

Se você estiver usando o Python versão 2 ou 3, talvez seja necessário usar o comando `pip2` ou `pip3`. Use o comando `aws --version` para determinar a versão do Python associada à AWS CLI versão 1 instalada.

```
$ pip3 uninstall awscli
```

Talvez seja necessário reiniciar a janela do prompt de comando ou o computador para remover todos os arquivos.

2. (Opcional) Remova as informações compartilhadas do AWS SDK e das configurações da AWS CLI na pasta `.aws`.

#### Warning

Essas configurações de configuração e credenciais são compartilhadas em todos os AWS SDKs e na AWS CLI. Se você remover essa pasta, elas não poderão ser acessadas por nenhum AWS SDK que ainda estiver em seu sistema.

O local padrão da pasta `.aws` difere entre plataformas. Por padrão, a pasta está localizada em `~/.aws/`. Se o seu usuário tiver permissão de gravação nesse diretório, não será necessário usar `sudo`.

```
$ sudo rm ~/.aws/
```

## Solução de problemas de erros de instalação e desinstalação da AWS CLI

Se você encontrar problemas após instalar ou desinstalar a AWS CLI, consulte [Solucionar erros](#) para obter os passos para a solução de problemas. Para obter os passos mais relevantes para a solução de problemas, consulte [the section called “Erros de comando não encontrado”](#), [the section called “O comando “aws --version” retorna uma versão diferente da que você instalou”](#) e [the section called “O comando “aws --version” retorna uma versão após a desinstalação da AWS CLI”](#).

## Instalar, atualizar e desinstalar a AWS CLI versão 1 no macOS

É possível instalar a AWS Command Line Interface (AWS CLI) versão 1 e suas dependências no macOS usando o instalador empacotado ou o `pip`.

## Seções

- [Pré-requisitos](#)
- [Instalar, atualizar e desinstalar a AWS CLI versão 1 no macOS usando o instalador empacotado](#)
- [Instalar, atualizar e desinstalar a AWS CLI versão 1 usando pip](#)
- [Solução de problemas de erros de instalação e desinstalação da AWS CLI](#)

## Pré-requisitos

Antes de instalar a AWS CLI versão 1 no macOS, o Python 3.8 ou posterior deverá ser instalado. Para obter instruções de instalação, consulte a página [Downloading Python](#) no Beginner's Guide do Python.

Matriz de suporte a versões do Python

| Versão do AWS CLI | Versão do Python com suporte                     |
|-------------------|--------------------------------------------------|
| 1.32.0: atual     | Python 3.8 e posterior                           |
| 1.27.0–1.31.x     | Python 3.7 e posterior                           |
| 1.20.0–1.26.x     | Python 3.6                                       |
| 1.19.0: 1.19.x    | Python 2.7+, Python 3.6+                         |
| 1.17 – 1.18.x     | Python 2.7+, Python 3.4+                         |
| 1.0 – 1.16.x      | Python 2.6 e anteriores, Python 3.3 e anteriores |

## Instalar, atualizar e desinstalar a AWS CLI versão 1 no macOS usando o instalador empacotado

No Linux ou macOS, é possível usar o instalador empacotado para instalar a versão 1 da AWS Command Line Interface (AWS CLI). O pacote de instalador inclui todas as dependências do pacote e pode ser usado offline.

O pacote de instalador fornecido não é compatível com a instalação em caminhos com espaços.

## Tópicos

- [Instalar a AWS CLI versão 1 usando o instalador empacotado com sudo](#)
- [Instalar a AWS CLI versão 1 usando o instalador empacotado sem sudo](#)
- [Desinstalar o instalador empacotado da AWS CLI versão 1](#)

## Instalar a AWS CLI versão 1 usando o instalador empacotado com **sudo**

As etapas a seguir permitem que você instale a AWS CLI versão 1 via linha de comando em qualquer compilação do macOS.

Veja a seguir um resumo dos comandos de instalação que você pode recortar e colar para executar como um único conjunto de comandos.

Para a versão mais recente da AWS CLI, use o seguinte bloco de comandos:

```
$ curl "https://s3.amazonaws.com/aws-cli/awscli-bundle.zip" -o "awscli-bundle.zip"
unzip awscli-bundle.zip
sudo ./awscli-bundle/install -i /usr/local/aws -b /usr/local/bin/aws
```

Para uma versão específica da AWS CLI, acrescente um hífen e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão **1.16.312** seria `awscli-bundle-1.16.312.zip`, o que resultaria no seguinte comando:

```
$ curl "https://s3.amazonaws.com/aws-cli/awscli-bundle-1.16.312.zip" -o "awscli-bundle.zip"
unzip awscli-bundle.zip
sudo ./awscli-bundle/install -i /usr/local/aws -b /usr/local/bin/aws
```

Para instalar a AWS CLI versão 1 usando o instalador empacotado

1. Baixe o instalador empacotado da AWS CLI versão 1 de uma das seguintes maneiras:

- Faça download usando o comando `curl`.

Para a versão mais recente da AWS CLI, use o seguinte bloco de comandos:

```
$ curl "https://s3.amazonaws.com/aws-cli/awscli-bundle.zip" -o "awscli-bundle.zip"
```

Para uma versão específica da AWS CLI, acrescente um hífen e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão **1.16.312** seria `awscli-bundle-1.16.312.zip`, o que resultaria no seguinte comando:

```
$ curl "https://s3.amazonaws.com/aws-cli/awscli-bundle-1.16.312.zip" -o "awscli-bundle.zip"
```

- Baixe usando o link direto.

Para a versão mais recente da AWS CLI: <https://s3.amazonaws.com/aws-cli/awscli-bundle.zip>

Para uma versão específica da AWS CLI, acrescente um hífen e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão **1.16.312** seria `awscli-bundle-1.16.312.zip`, o que resultaria no seguinte URL: <https://s3.amazonaws.com/aws-cli/awscli-bundle-1.16.312.zip>

2. Extraia (descompacte) os arquivos do pacote. Se você não tiver `unzip`, use o gerenciador de pacote integrado da distribuição do macOS para fazer a instalação.

```
$ unzip awscli-bundle.zip
```

3. Execute o programa de instalação. O instalador instala a AWS CLI no `/usr/local/aws` e cria o symlink `aws` na pasta `/usr/local/bin`. Usar a opção `-b` para a criação de um symlink elimina a necessidade de especificar a pasta de instalação na variável `$PATH` do usuário. Isso permite que todos os usuários chamem a AWS CLI ao inserir `aws` de qualquer diretório.

```
$ sudo ./awscli-bundle/install -i /usr/local/aws -b /usr/local/bin/aws
```

Por padrão, o script de instalação é executado sob a versão padrão do sistema do Python. Se uma versão diferente do Python estiver instalada e você quiser usá-la para instalar a AWS CLI, execute o script de instalação com essa versão por caminho absoluto para o Python executável, conforme indicado a seguir.

```
$ sudo /usr/local/bin/python3.7 awscli-bundle/install -i /usr/local/aws -b /usr/local/bin/aws
```

4. Verifique se o AWS CLI está instalado corretamente.

```
$ aws --version
```



```
aws-cli/1.27.69 Python/3.11.2 Linux/4.14.133-113.105.amzn2.x86_64 botocore/1.13
```

Se houver um erro, consulte [Solucionar erros da AWS CLI](#).

## Instalar a AWS CLI versão 1 usando o instalador empacotado sem **sudo**

Caso não tenha permissões sudo ou deseje instalar a AWS CLI apenas para o usuário atual, você pode usar uma versão modificada dos comandos anteriores. Os dois primeiros comandos são os mesmos.

Para a versão mais recente da AWS CLI, use o seguinte bloco de comandos:

```
$ curl "https://s3.amazonaws.com/aws-cli/awscli-bundle.zip" -o "awscli-bundle.zip"
unzip awscli-bundle.zip
./awscli-bundle/install -b ~/bin/aws
```

Para uma versão específica da AWS CLI, acrescente um hífen e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão **1.16.312** seria awscli-bundle-1.16.312.zip, o que resultaria no seguinte comando:

```
$ curl "https://s3.amazonaws.com/aws-cli/awscli-bundle-1.16.312.zip" -o "awscli-bundle.zip"
unzip awscli-bundle.zip
./awscli-bundle/install -b ~/bin/aws
```

Para instalar a AWS CLI versão 1 para o usuário atual

1. Baixe o instalador empacotado da AWS CLI versão 1 usando um dos seguintes métodos:

- Faça download usando o comando `curl`.

Para a versão mais recente da AWS CLI, use o seguinte bloco de comandos:

```
$ curl "https://s3.amazonaws.com/aws-cli/awscli-bundle.zip" -o "awscli-bundle.zip"
```

Para uma versão específica da AWS CLI, acrescente um hífen e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão **1.16.312** seria awscli-bundle-1.16.312.zip, o que resultaria no seguinte comando:

```
$ curl "https://s3.amazonaws.com/aws-cli/awscli-bundle-1.16.312.zip" -o "awscli-bundle.zip"
```

- Baixe usando o link direto.

Para a versão mais recente da AWS CLI: <https://s3.amazonaws.com/aws-cli/awscli-bundle.zip>

Para uma versão específica da AWS CLI, acrescente um hífen e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão **1.16.312** seria `awscli-bundle-1.16.312.zip`, o que resultaria no seguinte URL: <https://s3.amazonaws.com/aws-cli/awscli-bundle-1.16.312.zip>

2. Extraia os arquivos do pacote. Se você não tiver `unzip`, use a distribuição do Linux incluída no gerenciador de pacote para fazer a instalação.

```
$ unzip awscli-bundle.zip
```

3. Execute o programa de instalação. O instalador instala a AWS CLI no `/usr/local/aws` e cria o symlink `aws` no diretório `/usr/local/bin`. O comando usa o parâmetro `-b` para especificar o diretório no qual o instalador coloca o arquivo symlink `aws`. É necessário ter permissões de gravação no diretório especificado.

```
$ ./awscli-bundle/install -b ~/bin/aws
```

Isso instalará a AWS CLI no local padrão (`~/local/lib/aws`) e criará um link simbólico (symlink) em `~/bin/aws`. Verifique se o `~/bin` está na sua variável de ambiente `$PATH` para que o symlink funcione.

```
$ echo $PATH | grep ~/bin    // See if $PATH contains ~/bin (output will be empty if it doesn't)
$ export PATH=~/bin:$PATH    // Add ~/bin to $PATH if necessary
```

4. Verifique se a pasta na qual a AWS CLI versão 1 está instalada faz parte da variável `$PATH`.
  - a. Encontre o script de perfil do shell em sua pasta de usuário. Se não tiver certeza de qual shell você tem, execute `echo $SHELL`.

```
$ ls -a ~
.  ..  .bash_logout  .bash_profile  .bashrc  Desktop  Documents  Downloads
```

- Bash: `.bash_profile`, `.profile` ou `.bash_login`
- Zsh: `.zshrc`
- Tcsh: `.tcshrc`, `.cshrc` ou `.login`.

- b. Adicione um comando de exportação ao final do script de perfil que é semelhante ao exemplo a seguir.

```
export PATH=~/.local/bin:$PATH
```

Esse comando insere o caminho `~/.local/bin` neste exemplo na frente da variável `PATH` existente.

- c. Recarregue o perfil em sua sessão atual para colocar essas alterações em vigor.

```
$ source ~/.bash_profile
```

5. Verifique se o AWS CLI está instalado corretamente.

```
$ aws --version
aws-cli/1.27.69 Python/3.11.2 Linux/4.14.133-113.105.amzn2.x86_64 botocore/1.13
```

Se houver um erro, consulte [Solucionar erros da AWS CLI](#).

## Desinstalar o instalador empacotado da AWS CLI versão 1

1. O instalador empacotado coloca tudo dentro do diretório de instalação, exceto o symlink opcional, portanto, para desinstalar, basta excluir esses dois itens.

```
$ sudo rm -rf /usr/local/aws
$ sudo rm /usr/local/bin/aws
```

2. (Opcional) Remova as informações compartilhadas do AWS SDK e das configurações da AWS CLI na pasta `.aws`.

 Warning

Essas configurações de configuração e credenciais são compartilhadas em todos os AWS SDKs e na AWS CLI. Se você remover essa pasta, elas não poderão ser acessadas por nenhum AWS SDK que ainda estiver em seu sistema.

O local padrão da pasta `.aws` difere entre plataformas. Por padrão, a pasta está localizada em `~/.aws/`. Se o seu usuário tiver permissão de gravação nesse diretório, não será necessário usar `sudo`.

```
$ sudo rm ~/.aws/
```

## Instalar, atualizar e desinstalar a AWS CLI versão 1 usando pip

Também é possível usar o `pip` diretamente para instalar a AWS CLI.

### Tópicos

- [Instalar o pip](#)
- [Instalar e atualizar a AWS CLI usando pip](#)
- [Adicionar o executável da AWS CLI versão 1 ao caminho da linha de comando do macOS](#)
- [Desinstalar a AWS CLI usando pip](#)

### Instalar o pip

Se você ainda não tem o `pip` instalado, você pode instalá-lo usando o script que o Python Packaging Authority fornece. Execute o `pip --version` para ver se a sua versão do Linux já inclui o Python e o `pip`. Se o Python versão 3 ou posterior estiver instalado, recomendamos que você use o comando `pip3`.

1. Use o comando `curl` a seguir para baixar o script de instalação. O comando a seguir usa o parâmetro `-O` (letra “O” maiúscula) para especificar que o arquivo baixado deve ser armazenado na pasta atual usando o mesmo nome que ele tem no host remoto.

```
$ curl -O https://bootstrap.pypa.io/get-pip.py
```

2. Execute o script com o comando `python` ou `python3` para baixar e instalar a versão mais recente do `pip` e de outros pacotes de suporte necessários. Quando você inclui a chave `--user`, o script instala o `pip` no caminho `~/.local/bin`.

```
$ python3 get-pip.py --user
```

## Instalar e atualizar a AWS CLI usando pip

1. Use o comando `pip` ou `pip3` para instalar a AWS CLI. Se você usar o Python versão 3 ou posterior, recomendamos usar o comando `pip3`.

Para a versão mais recente da AWS CLI, use o seguinte bloco de comandos:

```
$ pip3 install awscli --upgrade --user
```

Para uma versão específica da AWS CLI, acrescente um hífen e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão **1.16.312** seria **`==1.16.312`**, o que resultaria no seguinte comando:

```
$ pip3 install awscli==1.16.312 --upgrade --user
```

### Note

Use as regras de aspas apropriadas para o terminal. Para usar o caractere `=`, você pode precisar usar aspas simples ou duplas para realizar o escape corretamente. O escape de exemplo a seguir usa aspas simples:

```
$ pip3 install 'awscli==1.16.312' --upgrade --user
```

2. Verifique se o AWS CLI está instalado corretamente.

```
$ aws --version  
aws-cli/1.27.69 Python/3.11.2 Darwin/18.7.0 botocore/1.13
```

Se o programa não for encontrado, [adicione-o ao caminho da linha de comando](#).

## Adicionar o executável da AWS CLI versão 1 ao caminho da linha de comando do macOS

Após a instalação com o `pip`, talvez seja necessário adicionar o programa `aws` à variável de ambiente `PATH` do seu sistema operacional. A localização do programa depende de onde o Python está instalado.

Exemplo Local de instalação da AWS CLI – macOS com Python 3.6 e **pip** (modo usuário)

```
~/Library/Python/3.7/bin
```

Substitua a versão do Python que você tem pela versão do exemplo anterior.

Se você não souber onde o Python está instalado, execute `which python`.

```
$ which python
/usr/local/bin/python
```

A saída pode ser o caminho para um symlink, e não o programa executável real. Execute `ls -al` para saber para onde ele aponta.

```
$ ls -al /usr/local/bin/python
~/Library/Python/3.7/bin/python3.7
```

O `pip` instala programas na mesma pasta que contém o aplicativo Python. Adicione esta pasta à variável `PATH`.

### Como modificar a variável **PATH**

1. Encontre o script de perfil do shell no diretório de usuário. Se não tiver certeza de qual shell você tem, execute `echo $SHELL`.

```
$ ls -a ~
.  ..  .bash_logout  .bash_profile  .bashrc  Desktop  Documents  Downloads
```

- Bash – `.bash_profile`, `.profile`, ou `.bash_login`
- Zsh – `.zshrc`
- Tcsh – `.tcshrc`, `.cshrc`, ou `.login`

2. Adicione um comando de exportação ao script de perfil.

```
export PATH=~/.local/bin:$PATH
```

Este comando adiciona um caminho, `~/.local/bin` neste exemplo, para a variável `PATH` atual.

3. Carregue o perfil atualizado em sua sessão atual.

```
$ source ~/.bash_profile
```

## Desinstalar a AWS CLI usando pip

1. Se você instalou a AWS CLI versão 1 usando `pip`, também será necessário desinstalá-la usando `pip`.

```
$ pip uninstall awscli
```

Se você estiver usando o Python versão 2 ou 3, talvez seja necessário usar o comando `pip2` ou `pip3`. Use o comando `aws --version` para determinar a versão do Python associada à AWS CLI versão 1 instalada.

```
$ pip3 uninstall awscli
```

Talvez seja necessário reiniciar a janela do prompt de comando ou o computador para remover todos os arquivos.

2. (Opcional) Remova as informações compartilhadas do AWS SDK e das configurações da AWS CLI na pasta `.aws`.

### Warning

Essas configurações de configuração e credenciais são compartilhadas em todos os AWS SDKs e na AWS CLI. Se você remover essa pasta, elas não poderão ser acessadas por nenhum AWS SDK que ainda estiver em seu sistema.

O local padrão da pasta `.aws` difere entre plataformas. Por padrão, a pasta está localizada em `~/.aws/`. Se o seu usuário tiver permissão de gravação nesse diretório, não será necessário usar `sudo`.

```
$ sudo rm ~/.aws/
```

## Solução de problemas de erros de instalação e desinstalação da AWS CLI

Se você encontrar problemas após instalar ou desinstalar a AWS CLI, consulte [Solucionar erros](#) para obter os passos para a solução de problemas. Para obter os passos mais relevantes para a solução de problemas, consulte [the section called “Erros de comando não encontrado”](#), [the section called “O comando “aws --version” retorna uma versão diferente da que você instalou”](#) e [the section called “O comando “aws --version” retorna uma versão após a desinstalação da AWS CLI”](#).

## Instalar, atualizar e desinstalar a AWS CLI versão 1 no Windows

Você pode instalar a versão 1 da AWS Command Line Interface (AWS CLI) no Windows usando um instalador independente (recomendado) ou o `pip`, que é um gerenciador de pacotes para Python.

Não inclua o símbolo de prompt (`C:\>`) ao digitar um comando. Eles são incluídos nas listas do programa para diferenciar comandos que você digita da saída retornada pela AWS CLI. O restante desse guia usa o símbolo de prompt genérico (`$`), exceto nos casos em que um comando é específico do Windows.

### Tópicos

- [Instalar, atualizar e desinstalar a AWS CLI versão 1 usando o instalador MSI](#)
- [Instalar, atualizar e desinstalar a AWS CLI versão 1 usando Python e pip no Windows](#)
- [Adicionar o executável da AWS CLI versão 1 ao caminho da linha de comando](#)
- [Solução de problemas de erros de instalação e desinstalação da AWS CLI](#)



# Instalar, atualizar e desinstalar a AWS CLI versão 1 usando o instalador MSI

A AWS CLI versão 1 é compatível com o Windows XP ou posterior. Para usuários do Windows, o pacote de instalação MSI oferece uma maneira familiar e conveniente de instalar a AWS CLI versão 1 sem necessitar de nenhum outro pré-requisito.

## Instalar e atualizar a AWS CLI versão 1 usando o instalador MSI

Confira a página de [Liberações](#) no GitHub para ver quando a versão mais recente foi lançada. Quando novas atualizações forem lançadas, você deverá repetir o processo de instalação para instalar a versão mais recente da AWS CLI versão 1.

### 1. Baixe o instalador MSI apropriado:

- Instalador MSI da AWS CLI para Windows (64 bits): <https://s3.amazonaws.com/aws-cli/AWSCLI64PY3.msi>
- Instalador MSI da AWS CLI para Windows (32 bits): <https://s3.amazonaws.com/aws-cli/AWSCLI32PY3.msi>
- Arquivo de configuração combinado da AWS CLI para Windows: <https://s3.amazonaws.com/aws-cli/AWSCLISetup.exe> (inclui os instaladores MSI de 32 bits e de 64 bits e instala automaticamente a versão correta)

### 2. Execute o instalador MSI baixado ou o arquivo de configuração.

### 3. Siga as instruções da tela. Por padrão, a AWS CLI versão 1 é instalada em C:\Program Files\Amazon\AWSCLI (versão de 64 bits) ou em C:\Program Files (x86)\Amazon\AWSCLI (versão de 32 bits).

### 4. Para confirmar a instalação, use o comando `aws --version` em um prompt de comando (abra o menu Start (Iniciar) e pesquise cmd para iniciar um prompt de comando).

```
C:\> aws --version
aws-cli/1.27.69 Python/3.11.2 Windows/10 botocore/1.13
```

Se o Windows não conseguir localizar o programa, talvez seja necessário fechar e abrir o prompt de comando novamente para atualizar o caminho ou [adicionar o diretório de instalação à variável de ambiente PATH](#) manualmente.

## Desinstalar a AWS CLI versão 1

Para usar as instruções de desinstalação a seguir, é necessário ter instalado a AWS CLI versão 1 com o instalador MSI ou com o arquivo de configuração.

1. Abra Programas e Recursos seguindo um destes procedimentos:

- Abra o Painel de Controle e selecione Programas e Recursos.
- Abra um prompt de comando e execute o comando a seguir.

```
C:\> appwiz.cpl
```

2. Selecione a entrada denominada AWS Command Line Interface e escolha Uninstall (Desinstalar) para executar o desinstalador.
3. Confirme que deseja desinstalar a AWS CLI.
4. (Opcional) Remova as informações compartilhadas do AWS SDK e das configurações da AWS CLI na pasta `.aws`.

### Warning

Essas configurações de configuração e credenciais são compartilhadas em todos os AWS SDKs e na AWS CLI. Se você remover essa pasta, elas não poderão ser acessadas por nenhum AWS SDK que ainda estiver em seu sistema.

O local padrão da pasta `.aws` difere entre plataformas. Por padrão, a pasta está localizada em `%UserProfile%\aws`.

```
$ rmdir %UserProfile%\aws
```

## Instalar, atualizar e desinstalar a AWS CLI versão 1 usando Python e pip no Windows

O Python Software Foundation fornece instaladores para Windows que incluem pip.

## Pré-requisitos

Você deve ter o Python 3.8 ou posterior instalado. Para obter instruções de instalação, consulte a página [Downloading Python](#) no Beginner's Guide do Python.

### Warning

O Python 2.7 foi descontinuado pela [Python Software Foundation](#) em 1º de janeiro de 2020. A partir da AWS CLI versão 1.20.0, é necessária uma versão mínima do Python 3.6. Para usar a AWS CLI versão 1 com uma versão anterior do Python, você precisará instalar uma versão anterior da AWS CLI versão 1. Para visualizar a matriz de suporte a versões do Python para a AWS CLI versão 1, consulte [the section called “Requisitos da versão do Python”](#).

## Instalar e atualizar a AWS CLI versão 1 usando pip

1. Para instalar a AWS CLI versão 1, use o comando `pip3` (se você usar o Python versão 3 ou posterior) ou o comando `pip`.

Para a versão mais recente da AWS CLI, use o seguinte bloco de comandos:

```
C:\> pip3 install awscli --upgrade --user
```

Para uma versão específica da AWS CLI, acrescente símbolo de menor que `<` e o número da versão ao nome do arquivo. Neste exemplo, o nome do arquivo para a versão **1.16.312** seria **<1.16.312**, o que resultaria no seguinte comando:

```
C:\> pip3 install awscli<1.16.312 --upgrade --user
```

2. Verifique se a AWS CLI versão 1 está instalada corretamente. Se não houver resposta, consulte a seção [Adicionar o executável da AWS CLI versão 1 ao caminho da linha de comando](#).

```
C:\> aws --version
aws-cli/1.27.69 Python/3.11.2 Windows/10 botocore/1.13
```

## Desinstalar a AWS CLI versão 1 usando pip

1. Se você instalou a AWS CLI versão 1 usando pip, também será necessário desinstalá-la usando pip.

```
C:\> pip uninstall awscli
```

Se você estiver usando o Python versão 2 ou 3, talvez seja necessário usar o comando pip2 ou pip3. Use o comando `aws --version` para determinar a versão do Python associada à AWS CLI versão 1 instalada.

```
C:\> pip3 uninstall awscli
```

Talvez seja necessário reiniciar a janela do prompt de comando ou o computador para remover todos os arquivos.

2. (Opcional) Remova as informações compartilhadas do AWS SDK e das configurações da AWS CLI na pasta `.aws`.

### Warning

Essas configurações de configuração e credenciais são compartilhadas em todos os AWS SDKs e na AWS CLI. Se você remover essa pasta, elas não poderão ser acessadas por nenhum AWS SDK que ainda estiver em seu sistema.

O local padrão da pasta `.aws` difere entre plataformas. Por padrão, a pasta está localizada em `%UserProfile%\aws`.

```
$ rmdir %UserProfile%\aws
```

## Adicionar o executável da AWS CLI versão 1 ao caminho da linha de comando

Após a instalação da AWS CLI versão 1 com o pip, adicione o programa `aws` à variável de ambiente PATH do seu sistema operacional. Com uma instalação MSI, isso deve acontecer automaticamente.

Mas se o comando `aws` não for executado após a instalação, talvez seja necessário configurá-lo manualmente.

1. Use o comando `where` para encontrar o local do arquivo `aws`. Por padrão, o comando `where` mostra onde um programa especificado é encontrado no PATH do sistema.

```
C:\> where aws
```

Os caminhos que aparecem dependem da plataforma e do método utilizado para instalar a AWS CLI. Os nomes de pasta que incluem os números de versão podem variar. Esses exemplos refletem o uso do Python versão 3.7. Substitua a versão pelo número de versão que você está usando, conforme necessário. Os caminhos típicos incluem o seguinte:

- Python 3 e **pip3** – C:\Program Files\Python37\Scripts\
- Python 3 e opção `--user` do **pip3** em versões anteriores do Windows – %USERPROFILE%\AppData\Local\Programs\Python\Python37\Scripts
- Python 3 e **pip3**: opção do usuário no Windows 10: %USERPROFILE%\AppData\Roaming\Python\Python37\Scripts
- Instalador MSI (64 bits): C:\Program Files\Amazon\AWSCLI\bin
- Instalador MSI (32 bits): C:\Program Files (x86)\Amazon\AWSCLI\bin

Use as etapas a seguir levando em conta se um caminho de arquivo é retornado.

A file path is returned

```
C:\> where aws
C:\Program Files\Amazon\AWSCLI\bin\aws.exe
```

É possível encontrar onde o programa `aws` está instalado executando o comando a seguir.

```
C:\> where c:\ aws
C:\Program Files\Python37\Scripts\aws
```

A file path is NOT returned

Se o comando `where` retornar o erro a seguir, ele não estará no PATH do sistema e você não poderá executá-lo inserindo seu nome.

```
C:\> where c:\ aws
INFO: Could not find files for the given pattern(s).
```

Nesse caso, execute o comando `where` com o parâmetro `/R` *path* para instruí-lo a pesquisar todas as pastas e adicione o caminho manualmente. Use a linha de comando ou o Explorador de Arquivos para descobrir onde ele está instalado no computador.

```
C:\> where /R c:\ aws
c:\Program Files\Amazon\AWSCLI\bin\aws.exe
c:\Program Files\Amazon\AWSCLI\bincompat\aws.cmd
c:\Program Files\Amazon\AWSCLI\runtime\Scripts\aws
c:\Program Files\Amazon\AWSCLI\runtime\Scripts\aws.cmd
...
```

2. Pressione a tecla Windows e digite **environment variables**.
3. Escolha Edit environment variables for your account (Editar variáveis de ambiente para sua conta).
4. Selecione PATH e, em seguida, Edit (Editar).
5. Adicione o caminho encontrado no campo Variable value (Valor da variável). Por exemplo, **C:\Program Files\Amazon\AWSCLI\bin\aws.exe**.
6. Escolha OK duas vezes para aplicar as novas configurações.
7. Feche todos os prompts de comando em execução e abra novamente a janela do prompt de comando.

## Solução de problemas de erros de instalação e desinstalação da AWS CLI

Se você encontrar problemas após instalar ou desinstalar a AWS CLI, consulte [Solucionar erros](#) para obter os passos para a solução de problemas. Para obter os passos mais relevantes para a solução de problemas, consulte [the section called “Erros de comando não encontrado”](#), [the section called “O comando “aws --version” retorna uma versão diferente da que você instalou”](#) e [the section called “O comando “aws --version” retorna uma versão após a desinstalação da AWS CLI”](#).

## Instalar e atualizar a AWS CLI versão 1 em um ambiente virtual

Instale a versão 1 da AWS Command Line Interface (AWS CLI) em um ambiente virtual para evitar conflitos de versão de requisito com outros pacotes pip.

## Tópicos

- [Pré-requisitos](#)
- [Instalar e atualizar a AWS CLI versão 1 em um ambiente virtual](#)
- [Solução de problemas de erros de instalação e desinstalação da AWS CLI](#)

## Pré-requisitos

- Python 3.8 ou posterior. Para obter instruções de instalação, consulte a página [Downloading Python](#) no Beginner's Guide do Python.

Matriz de suporte a versões do Python

| Versão do AWS CLI | Versão do Python com suporte                     |
|-------------------|--------------------------------------------------|
| 1.32.0: atual     | Python 3.8 e posterior                           |
| 1.27.0–1.31.x     | Python 3.7 e posterior                           |
| 1.20.0–1.26.x     | Python 3.6                                       |
| 1.19.0: 1.19.x    | Python 2.7+, Python 3.6+                         |
| 1.17 – 1.18.x     | Python 2.7+, Python 3.4+                         |
| 1.0 – 1.16.x      | Python 2.6 e anteriores, Python 3.3 e anteriores |

- pip ou pip3 está instalado.

## Instalar e atualizar a AWS CLI versão 1 em um ambiente virtual

1. Instale virtualenv usando o pip.

```
$ pip install --user virtualenv
```

2. Crie um ambiente virtual e nomeie-o.

```
$ virtualenv ~/cli-ve
```

Como alternativa, você pode usar a opção `-p` para especificar uma versão do Python que não seja a padrão.

```
$ virtualenv -p /usr/bin/python37 ~/cli-ve
```

3. Ative seu novo ambiente virtual.

Linux ou macOS

```
$ source ~/cli-ve/bin/activate
```

Windows

```
$ %USERPROFILE%\cli-ve\Scripts\activate
```

O prompt mudará para mostrar que o ambiente virtual está ativo.

```
(cli-ve)~$
```

4. Instale ou atualize a AWS CLI versão 1 em seu ambiente virtual.

```
(cli-ve)~$ pip install --upgrade awscli
```

5. Verifique se a AWS CLI versão 1 está instalada corretamente.

```
$ aws --version  
aws-cli/1.27.69 Python/3.11.2 Linux/4.14.133-113.105.amzn2.x86_64 botocore/1.13
```

6. Use o comando `deactivate` para sair do ambiente virtual. Sempre que você iniciar uma nova sessão, deverá ativar novamente o ambiente.

## Solução de problemas de erros de instalação e desinstalação da AWS CLI

Se você encontrar problemas após instalar ou desinstalar a AWS CLI, consulte [Solucionar erros](#) para obter os passos para a solução de problemas. Para obter os passos mais relevantes para a solução de problemas, consulte [the section called “Erros de comando não encontrado”](#), [the section called “O comando “aws --version” retorna uma versão diferente da que você instalou”](#) e [the section called “O comando “aws --version” retorna uma versão após a desinstalação da AWS CLI”](#).



# Configurar o AWS CLI

Esta seção explica como definir as configurações que a AWS Command Line Interface (AWS CLI) usa para interagir com a AWS. Incluindo o seguinte:

- As credenciais identificam quem está chamando a API. As credenciais de acesso são usadas para criptografar a solicitação aos servidores da AWS para confirmar sua identidade e recuperar as políticas de permissões associadas. Essas permissões determinam as ações que você pode realizar. Para obter informações sobre como configurar suas credenciais, consulte [Autenticação e credenciais de acesso](#).
- Outros detalhes de configuração para informar à AWS CLI como processar solicitações, como o formato de saída padrão e a região da AWS padrão.

## Note

A AWS exige que todas as solicitações recebidas sejam assinadas criptograficamente. O AWS CLI faz isso para você. A “assinatura” inclui uma data/time stamp. Portanto, você deve garantir que a data e a hora do seu computador estejam definidas corretamente. Se não fizer isso, e a data/hora na assinatura estiver muito longe da data/hora reconhecida pelo serviço da AWS, a AWS rejeitará a solicitação.

## Precedência de credenciais e configurações

As credenciais e as configurações estão localizadas em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de comando como um parâmetro. Certos locais têm precedência sobre outros. As credenciais da AWS CLI e as definições de configuração têm precedência na seguinte ordem:

1. [Opções da linha de comando](#): substituem as configurações em qualquer outro local, como nos parâmetros `--region`, `--output` e `--profile`.
2. [Variáveis de ambiente](#): você pode armazenar valores nas variáveis de ambiente do sistema.
3. [Assumir perfil](#): assumo as permissões de um perfil do IAM por meio da configuração ou do comando `aws sts assume-role`.

4. [Assumir perfil com identidade da web](#): assuma as permissões de um perfil do IAM usando uma identidade da web por meio da configuração ou do comando `aws sts assume-role`.
5. [Arquivo de credenciais](#): os arquivos `credentials` e `config` são atualizados quando você executa o comando `aws configure`. O arquivo `credentials` está localizado em `~/.aws/credentials` no Linux ou MacOS ou em `C:\Users\USERNAME\.aws\credentials` no Windows.
6. [Processo personalizado](#): obtenha suas credenciais de uma fonte externa.
7. [Arquivo de configuração](#): os arquivos `credentials` e `config` são atualizados quando você executa o comando `aws configure`. O arquivo `config` está localizado em `~/.aws/config` no Linux ou MacOS ou em `C:\Users\USERNAME\.aws\config` no Windows.
8. [Credenciais de perfil de instância](#): você pode associar um perfil do IAM a cada uma das suas instâncias do Amazon Elastic Compute Cloud (Amazon EC2). As credenciais temporárias para essa função estão disponíveis para o código em execução na instância. As credenciais são fornecidas por meio do serviço de metadados do Amazon EC2. Para obter mais informações, consulte [Funções do IAM para o Amazon EC2](#) no Manual do usuário do Amazon EC2 para instâncias do Linux e [Uso de perfis de instância](#) no Manual do usuário do IAM.
9. [Credenciais de container](#): você pode associar uma função do IAM a cada uma das suas definições de tarefa do Amazon Elastic Container Service (Amazon ECS). As credenciais temporárias para essa função estão disponíveis para os contêineres dessa tarefa. Para mais informações, consulte [Funções do IAM para Tarefas](#) no Guia de Desenvolvedor Amazon Elastic Container Service.

## Tópicos adicionais nesta seção

- [the section called “Configurações do arquivo de configuração e credenciais”](#)
- [the section called “Variáveis de ambiente”](#)
- [the section called “Opções de linha de comando”](#)
- [the section called “Conclusão de comando”](#)
- [the section called “Repetições”](#)
- [the section called “Usar um proxy HTTP”](#)

## Configurações do arquivo de configuração e credenciais

Você pode salvar as definições de configuração usadas com frequência e credenciais em arquivos que são mantidos pela AWS CLI.

Os arquivos são divididos em `profiles`. Por padrão, a AWS CLI usa as configurações encontradas no perfil chamado `default`. Para usar outras configurações, você pode criar e fazer referência a perfis adicionais.

Você pode substituir uma configuração definindo uma das variáveis de ambiente com suporte ou usando um parâmetro de linha de comando. Para obter mais informações sobre a precedência de configuração, consulte [Configurar o AWS CLI](#).

 **Note**

Para obter informações sobre como configurar suas credenciais, consulte [Autenticação e credenciais de acesso](#).

## Tópicos

- [Formato dos arquivos de credenciais e configuração](#)
- [Onde as definições de configuração ficam armazenadas?](#)
- [Usar perfis nomeados](#)
- [Definir e visualizar as configurações usando comandos](#)
- [Definir novos exemplos de comandos de configuração e credenciais](#)
- [Configurações de arquivo `config` compatíveis](#)

## Formato dos arquivos de credenciais e configuração

Os arquivos `config` e `credentials` são organizados em seções. As seções incluem perfis e serviços. Uma seção é um conjunto nomeado de configurações e continua até que outra linha de definição de seção seja encontrada. Vários perfis e seções podem ser armazenados nos arquivos `config` e `credentials`.

Esses arquivos são arquivos de texto simples que usam o seguinte formato:

- Os nomes das seções estão entre colchetes `[]`, como `[default]`, `[profile user1]` e `[sso-session]`.
- Todas as entradas em uma seção assumem a forma geral de `setting_name=value`.
- As linhas podem ser comentadas iniciando-as com um caractere de hashtag (`#`).

Os arquivos config e credentials contêm os seguintes tipos de seção:

- Tipo de seção: profile
- Tipo de seção: services

Tipo de seção: **profile**

## A AWS CLI armazena

Dependendo do arquivo, os nomes de seção de perfil usam o seguinte formato:

- Arquivo de configuração: [default] [profile *user1*]
- Arquivo de credenciais: [default] [*user1*]

Não use a palavra `profile` ao criar uma entrada no arquivo `credentials`.

Cada perfil especifica credenciais diferentes e também pode especificar regiões da AWS e formatos de saída diferentes. Ao nomear o perfil em um arquivo `config`, inclua a palavra de prefixo “`profile`”, mas não a inclua no arquivo `credentials`.

Os exemplos a seguir mostram um arquivo `credentials` e `config` com dois perfis, região e saída especificados. O primeiro [padrão] é usado quando você executa um comando da AWS CLI sem perfil especificado. O segundo é usado quando você executa um comando da AWS CLI com o parâmetro `--profile user1`.

## Short-term credentials

Este exemplo é para as credenciais de curto prazo do AWS Identity and Access Management. Para obter mais informações, consulte [the section called “Credenciais de curto prazo”](#).

## Arquivo de credenciais

```
[default]
aws_access_key_id=AKIAIOSFODNN7EXAMPLE
aws_secret_access_key=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
aws_session_token
= IqoJb3JpZ2luX2IqoJb3JpZ2luX2IqoJb3JpZ2luX2IqoJb3JpZ2luX2IqoJb3JpZVERYLONGSTRINGEXAMPLE

[user1]
aws_access_key_id=AKIAI44QH8DHBEXAMPLE
aws_secret_access_key=je7MtGbClwBF/2Zp9Utk/h3yCo8nvbEXAMPLEKEY
```

```
aws_session_token = fcZib3JpZ2luX2I0Jb3JpZ2luX2I0Jb3JpZ2luX2I0Jb3JpZ2luX2I0Jb3JpZVERYLONGSTRINGEXAMPLE
```

## Arquivo de configuração

```
[default]
region=us-west-2
output=json

[profile user1]
region=us-east-1
output=text
```

## IAM role

Este exemplo serve para assumir um perfil do IAM. Os perfis que usam perfis do IAM extraem credenciais de outro perfil e, depois, aplicam as permissões de perfil do IAM. Nos exemplos a seguir, `default` é o perfil de origem das credenciais, e `user1` empresta as mesmas credenciais e assume um novo perfil. Para obter mais informações, consulte [the section called “Perfis do IAM”](#).

## Arquivo de credenciais

O arquivo `credentials` depende da autenticação que o perfil de origem usa. No exemplo a seguir, o perfil de origem usa credenciais de curto prazo.

```
[default]
aws_access_key_id=AKIAIOSFODNN7EXAMPLE
aws_secret_access_key=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
aws_session_token
= IqoJb3JpZ2LuX2IqoJb3JpZ2LuX2IqoJb3JpZ2LuX2IqoJb3JpZ2LuX2IqoJb3JpZVERYLONGSTRINGEXAMPLE
```

## Arquivo de configuração

```
[default]
region=us-west-2
output=json

[profile user1]
role_arn=arn:aws:iam::777788889999:role/user1role
source_profile=default
role_session_name=session_user1
```

```
region=us-east-1
output=text
```

## Amazon EC2 instance metadata credentials

Este exemplo é para as credenciais obtidas dos metadados de instância do Amazon EC2 de host. Para obter mais informações, consulte [the section called “Uso de credenciais para metadados de instância do Amazon EC2.”](#).

### Arquivo de credenciais

O arquivo `credentials` não é usado para esse método de autenticação.

### Arquivo de configuração

```
[default]
role_arn=arn:aws:iam::123456789012:role/defaultrole
credential_source=Ec2InstanceMetadata
region=us-west-2
output=json

[profile user1]
role_arn=arn:aws:iam::777788889999:role/user1role
credential_source=Ec2InstanceMetadata
region=us-east-1
output=text
```

## Long-term credentials

### Warning

Para evitar riscos de segurança, não use usuários do IAM para autenticação ao desenvolver software com propósito específico ou trabalhar com dados reais. Em vez disso, use federação com um provedor de identidade, como [AWS IAM Identity Center](#).

Este exemplo é para as credenciais de longo prazo do AWS Identity and Access Management. Para obter mais informações, consulte [the section called “IAM users”](#).

### Arquivo de credenciais

```
[default]
```

```
aws_access_key_id=AKIAIOSFODNN7EXAMPLE
aws_secret_access_key=wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY

[user1]
aws_access_key_id=AKIAI44QH8DHBEXAMPLE
aws_secret_access_key=je7MtGbClwBF/2Zp9Utk/h3yCo8nvbEXAMPLEKEY
```

### Arquivo de configuração

```
[default]
region=us-west-2
output=json

[profile user1]
region=us-east-1
output=text
```

Para obter mais informações e métodos adicionais de autorização e credencial, consulte [the section called “IAM users”](#).

### Tipo de seção: **services**

A seção **services** é um grupo de configurações que configura endpoints personalizados para solicitações AWS service (Serviço da AWS). Depois, um perfil é vinculado a uma seção **services**.

```
[profile dev]
services = my-services
```

A seção **services** é separada em subseções por linhas `<SERVICE> =`, onde `<SERVICE>` é a chave de identificação do AWS service (Serviço da AWS). O identificador AWS service (Serviço da AWS) é baseado no `serviceId` do modelo de API, substituindo todos os espaços por sublinhados e colocando todas as letras em minúsculas. Para obter uma lista de todas as chaves de identificação de serviço a serem usadas na seção **services**, consulte [Usar endpoints na AWS CLI](#). A chave de identificação de serviço é seguida por configurações aninhadas, cada uma em sua própria linha e recuada por dois espaços.

O exemplo a seguir configura o endpoint a ser usado para solicitações feitas ao serviço Amazon DynamoDB na seção `my-services` que é usada no perfil `dev`. Todas as linhas imediatamente seguintes que estejam recuadas são incluídas nessa subseção e se aplicam a esse serviço.

```
[profile dev]
services = my-services

[services my-services]
dynamodb =
    endpoint_url = http://localhost:8000
```

Para obter mais informações sobre endpoints específicos de serviço, consulte [Usar endpoints na AWS CLI](#).

Se o seu perfil tiver credenciais baseadas em perfis configurados por meio de um parâmetro `source_profile` para a funcionalidade “assumir função” do IAM, o SDK usará somente configurações de serviço para o perfil especificado. Ele não usa perfis com funções vinculadas a ele. Por exemplo, usando o seguinte arquivo compartilhado config:

```
[profile A]
credential_source = Ec2InstanceMetadata
endpoint_url = https://profile-a-endpoint.aws/

[profile B]
source_profile = A
role_arn = arn:aws:iam::123456789012:role/roleB
services = profileB

[services profileB]
ec2 =
    endpoint_url = https://profile-b-ec2-endpoint.aws
```

Se você usar o perfil B e fizer uma chamada em seu código para o Amazon EC2, o endpoint será resolvido como `https://profile-b-ec2-endpoint.aws`. Se o seu código fizer uma solicitação para qualquer outro serviço, a resolução do endpoint não seguirá nenhuma lógica personalizada. O endpoint não é resolvido para o endpoint global definido no perfil A. Para que um endpoint global tenha efeito para o perfil B, você precisaria configurar `endpoint_url` diretamente no perfil B.

## Onde as definições de configuração ficam armazenadas?

A AWS CLI armazena informações confidenciais das credenciais com `aws configure` em um arquivo local chamado `credentials`, em uma pasta chamada `.aws`, no seu diretório inicial. As opções menos confidenciais de configuração especificadas com `aws configure` são armazenadas em um arquivo local chamado `config`, também armazenado na pasta `.aws`, no seu diretório inicial.



### Armazenar credenciais no arquivo de configuração

Você pode manter todas as configurações de perfil em um único arquivo, já que a AWS CLI pode ler as credenciais do arquivo `config`. Se houver credenciais nos dois arquivos para um perfil que compartilhe o mesmo nome, as chaves no arquivo de credenciais terão precedência. Sugerimos manter as credenciais nos arquivos `credentials`. Esses arquivos também são usados pelos vários kits de desenvolvimento de software de linguagem (SDKs). Se você usar um dos SDKs além da AWS CLI, confirme se as credenciais precisarão ser armazenadas no próprio arquivo.

O local do diretório inicial varia de acordo com o sistema operacional, mas é acessado usando as variáveis de ambiente `%UserProfile%` no Windows e `$HOME` ou `~` (til) em sistemas baseados em Unix. Você pode especificar um local não padrão para os arquivos configurando as variáveis de ambiente `AWS_CONFIG_FILE` e `AWS_SHARED_CREDENTIALS_FILE` para outro caminho local. Para mais detalhes, consulte [Variáveis de ambiente para configurar a AWS CLI](#).

Quando você usa um perfil compartilhado que especifica uma função do AWS Identity and Access Management (IAM), a AWS CLI chama a operação `AssumeRole` do AWS STS para recuperar as credenciais temporárias. Essas credenciais são armazenadas (no `~/.aws/cli/cache`). Os comandos subsequentes da AWS CLI usam as credenciais temporárias armazenadas em cache até que expirem e, nesse ponto, a AWS CLI atualiza automaticamente as credenciais.

## Usar perfis nomeados

Se nenhum perfil for definido explicitamente, será usado o perfil `default`.

Para usar um perfil designado, adicione a opção `--profile` *profile-name* para o comando. O exemplo a seguir lista todas as instâncias do Amazon EC2 que usam as credenciais e as configurações definidas no perfil `user1`.

```
$ aws ec2 describe-instances --profile user1
```

Para usar um perfil designado para vários comandos, evite especificar o perfil ao configurar o comando em cada variável de ambiente `AWS_PROFILE` como o perfil padrão. É possível substituir essa configuração usando o parâmetro `--profile`.

## Linux or macOS

```
$ export AWS_PROFILE=user1
```

## Windows

```
C:\> setx AWS_PROFILE user1
```

O uso de [set](#) para definir uma variável de ambiente altera o valor usado até o final da sessão de prompt de comando atual ou até que você defina a variável como um valor diferente.

Usar [setx](#) para definir uma variável de ambiente altera o valor em todos os shells de comando criados depois de executar o comando. Isso não afeta nenhum shell de comando que já esteja em execução no momento em que você executa o comando. Feche e reinicie o shell de comando para ver os efeitos da alteração.

Configurar a variável de ambiente altera o perfil padrão até o final do seu shell sessão ou até que você defina a variável a um valor diferente. Você pode tornar as variáveis de ambiente persistentes em sessões futuras colocando-as no script de inicialização do shell. Para obter mais informações, consulte [Variáveis de ambiente para configurar a AWS CLI](#).

## Definir e visualizar as configurações usando comandos

Existem várias formas de visualizar e definir as configurações usando comandos.

### [aws configure](#)

Execute esse comando para definir e visualizar rapidamente as credenciais do , a região e o formato de saída. O exemplo a seguir mostra valores de amostra.

```
$ aws configure
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY
Default region name [None]: us-west-2
Default output format [None]: json
```

### [aws configure set](#)

Você pode definir quaisquer credenciais ou configurações usando `aws configure set`. Especifique o perfil que você deseja visualizar ou modificar com a configuração `--profile`.

Por exemplo, o comando a seguir define a `region` no perfil chamado `integ`.

```
$ aws configure set region us-west-2 --profile integ
```

Para remover uma configuração, use uma string vazia como o valor ou exclua manualmente a configuração nos arquivos `config` e `credentials` em um editor de texto.

```
$ aws configure set cli_pager "" --profile integ
```

## aws configure get

Você pode recuperar quaisquer credenciais ou configurações definidas usando `aws configure get`. Especifique o perfil que você deseja visualizar ou modificar com a configuração `--profile`.

Por exemplo, o comando a seguir recupera a configuração `region` no perfil chamado `integ`.

```
$ aws configure get region --profile integ
us-west-2
```

Se a saída estiver vazia, a configuração não estará explicitamente definida e usará o valor padrão.

## aws configure list

Para listar os dados de configuração, use o comando `aws configure list`. Esse comando lista as informações de perfil, chave de acesso, chave secreta e configuração da região usadas para o perfil especificado. Para cada item de configuração, ele mostra o valor, onde o valor da configuração foi recuperado e o nome da variável de configuração.

Por exemplo, se você fornecer a Região da AWS em uma variável de ambiente, esse comando mostrará o nome da região que você configurou, que esse valor veio de uma variável de ambiente e o nome da variável de ambiente.

No caso de métodos de credenciais temporárias, como perfis e o Centro de Identidade do IAM, esse comando exibe a chave de acesso armazenada temporariamente em cache e a chave de acesso secreta é exibida.

```
$ aws configure list
```

| Name | Value | Type | Location |
|------|-------|------|----------|
| ---- | ----- | ---- | -----    |



## Amazon EC2 instance metadata credentials

Este exemplo é para as credenciais obtidas dos metadados de instância do Amazon EC2 de host. Não há assistente para esse processo, portanto, cada valor é definido usando o comando `aws configure set`. Para obter mais informações, consulte [the section called “Uso de credenciais para metadados de instância do Amazon EC2.”](#).

```
$ aws configure set role_arn arn:aws:iam::123456789012:role/defaultrole
$ aws configure set credential_source Ec2InstanceMetadata
$ aws configure set region us-west-2
$ aws configure set output json
```

## Long-term credentials

### Warning

Para evitar riscos de segurança, não use usuários do IAM para autenticação ao desenvolver software com propósito específico ou trabalhar com dados reais. Em vez disso, use federação com um provedor de identidade, como [AWS IAM Identity Center](#).

Este exemplo é para as credenciais de longo prazo do AWS Identity and Access Management. Para obter mais informações, consulte [the section called “IAM users”](#).

```
$ aws configure
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY
Default region name [None]: us-west-2
Default output format [None]: json
```

## Configurações de arquivo **config** compatíveis

### Tópicos

- [Configurações globais](#)
- [Configurações de comando personalizadas do S3](#)

As configurações a seguir têm suporte no arquivo `config`. Os valores listados no perfil especificado (ou padrão) serão usados, a menos que eles sejam ignorados pela presença de uma variável de

ambiente ou uma opção de linha de comando com o mesmo nome. Para obter informações sobre a ordem de precedência das configurações, consulte [Configurar o AWS CLI](#)

## Configurações globais

### *api\_versions*

Alguns serviços da AWS mantêm várias versões de API para oferecer suporte à compatibilidade com versões anteriores. Por padrão, os comandos da AWS CLI usam a versão de API mais recente disponível. Você pode especificar uma versão de API a ser usada para um perfil incluindo a configuração `api_versions` no arquivo `config`.

Essa é uma configuração “aninhada” que é seguida por uma ou mais linhas recuadas, cada uma identificando um serviço da AWS e a versão de API a ser usada. Consulte a documentação de cada serviço para saber quais versões de API estão disponíveis.

O exemplo a seguir mostra como especificar uma versão de API para dois serviços da AWS. Essas versões de API são usadas apenas para comandos que são executados sob o perfil que contém essas configurações.

```
api_versions =  
    ec2 = 2015-03-01  
    cloudfront = 2015-09-017
```

Essa configuração não tem uma variável de ambiente nem parâmetro de linha de comando equivalente.

### **aws\_access\_key\_id**

Especifica a chave de acesso da AWS usada como parte das credenciais para autenticar a solicitação do comando. Embora ela possa ser armazenada no arquivo `config`, recomendamos armazená-la no arquivo `credentials`.

Pode ser substituída pela variável de ambiente `AWS_ACCESS_KEY_ID`. Você não pode especificar o ID de chave de acesso como uma opção de linha de comando.

```
aws_access_key_id = AKIAIOSFODNN7EXAMPLE
```

## aws\_secret\_access\_key

Especifica a chave secreta da AWS usada como parte das credenciais para autenticar a solicitação do comando. Embora ela possa ser armazenada no arquivo `config`, recomendamos armazená-la no arquivo `credentials`.

Pode ser substituída pela variável de ambiente `AWS_SECRET_ACCESS_KEY`. Você não pode especificar a chave de acesso secreta como uma opção de linha de comando.

```
aws_secret_access_key = wJa1rXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY
```

## aws\_session\_token

Especifica um token de sessão da AWS. Um token de sessão só será necessário se você especificar manualmente credenciais de segurança temporárias. Embora ela possa ser armazenada no arquivo `config`, recomendamos armazená-la no arquivo `credentials`.

Pode ser substituída pela variável de ambiente `AWS_SESSION_TOKEN`. Você não pode especificar o token de sessão como uma opção de linha de comando.

```
aws_session_token = AqoEXAMPLEH4aoAH0gNCAPyJxz4BlCFFxWNE1OPTgk5TthT  
+FvwnKwRcOIfrRh3c/LTo6UDdyJw00vEVPvLXCrrrUtdnniCEXAMPLE/  
IvU1dYUg2RVAJBanLiHb4IgRmpRV3zrkuWJ0gQs8IZZaIv2BXIa2R40lgk
```

## ca\_bundle

Especifica um pacote de certificado CA (um arquivo com a extensão `.pem`) que é usado para verificar certificados SSL.

Pode ser substituído pela variável de ambiente [`AWS\_CA\_BUNDLE`](#) ou pela opção de linha de comando [`--ca-bundle`](#).

```
ca_bundle = dev/apps/ca-certs/cabundle-2019mar05.pem
```

## cli\_follow\_urlparam

Especifica se a AWS CLI tentará seguir links de URL em parâmetros de linha de comando que começam com `http://` ou `https://`. Quando ativado, o conteúdo recuperado é usado como o valor do parâmetro, em vez do URL.

- **true:** o valor padrão. Se especificado, todos os parâmetros de string que comecem com `http://` ou `https://` são obtidos e todo o conteúdo baixado é usado como o valor do parâmetro para o comando.
- **false:** se especificado, a AWS CLI não tratará valores de string de parâmetros que comecem com `http://` ou `https://` de forma diferente de outras strings.

Essa entrada não tem uma variável de ambiente equivalente nem uma opção de linha de comando.

```
cli_follow_urlparam = false
```

## **cli\_history**

Desabilitado por padrão. Essa configuração ativa o histórico de comandos para a AWS CLI. Após habilitar essa configuração, a AWS CLI registra o histórico de comandos da aws.

```
cli_history = enabled
```

Você pode listar seu histórico usando o comando `aws history list` e usar os `command_ids` resultantes no comando `aws history show` para obter detalhes. Para obter mais informações, consulte [aws history](#) no guia de referência da AWS CLI.

## **cli\_timestamp\_format**

Especifica o formato dos valores de timestamp incluídos no resultado. Você pode especificar qualquer um dos seguintes valores:

- **iso8601:** o valor padrão para a AWS CLI versão 2. Se especificado, a AWS CLI reformata todos os time stamps de acordo com a [ISO 8601](#).

Os time stamps formatados como ISO 8601 são parecidos com os exemplos a seguir. O primeiro exemplo mostra a hora no [Tempo Universal Coordenado \(UTC\)](#) incluindo um Z após o horário. A data e a hora são separadas por T.

```
2019-10-31T22:21:41Z
```

Para especificar um fuso horário diferente, em vez do Z, especifique + ou - e o número de horas em que o fuso horário desejado está à frente ou atrás do UTC, como um valor de dois dígitos. O exemplo a seguir mostra o mesmo tempo que o exemplo anterior, mas ajustado para o horário padrão do Pacífico, que está oito horas atrás do UTC:



```
2019-10-31T14:21:41-08
```

- `wire`: o valor padrão para a AWS CLI versão 1. Se especificado, a AWS CLI exibe todos os valores de time stamp exatamente como recebidos na resposta de consulta HTTP.

Essa entrada não tem uma variável de ambiente equivalente nem uma opção de linha de comando.

```
cli_timestamp_format = iso8601
```

## credential\_process

Especifica um comando externo que a AWS CLI executa para gerar ou recuperar credenciais de autenticação a serem usadas para esse comando. O comando deve retornar as credenciais em um formato específico. Para obter mais informações sobre como usar essa configuração, consulte [Credenciais de origem com um processo externo](#).

Essa entrada não tem uma variável de ambiente equivalente nem uma opção de linha de comando.

```
credential_process = /opt/bin/awscreds-retriever --username susan
```

## credential\_source

Usado em instâncias ou contêineres do Amazon EC2 para especificar onde a AWS CLI pode encontrar credenciais a serem usadas para assumir a função especificada com o parâmetro `role_arn`. Não é possível especificar `source_profile` e `credential_source` no mesmo perfil.

Esse parâmetro pode ter um dos três valores:

- `Environment`: especifica que a AWS CLI deve recuperar credenciais de origem de variáveis de ambiente.
- `Ec2InstanceMetadata`: especifica que a AWS CLI deve usar a função do IAM associada ao [perfil de instância do EC2](#) para obter credenciais de origem.
- `EcsContainer`: especifica que a AWS CLI deve usar a função do IAM associada ao contêiner do ECS como credenciais de origem.

```
credential_source = Ec2InstanceMetadata
```

## **duration\_seconds**

Especifica a duração máxima da sessão da função, em segundos. Esse valor pode variar de 900 segundos (15 minutos) até o valor configurado de duração máxima da sessão para a função (que pode ser até 43200). Esse é um parâmetro opcional e, por padrão, o valor é definido como 3600 segundos.

## **endpoint\_url**

Especifica o endpoint usado para todas as solicitações de serviço. Se essa configuração for usada na seção [services](#) do arquivo config, o endpoint será usado somente para o serviço especificado.

O exemplo a seguir usa o endpoint global `http://localhost:1234` e um endpoint específico de serviço de `http://localhost:4567` para o Amazon S3.

```
[profile dev]
endpoint_url = http://localhost:1234
services = s3-specific

[services s3-specific]
s3 =
    endpoint_url = http://localhost:4567
```

As configurações do endpoint estão em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de comando como um parâmetro. As configurações do endpoint da AWS CLI têm precedência na seguinte ordem:

1. A opção da linha de comando [--endpoint-url](#).
2. Se habilitada, a variável de ambiente global [AWS\\_IGNORE\\_CONFIGURED\\_ENDPOINT\\_URLS](#) ou a configuração do perfil [ignore\\_configure\\_endpoint\\_urls](#) para ignorar os endpoints personalizados.
3. O valor fornecido por uma variável de ambiente específica do serviço [AWS\\_ENDPOINT\\_URL\\_<SERVICE>](#), como `AWS_ENDPOINT_URL_DYNAMODB`.
4. Os valores fornecidos pelas variáveis de ambiente [AWS\\_USE\\_DUALSTACK\\_ENDPOINT](#), [AWS\\_USE\\_FIPS\\_ENDPOINT](#) e [AWS\\_ENDPOINT\\_URL](#).
5. O valor do endpoint específico de serviço fornecido pela configuração [endpoint\\_url](#) em uma seção `services` do arquivo compartilhado config.

6. O valor fornecido pela configuração [endpoint\\_url](#) em um profile do arquivo compartilhado config.
7. Configurações [use\\_dualstack\\_endpoint](#), [use\\_fips\\_endpoint](#) e [endpoint\\_url](#).
8. Qualquer URL de endpoint padrão para o respectivo AWS service (Serviço da AWS) é usado por último. Para obter uma lista dos endpoints de serviços padrão disponíveis em cada região, consulte [Regiões e endpoints da AWS](#) no Referência geral da Amazon Web Services.

## **ignore\_configure\_endpoint\_urls**

Se habilitada, a AWS CLI ignorará todas as configurações de endpoint personalizadas especificadas no arquivo config. Os valores válidos são **true** e **false**.

```
ignore_configure_endpoint_urls = true
```

As configurações do endpoint estão em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de comando como um parâmetro. As configurações do endpoint da AWS CLI têm precedência na seguinte ordem:

1. A opção da linha de comando [--endpoint-url](#).
2. Se habilitada, a variável de ambiente global [AWS\\_IGNORE\\_CONFIGURED\\_ENDPOINT\\_URLS](#) ou a configuração do perfil [ignore\\_configure\\_endpoint\\_urls](#) para ignorar os endpoints personalizados.
3. O valor fornecido por uma variável de ambiente específica do serviço [AWS\\_ENDPOINT\\_URL\\_<SERVICE>](#), como [AWS\\_ENDPOINT\\_URL\\_DYNAMODB](#).
4. Os valores fornecidos pelas variáveis de ambiente [AWS\\_USE\\_DUALSTACK\\_ENDPOINT](#), [AWS\\_USE\\_FIPS\\_ENDPOINT](#) e [AWS\\_ENDPOINT\\_URL](#).
5. O valor do endpoint específico de serviço fornecido pela configuração [endpoint\\_url](#) em uma seção services do arquivo compartilhado config.
6. O valor fornecido pela configuração [endpoint\\_url](#) em um profile do arquivo compartilhado config.
7. Configurações [use\\_dualstack\\_endpoint](#), [use\\_fips\\_endpoint](#) e [endpoint\\_url](#).
8. Qualquer URL de endpoint padrão para o respectivo AWS service (Serviço da AWS) é usado por último. Para obter uma lista dos endpoints de serviços padrão disponíveis em cada região, consulte [Regiões e endpoints da AWS](#) no Referência geral da Amazon Web Services.

## external\_id

Especifica um identificador exclusivo que é usado por terceiros para assumir uma função em suas contas de clientes. Isso é mapeado para o parâmetro `ExternalId` na operação `AssumeRole`. Esse parâmetro será necessário somente se a política de confiança para a função especificar um valor para `ExternalId`. Para obter mais informações, consulte [Como usar um ID externo ao conceder acesso aos seus recursos da AWS a terceiros](#) no Guia do usuário do IAM.

## max\_attempts

Especifica um valor para o máximo de novas tentativas utilizadas pelo manipulador de novas tentativas da AWS CLI, onde a chamada inicial conta para o valor de `max_attempts` fornecido por você.

Você pode substituir esse valor usando a variável de ambiente `AWS_MAX_ATTEMPTS`.

```
max_attempts = 3
```

## mfa\_serial

O número de identificação de um dispositivo MFA a ser usado ao assumir uma função. Isso será obrigatório apenas se a política de confiança da função que está sendo assumida incluir uma condição que exija a autenticação MFA. O valor pode ser um número de série de um dispositivo de hardware (como GAHT12345678) ou um nome de recurso da Amazon (ARN) de um dispositivo MFA virtual (como `arn:aws:iam::123456789012:mfa/user`).

## **output**

Especifica o formato de saída padrão para comandos solicitados usando esse perfil. Você pode especificar qualquer um dos seguintes valores:

- **json**: a saída é formatada como uma string [JSON](#).
- **text**: a saída é formatada como várias linhas de valores de string separados por tabulação. Isso pode ser útil para passar a saída para um processador de texto, como `grep`, `sed` ou `awk`.
- **table**: a saída é formatada como uma tabela usando os caracteres `+|-` para formar as bordas da célula. Geralmente, a informação é apresentada em um formato "amigável", que é muito mais fácil de ler do que outros, mas não tão útil programaticamente.

Pode ser substituído pela variável de ambiente `AWS_DEFAULT_OUTPUT` ou pela opção de linha de comando `--output`.

```
output = table
```

## parameter\_validation

Especifica se o cliente da AWS CLI tentará validar os parâmetros antes de enviá-los para o endpoint de serviço da AWS.

- **true**: o valor padrão. Se especificado, a AWS CLI executa a validação local de parâmetros da linha de comando.
- **false**: se especificado, a AWS CLI não valida parâmetros da linha de comando antes de enviá-los para o endpoint de serviço da AWS.

Essa entrada não tem uma variável de ambiente equivalente nem uma opção de linha de comando.

```
parameter_validation = false
```

## region

Especifica a Região da AWS para a qual enviar solicitações para os comandos solicitados usando esse perfil.

- É possível especificar qualquer um dos códigos de região disponíveis para o serviço escolhido, conforme listado em [Regiões e endpoints da AWS](#) na Referência geral da Amazon Web Services.
- **aws\_global** permite especificar o endpoint global para serviços compatíveis com um endpoint global, além de endpoints regionais, como AWS Security Token Service (AWS STS) e o Amazon Simple Storage Service (Amazon S3).

Você pode substituir esse valor usando a variável de ambiente **AWS\_DEFAULT\_REGION** ou a opção da linha de comando **--region**.

```
region = us-west-2
```

## retry\_mode

Especifica qual modo de repetição a AWS CLI utiliza. Existem três modos de repetição disponíveis: herdado (o modo usado por padrão), padrão e adaptive. Para obter mais informações sobre novas tentativas, consulte [Novas tentativas da AWS CLI](#).

Você pode substituir esse valor usando a variável de ambiente `AWS_RETRY_MODE`.

```
retry_mode = standard
```

## role\_arn

Especifica o nome do recurso da Amazon (ARN) de uma função do IAM que você deseja usar para executar os comandos da AWS CLI. Também é necessário especificar um dos seguintes parâmetros para identificar as credenciais que têm permissão para assumir essa função:

- `source_profile`
- `credential_source`

```
role_arn = arn:aws:iam::123456789012:role/role-name
```

A variável de ambiente `AWS_ROLE_ARN` sobrescreve essa configuração.

Para obter mais informações sobre o uso de identidades da Web, consulte [the section called “Assumir a função com a identidade da web”](#).

## role\_session\_name

Especifica o nome a ser associado à sessão da função. Esse valor é fornecido ao parâmetro `RoleSessionName` quando a AWS CLI chama a operação `AssumeRole` e se torna parte do ARN do usuário da função assumida: `arn:aws:sts::123456789012:assumed-role/role_name/role_session_name`. Esse parâmetro é opcional. Se você não fornecer esse valor, um nome de sessão será gerado automaticamente. Esse nome aparece nos logs do AWS CloudTrail para entradas associadas a essa sessão.

```
role_session_name = maria_garcia_role
```

A variável de ambiente `AWS_ROLE_SESSION_NAME` sobrescreve essa configuração.

Para obter mais informações sobre o uso de identidades da Web, consulte [the section called “Assumir a função com a identidade da web”](#).

## services

Especifica a configuração de serviço a ser usada em seu perfil.

```
[profile dev-s3-specific-and-global]
```

```
endpoint_url = http://localhost:1234
services = s3-specific

[services s3-specific]
s3 =
    endpoint_url = http://localhost:4567
```

Para obter mais informações sobre a seção `services`, consulte [the section called “services”](#).

A variável de ambiente [AWS\\_ROLE\\_SESSION\\_NAME](#) sobrescreve essa configuração.

Para obter mais informações sobre o uso de identidades da Web, consulte [the section called “Assumir a função com a identidade da web”](#).

## source\_profile

Especifica um perfil nomeado com credenciais de longo prazo que a AWS CLI pode usar para assumir uma função que você especificou com o parâmetro `role_arn`. Não é possível especificar `source_profile` e `credential_source` no mesmo perfil.

```
source_profile = production-profile
```

## *sts\_regional\_endpoints*

Especifica como a AWS CLI determina o endpoint do serviço da AWS que o cliente da AWS CLI usa para se comunicar com o AWS Security Token Service (AWS STS). O valor padrão para a AWS CLI versão 1 é `legacy`.

É possível especificar um dos dois valores:

- **legacy**: usa o endpoint global do STS, `sts.amazonaws.com`, para as seguintes regiões da AWS: `ap-northeast-1`, `ap-south-1`, `ap-southeast-1`, `ap-southeast-2`, `aws-global`, `ca-central-1`, `eu-central-1`, `eu-north-1`, `eu-west-1`, `eu-west-2`, `eu-west-3`, `sa-east-1`, `us-east-1`, `us-east-2`, `us-west-1` e `us-west-2`. Todas as demais regiões usam automaticamente o respectivo endpoint regional.
- **regional**: a AWS CLI sempre usa o endpoint do AWS STS para a região configurada no momento. Por exemplo, se o cliente estiver configurado para usar `us-west-2`,

todas as chamadas ao AWS STS serão feitas para o endpoint regional `sts.us-west-2.amazonaws.com` em vez do endpoint global `sts.amazonaws.com`. Para enviar uma solicitação para o endpoint global enquanto a configuração é habilitada, você pode definir a Região como `aws-global`.

Essa configuração pode ser substituída usando a variável de ambiente `AWS_STS_REGIONAL_ENDPOINTS`. Você não pode definir esse valor como um parâmetro de linha de comando.

### *`use_dualstack_endpoint`*

Permite o uso de endpoints de pilha dupla para enviar solicitações da AWS. Para saber mais sobre endpoints de pilha dupla, que suportam tráfego IPv4 e IPv6, consulte [Como usar endpoints de pilha dupla do Amazon S3](#) no Guia do usuário do Amazon Simple Storage Service. Endpoints de pilha dupla estão disponíveis para alguns serviços em algumas regiões. Se não existir um endpoint de pilha dupla para o serviço ou Região da AWS, a solicitação falhará. Ela fica desabilitada por padrão.

Isso é mutuamente exclusivo com a configuração `use_accelerate_endpoint`.

As configurações do endpoint estão em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de comando como um parâmetro. As configurações do endpoint da AWS CLI têm precedência na seguinte ordem:

1. A opção da linha de comando `--endpoint-url`.
2. Se habilitada, a variável de ambiente global `AWS_IGNORE_CONFIGURED_ENDPOINT_URLS` ou a configuração do perfil `ignore_configure_endpoint_urls` para ignorar os endpoints personalizados.
3. O valor fornecido por uma variável de ambiente específica do serviço `AWS_ENDPOINT_URL_<SERVICE>`, como `AWS_ENDPOINT_URL_DYNAMODB`.
4. Os valores fornecidos pelas variáveis de ambiente `AWS_USE_DUALSTACK_ENDPOINT`, `AWS_USE_FIPS_ENDPOINT` e `AWS_ENDPOINT_URL`.
5. O valor do endpoint específico de serviço fornecido pela configuração `endpoint_url` em uma seção `services` do arquivo compartilhado `config`.
6. O valor fornecido pela configuração `endpoint_url` em um `profile` do arquivo compartilhado `config`.
7. Configurações `use_dualstack_endpoint`, `use_fips_endpoint` e `endpoint_url`.



- Qualquer URL de endpoint padrão para o respectivo AWS service (Serviço da AWS) é usado por último. Para obter uma lista dos endpoints de serviços padrão disponíveis em cada região, consulte [Regiões e endpoints da AWS](#) no Referência geral da Amazon Web Services.

### *use\_fips\_endpoint*

Alguns serviços da AWS oferecem endpoints compatíveis com o [Federal Information Processing Standard \(FIPS\) 140-2](#) em algumas Regiões da AWS. Quando o serviço da AWS é compatível com o FIPS, essa configuração especifica qual endpoint do FIPS a AWS CLI deve usar. Ao contrário dos endpoints-padrão da AWS, os endpoints do FIPS usam uma biblioteca de software TLS compatível com o FIPS 140-2. Esses endpoints podem ser necessários por empresas que interagem com o governo dos Estados Unidos.

Se essa configuração estiver ativada, mas não existir um endpoint FIPS para o serviço em sua Região da AWS, o comando AWS poderá falhar. Nesse caso, especifique manualmente o endpoint a ser usado no comando usando a opção [--endpoint-url](#) ou use [endpoints específicos do serviço](#).

Para saber mais sobre como especificar endpoints FIPS por Região da AWS, consulte [Endpoints FIPS por serviço](#).

As configurações do endpoint estão em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de comando como um parâmetro. As configurações do endpoint da AWS CLI têm precedência na seguinte ordem:

- A opção da linha de comando [--endpoint-url](#).
- Se habilitada, a variável de ambiente global [AWS\\_IGNORE\\_CONFIGURED\\_ENDPOINT\\_URLS](#) ou a configuração do perfil [ignore\\_configure\\_endpoint\\_urls](#) para ignorar os endpoints personalizados.
- O valor fornecido por uma variável de ambiente específica do serviço [AWS\\_ENDPOINT\\_URL\\_<SERVICE>](#), como [AWS\\_ENDPOINT\\_URL\\_DYNAMODB](#).
- Os valores fornecidos pelas variáveis de ambiente [AWS\\_USE\\_DUALSTACK\\_ENDPOINT](#), [AWS\\_USE\\_FIPS\\_ENDPOINT](#) e [AWS\\_ENDPOINT\\_URL](#).
- O valor do endpoint específico de serviço fornecido pela configuração [endpoint\\_url](#) em uma seção `services` do arquivo compartilhado `config`.
- O valor fornecido pela configuração [endpoint\\_url](#) em um `profile` do arquivo compartilhado `config`.
- Configurações [use\\_dualstack\\_endpoint](#), [use\\_fips\\_endpoint](#) e [endpoint\\_url](#).

- Qualquer URL de endpoint padrão para o respectivo AWS service (Serviço da AWS) é usado por último. Para obter uma lista dos endpoints de serviços padrão disponíveis em cada região, consulte [Regiões e endpoints da AWS](#) no Referência geral da Amazon Web Services.

### `web_identity_token_file`

Especifica o caminho para um arquivo que contém um token de acesso OAuth 2.0 ou token de ID OpenID Connect fornecido por um provedor de identidade. A AWS CLI carrega o conteúdo desse arquivo e o transmite como o argumento `WebIdentityToken` para a operação `AssumeRoleWithWebIdentity`.

A variável de ambiente [`AWS\_WEB\_IDENTITY\_TOKEN\_FILE`](#) sobrescreve essa configuração.

Para obter mais informações sobre o uso de identidades da Web, consulte [the section called “Assumir a função com a identidade da web”](#).

### `tcp_keepalive`

Especifica se o cliente de AWS CLI usará pacotes keep-alive de TCP.

Essa entrada não tem uma variável de ambiente equivalente nem uma opção de linha de comando.

```
tcp_keepalive = false
```

## Configurações de comando personalizadas do S3

O Amazon S3 oferece suporte a várias configurações que definem como a AWS CLI executa operações do Amazon S3. Algumas se aplicam a todos os comandos do S3 nos namespaces `s3api` e `s3`. Outras são especificamente para os comandos “personalizados” do S3 que abstraem operações comuns e fazem mais do que um mapeamento individual para uma operação de API. Os comandos de transferência `aws s3 cp`, `sync`, `mv` e `rm` têm configurações adicionais que você pode usar para controlar as transferências do S3.

Todas essas opções podem ser configuradas especificando a configuração `s3` aninhada em seu arquivo `config`. Depois, cada configuração é recuada em sua própria linha.

### Note

Essas configurações são totalmente opcionais. Você deve ser capaz de usar com êxito os comandos de transferência `aws s3` sem definir nenhuma dessas configurações. Essas

configurações são fornecidas para permitir que você ajuste a performance ou considere o ambiente específico no qual está executando esses comandos `aws s3`.

Essas configurações são todas definidas em uma chave `s3` de nível superior no arquivo `config`, conforme mostrado no exemplo a seguir para o perfil `development`.

```
[profile development]
s3 =
    max_concurrent_requests = 20
    max_queue_size = 10000
    multipart_threshold = 64MB
    multipart_chunksize = 16MB
    max_bandwidth = 50MB/s
    use_accelerate_endpoint = true
    addressing_style = path
```

As configurações a seguir se aplicam a qualquer comando do S3 nos namespaces `s3` ou `s3api`.

## **addressing\_style**

Especifica qual estilo de endereçamento usar. Isso controla se o nome do bucket está no nome do host ou em parte do URL. Os valores válidos são: `path`, `virtual` e `auto`. O valor padrão é `auto`.

Há dois estilos de criação de um endpoint do Amazon S3. O primeiro é chamado `virtual` e inclui o nome do bucket como parte do nome do host. Por exemplo: `https://bucketname.s3.amazonaws.com`. Como alternativa, com o estilo `path`, você trata o nome do bucket como se fosse um caminho no URI. Por exemplo, `https://s3.amazonaws.com/bucketname`. O valor padrão na CLI é usar `auto`, que tentará empregar o estilo `virtual` onde puder, mas voltará para o estilo `path` quando necessário. Por exemplo, se o nome do bucket não for compatível com DNS, ele não poderá fazer parte do nome do host e deverá estar no caminho. Com `auto`, a CLI detectará essa condição e alternará automaticamente para o estilo `path`. Se você definir o estilo de endereçamento como `path`, garanta que a região da AWS que você configurou na AWS CLI corresponda à região do bucket.

## **payload\_signing\_enabled**

Especifica se o SHA256 deverá assinar cargas sigv4. Por padrão, ele fica desativado para uploads de streaming (`UploadPart` e `PutObject`) ao usar HTTPS. Por padrão, esse valor é

definido como `false` para uploads de streaming (`UploadPart` e `PutObject`), mas somente se um `ContentMD5` estiver presente (é gerado por padrão) e o endpoint usar HTTPS.

Se definido como `true`, as solicitações do S3 receberão validação de conteúdo adicional na forma de uma soma de verificação SHA256 que é calculada para você e será incluída na assinatura da solicitação. Se for definido como `false`, a soma de verificação não será calculada. Desativar essa opção poderá ser útil para reduzir a sobrecarga de performance criada pela soma da soma de verificação.

## **use\_accelerate\_endpoint**

Use o endpoint Amazon S3 Accelerate para todos os comandos `s3` e `s3api`. O valor padrão é falso. Isso é mutuamente exclusivo com a configuração `use_dualstack_endpoint`.

Se definido como `true`, a AWS CLI direcionará todas as solicitações do Amazon S3 para o endpoint `s3-accelerate.amazonaws.com` em `S3 Accelerate`. Para usar esse endpoint, é necessário ativar o bucket para usar o `S3 Accelerate`. Todas as solicitações são enviadas usando o estilo virtual de endereçamento de bucket: *my-bucket.s3-accelerate.amazonaws.com*. As solicitações `ListBuckets`, `CreateBucket` e `DeleteBucket` não são enviadas ao endpoint do S3 Accelerate porque esse endpoint não oferece suporte a essas operações. Esse comportamento também poderá ser definido se o parâmetro `--endpoint-url` estiver definido como `https://s3-accelerate.amazonaws.com` ou `http://s3-accelerate.amazonaws.com` para qualquer comando `s3` ou `s3api`.

As seguintes configurações se aplicam somente aos comandos do conjunto de comandos do namespace `s3`.

## **max\_bandwidth**

Especifica a largura de banda máxima que pode ser consumida para carregar e baixar dados de e para o Amazon S3. O padrão é sem limite.

Isso limita a largura de banda máxima que os comandos do S3 podem usar para transferir dados de e para o Amazon S3. Esse valor se aplica apenas a uploads e downloads; ele não se aplica a cópias nem as exclui. O valor é expresso em bytes por segundo. O valor pode ser especificado como:

- Um valor inteiro. Por exemplo, `1048576` define o uso máximo da largura de banda como 1 megabyte por segundo.

- Um valor inteiro seguido por um sufixo de taxa. Você pode especificar sufixos de taxa usando: KB/s, MB/s ou GB/s. Por exemplo, 300KB/s, 10MB/s.

Em geral, recomendamos que você primeiro tente reduzir o consumo de largura de banda diminuindo `max_concurrent_requests`. Se isso não adequar o consumo de largura de banda limite para a taxa desejada, você poderá usar a configuração `max_bandwidth` para limitar ainda mais o consumo de largura de banda. Isso ocorre porque `max_concurrent_requests` controla o número de threads em execução no momento. Se, em vez disso, você baixar primeiro, `max_bandwidth` mas deixar uma configuração `max_concurrent_requests` alta, isso pode fazer com que threads tenham que esperar desnecessariamente. Isso pode levar ao excesso de consumo de recursos e tempos limite de conexão.

### **`max_concurrent_requests`**

Especifica o número máximo de solicitações simultâneas. O valor padrão é 10.

Os comandos de transferência `aws s3` são multithread. Em um determinado momento, várias solicitações do Amazon S3 podem estar em execução. Por exemplo, quando você usa o comando `aws s3 cp localdir s3://bucket/ --recursive` para fazer upload de arquivos para um bucket do S3, a AWS CLI pode carregar os arquivos `localdir/file1`, `localdir/file2` e `localdir/file3` paralelamente. A configuração `max_concurrent_requests` especifica o número máximo de operações de transferência que podem ser executadas ao mesmo tempo.

Talvez você precise alterar esse valor por alguns motivos:

- Diminuir esse valor: em alguns ambientes, o padrão de 10 solicitações simultâneas pode sobrecarregar um sistema. Isso pode causar erros de tempo limite de conexão ou diminuir a capacidade de resposta do sistema. Reduzir esse valor faz com que os comandos de transferência do S3 usem menos recursos. A desvantagem é que as transmissões do S3 podem levar mais tempo para serem concluídas. Reduzir esse valor poderá ser necessário se você usar uma ferramenta para limitar a largura de banda.
- Aumentar esse valor: em alguns cenários, talvez você queira que as transferências do Amazon S3 sejam concluídas com o máximo de rapidez possível, usando tanta largura de banda de rede quanto necessário. Nesse cenário, o número padrão de solicitações simultâneas pode não ser suficiente para utilizar toda a largura de banda de rede disponível. O aumento desse valor pode melhorar o tempo necessário para concluir uma transferência do Amazon S3.

### **`max_queue_size`**

Especifica o número máximo de tarefas na fila de tarefas. O valor padrão é 1000.

A AWS CLI usa internamente um modelo que coloca na fila as tarefas do Amazon S3 que, em seguida, são executadas por consumidores cujos números são limitados por `max_concurrent_requests`. Em geral, uma tarefa é mapeada para uma única operação do Amazon S3. Por exemplo, a tarefa pode ser `PutObjectTask`, `GetObjectTask` ou `UploadPartTask`. A taxa na qual as tarefas são adicionadas à fila pode ser muito mais rápida do que a taxa na qual os consumidores concluem as tarefas. Para evitar o crescimento não vinculado, o tamanho da fila de tarefas é limitada a um tamanho específico. Essa configuração altera o valor desse número máximo.

Em geral, você não precisa alterar essa configuração. Essa configuração também corresponde ao número de tarefas que a AWS CLI está ciente de que precisam ser executadas. Isso significa que, por padrão, a AWS CLI só pode ver 1000 tarefas à frente. Aumentar esse valor significa que a AWS CLI pode saber com mais rapidez o número total de tarefas necessárias, supondo que a taxa de enfileiramento seja mais rápida do que a taxa de conclusão de tarefas. A desvantagem é que um `max_queue_size` maior requer mais memória.

### **`multipart_chunksize`**

Especifica o tamanho de bloco que a AWS CLI usa para transferências multipart de arquivos individuais. O valor padrão é de 8 MB, com um mínimo de 5 MB.

Quando uma transferência de arquivos excede o `multipart_threshold`, a AWS CLI divide o arquivo em blocos desse tamanho. Esse valor pode ser especificado usando a mesma sintaxe que `multipart_threshold`, como o número de bytes como um número inteiro ou usando um tamanho e um sufixo.

### **`multipart_threshold`**

Especifica o limite de tamanho que a AWS CLI usa para transferências multipart de arquivos individuais. O valor padrão é de 8 MB.

Ao carregar, baixar ou copiar um arquivo, os comandos do Amazon S3 alternarão para operações multipart se o arquivo exceder esse tamanho. É possível especificar esse valor de duas formas:

- O tamanho do arquivo em bytes. Por exemplo, 1048576.
- O tamanho do arquivo com um sufixo de tamanho. Você pode usar KB, MB, GB ou TB. Por exemplo: 10MB, 1GB.

**Note**

O S3 pode impor restrições quanto a valores válidos que podem ser usados para operações multipart. Para obter mais informações, consulte a [documentação do Carregamento fracionado do S3](#) no Manual do usuário do Amazon Simple Storage Service.

## Variáveis de ambiente para configurar a AWS CLI

Variáveis de ambiente fornecem outra maneira de especificar opções de configuração e credenciais e podem ser úteis para criação de scripts ou configuração temporária de um perfil nomeado como o padrão.

### Precedência de opções

- Se você especificar uma opção usando uma das variáveis de ambiente descritas nesse tópico, ela substituirá qualquer valor carregado de um perfil no arquivo de configuração.
- Se você especificar uma opção usando um parâmetro na linha de comando da AWS CLI, ela sobrescreverá qualquer valor da variável de ambiente correspondente ou de um perfil no arquivo de configuração.

Para obter mais informações sobre precedência e como a AWS CLI determina quais credenciais usar, consulte [Configurar o AWS CLI](#).

### Tópicos

- [Como definir variáveis de ambiente](#)
- [Variáveis de ambiente compatíveis da AWS CLI](#)

## Como definir variáveis de ambiente

Os exemplos a seguir mostram como configurar variáveis de ambiente para o usuário padrão.

### Linux or macOS

```
$ export AWS_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE
$ export AWS_SECRET_ACCESS_KEY=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
```

```
$ export AWS_DEFAULT_REGION=us-west-2
```

Configurar a variável de ambiente altera o valor usado até o final da sua sessão de shell ou até que você defina a variável como um valor diferente. Você pode tornar as variáveis persistentes em sessões futuras definindo-as no script de inicialização do shell.

## Windows Command Prompt

Como definir para todas as sessões

```
C:\> setx AWS_ACCESS_KEY_ID AKIAIOSFODNN7EXAMPLE  
C:\> setx AWS_SECRET_ACCESS_KEY wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY  
C:\> setx AWS_DEFAULT_REGION us-west-2
```

O uso de [setx](#) para definir uma variável de ambiente altera o valor usado na sessão de prompt de comando atual e todas as sessões de prompt de comando que você criar após a execução do comando. Não afeta outros shells de comando que já estejam em execução no momento em que você executar o comando. Talvez seja necessário reiniciar o terminal para que as configurações sejam carregadas.

Como definir somente para a sessão atual

O uso de [set](#) para definir uma variável de ambiente altera o valor usado até o final da sessão de prompt de comando atual ou até que você defina a variável como um valor diferente.

```
C:\> set AWS_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE  
C:\> set AWS_SECRET_ACCESS_KEY=wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY  
C:\> set AWS_DEFAULT_REGION=us-west-2
```

## PowerShell

```
PS C:\> $Env:AWS_ACCESS_KEY_ID="AKIAIOSFODNN7EXAMPLE"  
PS C:\> $Env:AWS_SECRET_ACCESS_KEY="wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY"  
PS C:\> $Env:AWS_DEFAULT_REGION="us-west-2"
```

Se você definir uma variável de ambiente no prompt do PowerShell conforme mostrado nos exemplos anteriores, ela salvará o valor somente pela duração da sessão atual. Para fazer com que a configuração da variável de ambiente seja persistente em todas as sessões do prompt de comando e do PowerShell, armazene-a usando o aplicativo System (Sistema) no Control Panel (Painel de controle). Como alternativa, você pode definir a variável para todas as futuras sessões do PowerShell adicionando-a ao seu perfil do PowerShell. Consulte a documentação do



[PowerShell](#) para obter mais informações sobre como armazenar variáveis de ambiente ou como persisti-las nas sessões.

## Variáveis de ambiente compatíveis da AWS CLI

A AWS CLI é compatível com as seguintes variáveis de ambiente.

### **AWS\_ACCESS\_KEY\_ID**

Especifica uma chave de acesso da AWS associada a uma conta do IAM.

Se definida, essa variável de ambiente sobrescreverá o valor da configuração de perfil `aws_access_key_id`. Você não pode especificar o ID de chave de acesso com uma opção de linha de comando.

### **AWS\_CA\_BUNDLE**

Especifica o caminho para um pacote de certificado a ser usado para a validação de certificado HTTPS.

Se definida, essa variável de ambiente sobrescreverá o valor da configuração de perfil `ca_bundle`. Você pode substituir essa variável de ambiente usando o parâmetro de linha de comando `--ca-bundle`.

### **AWS\_CONFIG\_FILE**

Especifica o local do arquivo que a AWS CLI usa para armazenar perfis de configuração. O caminho padrão é `~/.aws/config`.

Você não pode especificar esse valor em uma configuração de perfil nomeado nem usando um parâmetro de linha de comando.

### **AWS\_DATA\_PATH**

Uma lista de diretórios adicionais a serem verificados fora do caminho de pesquisa interno do `~/.aws/models` ao carregar dados da AWS CLI. A definição dessa variável de ambiente indica os diretórios adicionais a serem verificados primeiro antes de voltar para os caminhos de pesquisa internos. Várias entradas devem ser separadas com o caractere `.pathsep`, que é : no Linux ou no macOS e ; no Windows.

### **AWS\_DEFAULT\_OUTPUT**

Especifica o [formato de saída](#) a ser usado.

Se definida, essa variável de ambiente sobrescreverá o valor da configuração de perfil `output`. Você pode substituir essa variável de ambiente usando o parâmetro de linha de comando `--output`.

## **AWS\_DEFAULT\_REGION**

O `Default region name` identifica a região da AWS dos servidores para os quais você deseja enviar suas solicitações por padrão. Normalmente, é a região mais próxima de você, mas pode ser qualquer região. Por exemplo, você pode digitar `us-west-2` para usar a região Oeste dos EUA (Oregon). Essa é a região para a qual todas as solicitações posteriores são enviadas, a menos que você especifique o contrário em um comando individual.

### Note

Você deve especificar uma região da AWS ao usar a AWS CLI, explicitamente ou definindo uma região padrão. Para obter uma lista das regiões disponíveis, consulte [Regiões e endpoints](#). Os designadores de região usados pela AWS CLI têm os mesmos nomes que você vê nos URLs nos endpoints de serviço do AWS Management Console.

Se definida, essa variável de ambiente sobrescreverá o valor da configuração de perfil `region`. Você pode substituir essa variável de ambiente usando o parâmetro de linha de comando `--region`.

## **AWS\_EC2\_METADATA\_DISABLED**

Desabilita o uso do serviço de metadados da instância do Amazon EC2 (IMDS).

Se definido como `true`, as credenciais do usuário ou a configuração (como a região) não são solicitadas do IMDS.

## **AWS\_ENDPOINT\_URL**

Especifica o endpoint usado para todas as solicitações de serviço.

As configurações do endpoint estão em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de comando como um parâmetro. As configurações do endpoint da AWS CLI têm precedência na seguinte ordem:

1. A opção da linha de comando [`--endpoint-url`](#).

2. Se habilitada, a variável de ambiente global [AWS\\_IGNORE\\_CONFIGURED\\_ENDPOINT\\_URLS](#) ou a configuração do perfil [ignore\\_configure\\_endpoint\\_urls](#) para ignorar os endpoints personalizados.
3. O valor fornecido por uma variável de ambiente específica do serviço [AWS\\_ENDPOINT\\_URL\\_<SERVICE>](#), como `AWS_ENDPOINT_URL_DYNAMODB`.
4. Os valores fornecidos pelas variáveis de ambiente [AWS\\_USE\\_DUALSTACK\\_ENDPOINT](#), [AWS\\_USE\\_FIPS\\_ENDPOINT](#) e [AWS\\_ENDPOINT\\_URL](#).
5. O valor do endpoint específico de serviço fornecido pela configuração [endpoint\\_url](#) em uma seção `services` do arquivo compartilhado `config`.
6. O valor fornecido pela configuração [endpoint\\_url](#) em um `profile` do arquivo compartilhado `config`.
7. Configurações [use\\_dualstack\\_endpoint](#), [use\\_fips\\_endpoint](#) e [endpoint\\_url](#).
8. Qualquer URL de endpoint padrão para o respectivo AWS service (Serviço da AWS) é usado por último. Para obter uma lista dos endpoints de serviços padrão disponíveis em cada região, consulte [Regiões e endpoints da AWS](#) no Referência geral da Amazon Web Services.

## **AWS\_ENDPOINT\_URL\_<SERVICE>**

Especifica um endpoint personalizado usado para um serviço específico, onde `<SERVICE>` é substituído pelo identificador AWS service (Serviço da AWS). Por exemplo, Amazon DynamoDB tem um `serviceId` do [DynamoDB](#). Para esse serviço, a variável de ambiente do URL do endpoint é `AWS_ENDPOINT_URL_DYNAMODB`.

Para obter uma lista de todas as variáveis de ambiente específicas do serviço, consulte [Lista de identificadores específicos de serviço](#).

As configurações do endpoint estão em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de comando como um parâmetro. As configurações do endpoint da AWS CLI têm precedência na seguinte ordem:

1. A opção da linha de comando [--endpoint-url](#).
2. Se habilitada, a variável de ambiente global [AWS\\_IGNORE\\_CONFIGURED\\_ENDPOINT\\_URLS](#) ou a configuração do perfil [ignore\\_configure\\_endpoint\\_urls](#) para ignorar os endpoints personalizados.
3. O valor fornecido por uma variável de ambiente específica do serviço [AWS\\_ENDPOINT\\_URL\\_<SERVICE>](#), como `AWS_ENDPOINT_URL_DYNAMODB`.

4. Os valores fornecidos pelas variáveis de ambiente [AWS\\_USE\\_DUALSTACK\\_ENDPOINT](#), [AWS\\_USE\\_FIPS\\_ENDPOINT](#) e [AWS\\_ENDPOINT\\_URL](#).
5. O valor do endpoint específico de serviço fornecido pela configuração [endpoint\\_url](#) em uma seção `services` do arquivo compartilhado `config`.
6. O valor fornecido pela configuração [endpoint\\_url](#) em um `profile` do arquivo compartilhado `config`.
7. Configurações [use\\_dualstack\\_endpoint](#), [use\\_fips\\_endpoint](#) e [endpoint\\_url](#).
8. Qualquer URL de endpoint padrão para o respectivo AWS service (Serviço da AWS) é usado por último. Para obter uma lista dos endpoints de serviços padrão disponíveis em cada região, consulte [Regiões e endpoints da AWS](#) no Referência geral da Amazon Web Services.

## AWS\_IGNORE\_CONFIGURED\_ENDPOINT\_URLS

Se habilitada, a AWS CLI ignora todas as configurações personalizadas de endpoint. Os valores válidos são **true** e **false**.

As configurações do endpoint estão em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de comando como um parâmetro. As configurações do endpoint da AWS CLI têm precedência na seguinte ordem:

1. A opção da linha de comando [--endpoint-url](#).
2. Se habilitada, a variável de ambiente global [AWS\\_IGNORE\\_CONFIGURED\\_ENDPOINT\\_URLS](#) ou a configuração do perfil [ignore\\_configure\\_endpoint\\_urls](#) para ignorar os endpoints personalizados.
3. O valor fornecido por uma variável de ambiente específica do serviço [AWS\\_ENDPOINT\\_URL\\_<SERVICE>](#), como `AWS_ENDPOINT_URL_DYNAMODB`.
4. Os valores fornecidos pelas variáveis de ambiente [AWS\\_USE\\_DUALSTACK\\_ENDPOINT](#), [AWS\\_USE\\_FIPS\\_ENDPOINT](#) e [AWS\\_ENDPOINT\\_URL](#).
5. O valor do endpoint específico de serviço fornecido pela configuração [endpoint\\_url](#) em uma seção `services` do arquivo compartilhado `config`.
6. O valor fornecido pela configuração [endpoint\\_url](#) em um `profile` do arquivo compartilhado `config`.
7. Configurações [use\\_dualstack\\_endpoint](#), [use\\_fips\\_endpoint](#) e [endpoint\\_url](#).
8. Qualquer URL de endpoint padrão para o respectivo AWS service (Serviço da AWS) é usado por último. Para obter uma lista dos endpoints de serviços padrão disponíveis em cada região, consulte [Regiões e endpoints da AWS](#) no Referência geral da Amazon Web Services.

## **AWS\_MAX\_ATTEMPTS**

Especifica um valor para o máximo de novas tentativas utilizadas pelo manipulador de novas tentativas da AWS CLI, onde a chamada inicial conta para o valor fornecido por você. Para obter mais informações sobre novas tentativas, consulte [Novas tentativas da AWS CLI](#).

Se definida, essa variável de ambiente sobrescreverá o valor da configuração de perfil `max_attempts`.

## **AWS\_METADATA\_SERVICE\_NUM\_ATTEMPTS**

Ao tentar recuperar credenciais em uma instância do Amazon EC2 que foi configurada com uma função do IAM, a AWS CLI fará apenas uma tentativa para recuperar as credenciais do serviço de metadados da instância antes de interromper. Se souber que seus comandos serão executados em uma instância do Amazon EC2, você poderá aumentar esse valor para possibilitar que a AWS CLI faça várias tentativas antes de desistir.

## **AWS\_METADATA\_SERVICE\_TIMEOUT**

O número de segundos antes de uma conexão ao serviço de metadados da instância atingir o tempo limite. Ao tentar recuperar credenciais em uma instância do Amazon EC2 que foi configurada com uma função do IAM, por padrão uma conexão ao serviço de metadados da instância atingirá o tempo limite depois de um segundo. Se souber que está executando em uma instância do Amazon EC2 com uma função do IAM configurada, você poderá aumentar esse valor, se necessário.

## **AWS\_PROFILE**

Especifica o nome do perfil da AWS CLI com as credenciais e as opções a serem usadas. Esse pode ser o nome de um perfil armazenado em um arquivo `credentials` ou `config` ou o valor `default` para usar o perfil padrão.

Se você especificar essa variável de ambiente, ela substituirá o comportamento de usar o perfil nomeado `[default]` no arquivo de configuração. Você pode substituir essa variável de ambiente usando o parâmetro de linha de comando `--profile`.

## **AWS\_RETRY\_MODE**

Especifica qual modo de repetição a AWS CLI utiliza. Existem três modos de repetição disponíveis: herdado (o modo usado por padrão), padrão e `adaptive`. Para obter mais informações sobre novas tentativas, consulte [Novas tentativas da AWS CLI](#).

Se definida, essa variável de ambiente sobrescreverá o valor da configuração de perfil `retry_mode`.

## **AWS\_ROLE\_ARN**

Especifica o nome do recurso da Amazon (ARN) de uma função do IAM com um provedor de identidade da Web que você deseja usar para executar os comandos da AWS CLI.

Usado com as variáveis de ambiente `AWS_WEB_IDENTITY_TOKEN_FILE` e `AWS_ROLE_SESSION_NAME`.

Se definida, essa variável de ambiente sobrescreverá o valor da configuração de perfil [role\\_arn](#). Você não pode especificar um nome de sessão de função como um parâmetro de linha de comando.

### Note

Essa variável de ambiente se aplica somente a uma função assumida com o provedor de identidade da Web, mas não se aplica à configuração do provedor de função assumida geral.

Para obter mais informações sobre o uso de identidades da Web, consulte [the section called “Assumir a função com a identidade da web”](#).

## **AWS\_ROLE\_SESSION\_NAME**

Especifica o nome a ser associado à sessão da função. Esse valor é fornecido ao parâmetro `RoleSessionName` quando a AWS CLI chama a operação `AssumeRole` e se torna parte do ARN do usuário da função assumida: `arn:aws:sts::123456789012:assumed-role/role_name/role_session_name`. Esse parâmetro é opcional. Se você não fornecer esse valor, um nome de sessão será gerado automaticamente. Esse nome aparece nos logs do AWS CloudTrail para entradas associadas a essa sessão.

Se definida, essa variável de ambiente sobrescreverá o valor da configuração de perfil [role\\_session\\_name](#).

Usado com as variáveis de ambiente `AWS_ROLE_ARN` e `AWS_WEB_IDENTITY_TOKEN_FILE`.

Para obter mais informações sobre o uso de identidades da Web, consulte [the section called “Assumir a função com a identidade da web”](#).

 Note

Essa variável de ambiente se aplica somente a uma função assumida com o provedor de identidade da Web, mas não se aplica à configuração do provedor de função assumida geral.

## **AWS\_SECRET\_ACCESS\_KEY**

Especifica a chave secreta associada à chave de acesso. Essencialmente, essa é a “senha” para a chave de acesso.

Se definida, essa variável de ambiente sobrescreverá o valor da configuração de perfil `aws_secret_access_key`. Você não pode especificar o ID chave de acesso secreta como uma opção de linha de comando.

## **AWS\_SESSION\_TOKEN**

Especifica o valor de token de sessão que é necessário se você estiver usando credenciais de segurança temporárias recuperadas diretamente das operações do AWS STS. Para obter mais informações, consulte a [Seção de saída do comando `assume-role`](#) na Referência de comandos da AWS CLI.

Se definida, essa variável de ambiente sobrescreverá o valor da configuração de perfil `aws_session_token`.

## **AWS\_SHARED\_CREDENTIALS\_FILE**

Especifica o local do arquivo que a AWS CLI usa para armazenar chaves de acesso. O caminho padrão é `~/.aws/credentials`.

Você não pode especificar esse valor em uma configuração de perfil nomeado nem usando um parâmetro de linha de comando.

## **AWS\_STS\_REGIONAL\_ENDPOINTS**

Especifica como a AWS CLI determina o endpoint do serviço da AWS que o cliente da AWS CLI usa para se comunicar com o AWS Security Token Service (AWS STS). O valor padrão para a AWS CLI versão 1 é `legacy`.

É possível especificar um dos dois valores:

- **legacy:** usa o endpoint global do STS, `sts.amazonaws.com`, para as seguintes regiões da AWS: `ap-northeast-1`, `ap-south-1`, `ap-southeast-1`, `ap-southeast-2`, `aws-global`, `ca-central-1`, `eu-central-1`, `eu-north-1`, `eu-west-1`, `eu-west-2`, `eu-west-3`, `sa-east-1`, `us-east-1`, `us-east-2`, `us-west-1` e `us-west-2`. Todas as demais regiões usam automaticamente o respectivo endpoint regional.
- **regional:** a AWS CLI sempre usa o endpoint do AWS STS para a região configurada no momento. Por exemplo, se o cliente estiver configurado para usar `us-west-2`, todas as chamadas ao AWS STS serão feitas para o endpoint regional `sts.us-west-2.amazonaws.com` em vez do endpoint global `sts.amazonaws.com`. Para enviar uma solicitação para o endpoint global enquanto a configuração é habilitada, você pode definir a Região como `aws-global`.

## AWS\_USE\_DUALSTACK\_ENDPOINT

Permite o uso de endpoints de pilha dupla para enviar solicitações da AWS. Para saber mais sobre endpoints de pilha dupla, que suportam tráfego IPv4 e IPv6, consulte [Como usar endpoints de pilha dupla do Amazon S3](#) no Guia do usuário do Amazon Simple Storage Service. Endpoints de pilha dupla estão disponíveis para alguns serviços em algumas regiões. Se não existir um endpoint de pilha dupla para o serviço ou Região da AWS, a solicitação falhará. Ela fica desabilitada por padrão.

As configurações do endpoint estão em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de comando como um parâmetro. As configurações do endpoint da AWS CLI têm precedência na seguinte ordem:

1. A opção da linha de comando `--endpoint-url`.
2. Se habilitada, a variável de ambiente global `AWS_IGNORE_CONFIGURED_ENDPOINT_URLS` ou a configuração do perfil `ignore_configure_endpoint_urls` para ignorar os endpoints personalizados.
3. O valor fornecido por uma variável de ambiente específica do serviço `AWS_ENDPOINT_URL_<SERVICE>`, como `AWS_ENDPOINT_URL_DYNAMODB`.
4. Os valores fornecidos pelas variáveis de ambiente `AWS_USE_DUALSTACK_ENDPOINT`, `AWS_USE_FIPS_ENDPOINT` e `AWS_ENDPOINT_URL`.
5. O valor do endpoint específico de serviço fornecido pela configuração `endpoint_url` em uma seção `services` do arquivo compartilhado `config`.
6. O valor fornecido pela configuração `endpoint_url` em um `profile` do arquivo compartilhado `config`.



7. Configurações [use\\_dualstack\\_endpoint](#), [use\\_fips\\_endpoint](#) e [endpoint\\_url](#).
8. Qualquer URL de endpoint padrão para o respectivo AWS service (Serviço da AWS) é usado por último. Para obter uma lista dos endpoints de serviços padrão disponíveis em cada região, consulte [Regiões e endpoints da AWS](#) no Referência geral da Amazon Web Services.

## AWS\_USE\_FIPS\_ENDPOINT

Alguns serviços da AWS oferecem endpoints compatíveis com o [Federal Information Processing Standard \(FIPS\) 140-2](#) em algumas Regiões da AWS. Quando o serviço da AWS é compatível com o FIPS, essa configuração especifica qual endpoint do FIPS a AWS CLI deve usar. Ao contrário dos endpoints-padrão da AWS, os endpoints do FIPS usam uma biblioteca de software TLS compatível com o FIPS 140-2. Esses endpoints podem ser necessários por empresas que interagem com o governo dos Estados Unidos.

Se essa configuração estiver ativada, mas não existir um endpoint FIPS para o serviço em sua Região da AWS, o comando AWS poderá falhar. Nesse caso, especifique manualmente o endpoint a ser usado no comando usando a opção [--endpoint-url](#) ou use [endpoints específicos do serviço](#).

Para saber mais sobre como especificar endpoints FIPS por Região da AWS, consulte [Endpoints FIPS por serviço](#).

As configurações do endpoint estão em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de comando como um parâmetro. As configurações do endpoint da AWS CLI têm precedência na seguinte ordem:

1. A opção da linha de comando [--endpoint-url](#).
2. Se habilitada, a variável de ambiente global [AWS\\_IGNORE\\_CONFIGURED\\_ENDPOINT\\_URLS](#) ou a configuração do perfil [ignore\\_configure\\_endpoint\\_urls](#) para ignorar os endpoints personalizados.
3. O valor fornecido por uma variável de ambiente específica do serviço [AWS\\_ENDPOINT\\_URL\\_<SERVICE>](#), como [AWS\\_ENDPOINT\\_URL\\_DYNAMODB](#).
4. Os valores fornecidos pelas variáveis de ambiente [AWS\\_USE\\_DUALSTACK\\_ENDPOINT](#), [AWS\\_USE\\_FIPS\\_ENDPOINT](#) e [AWS\\_ENDPOINT\\_URL](#).
5. O valor do endpoint específico de serviço fornecido pela configuração [endpoint\\_url](#) em uma seção `services` do arquivo compartilhado `config`.
6. O valor fornecido pela configuração [endpoint\\_url](#) em um `profile` do arquivo compartilhado `config`.

7. Configurações [use\\_dualstack\\_endpoint](#), [use\\_fips\\_endpoint](#) e [endpoint\\_url](#).
8. Qualquer URL de endpoint padrão para o respectivo AWS service (Serviço da AWS) é usado por último. Para obter uma lista dos endpoints de serviços padrão disponíveis em cada região, consulte [Regiões e endpoints da AWS](#) no Referência geral da Amazon Web Services.

## [AWS\\_WEB\\_IDENTITY\\_TOKEN\\_FILE](#)

Especifica o caminho para um arquivo que contém um token de acesso OAuth 2.0 ou token de ID OpenID Connect fornecido por um provedor de identidade. A AWS CLI carrega o conteúdo desse arquivo e o transmite como o argumento `WebIdentityToken` para a operação `AssumeRoleWithWebIdentity`.

Usado com as variáveis de ambiente `AWS_ROLE_ARN` e `AWS_ROLE_SESSION_NAME`.

Se definida, essa variável de ambiente sobrescreverá o valor da configuração de perfil `web_identity_token_file`.

Para obter mais informações sobre o uso de identidades da Web, consulte [the section called “Assumir a função com a identidade da web”](#).

### Note

Essa variável de ambiente se aplica somente a uma função assumida com o provedor de identidade da Web, mas não se aplica à configuração do provedor de função assumida geral.

## Opções de linha de comando

Na AWS CLI, as opções de linha de comando são parâmetros globais que você pode usar para substituir as configurações padrão, qualquer configuração de perfil correspondente ou a configuração de variável de ambiente para esse único comando. Você não pode usar opções de linha de comando para especificar diretamente credenciais, embora seja possível especificar qual perfil usar.

### Tópicos

- [Como usar as opções de linha de comando](#)
- [A AWS CLI comporta opções de linha de comando globais](#)
- [Usos comuns das opções de linha de comando](#)

## Como usar as opções de linha de comando

As opções de linha de comando são, na maioria, strings simples, como o nome do perfil `profile1` no exemplo a seguir:

```
$ aws s3 ls --profile profile1
example-bucket-1
example-bucket-2
...
```

Cada opção que obtém um argumento requer com um espaço ou sinal de igual (=) separando o argumento do nome da opção. Se o valor do argumento for uma string que contenha um espaço, coloque o argumento entre aspas. Para obter detalhes sobre tipos de argumento e formatação de parâmetros, consulte [Especificar valores de parâmetro para a AWS CLI](#).

## A AWS CLI comporta opções de linha de comando globais

Na AWS CLI, você pode usar as opções de linha de comando a seguir para substituir as configurações padrão, qualquer configuração de perfil correspondente ou a configuração de variável de ambiente para esse único comando.

`--ca-bundle` **<string>**

Especifica o pacote de certificados da autoridade de certificação (CA) a ser usado ao verificar certificados SSL.

Se definida, essa opção substituirá o valor da configuração de perfil [ca\\_bundle](#) e [AWS\\_CA\\_BUNDLE](#) da variável de ambiente.

`--cli-connect-timeout` **<integer>**

Especifica o tempo de conexão de soquete máximo em segundos. Se o valor for definido como zero (0), a conexão de soquete aguardará indefinidamente (será bloqueada) e não atingirá o tempo limite.

`--cli-read-timeout` **<integer>**

Especifica o tempo de leitura de soquete máximo em segundos. Se o valor for definido como zero (0), a leitura do soquete aguardará indefinidamente (será bloqueada) e não atingirá o tempo limite.

**--color <string>**

Especifica o suporte para saída de cores. Os valores válidos são on, off e auto. O valor padrão é auto.

**--debug**

Uma operação booliana que ativa o registro em log de depuração. Por padrão, a AWS CLI fornece informações limpas sobre quaisquer sucessos ou falhas em relação aos resultados do comando na saída do comando. A opção `--debug` fornece os logs completos do Python. Isso inclui informações adicionais de diagnóstico de `stderr` sobre a operação do comando que podem ser úteis para a solução de problemas de um comando que gera resultados inesperados. Para visualizar facilmente os logs de depuração, sugerimos enviar os logs para um arquivo para que seja possível pesquisar as informações de forma mais fácil. Isso pode ser feito de uma das formas a seguir.

Para enviar somente as informações de diagnóstico de `stderr`, anexe `2> debug.txt`, onde `debug.txt` é o nome que você deseja usar para o seu arquivo de depuração:

```
$ aws servicename commandname options --debug 2> debug.txt
```

Para enviar ambas a saída e as informações de diagnóstico de `stderr`, anexe `&> debug.txt`, onde `debug.txt` é o nome que você deseja usar para o seu arquivo de depuração:

```
$ aws servicename commandname options --debug &> debug.txt
```

**--endpoint-url <string>**

Especifica o URL para o qual enviar a solicitação. Para a maioria dos comandos, a AWS CLI determina automaticamente o URL com base no serviço selecionado e na região da AWS especificada. No entanto, alguns comandos exigem que você especifique um URL específico para a conta. Você também pode configurar alguns serviços da AWS para [hospedar um endpoint diretamente dentro de sua VPC privada](#), que talvez precise ser especificada.

O exemplo de comando a seguir usa um URL personalizado do endpoint do Amazon S3.

```
$ aws s3 ls --endpoint-url http://localhost:4567
```

As configurações do endpoint estão em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de

comando como um parâmetro. As configurações do endpoint da AWS CLI têm precedência na seguinte ordem:

1. A opção da linha de comando [`--endpoint-url`](#).
2. Se habilitada, a variável de ambiente global [`AWS\_IGNORE\_CONFIGURED\_ENDPOINT\_URLS`](#) ou a configuração do perfil [`ignore\_configure\_endpoint\_urls`](#) para ignorar os endpoints personalizados.
3. O valor fornecido por uma variável de ambiente específica do serviço [`AWS\_ENDPOINT\_URL\_<SERVICE>`](#), como `AWS_ENDPOINT_URL_DYNAMODB`.
4. Os valores fornecidos pelas variáveis de ambiente [`AWS\_USE\_DUALSTACK\_ENDPOINT`](#), [`AWS\_USE\_FIPS\_ENDPOINT`](#) e [`AWS\_ENDPOINT\_URL`](#).
5. O valor do endpoint específico de serviço fornecido pela configuração [`endpoint\_url`](#) em uma seção `services` do arquivo compartilhado `config`.
6. O valor fornecido pela configuração [`endpoint\_url`](#) em um `profile` do arquivo compartilhado `config`.
7. Configurações [`use\_dualstack\_endpoint`](#), [`use\_fips\_endpoint`](#) e [`endpoint\_url`](#).
8. Qualquer URL de endpoint padrão para o respectivo AWS service (Serviço da AWS) é usado por último. Para obter uma lista dos endpoints de serviços padrão disponíveis em cada região, consulte [Regiões e endpoints da AWS](#) no Referência geral da Amazon Web Services.

#### `--no-paginate`

Uma opção booleana que desabilita as chamadas múltiplas que a AWS CLI faz automaticamente para receber todos os resultados de comandos que criam a paginação da saída. Isso significa que apenas a primeira página da sua saída é exibida.

#### `--no-sign-request`

Uma operação booliana que desativa a assinatura de solicitações HTTP para o endpoint de serviço da AWS. Isso evita que as credenciais sejam carregadas.

#### `--no-verify-ssl`

Por padrão, a AWS CLI usa SSL ao se comunicar com serviços da AWS. Para cada chamada e conexão SSL, a AWS CLI verifica os certificados SSL. O uso dessa opção substitui o comportamento padrão de verificação de certificados SSL.

**⚠ Warning**

Essa opção não é uma prática recomendada. Se você usa `--no-verify-ssl`, o tráfego entre seu cliente e os serviços da AWS não está mais protegido. Isso significa que o tráfego é um risco de segurança e está vulnerável a explorações man-in-the-middle. Se estiver tendo problemas com certificados, é melhor resolvê-los. Para ver as etapas de resolução de problemas de certificados, consulte [the section called “Erros de certificado SSL”](#).

**--output <string>**

Especifica o formato de saída a ser usado para este comando. Você pode especificar qualquer um dos seguintes valores:

- **json**: a saída é formatada como uma string [JSON](#).
- **text**: a saída é formatada como várias linhas de valores de string separados por tabulação. Isso pode ser útil para passar a saída para um processador de texto, como `grep`, `sed` ou `awk`.
- **table**: a saída é formatada como uma tabela usando os caracteres `+` e `|` para formar as bordas da célula. Geralmente, a informação é apresentada em um formato "amigável", que é muito mais fácil de ler do que outros, mas não tão útil programaticamente.

**--profile <string>**

Especifica o [perfil nomeado](#) para usar esse comando. Para configurar perfis nomeados adicionais, você pode usar o comando `aws configure` com a opção `--profile`.

```
$ aws configure --profile <profilename>
```

**--query <string>**

Especifica uma [consulta JMESPath](#) para uso na filtragem dos dados de resposta. Para obter mais informações, consulte [Filtragem da saída da AWS CLI](#).

**--region <string>**

Especifica para qual região da AWS enviar a solicitação da AWS desse comando. Para obter uma lista de todas as regiões que você pode especificar, consulte [Regiões e endpoints da AWS](#) no Referência geral da Amazon Web Services.

## --version

Uma operação booliana que exibe a versão atual do programa de AWS CLI que está em execução.

## Usos comuns das opções de linha de comando

Os usos comuns das opções de linha de comando incluem a verificação de seus recursos em várias regiões da AWS e a alteração do formato de saída para legibilidade ou a facilidade de uso ao criar scripts. Nos exemplos a seguir, executamos o comando `describe-instances` em cada região até descobrirmos em qual delas nossa instância está.

```
$ aws ec2 describe-instances --output table --region us-west-1
-----
|DescribeInstances|
+-----+
$ aws ec2 describe-instances --output table --region us-west-2
-----
|                                     DescribeInstances                                     |
+-----+
||                                     Reservations                                     ||
|+-----+|
||   OwnerId                           | 012345678901                                ||
||   ReservationId                     | r-abcdefgh                                ||
|+-----+|
||                                     Instances                                     ||
||+-----+||
|||   AmiLaunchIndex                   | 0                                           |||
|||   Architecture                     | x86_64                                    |||
...

```

## Conclusão de comando

A AWS Command Line Interface (AWS CLI) inclui um recurso de preenchimento automático de comandos que permite usar a tecla Tab para concluir o preenchimento um comando inserido parcialmente. Na maioria dos sistemas, esse recurso deve ser configurado manualmente.

### Tópicos

- [Como funciona](#)

- [Configuração do preenchimento automático de comando no Linux ou no macOS](#)
- [Configuração do preenchimento de comandos no Windows](#)

## Como funciona

Ao inserir parcialmente um comando, um parâmetro ou uma opção, o recurso de conclusão de comando conclui automaticamente o comando ou exibe uma lista sugerida de comandos. Para solicitar a conclusão de comandos, insira parcialmente um comando e pressione a tecla de conclusão, que normalmente é **Tab** na maioria dos shells.

Os exemplos a seguir mostram diferentes maneiras de usar a conclusão de comando:

- Insira parcialmente um comando e pressione **Tab** para exibir uma lista sugerida de comandos.

```
$ aws dynamodb dTAB
delete-backup                describe-global-table
delete-item                  describe-global-table-settings
delete-table                 describe-limits
describe-backup              describe-table
describe-continuous-backups  describe-table-replica-auto-scaling
describe-contributor-insights describe-time-to-live
describe-endpoints
```

- Insira parcialmente um parâmetro e pressione **Tab** para exibir uma lista sugerida de parâmetros.

```
$ aws dynamodb delete-table --TAB
--ca-bundle                --endpoint-url            --profile
--cli-connect-timeout      --generate-cli-skeleton   --query
--cli-input-json           --no-paginate             --region
--cli-read-timeout         --no-sign-request         --table-name
--color                    --no-verify-ssl           --version
--debug                    --output
```

- Insira um parâmetro e pressione **Tab** para exibir uma lista sugerida de valores de recursos. Esse recurso só está disponível na AWS CLI versão 2.

```
$ aws dynamodb db delete-table --table-name TAB
Table 1                Table 2                Table 3
```



# Configuração do preenchimento automático de comando no Linux ou no macOS

Para configurar o preenchimento automático de comandos no Linux ou macOS, você deve conhecer o nome do shell que está usando e a localização do script `aws_completer`.

## Note

O preenchimento do comando é configurado automaticamente e habilitado por padrão nas instâncias do Amazon EC2 que executam o Amazon Linux.

## Tópicos

- [Confirme se a pasta do completer está na variável path](#)
- [Habilitar a conclusão de comando](#)
- [Verifique o preenchimento de comandos](#)

## Confirme se a pasta do completer está na variável path

Para o completer da AWS funcionar corretamente, o `aws_completer` deverá estar na variável `path` do seu shell. O comando `which` pode verificar se o completer está na variável `path`.

```
$ which aws_completer
/usr/local/bin/aws_completer
```

Se o comando não conseguir encontrar o completer, use as etapas a seguir para adicionar a pasta do completer à variável `PATH`.

## Etapa 1: Localizar o completer da AWS

A localização do completer da AWS pode variar, dependendo do método de instalação usado.

- Gerenciador de pacotes: programas como `pip`, `yum`, `brew` e `apt-get` normalmente instalam o completer da AWS (ou um symlink para ele) em um local que faz parte do caminho padrão.
  - Se você usou o `pip` sem o parâmetro `--user`, o caminho padrão é `/usr/local/bin/aws_completer`.

- Se você usou o pip com o parâmetro `--user`, o caminho padrão é `home/username/.local/bin/aws_completer`.
- Instalador empacotado: se você usou o instalador empacotado, o caminho padrão é `/usr/local/bin/aws_completer`.

Se tudo falhar, você poderá usar o comando `find` para pesquisar o `completer` da AWS em todo o seu sistema de arquivos.

```
$ find / -name aws_completer
/usr/local/bin/aws_completer
```

## Etapa 2: Identificar o shell

Para identificar qual shell você está usando, é possível usar um dos seguintes comandos.

- `echo $SHELL`: exibe o nome do arquivo do programa de shell. Isso geralmente corresponde ao nome do shell em uso, a menos que você tenha iniciado um shell diferente após fazer o login.

```
$ echo $SHELL
/bin/bash
```

- `ps`: exibe os processos em execução para o usuário atual. Um deles é o shell.

```
$ ps
  PID TTY          TIME CMD
 2148 pts/1    00:00:00 bash
 8756 pts/1    00:00:00 ps
```

## Etapa 3: Adicionar o `completer` ao caminho

1. Encontre o script de perfil do shell em sua pasta de usuário.

```
$ ls -a ~/
.  ..  .bash_logout  .bash_profile  .bashrc  Desktop  Documents  Downloads
```

- Bash: `.bash_profile`, `.profile` ou `.bash_login`
- Zsh: `.zshrc`
- Tcsh: `.tcshrc`, `.cshrc` ou `.login`

2. Adicione um comando de exportação ao final do script de perfil que é semelhante ao exemplo a seguir. Substitua `/usr/local/bin/` pela pasta que você descobriu na seção anterior.

```
export PATH=/usr/local/bin/:$PATH
```

3. Recarregue o perfil na sessão atual para colocar essas alterações em vigor. Substitua `.bash_profile` pelo nome do script de shell que você descobriu na primeira seção.

```
$ source ~/.bash_profile
```

## Habilitar a conclusão de comando

Depois de confirmar que o `completer` está no caminho, habilite a conclusão de comando executando o comando apropriado para o shell que você está usando. Você pode adicionar o comando ao perfil do shell para executá-lo cada vez que você abrir um novo shell. Em cada comando, substitua o caminho `/usr/local/bin/` pelo encontrado no seu sistema em [Confirme se a pasta do `completer` está na variável `path`](#).

- **bash:** usa o comando interno `complete`.

```
$ complete -C '/usr/local/bin/aws_completer' aws
```

Adicione o comando anterior ao `~/.bashrc` para executá-lo cada vez que você abrir um novo shell. O `~/.bash_profile` deve se basear em `~/.bashrc` para garantir que o comando também seja executado em shells de login.

- **zsh:** para executar o preenchimento de comandos, é necessário executar `bashcompinit` adicionando a seguinte linha de carregamento automático ao final do script do perfil `~/.zshrc`.

```
$ autoload bashcompinit && bashcompinit  
$ autoload -Uz compinit && compinit
```

Para habilitar a conclusão de comando, use o comando interno `complete`.

```
$ complete -C '/usr/local/bin/aws_completer' aws
```

Adicione os comandos anteriores ao `~/.zshrc` para executá-los cada vez que você abrir um novo shell.

- **tcsh**: o preenchimento para tcsh utiliza um tipo de palavra e padrão para definir o comportamento do preenchimento.

```
> complete aws 'p/*/'`aws_completer`/'
```

Adicione o comando anterior ao `~/.tcshrc` para executá-lo cada vez que você abrir um novo shell.

Depois de habilitar o preenchimento automático de comandos, [Verifique o preenchimento de comandos](#) está funcionando.

## Verifique o preenchimento de comandos

Após habilitar o preenchimento de comandos, insira um comando parcial e pressione Tab para ver os comandos disponíveis.

```
$ aws sTAB
s3          ses          sqs          sts          swf
s3api       sns          storagegateway support
```

## Configuração do preenchimento de comandos no Windows

### Note

Para obter informações sobre como o PowerShell lida com a conclusão, incluindo as várias teclas de conclusão, consulte [about\\_tab\\_Expansion](#) na documentação do Microsoft PowerShell.

Para habilitar o preenchimento de comandos para o PowerShell no Windows, conclua as etapas a seguir no PowerShell.

1. Abra o arquivo `$PROFILE` com o comando a seguir.

```
PS C:\> Notepad $PROFILE
```

Se você ainda não tiver um `$PROFILE`, use o procedimento a seguir para criar um perfil de usuário.

```
PS C:\> if (!(Test-Path -Path $PROFILE ))
{ New-Item -Type File -Path $PROFILE -Force }
```

Para obter mais informações sobre os perfis do PowerShell, consulte [Como usar perfis no Windows PowerShell ISE](#) no site de Documentação da Microsoft.

2. Para ativar o preenchimento de comandos, adicione o seguinte bloco de código ao seu perfil, salve e, em seguida, feche o arquivo.

```
Register-ArgumentCompleter -Native -CommandName aws -ScriptBlock {
    param($commandName, $wordToComplete, $cursorPosition)
    $env:COMP_LINE=$wordToComplete
    if ($env:COMP_LINE.Length -lt $cursorPosition){
        $env:COMP_LINE=$env:COMP_LINE + " "
    }
    $env:COMP_POINT=$cursorPosition
    aws_completer.exe | ForEach-Object {
        [System.Management.Automation.CompletionResult]::new($_, $_,
        'ParameterValue', $_)
    }
    Remove-Item Env:\COMP_LINE
    Remove-Item Env:\COMP_POINT
}
```

3. Depois de habilitar a conclusão dos comandos, recarregue o shell, insira um comando parcial e pressione Tab para percorrer os comandos disponíveis.

```
$ aws sTab
```

```
$ aws s3
```

Para ver todos os comandos disponíveis para conclusão, insira um comando parcial e pressione Ctrl + Espaço.

```
$ aws sCtrl + Space
s3          ses          sqs          sts          swf
s3api       sns          storagegateway support
```

# Novas tentativas da AWS CLI

Este tópico descreve como a AWS CLI pode observar chamadas para AWS falharem devido a problemas inesperados. Esses problemas podem ocorrer no lado do servidor ou podem falhar devido à limitação de taxa do serviço da AWS que você está tentando chamar. Esses tipos de falhas geralmente não exigem tratamento especial e a chamada é feita automaticamente novamente, em geral após um breve período de espera. A AWS CLI oferece muitos recursos para ajudar a repetir chamadas de cliente para os serviços da AWS quando esses tipos de erros ou exceções ocorrem.

## Tópicos

- [Modos de novas tentativas disponíveis](#)
- [Configuração um modo de nova tentativa](#)
- [Visualização de logs de novas tentativas](#)

## Modos de novas tentativas disponíveis

A AWS CLI oferece vários modos para escolha dependendo da sua versão:

- [Modo de novas tentativas herdado](#)
- [Modo de nova tentativa padrão](#)
- [Modo de nova tentativa adaptável](#)

## Modo de novas tentativas herdado

O modo herdado é o modo padrão usado pela AWS CLI versão 1. O modo herdado usa um manipulador de novas tentativas mais antigo e com funcionalidade limitada que inclui:

- Um valor padrão de 4 para o máximo de novas tentativas, totalizando 5 tentativas de chamada. Esse valor pode ser sobrescrito por meio do parâmetro de configuração `max_attempts`.
- No DynamoDB, o valor padrão máximo de novas tentativas é nove, totalizando dez tentativas de chamada. Esse valor pode ser sobrescrito por meio do parâmetro de configuração `max_attempts`.
- Novas tentativas para o seguinte número limitado de erros e exceções:
  - Erros gerais de soquete e conexão:
    - `ConnectionError`
    - `ConnectionClosedError`

- `ReadTimeoutError`
- `EndpointConnectionError`
- Erros e exceções de controle de utilização ou limitação no lado do serviço:
  - `Throttling`
  - `ThrottlingException`
  - `ThrottledException`
  - `RequestThrottledException`
  - `ProvisionedThroughputExceededException`
- Novas tentativas em vários códigos de status HTTP, incluindo 429, 500, 502, 503, 504 e 509.
- Qualquer tentativa de repetição incluirá um recuo exponencial por um fator de base 2.

## Modo de nova tentativa padrão

O modo padrão é um conjunto padrão de regras de novas tentativas nos AWS SDKs com mais funcionalidade do que herdado. O modo padrão foi criado para a AWS CLI versão 2 e é compatível com a AWS CLI versão 1. A funcionalidade do modo padrão inclui:

- Um valor padrão de 2 para o máximo de novas tentativas, totalizando 3 tentativas de chamada. Esse valor pode ser sobrescrito por meio do parâmetro de configuração `max_attempts`.
- Novas tentativas para a seguinte lista estendida de erros/exceções:
  - Erros e exceções transientes
    - `RequestTimeout`
    - `RequestTimeoutException`
    - `PriorRequestNotComplete`
    - `ConnectionError`
    - `HTTPClientError`
  - Erros e exceções de controle de utilização ou limitação no lado do serviço:
    - `Throttling`
    - `ThrottlingException`
    - `ThrottledException`
    - `RequestThrottledException`
    - `TooManyRequestsException`

- `ProvisionedThroughputExceededException`
  - `TransactionInProgressException`
  - `RequestLimitExceeded`
  - `BandwidthLimitExceeded`
  - `LimitExceededException`
  - `RequestThrottled`
  - `SlowDown`
  - `EC2ThrottledException`
- Novas tentativas em códigos de erro não descritivos transientes. Especificamente, estes códigos de status HTTP: 500, 502, 503, 504.
  - Qualquer nova tentativa incluirá um recuo exponencial por um fator de base 2 para um tempo máximo de recuo de 20 segundos.

## Modo de nova tentativa adaptável

### Warning

O modo adaptativo é um experimental e está sujeito a modificações, tanto em suas características quanto em seu comportamento.

O modo de nova tentativa adaptativo é um modo de repetição experimental que inclui todos os recursos do modo padrão. Além dos recursos do modo padrão, o modo adaptativo também introduz limitação de taxa no lado do cliente através do uso de um bucket de token e variáveis de limite de taxa que são atualizadas dinamicamente com cada tentativa de repetição. Esse modo oferece flexibilidade de novas tentativas no lado do cliente que se adapta à resposta de estado de erro e exceção de um serviço da AWS.

Com cada nova tentativa, o modo adaptativo modifica as variáveis de limite de taxa com base no erro, exceção ou código de status HTTP apresentado na resposta do serviço da AWS. Essas variáveis de limite de taxa são usadas para calcular uma nova taxa de chamada para o cliente. Cada resposta HTTP de exceção/erro ou não bem-sucedida (fornecida na lista acima) de um serviço da AWS atualiza as variáveis de limite de taxa à medida que as tentativas ocorrerem até alcançarem êxito, o bucket de token se esgotar ou o valor máximo de tentativas configurado ser atingido.



## Configuração um modo de nova tentativa

A AWS CLI inclui uma variedade de configurações de novas tentativas, bem como métodos de configuração a serem considerados ao criar seu objeto cliente.

### Métodos de configuração disponíveis

Na AWS CLI, os usuários podem configurar novas tentativas das seguintes formas:

- Variáveis de ambiente
- Arquivo de configuração da AWS CLI

Os usuários podem personalizar as seguintes opções de novas tentativas:

- Modo de repetição: especifica qual modo de repetição será usado pela AWS CLI. Conforme descrito anteriormente existem três modos de repetição disponíveis: herdado (o modo usado por padrão), padrão e adaptativo. O valor padrão para a AWS CLI versão 1 é herdado.
- Máximo de tentativas: especifica um valor para o máximo de novas tentativas utilizadas pelo manipulador de novas tentativas da AWS CLI, onde a chamada inicial conta para o valor fornecido por você. O valor padrão é 5.

### Definição de uma configuração de novas tentativas em suas variáveis de ambiente

Para definir a configuração de novas tentativas para o AWS CLI, atualize as variáveis de ambiente do seu sistema operacional.

As variáveis de ambiente de novas tentativas são:

- AWS\_RETRY\_MODE
- AWS\_MAX\_ATTEMPTS

Para obter mais informações sobre variáveis de ambiente, consulte [Variáveis de ambiente para configurar a AWS CLI](#).

## Visualização de logs de novas tentativas

A AWS CLI usa a metodologia de novas tentativas e log do Boto3. Você pode usar a opção `--debug` em qualquer comando para receber logs de depuração. Para obter informações sobre como usar a opção `--debug`, consulte [Opções de linha de comando](#).

Se você procurar “retry” em seus logs de depuração, encontrará as informações de repetição de que necessita. As entradas de log do cliente para tentativas de repetição dependem do modo de repetição ativado.

Modo herdado:

As mensagens de novas tentativas são geradas por `botocore.retryhandler`. Você verá uma de três mensagens:

- `No retry needed`
- `Retry needed, action of: <action_name>`
- `Reached the maximum number of retry attempts: <attempt_number>`

Modo padrão ou adaptativo:

As mensagens de novas tentativas são geradas por `botocore.retries.standard`. Você verá uma de três mensagens:

- `No retrying request`
- `Retry needed, retrying request after delay of: <delay_value>`
- `Retry needed but retry quota reached, not retrying request`

Para obter o arquivo de definição completa de novas tentativas do `botocore`, consulte [\\_retry.json](#) no Repositório do `botocore` no GitHub.

## Usar um proxy HTTP

Para acessar a AWS por meio de servidores de proxy, é possível configurar as variáveis de ambiente `HTTP_PROXY` e `HTTPS_PROXY` com os nomes de domínio DNS ou endereços IP e números de porta usados pelos servidores de proxy.

Tópicos

- [Como usar os exemplos da](#)
- [Autenticar para um proxy](#)
- [Uso de proxy em instâncias do Amazon EC2](#)
- [Solução de problemas](#)

## Como usar os exemplos da

### Note

Os exemplos a seguir mostram o nome da variável de ambiente com todas as letras maiúsculas. No entanto, se você especificar uma variável duas vezes usando letras maiúsculas e minúsculas, as minúsculas terão precedência. Recomendamos que você defina cada variável somente uma vez para evitar confusão e comportamento inesperado do sistema.

Os exemplos a seguir mostram como você pode usar o endereço IP explícito do proxy ou um nome de DNS que seja resolvido para o endereço IP do proxy. Também pode ser seguido por uma vírgula e o número da porta para a qual as consultas devem ser enviadas.

### Linux or macOS

```
$ export HTTP_PROXY=http://10.15.20.25:1234
$ export HTTP_PROXY=http://proxy.example.com:1234
$ export HTTPS_PROXY=http://10.15.20.25:5678
$ export HTTPS_PROXY=http://proxy.example.com:5678
```

### Windows Command Prompt

#### Como definir para todas as sessões

```
C:\> setx HTTP_PROXY http://10.15.20.25:1234
C:\> setx HTTP_PROXY http://proxy.example.com:1234
C:\> setx HTTPS_PROXY http://10.15.20.25:5678
C:\> setx HTTPS_PROXY http://proxy.example.com:5678
```

O uso de [setx](#) para definir uma variável de ambiente altera o valor usado na sessão de prompt de comando atual e todas as sessões de prompt de comando que você criar após a execução do

comando. Não afeta outros shells de comando que já estejam em execução no momento em que você executar o comando.

Como definir somente para a sessão atual

O uso de [set](#) para definir uma variável de ambiente altera o valor usado até o final da sessão de prompt de comando atual ou até que você defina a variável como um valor diferente.

```
C:\> set HTTP_PROXY=http://10.15.20.25:1234
C:\> set HTTP_PROXY=http://proxy.example.com:1234
C:\> set HTTPS_PROXY=http://10.15.20.25:5678
C:\> set HTTPS_PROXY=http://proxy.example.com:5678
```

## Autenticar para um proxy

### Note

A AWS CLI não é compatível com proxies NTLM. Se você usa um proxy de protocolo NTLM ou Kerberos, talvez seja possível se conectar por meio de um proxy de autenticação, como [Cntlm](#).

O AWS CLI é compatível com a autenticação básica HTTP. Especifique o nome do usuário e uma senha no URL de proxy da forma a seguir.

Linux or macOS

```
$ export HTTP_PROXY=http://username:password@proxy.example.com:1234
$ export HTTPS_PROXY=http://username:password@proxy.example.com:5678
```

Windows Command Prompt

Como definir para todas as sessões

```
C:\> setx HTTP_PROXY http://username:password@proxy.example.com:1234
C:\> setx HTTPS_PROXY http://username:password@proxy.example.com:5678
```

Como definir somente para a sessão atual

```
C:\> set HTTP_PROXY=http://username:password@proxy.example.com:1234  
C:\> set HTTPS_PROXY=http://username:password@proxy.example.com:5678
```

## Uso de proxy em instâncias do Amazon EC2

Se você configurar um proxy em uma instância do Amazon EC2 iniciada com uma função do IAM anexada, certifique-se de isentar o endereço usado do acesso aos [metadados da instância](#). Para fazer isso, defina a variável de ambiente NO\_PROXY como o endereço IP do serviço de metadados da instância 169.254.169.254. Esse endereço não varia.

Linux or macOS

```
$ export NO_PROXY=169.254.169.254
```

Windows Command Prompt

Como definir para todas as sessões

```
C:\> setx NO_PROXY 169.254.169.254
```

Como definir somente para a sessão atual

```
C:\> set NO_PROXY=169.254.169.254
```

## Solução de problemas

Se você encontrar problemas com a AWS CLI, consulte [Solucionar erros](#) para obter as etapas de solução de problemas. Para obter as etapas mais relevantes de solução de problemas, consulte [the section called “Erros de certificado SSL”](#).

## Usar endpoints na AWS CLI

Para se conectar a um AWS service (Serviço da AWS) de forma programática, use um endpoint. Um endpoint é o URL do ponto de entrada para um serviço da Web da AWS. A AWS Command Line Interface (AWS CLI) usa automaticamente o endpoint padrão para cada serviço em uma Região da AWS, mas você pode especificar um endpoint alternativo para suas solicitações de API.

## Tópicos de endpoint

- [Definir o endpoint para um único comando](#)
- [Definir um endpoint global para todos os Serviços da AWS](#)
- [Definido para usar endpoints FIPs para todos os Serviços da AWS](#)
- [Configurado para usar os endpoints de pilha dupla para todos os Serviços da AWS](#)
- [Definir endpoints específicos de serviço](#)
  - [Endpoints específicos de serviço: variáveis de ambiente](#)
  - [Endpoints específicos de serviço: arquivo compartilhado config](#)
  - [Endpoints específicos de serviço: lista de identificadores específicos de serviço](#)
- [Precedência de configurações e definições do endpoint](#)

## Definir o endpoint para um único comando

Para substituir qualquer configuração de endpoint ou variável de ambiente referente a um único comando, use a opção de linha de comando [--endpoint-url](#). O exemplo de comando a seguir usa um URL personalizado do endpoint do Amazon S3.

```
$ aws s3 ls --endpoint-url http://localhost:4567
```

## Definir um endpoint global para todos os Serviços da AWS

Para rotear solicitações de todos os serviços para um URL de endpoint personalizado, use uma das seguintes configurações:

- Variáveis de ambiente:
  - [AWS\\_IGNORE\\_CONFIGURED\\_ENDPOINT\\_URLS](#): ignore os URLs de endpoints configurados.

Linux or macOS

```
$ export AWS_IGNORE_CONFIGURED_ENDPOINT_URLS=true
```

Windows Command Prompt

Como definir para todas as sessões

```
C:\> setx AWS_IGNORE_CONFIGURED_ENDPOINT_URLS true
```

Como definir somente para a sessão atual

```
C:\> set AWS_IGNORE_CONFIGURED_ENDPOINT_URLS=true
```

PowerShell

```
PS C:\> $Env:AWS_IGNORE_CONFIGURED_ENDPOINT_URLS="true"
```

- [AWS\\_ENDPOINT\\_URL](#): defina o URL do endpoint global.

Linux or macOS

```
$ export AWS_ENDPOINT_URL=http://localhost:4567
```

Windows Command Prompt

Como definir para todas as sessões

```
C:\> setx AWS_ENDPOINT_URL http://localhost:4567
```

Como definir somente para a sessão atual

```
C:\> set AWS_ENDPOINT_URL=http://localhost:4567
```

PowerShell

```
PS C:\> $Env:AWS_ENDPOINT_URL="http://localhost:4567"
```

- O arquivo config:
  - [ignore\\_configure\\_endpoint\\_urls](#): ignore os URLs de endpoints configurados.

```
ignore_configure_endpoint_urls = true
```

- [endpoint\\_url](#): defina o URL do endpoint global.

```
endpoint_url = http://localhost:4567
```

Os endpoints específicos de serviço e a opção de linha de comando `--endpoint-url` substituem qualquer endpoint global.

## Definido para usar endpoints FIPs para todos os Serviços da AWS

Para rotear solicitações para todos os serviços usarem endpoints FIPS, use uma das seguintes opções:

- variável de ambiente [AWS\\_USE\\_FIPS\\_ENDPOINT](#).

Linux or macOS

```
$ export AWS_USE_FIPS_ENDPOINT=true
```

Windows Command Prompt

Como definir para todas as sessões

```
C:\> setx AWS_USE_FIPS_ENDPOINT true
```

Como definir somente para a sessão atual

```
C:\> set AWS_USE_FIPS_ENDPOINT=true
```

PowerShell

```
PS C:\> $Env:AWS_USE_FIPS_ENDPOINT="true"
```

- configuração do arquivo [use\\_fips\\_endpoint](#).

```
use_fips_endpoint = true
```

Alguns serviços da AWS oferecem endpoints compatíveis com o [Federal Information Processing Standard \(FIPS\) 140-2](#) em algumas Regiões da AWS. Quando o serviço da AWS é compatível com o FIPS, essa configuração especifica qual endpoint do FIPS a AWS CLI deve usar. Ao contrário dos endpoints-padrão da AWS, os endpoints do FIPS usam uma biblioteca de software TLS compatível com o FIPS 140-2. Esses endpoints podem ser necessários por empresas que interagem com o governo dos Estados Unidos.



Se essa configuração estiver ativada, mas não existir um endpoint FIPS para o serviço em sua Região da AWS, o comando AWS poderá falhar. Nesse caso, especifique manualmente o endpoint a ser usado no comando usando a opção `--endpoint-url` ou use [endpoints específicos do serviço](#).

Para saber mais sobre como especificar endpoints FIPS por Região da AWS, consulte [Endpoints FIPS por serviço](#).

## Configurado para usar os endpoints de pilha dupla para todos os Serviços da AWS

Para rotear solicitações para todos os serviços usarem endpoints de pilha dupla, use uma das seguintes configurações:

- variável de ambiente [AWS\\_USE\\_DUALSTACK\\_ENDPOINT](#).

Linux or macOS

```
$ export AWS_USE_DUALSTACK_ENDPOINT=true
```

Windows Command Prompt

Como definir para todas as sessões

```
C:\> setx AWS_USE_DUALSTACK_ENDPOINT true
```

Como definir somente para a sessão atual

```
C:\> set AWS_USE_DUALSTACK_ENDPOINT=true
```

PowerShell

```
PS C:\> $Env:AWS_USE_DUALSTACK_ENDPOINT="true"
```

- configuração do arquivo [use\\_dualstack\\_endpoint](#).

```
use_dualstack_endpoint = true
```

Permite o uso de endpoints de pilha dupla para enviar solicitações da AWS. Para saber mais sobre endpoints de pilha dupla, que suportam tráfego IPv4 e IPv6, consulte [Como usar endpoints de pilha](#)

[dupla do Amazon S3](#) no Guia do usuário do Amazon Simple Storage Service. Endpoints de pilha dupla estão disponíveis para alguns serviços em algumas regiões. Se não existir um endpoint de pilha dupla para o serviço ou Região da AWS, a solicitação falhará. Ela fica desabilitada por padrão.

## Definir endpoints específicos de serviço

A configuração de endpoint específico de serviço oferece a opção de usar um endpoint persistente de sua escolha para solicitações da AWS CLI. Essas configurações oferecem flexibilidade para permitir endpoints locais, endpoints da VPC e ambientes de desenvolvimento da AWS local de terceiros. Diferentes endpoints podem ser usados para ambientes de teste e produção. Você pode especificar um URL de endpoint para Serviços da AWS individuais.

Os endpoints específicos de serviço podem ser designados das seguintes maneiras:

- A opção de linha de comando [--endpoint-url](#) para um único comando.
- Variáveis de ambiente:
  - [AWS\\_IGNORE\\_CONFIGURED\\_ENDPOINT\\_URLS](#): ignora todos os URLs de endpoint configurados, a menos que especificado na linha de comando.
  - [AWS\\_ENDPOINT\\_URL\\_<SERVICE>](#): especifica um endpoint personalizado usado para um serviço específico, onde <SERVICE> é substituído pelo identificador AWS service (Serviço da AWS). Para todas as variáveis específicas de serviço, consulte [the section called “Lista de identificadores específicos de serviço”](#).
- Arquivo config:
  - [ignore\\_configure\\_endpoint\\_urls](#): ignora todos os URLs de endpoint configurados, a menos que especificado por meio de variáveis de ambiente ou na linha de comando.
  - A seção [services](#) do arquivo config combinada com a configuração do arquivo [endpoint\\_url](#).

Tópicos de endpoints específicos de serviço:

- [Endpoints específicos de serviço: variáveis de ambiente](#)
- [Endpoints específicos de serviço: arquivo compartilhado config](#)
- [Endpoints específicos de serviço: lista de identificadores específicos de serviço](#)

## Endpoints específicos de serviço: variáveis de ambiente

As variáveis de ambiente substituem as configurações no arquivo de configuração, mas não substituem as opções especificadas na linha de comando. Use variáveis de ambiente se quiser que todos os perfis usem os mesmos endpoints no dispositivo.

Veja a seguir as variáveis de ambiente específicas de serviço:

- [AWS\\_IGNORE\\_CONFIGURED\\_ENDPOINT\\_URLS](#): ignora todos os URLs de endpoint configurados, a menos que especificado na linha de comando.

Linux or macOS

```
$ export AWS_IGNORE_CONFIGURED_ENDPOINT_URLS=true
```

Windows Command Prompt

Como definir para todas as sessões

```
C:\> setx AWS_IGNORE_CONFIGURED_ENDPOINT_URLS true
```

Como definir somente para a sessão atual

```
C:\> set AWS_IGNORE_CONFIGURED_ENDPOINT_URLS=true
```

PowerShell

```
PS C:\> $Env:AWS_IGNORE_CONFIGURED_ENDPOINT_URLS="true"
```

- [AWS\\_ENDPOINT\\_URL\\_<SERVICE>](#): especifica um endpoint personalizado usado para um serviço específico, onde <SERVICE> é substituído pelo identificador AWS service (Serviço da AWS). Para todas as variáveis específicas de serviço, consulte [the section called “Lista de identificadores específicos de serviço”](#).

O exemplo a seguir de variável de ambiente define um endpoint para o AWS Elastic Beanstalk:

Linux or macOS

```
$ export AWS_ENDPOINT_URL_ELASTIC_BEANSTALK=http://localhost:4567
```

## Windows Command Prompt

Como definir para todas as sessões

```
C:\> setx AWS_ENDPOINT_URL_ELASTIC_BEANSTALK http://localhost:4567
```

Como definir somente para a sessão atual

```
C:\> set AWS_ENDPOINT_URL_ELASTIC_BEANSTALK=http://localhost:4567
```

## PowerShell

```
PS C:\> $Env:AWS_ENDPOINT_URL_ELASTIC_BEANSTALK="http://localhost:4567"
```

Para obter mais informações sobre como definir variáveis de ambiente, consulte [the section called “Variáveis de ambiente”](#).

## Endpoints específicos de serviço: arquivo compartilhado **config**

No arquivo compartilhado `config`, `endpoint_url` é usado em várias seções. Para definir um endpoint específico de serviço, use a configuração `endpoint_url` aninhada em uma chave de identificação de serviço em uma seção `services`. Para obter detalhes sobre como definir uma seção `services` no arquivo compartilhado [the section called “services”](#), consulte `config`.

O exemplo a seguir usa uma seção `services` para configurar um URL de endpoint específico do serviço para o Amazon S3 e um endpoint global personalizado usado para todos os outros serviços:

```
[profile dev1]
endpoint_url = http://localhost:1234
services = s3-specific

[services testing-s3]
s3 =
    endpoint_url = http://localhost:4567
```

Um único perfil pode configurar endpoints para vários serviços. O exemplo a seguir define os URLs de endpoint específicos de serviço para o Amazon S3 e o AWS Elastic Beanstalk no mesmo perfil.

Para obter uma lista de todas as chaves de identificação de serviço a serem usadas na seção `services`, consulte [Lista de identificadores específicos de serviço](#).

```
[profile dev1]
services = testing-s3-and-eb

[services testing-s3-and-eb]
s3 =
    endpoint_url = http://localhost:4567
elastic_beanstalk =
    endpoint_url = http://localhost:8000
```

A seção de configuração de serviço pode ser usada em vários perfis. O exemplo a seguir tem dois perfis que usam a mesma definição de `services`:

```
[profile dev1]
output = json
services = testing-s3

[profile dev2]
output = text
services = testing-s3

[services testing-s3]
s3 =
    endpoint_url = https://localhost:4567
```

## Endpoints específicos de serviço: lista de identificadores específicos de serviço

O identificador AWS service (Serviço da AWS) é baseado no `serviceId` do modelo de API, substituindo todos os espaços por sublinhados e colocando todas as letras em minúsculas.

O exemplo de identificador de serviço a seguir usa o AWS Elastic Beanstalk. O AWS Elastic Beanstalk tem um `serviceId` de [Elastic Beanstalk](#); portanto, a chave de identificação de serviço é `elastic_beanstalk`.

A tabela a seguir lista todos os identificadores, chaves do arquivo `config` e variáveis de ambiente específicos de serviço.

## Precedência de configurações e definições do endpoint

As configurações do endpoint estão em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de comando como um parâmetro. As configurações do endpoint da AWS CLI têm precedência na seguinte ordem:

1. A opção da linha de comando [--endpoint-url](#).
2. Se habilitada, a variável de ambiente global [AWS\\_IGNORE\\_CONFIGURED\\_ENDPOINT\\_URLS](#) ou a configuração do perfil [ignore\\_configure\\_endpoint\\_urls](#) para ignorar os endpoints personalizados.
3. O valor fornecido por uma variável de ambiente específica do serviço [AWS\\_ENDPOINT\\_URL\\_<SERVICE>](#), como [AWS\\_ENDPOINT\\_URL\\_DYNAMODB](#).
4. Os valores fornecidos pelas variáveis de ambiente [AWS\\_USE\\_DUALSTACK\\_ENDPOINT](#), [AWS\\_USE\\_FIPS\\_ENDPOINT](#) e [AWS\\_ENDPOINT\\_URL](#).
5. O valor do endpoint específico de serviço fornecido pela configuração [endpoint\\_url](#) em uma seção `services` do arquivo compartilhado `config`.
6. O valor fornecido pela configuração [endpoint\\_url](#) em um `profile` do arquivo compartilhado `config`.
7. Configurações [use\\_dualstack\\_endpoint](#), [use\\_fips\\_endpoint](#) e [endpoint\\_url](#).
8. Qualquer URL de endpoint padrão para o respectivo AWS service (Serviço da AWS) é usado por último. Para obter uma lista dos endpoints de serviços padrão disponíveis em cada região, consulte [Regiões e endpoints da AWS](#) no Referência geral da Amazon Web Services.

# Autenticação e credenciais de acesso

É necessário estabelecer como a AWS CLI faz a autenticação com a AWS quando você desenvolve com serviços da AWS. Para configurar credenciais para acesso programático para a AWS CLI, escolha uma das opções a seguir. As opções estão em ordem de recomendação.

| Qual usuário precisa de acesso programático? | Finalidade                                        | Instruções                                                      |
|----------------------------------------------|---------------------------------------------------|-----------------------------------------------------------------|
| IAM                                          | Use credenciais de curto prazo.                   | <a href="#">the section called “Credenciais de curto prazo”</a> |
| IAM                                          | Use perfis como credenciais.                      | <a href="#">the section called “Perfis do IAM”</a>              |
| IAM                                          | (Não recomendado) Use credenciais de longo prazo. | <a href="#">the section called “IAM users”</a>                  |

## Precedência de credenciais e configurações

As credenciais e as configurações estão localizadas em vários locais, como variáveis de ambiente do sistema ou do usuário, arquivos de configuração local da AWS, ou explicitamente declaradas na linha de comando como um parâmetro. Certos locais têm precedência sobre outros. As credenciais da AWS CLI e as definições de configuração têm precedência na seguinte ordem:

1. [Opções da linha de comando](#): substituem as configurações em qualquer outro local, como nos parâmetros `--region`, `--output` e `--profile`.
2. [Variáveis de ambiente](#): você pode armazenar valores nas variáveis de ambiente do sistema.
3. [Assumir perfil](#): assumo as permissões de um perfil do IAM por meio da configuração ou do comando `aws sts assume-role`.
4. [Assumir perfil com identidade da web](#): assumo as permissões de um perfil do IAM usando uma identidade da web por meio da configuração ou do comando `aws sts assume-role`.
5. [Arquivo de credenciais](#): os arquivos `credentials` e `config` são atualizados quando você executa o comando `aws configure`. O arquivo `credentials` está localizado em `~/.aws/`

credentials no Linux ou MacOS ou em C:\Users\**USERNAME**\.aws\credentials no Windows.

6. [Processo personalizado](#): obtenha suas credenciais de uma fonte externa.
7. [Arquivo de configuração](#): os arquivos credentials e config são atualizados quando você executa o comando `aws configure`. O arquivo config está localizado em ~/.aws/config no Linux ou MacOS ou em C:\Users\**USERNAME**\.aws\config no Windows.
8. [Credenciais de perfil de instância](#): você pode associar um perfil do IAM a cada uma das suas instâncias do Amazon Elastic Compute Cloud (Amazon EC2). As credenciais temporárias para essa função estão disponíveis para o código em execução na instância. As credenciais são fornecidas por meio do serviço de metadados do Amazon EC2. Para obter mais informações, consulte [Funções do IAM para o Amazon EC2](#) no Manual do usuário do Amazon EC2 para instâncias do Linux e [Uso de perfis de instância](#) no Manual do usuário do IAM.
9. [Credenciais de container](#): você pode associar uma função do IAM a cada uma das suas definições de tarefa do Amazon Elastic Container Service (Amazon ECS). As credenciais temporárias para essa função estão disponíveis para os contêineres dessa tarefa. Para mais informações, consulte [Funções do IAM para Tarefas](#) no Guia de Desenvolvedor Amazon Elastic Container Service.

## Tópicos adicionais nesta seção

- [the section called “Credenciais de curto prazo”](#)
- [the section called “Perfis do IAM”](#)
- [the section called “IAM users”](#)
- [the section called “Uso de credenciais para metadados de instância do Amazon EC2.”](#)
- [the section called “Credenciais externas”](#)

## Autenticar com credenciais de curto prazo

Recomendamos configurar o SDK ou a ferramenta para usar a [autenticação do IAM Identity Center](#) com opções de duração de sessão estendida. No entanto, você pode copiar e usar credenciais temporárias que estão disponíveis no portal de acesso da AWS. As novas credenciais precisarão ser copiadas quando essas expirarem. É possível usar as credenciais temporárias em um perfil ou usá-las como valores para propriedades do sistema e variáveis de ambiente.

1. [Faça login no portal de acesso da AWS.](#)



2. Siga [estas instruções](#) para copiar as credenciais de perfil do IAM do portal de acesso da AWS.
  1. Na etapa 2 das instruções vinculadas, escolha a conta da AWS e o nome do perfil do IAM que concede acesso para suas necessidades de desenvolvimento. Esse perfil normalmente tem um nome como PowerUserAccess ou Developer.
  2. Na etapa 4, selecione a opção Adicionar um perfil ao seu arquivo de credenciais da AWS e copie o conteúdo.
3. Crie ou abra o arquivo `credentials` compartilhado. Esse arquivo é `~/.aws/credentials` em sistemas Linux e macOS e `%USERPROFILE%\aws\credentials` no Windows. Para obter mais informações, consulte [the section called “Configurações do arquivo de configuração e credenciais”](#).
4. Adicione o texto a seguir ao arquivo `credentials` compartilhado. Substitua os valores da amostra pelas credenciais que você copiou.

```
[default]
aws_access_key_id = AKIAIOSFODNN7EXAMPLE
aws_secret_access_key = wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
aws_session_token =
    IqoJb3JpZ2luX2IqoJb3JpZ2luX2IqoJb3JpZ2luX2IqoJb3JpZ2luX2IqoJb3JpZVERYLONGSTRINGEXAMPLE
```

5. Adicione a região e o formato padrão preferidos ao arquivo `config` compartilhado.

```
[default]
region=us-west-2
output=json

[profile user1]
region=us-east-1
output=text
```

Quando o SDK cria um cliente de serviço, ele acessa essas credenciais temporárias e as usa para cada solicitação. As configurações do perfil do IAM escolhidas na etapa 2a determinam [por quanto tempo as credenciais temporárias são válidas](#). A duração máxima é de doze horas.

Repita essas etapas sempre que suas credenciais expirarem.

# Uso de perfis do IAM na AWS CLI

Um [perfil do AWS Identity and Access Management \(IAM\)](#) é uma ferramenta de autorização que permite que um usuário obtenha permissões adicionais (ou diferentes) ou obtenha permissões para executar ações em uma conta diferente da AWS.

## Tópicos

- [Pré-requisitos](#)
- [Visão geral do uso de funções do IAM](#)
- [Configurar e usar uma função](#)
- [Usar autenticação multifator](#)
- [Funções entre contas e ID externo](#)
- [Especificar um nome de sessão de função para facilitar a auditoria](#)
- [Assumir a função com a identidade da web](#)
- [Limpar as credenciais em cache](#)

## Pré-requisitos

Para executar os comandos `iam`, você precisa instalar e configurar a AWS CLI. Para obter mais informações, consulte [Instalar a AWS CLI](#).

## Visão geral do uso de funções do IAM

Você pode configurar a AWS Command Line Interface (AWS CLI) para usar uma função do IAM definindo um perfil para a função no arquivo `~/.aws/config`.

O exemplo a seguir mostra um perfil de função chamado `marketingadmin`. Se você executar comandos com `--profile marketingadmin` (ou especificá-los com a [variável de ambiente `AWS\_PROFILE`](#)), a AWS CLI usará as credenciais definidas em um perfil do `user1` separado para assumir a função com o nome de recurso da Amazon (ARN) `arn:aws:iam::123456789012:role/marketingadminrole`. É possível executar quaisquer operações que forem permitidas pelas permissões atribuídas a essa função.

```
[profile marketingadmin]  
role_arn = arn:aws:iam::123456789012:role/marketingadminrole
```

```
source_profile = user1
```

Então, é possível especificar um `source_profile` que aponte para um perfil nomeado separado que contenha credenciais de usuário com permissão para usar o perfil. No exemplo anterior, o perfil `marketingadmin` usa as credenciais no perfil `user1`. Quando você especifica que um comando da AWS CLI deve usar o perfil `marketingadmin`, a AWS CLI procura automaticamente as credenciais para o perfil `user1` vinculado e as utiliza para solicitar credenciais temporárias para a função especificada do IAM. A CLI usa a operação [sts:AssumeRole](#) em segundo plano para fazer isso. Essas credenciais temporárias são então usadas para executar o comando da AWS CLI solicitado. A função especificada deve ter políticas de permissão do IAM associadas que permitam a execução do comando da AWS CLI solicitado.

Para executar um comando da AWS CLI em uma instância do Amazon Elastic Compute Cloud (Amazon EC2) ou um contêiner do Amazon Elastic Container Service (Amazon ECS), você pode usar uma função do IAM anexada ao perfil de instância ou ao contêiner. Se você não especificar nenhum perfil ou não definir variáveis de ambiente, essa função será usada diretamente. Isso permite que você evite armazenar chaves de acesso de longa duração em suas instâncias. Também é possível usar essas funções de instância ou contêiner apenas para obter credenciais para outra função. Para isso, use `credential_source` (em vez de `source_profile`) para especificar como localizar as credenciais. O atributo `credential_source` oferece suporte aos seguintes valores:

- `Environment`: recupera as credenciais de origem de variáveis de ambiente.
- `Ec2InstanceMetadata`: usa a função do IAM anexada ao perfil de instância do Amazon EC2.
- `EcsContainer`: usa a função do IAM anexada ao contêiner do Amazon ECS.

O exemplo a seguir mostra a mesma função `marketingadminrole` usada fazendo referência a um perfil de instância do Amazon EC2.

```
[profile marketingadmin]
role_arn = arn:aws:iam::123456789012:role/marketingadminrole
credential_source = Ec2InstanceMetadata
```

Ao invocar uma função, você tem opções adicionais que podem ser exigidas, como o uso de autenticação multifator e um ID externo (usado por empresas de terceiros para acessar os recursos de seus clientes). Também é possível especificar nomes de sessão de função exclusivos que possam ser auditados com mais facilidade nos logs do AWS CloudTrail.

## Configurar e usar uma função

Ao executar comandos usando um perfil que especifica uma função do IAM, a AWS CLI usa as credenciais do perfil de origem para chamar AWS Security Token Service (AWS STS) e solicitar credenciais temporárias para a função especificada. O usuário no perfil de origem deve ter permissão para chamar `sts:assume-role` para a função no perfil especificado. A função deve ter uma relação de confiança que permita que o usuário no perfil de origem use a função. Geralmente, o processo de recuperar e depois usar credenciais temporárias para uma função é chamado de assumir a função.

É possível criar um perfil no IAM com as permissões que você deseja que os usuários assumam seguindo o procedimento em [Criação de um perfil para conceder permissões a um usuário do IAM](#) no Manual do usuário do AWS Identity and Access Management. Se a função e o usuário do perfil de origem estão na mesma conta, é possível inserir seu próprio ID de conta ao configurar a relação de confiança da função.

Depois de criar a função, modifique a relação de confiança para permitir que o usuário assumir .

O exemplo a seguir mostra uma política de confiança que pode ser associada a uma função. Essa política permite que o perfil seja assumido por qualquer usuário na conta 123456789012, se o administrador dessa conta conceder explicitamente a permissão `sts:AssumeRole` ao usuário.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::123456789012:root"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

A política de confiança não concede permissões. O administrador da conta deve delegar a permissão para assumir a função para usuários individuais, anexando uma política com as permissões apropriadas. O exemplo a seguir mostra uma política que pode ser associada a um usuário, permitindo que ele assuma apenas o perfil `marketingadminrole`. Para obter mais informações

sobre como conceder a um usuário acesso para assumir uma função, consulte [Como conceder a um usuário permissão para alternar funções no IAM](#) no Manual do usuário do IAM.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "sts:AssumeRole",
      "Resource": "arn:aws:iam::123456789012:role/marketingadminrole"
    }
  ]
}
```

O usuário não precisa ter permissões adicionais para executar os comandos da AWS CLI usando o perfil. Em vez disso, as permissões para executar o comando vêm das associadas à função. Você pode associar políticas à função para especificar quais ações podem ser executadas em quais recursos da AWS. Para obter mais informações sobre como associar permissões a um perfil (que funciona de maneira idêntica a um usuário), consulte [Alteração de permissões de um usuário do IAM](#) no Manual do usuário do IAM.

Agora que o perfil de função, as permissões de função, a relação de confiança de função e as permissões de usuário estão configurados corretamente, é possível usar a função na linha de comando invocando a opção `--profile`. Por exemplo, o indicado a seguir chama o comando `ls` do Amazon S3 usando as permissões anexadas à função `marketingadmin`, conforme definido no exemplo no início desse tópico.

```
$ aws s3 ls --profile marketingadmin
```

Para usar a função para várias chamadas, defina a variável de ambiente `AWS_PROFILE` para a sessão atual na linha de comando. Enquanto essa variável de ambiente é definida, você não precisa especificar a opção `--profile` em cada comando.

Linux ou macOS

```
$ export AWS_PROFILE=marketingadmin
```

Windows

```
C:\> setx AWS_PROFILE marketingadmin
```

Para obter mais informações sobre como configurar usuários e perfis, consulte [Usuários e grupos](#) e [Perfis](#) no Manual do usuário do IAM.

## Usar autenticação multifator

Para segurança adicional, você pode exigir que os usuários forneçam uma chave única gerada a partir de um dispositivo de autenticação multifator (MFA), um dispositivo U2F ou um aplicativo móvel quando eles tentarem fazer uma chamada usando o perfil de função.

Primeiro, você pode optar por modificar a relação de confiança na função do IAM para exigir MFA. Isso impede que alguém usando a função sem antes autenticar usando MFA. Por exemplo, veja a linha `Condition` no exemplo a seguir. Essa política permite que o usuário chamado `anika` assuma o perfil ao qual a política está anexada, mas somente se ele se autenticar usando MFA.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "",
      "Effect": "Allow",
      "Principal": { "AWS": "arn:aws:iam::123456789012:user/anika" },
      "Action": "sts:AssumeRole",
      "Condition": { "Bool": { "aws:multifactorAuthPresent": true } }
    }
  ]
}
```

Depois, adicione uma linha à função que especifica o perfil do usuário ARN do dispositivo MFA. As entradas do exemplo de arquivo `config` a seguir mostram dois perfis que usam as chaves de acesso para o usuário `anika` solicitar credenciais temporárias para o perfil `cli-role`. O usuário `anika` tem permissão para assumir a função, concedida pela política de confiança da função.

```
[profile role-without-mfa]
region = us-west-2
role_arn= arn:aws:iam::128716708097:role/cli-role
source_profile=cli-user
```

```
[profile role-with-mfa]
region = us-west-2
role_arn= arn:aws:iam::128716708097:role/cli-role
source_profile = cli-user
mfa_serial = arn:aws:iam::128716708097:mfa/cli-user

[profile cli-user]
region = us-west-2
output = json
```

A configuração `mfa_serial` pode usar um ARN, conforme mostrado, ou o número de série de um token MFA de hardware.

O primeiro perfil, `role-without-mfa`, não exige MFA. No entanto, como a política de confiança do exemplo anterior anexada à função requer MFA, qualquer tentativa de executar um comando com esse perfil vai falhar.

```
$ aws iam list-users --profile role-without-mfa
```

```
An error occurred (AccessDenied) when calling the AssumeRole operation: Access denied
```

A segunda entrada do perfil, `role-with-mfa`, identifica um dispositivo MFA para usar. Quando o usuário tenta executar um comando da AWS CLI com esse perfil, a AWS CLI solicita que o usuário insira a senha de uso único (OTP) fornecida pelo dispositivo MFA. Se a autenticação de MFA for bem-sucedida, o comando executará a operação solicitada. A OTP não é exibida na tela.

```
$ aws iam list-users --profile role-with-mfa
Enter MFA code for arn:aws:iam::123456789012:mfa/cli-user:
{
  "Users": [
    {
      ...
```

## Funções entre contas e ID externo

É possível permitir que os usuários usem funções que pertençam a diferentes contas ao configurar a função como uma função entre contas. Durante a criação da função, defina o tipo de função como Another AWS account (Outra conta da AWS), conforme descrito em [Criar uma função para delegar permissões a um usuário do IAM](#). Como opção, selecione Require MFA (Exigir MFA).

Require MFA (Exigir MFA) configura a condição apropriada na relação de confiança, conforme descrito em [Usar autenticação multifator](#).

Se usar um [ID externo](#) para fornecer mais controle sobre quem pode usar uma função em todas as contas, você também deverá adicionar o parâmetro `external_id` ao perfil da função. Normalmente, isso é usado somente quando a outra conta é controlada por alguém de fora da sua empresa ou organização.

```
[profile crossaccountrole]
role_arn = arn:aws:iam::234567890123:role/SomeRole
source_profile = default
mfa_serial = arn:aws:iam::123456789012:mfa/saanvi
external_id = 123456
```

## Especificar um nome de sessão de função para facilitar a auditoria

Quando muitos indivíduos compartilham uma função, a auditoria torna-se um desafio. É recomendável associar cada operação invocada ao indivíduo que invocou a ação. No entanto, quando o indivíduo usa uma função, a assunção da função por ele é uma ação separada da invocação de uma operação, e é necessário correlacionar manualmente as duas.

É possível simplificar isso especificando nomes de sessão de função exclusivos quando os usuários assumem uma função. Para fazer isso, adicione um parâmetro `role_session_name` a cada perfil nomeado no arquivo `config` que especifica uma função. O valor `role_session_name` é transmitido para a operação `AssumeRole` e se torna parte do ARN da sessão da função. Ele também é incluído nos logs do AWS CloudTrail de todas as operações registradas.

Por exemplo, é possível criar um perfil baseado em função da maneira a seguir.

```
[profile namedsessionrole]
role_arn = arn:aws:iam::234567890123:role/SomeRole
source_profile = default
role_session_name = Session_Maria_Garcia
```

Isso faz com que a sessão da função tenha o ARN a seguir.

```
arn:aws:iam::234567890123:assumed-role/SomeRole/Session_Maria_Garcia
```

Além disso, todos os logs do AWS CloudTrail incluem o nome da sessão da função nas informações capturadas para cada operação.



## Assumir a função com a identidade da web

É possível configurar um perfil para indicar que a AWS CLI deve assumir uma função usando a [federação de identidades da web e o Open ID Connect \(OIDC\)](#). Quando isso é especificado em um perfil, a AWS CLI faz automaticamente a chamada AWS STS AssumeRoleWithWebIdentity correspondente para você.

### Note

Quando você especifica um perfil que usa uma função do IAM, a AWS CLI faz as chamadas apropriadas para recuperar credenciais temporárias. Essas credenciais são armazenadas em `~/.aws/cli/cache`. Os comandos subsequentes da AWS CLI que especificam o mesmo perfil usam as credenciais temporárias armazenadas em cache até que elas expirem. Nesse ponto, a AWS CLI atualiza automaticamente as credenciais.

Para recuperar e usar credenciais temporárias usando a federação de identidades da web, é possível especificar os valores de configuração a seguir em um perfil compartilhado.

### `role_arn`

Especifica o ARN da função a assumir.

### `web_identity_token_file`

Especifica o caminho para um arquivo que contém um token de acesso OAuth 2.0 ou token de ID OpenID Connect fornecido pelo provedor de identidade. A AWS CLI carrega esse arquivo e transmite seu conteúdo como o argumento `WebIdentityToken` da operação `AssumeRoleWithWebIdentity`.

### `role_session_name`

Especifica um nome opcional aplicado a essa sessão `assume-role`.

Veja a seguir um exemplo da configuração mínima necessária para uma função de admissão com o perfil de identidade da web:

```
# In ~/.aws/config

[profile web-identity]
role_arn=arn:aws:iam:123456789012:role/RoLeNameToAssume
```

```
web_identity_token_file=/path/to/a/token
```

Também é possível fornecer essa configuração usando [variáveis de ambiente](#).

#### AWS\_ROLE\_ARN

O ARN da função a ser assumida

#### AWS\_WEB\_IDENTITY\_TOKEN\_FILE

O caminho para o arquivo de token de identidade da web.

#### AWS\_ROLE\_SESSION\_NAME

O nome aplicado a essa sessão assume-role.

#### Note

No momento, essas variáveis de ambiente se aplicam somente à função de admissão com o provedor de identidade da web. Elas não se aplicam à configuração geral do provedor de função.

## Limpar as credenciais em cache

Quando você usa uma função, a AWS CLI armazena em cache as credenciais temporárias localmente até que elas expirem. Na próxima vez que você tentar usá-las, a AWS CLI tentará renová-las em seu nome.

Se as credenciais temporárias da função forem [revogadas](#), elas não serão renovadas automaticamente, e as tentativas de usá-las falharão. No entanto, é possível excluir o cache para forçar a AWS CLI a recuperar novas credenciais.

### Linux ou macOS

```
$ rm -r ~/.aws/cli/cache
```

### Windows

```
C:\> del /s /q %UserProfile%\aws\cli\cache
```

# Autenticar com credenciais de usuário do IAM

## Warning

Para evitar riscos de segurança, não use usuários do IAM para autenticação ao desenvolver software com propósito específico ou trabalhar com dados reais. Em vez disso, use federação com um provedor de identidade, como [AWS IAM Identity Center](#).

Essa seção explica como definir as configurações básicas com um usuário do IAM. Isso inclui suas credenciais de segurança usando os arquivos `config` e `credentials`.

## Tópicos

- [Etapa 1: criar o usuário do IAM](#)
- [Etapa 2: obter as chaves de acesso](#)
- [Configurar o AWS CLI](#)
  - [Usar o `aws configure`](#)

## Etapa 1: criar o usuário do IAM

Crie o usuário do IAM seguindo o procedimento de [Criação de usuários do IAM \(console\)](#) no Guia do usuário do IAM.

- Em Opções de permissão, escolha Anexar políticas diretamente para como você deseja atribuir permissões a esse usuário.
- A maioria dos tutoriais de “Conceitos básicos” do SDK usa o serviço Amazon S3 como exemplo. Para fornecer à aplicação acesso total ao Amazon S3, selecione a política `AmazonS3FullAccess` para anexar a esse usuário.

## Etapa 2: obter as chaves de acesso

1. Faça login no AWS Management Console e abra o console do IAM em <https://console.aws.amazon.com/iam/>.
2. No painel de navegação do console do IAM, selecione Usuários e escolha o usuário **User name** que você criou anteriormente.

3. Na página do usuário, selecione a página Credenciais de segurança. Depois, em Chaves de acesso, selecione Criar chave de acesso.
4. Em Criar chave de acesso: etapa 1, escolha Command Line Interface (CLI).
5. Em Criar chave de acesso: etapa 2, insira uma tag opcional e selecione Próximo.
6. Em Criar chave de acesso: etapa 3, selecione Baixar arquivo .csv para salvar um arquivo .csv com a chave de acesso e a chave de acesso secreta do usuário do IAM. Você precisará dessas informações posteriormente.
7. Selecione Concluído.

## Configurar o AWS CLI

Para uso geral, a AWS CLI precisa das seguintes informações:

- Access key ID (ID da chave de acesso)
- Secret access key (Chave de acesso secreta)
- Região da AWS
- Formato de saída

A AWS CLI armazena essas informações em um perfil (um conjunto de configurações) chamado `default` no arquivo `credentials`. Por padrão, as informações neste perfil são usadas quando você executa um comando da AWS CLI que não especifica explicitamente um perfil a ser usado. Para obter mais informações sobre o arquivo `credentials`, consulte [Configurações do arquivo de configuração e credenciais](#).

Para configurar a AWS CLI, use um dos seguintes procedimentos:

### Tópicos

- [Usar o `aws configure`](#)

## Usar o `aws configure`

Para uso geral, o comando `aws configure` é a maneira mais rápida de configurar sua instalação da AWS CLI. Esse assistente de configuração solicita cada informação necessária para começar. A menos que especificado de outra forma usando a opção `--profile`, a AWS CLI armazena essas informações no perfil `default`.

O exemplo a seguir configura um perfil default usando valores de amostra. Substitua-os por seus próprios valores, conforme descrito nas seções a seguir.

```
$ aws configure
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY
Default region name [None]: us-west-2
Default output format [None]: json
```

O exemplo a seguir configura um perfil chamado userprod usando valores de amostra. Substitua-os por seus próprios valores, conforme descrito nas seções a seguir.

```
$ aws configure --profile userprod
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY
Default region name [None]: us-west-2
Default output format [None]: json
```

## Uso de credenciais para metadados de instância do Amazon EC2.

Quando você executa a AWS CLI em uma instância do Amazon Elastic Compute Cloud (Amazon EC2), é possível simplificar o fornecimento de credenciais aos seus comandos. Cada instância do Amazon EC2 contém metadados que a AWS CLI pode consultar diretamente por credenciais temporárias. Quando uma função do IAM é associada à instância, a AWS CLI recupera de forma automática e segura as credenciais dos metadados da instância.

Para desabilitar esse serviço, use a variável de ambiente [AWS\\_EC2\\_METADATA\\_DISABLED](#).

### Tópicos

- [Pré-requisitos](#)
- [Configuração de um perfil para metadados do Amazon EC2](#)

## Pré-requisitos

Para usar credenciais do Amazon EC2 com a AWS CLI, é necessário concluir o seguinte:

- Instale e configure a AWS CLI. Para obter mais informações, consulte [Instalar a AWS CLI](#) e [Autenticação e credenciais de acesso](#).

- Conhecer os arquivos de configuração e os perfis nomeados. Para obter mais informações, consulte [Configurações do arquivo de configuração e credenciais](#).
- Você criou uma função do AWS Identity and Access Management (IAM) que tem acesso aos recursos necessários e associou essa função à instância do Amazon EC2 ao iniciá-la. Para obter mais informações, consulte [Políticas do IAM para Amazon EC2](#) no Manual do usuário do Amazon EC2 para instâncias do Linux e [Concessão de acesso a recursos da AWS para aplicações executadas em instâncias do Amazon EC2](#) no Manual do usuário do IAM.

## Configuração de um perfil para metadados do Amazon EC2

Para especificar que você deseja usar as credenciais disponíveis no perfil de instância de hospedagem do Amazon EC2, use a sintaxe a seguir no perfil nomeado em seu arquivo de configuração. Veja as etapas a seguir para obter mais instruções.

```
[profile profilename]  
role_arn = arn:aws:iam::123456789012:role/rolename  
credential_source = Ec2InstanceMetadata  
region = region
```

1. Crie um perfil em seu arquivo de configuração.

```
[profile profilename]
```

2. Adicione a função arn do IAM que tem acesso aos recursos necessários.

```
role_arn = arn:aws:iam::123456789012:role/rolename
```

3. Especifique Ec2InstanceMetadata como sua fonte de credencial.

```
credential_source = Ec2InstanceMetadata
```

4. Defina sua região.

```
region = region
```

### Exemplo

O exemplo a seguir assume o perfil *marketingadminrole* e usa a região *us-west-2* em um perfil de instância do Amazon EC2 chamado *marketingadmin*.

```
[profile marketingadmin]
role_arn = arn:aws:iam::123456789012:role/marketingadminrole
credential_source = Ec2InstanceMetadata
region = us-west-2
```

## Credenciais de origem com um processo externo

### Warning

Este tópico discute as credenciais de fornecimento de um processo externo. Isso poderá ser um risco de segurança se o comando para gerar as credenciais se tornar acessível por processos ou usuários não aprovados. Recomendamos que você use as alternativas seguras compatíveis fornecidas pela AWS CLI e pela AWS para reduzir o risco de comprometer suas credenciais. Certifique-se de proteger o arquivo `config` e todos os arquivos e ferramentas com suporte para impedir a divulgação.

Certifique-se de que sua ferramenta de credenciais personalizada não grave informações secretas em `StdErr` porque os SDKs e a AWS CLI podem capturar e registrar essas informações, possivelmente expondo-as a usuários não autorizados.

Se você tiver um método para gerar ou procurar credenciais não compatíveis diretamente com a AWS CLI, poderá configurar a AWS CLI para usá-lo definindo a configuração `credential_process` no arquivo `config`.

Por exemplo, você pode incluir uma entrada semelhante à seguinte no arquivo `config`.

```
[profile developer]
credential_process = /opt/bin/awscreds-custom --username helen
```

### Sintaxe

Para criar essa string de uma forma compatível com qualquer sistema operacional, siga estas regras:

- Se o caminho ou o nome do arquivo contiver um espaço, coloque o caminho completo e o nome do arquivo entre aspas duplas (" "). O caminho e o nome do arquivo podem ter somente os caracteres: A–Z a–z 0–9 - \_ . espaço

- Se um nome de parâmetro ou um valor de parâmetro tiver um espaço, coloque esse elemento entre aspas duplas (" "). Coloque somente o nome ou o valor entre aspas, não o par.
- Não inclua variáveis de ambiente nas strings. Por exemplo, não inclua \$HOME ou %USERPROFILE %.
- Não especifique a pasta base como ~. É necessário especificar o caminho completo.

### Exemplo para Windows

```
credential_process = "C:\Path\To\credentials.cmd" parameterWithoutSpaces "parameter with spaces"
```

### Exemplo para Linux ou macOS

```
credential_process = "/Users/Dave/path/to/credentials.sh" parameterWithoutSpaces "parameter with spaces"
```

### Saída esperada do programa de credenciais

A AWS CLI executa o comando conforme especificado no perfil e lê os dados de STDOUT. O comando especificado deverá gerar a saída JSON em STDOUT que corresponde à sintaxe a seguir.

```
{
  "Version": 1,
  "AccessKeyId": "an AWS access key",
  "SecretAccessKey": "your AWS secret access key",
  "SessionToken": "the AWS session token for temporary credentials",
  "Expiration": "ISO8601 timestamp when the credentials expire"
}
```

#### Note

No momento da elaboração deste documento, a chave `Version` deve ser definida como 1. Isso pode aumentar ao longo do tempo conforme a estrutura evolui.

A chave `Expiration` é um timestamp no formato [ISO8601](#). Se a chave `Expiration` não estiver presente na saída da ferramenta, a CLI vai supor que as credenciais são em longo prazo que não



são atualizadas. Caso contrário, as credenciais serão consideradas temporárias e serão atualizadas automaticamente com a nova execução do comando `credential_process` antes de expirarem.

 Note

A AWS CLI não armazena em cache as credenciais do processo como faz com credenciais `assume-role`. Se o armazenamento em cache for obrigatório, implemente-o no processo externo.

O processo externo pode retornar um código de retorno diferente de zero para indicar que ocorreu um erro ao recuperar as credenciais.

# Usar a AWS CLI

Esta seção fornece informações sobre uso geral, recursos comuns e opções disponíveis na AWS Command Line Interface (AWS CLI), além do que está escrito na seção Configuração [the section called “Endpoints”](#). As instruções incluem como escrever um comando, estrutura básica, formatação, filtragem e localização do conteúdo de ajuda ou da documentação de um comando.

Para obter exemplos específicos do AWS service (Serviço da AWS), consulte [Exemplos de código](#) ou [o guia de referência da AWS CLI](#).

## Note

Por padrão, a AWS CLI envia solicitações para serviços do AWS usando HTTPS na porta TCP 443. Para usar a AWS CLI com êxito, você deve ser capaz de fazer as conexões de saída na porta TCP 443.

Tópicos neste guia

- [Obter ajuda com a AWS CLI](#)
- [Estrutura do comando na AWS CLI](#)
- [Especificar valores de parâmetro para a AWS CLI](#)
- [Controlar a saída do comando da AWS CLI](#)
- [Códigos de retorno da AWS CLI](#)
- [Criar e usar aliases da AWS CLI](#)

## Obter ajuda com a AWS CLI

Este tópico descreve como acessar o conteúdo da ajuda da AWS Command Line Interface (AWS CLI).

Tópicos

- [O comando de ajuda integrado da AWS CLI](#)
- [Guia de referência da AWS CLI](#)
- [Documentação de API](#)
- [Solucionar de problemas de erros](#)

- [Ajuda adicional](#)

## O comando de ajuda integrado da AWS CLI

Você pode obter ajuda com qualquer comando ao usar a AWS Command Line Interface (AWS CLI). Para fazer isso, basta digitar `help` no final de um nome do comando.

Por exemplo, o comando a seguir exibe a ajuda para as opções gerais da AWS CLI e os comandos de nível superior disponíveis.

```
$ aws help
```

O comando a seguir exibe os comandos específicos do Amazon Elastic Compute Cloud (Amazon EC2) disponíveis.

```
$ aws ec2 help
```

O exemplo a seguir exibe ajuda detalhada para a operação `DescribeInstances` do Amazon EC2. A ajuda inclui descrições de seus parâmetros de entrada, filtros disponíveis, e o que é incluído como saída. Ela também inclui exemplos que mostram como digitar as variações comuns do comando.

```
$ aws ec2 describe-instances help
```

A ajuda para cada comando é dividida em seis seções:

### Nome

O nome do comando.

```
NAME
    describe-instances -
```

### Descrição

Uma descrição da operação da API que o comando invoca.

```
DESCRIPTION
    Describes one or more of your instances.

    If you specify one or more instance IDs, Amazon EC2 returns information
```

```
for those instances. If you do not specify instance IDs, Amazon EC2
returns information for all relevant instances. If you specify an
instance ID that is not valid, an error is returned. If you specify an
instance that you do not own, it is not included in the returned
results.
```

```
...
```

## Resumo

A sintaxe básica para usar o comando e suas opções. Se uma opção estiver entre colchetes, significa que ela é opcional, tem um valor padrão ou tem uma opção alternativa que você pode usar.

### SYNOPSIS

```
describe-instances
[--dry-run | --no-dry-run]
[--instance-ids <value>]
[--filters <value>]
[--cli-input-json <value>]
[--starting-token <value>]
[--page-size <value>]
[--max-items <value>]
[--generate-cli-skeleton]
```

Por exemplo, `describe-instances` tem um comportamento padrão que descreve todas as instâncias na conta e na região atuais da AWS. Se preferir, você poderá especificar uma lista de `instance-ids` para descrever uma ou mais instâncias. O `dry-run` é um sinalizador booleano opcional que não tem um valor. Para usar um sinalizador booleano, especifique um valor apresentado, nesse caso `--dry-run` ou `--no-dry-run`. Da mesma forma, `--generate-cli-skeleton` não tem um valor. Se houver condições no uso de uma opção, elas serão descritas na seção `OPTIONS` ou mostradas nos exemplos.

## Opções

A descrição de cada uma das opções mostradas na sinopse.

### OPTIONS

```
--dry-run | --no-dry-run (boolean)
Checks whether you have the required permissions for the action,
without actually making the request, and provides an error response.
If you have the required permissions, the error response is DryRun-
Operation . Otherwise, it is UnauthorizedOperation .
```

```
--instance-ids (list)
    One or more instance IDs.

    Default: Describes all your instances.
```

...

## Exemplos

Os exemplos que mostram o uso do comando e suas opções. Se nenhum exemplo estiver disponível para um comando ou caso de uso que você precisa, solicite um usando o link de comentários nesta página ou na referência de comandos da AWS CLI na página de ajuda para o comando.

### EXAMPLES

#### **To describe an Amazon EC2 instance**

Command:

```
aws ec2 describe-instances --instance-ids i-5203422c
```

#### **To describe all instances with the instance type m1.small**

Command:

```
aws ec2 describe-instances --filters "Name=instance-type,Values=m1.small"
```

#### **To describe all instances with an Owner tag**

Command:

```
aws ec2 describe-instances --filters "Name=tag-key,Values=Owner"
```

...

## Saída

As descrições de cada um dos campos e os tipos de dados incluídos na resposta da AWS.

Para `describe-instances`, a saída é uma lista de objetos de reserva, cada um com vários campos e objetos que contêm informações sobre as instâncias associadas a ele. Essas informações são provenientes da [documentação da API para o tipo de dados de reserva](#) usada pelo Amazon EC2.

**OUTPUT**

Reservations -> (list)

One or more reservations.

(structure)

Describes a reservation.

ReservationId -> (string)

The ID of the reservation.

OwnerId -> (string)

The ID of the AWS account that owns the reservation.

RequesterId -> (string)

The ID of the requester that launched the instances on your behalf (for example, AWS Management Console or Auto Scaling).

Groups -> (list)

One or more security groups.

(structure)

Describes a security group.

GroupName -> (string)

The name of the security group.

GroupId -> (string)

The ID of the security group.

Instances -> (list)

One or more instances.

(structure)

Describes an instance.

InstanceId -> (string)

The ID of the instance.

ImageId -> (string)

The ID of the AMI used to launch the instance.

State -> (structure)

The current state of the instance.

```
Code -> (integer)
    The low byte represents the state. The high byte
    is an opaque internal value and should be ignored.
```

```
...
```

Quando a AWS CLI renderiza a saída em JSON, ela se torna uma matriz de objetos de reserva, semelhante ao exemplo a seguir.

```
{
  "Reservations": [
    {
      "OwnerId": "012345678901",
      "ReservationId": "r-4c58f8a0",
      "Groups": [],
      "RequesterId": "012345678901",
      "Instances": [
        {
          "Monitoring": {
            "State": "disabled"
          },
          "PublicDnsName": "ec2-52-74-16-12.us-
west-2.compute.amazonaws.com",
          "State": {
            "Code": 16,
            "Name": "running"
          },
          ...
        }
      ]
    }
  ]
}
```

Cada objeto de reserva contém campos que descrevem a reserva e uma série de objetos de instância, cada um com seus próprios campos (por exemplo, `PublicDnsName`) e objetos (por exemplo, `State`) que os descrevem.

### Usuários do Windows

Você pode usar pipe (`|`) na saída do comando de ajuda do comando `more` para visualizar o arquivo de ajuda uma página por vez. Pressione a barra de espaço ou `PgDn` para visualizar mais detalhes do documento ou `q` se quiser sair.

```
C:\> aws ec2 describe-instances help | more
```

# Guia de referência da AWS CLI

Os arquivos de ajuda contêm links que não podem ser visualizados a partir da linha de comando nem é possível navegar até eles a partir dela. Você pode visualizar e interagir com esses links usando o [Guia de referência da AWS CLI versão 1](#) e o online. A referência também inclui o conteúdo da ajuda para todos os comandos da AWS CLI. As descrições são apresentadas para facilitar a navegação e a visualização em dispositivos móveis, tablets e telas de desktop.

## Documentação de API

Todos os comandos na AWS CLI correspondem a solicitações feitas para uma API pública de serviço do AWS. Cada serviço com uma API pública tem uma referência da API que pode ser encontrada na página inicial do serviço no [site de documentação da AWS](#). O conteúdo de uma referência da API varia de acordo com a forma como a API é criada e qual protocolo é usado. Normalmente, uma referência da API contém informações detalhadas sobre as operações compatíveis com a API, os dados enviados para e do serviço e qualquer condição de erro que o serviço pode relatar.

### Seções da documentação de API

- **Ações:** informações detalhadas sobre cada operação e seus parâmetros (incluindo restrições de tamanho ou conteúdo e valores padrão). Ela lista os erros que podem ocorrer nessa operação. Cada operação corresponde a um subcomando na AWS CLI.
- **Tipos de dados:** informações detalhadas sobre estruturas que um comando pode exigir como um parâmetro ou retornar em resposta a uma solicitação.
- **Parâmetros comuns:** informações detalhadas sobre os parâmetros que são compartilhados por toda a ação para o serviço.
- **Erros comuns:** informações detalhadas sobre erros que podem ser retornados por todas as operações de um serviço.

O nome e a disponibilidade de cada seção podem variar de acordo com o serviço.

#### CLIs específicas para o serviço

Alguns serviços têm uma CLI separado desde antes da criação de uma única AWS CLI que funciona com todos os serviços. Essas CLIs específicas do serviço têm documentação



separada vinculada a partir da página de documentação do serviço. A documentação para CLIs específicas de serviço não se aplica à AWS CLI.

## Solucionar de problemas de erros

Para obter ajuda para diagnosticar e corrigir erros da AWS CLI, consulte [Solucionar erros](#).

## Ajuda adicional

Para obter ajuda adicional para problemas da AWS CLI, visite a [Comunidade da AWS CLI](#) no GitHub.

## Estrutura do comando na AWS CLI

Este tópico aborda como o comando da AWS Command Line Interface (AWS CLI) é estruturado e como usar comandos de espera.

### Tópicos

- [Estrutura do comando](#)
- [Comandos de espera](#)

## Estrutura do comando

A AWS CLI usa uma estrutura em várias partes na linha de comando que deve ser especificada nesta ordem:

1. A chamada básica para o programa aws.
2. O comando de nível superior que normalmente corresponde a um serviço do AWS compatível com a AWS CLI.
3. O subcomando que especifica a operação a ser realizada.
4. As opções gerais da AWS CLI ou os parâmetros necessários para a operação. Você pode especificá-los em qualquer ordem, desde que siga as três primeiras partes. Se um parâmetro exclusivo for especificado várias vezes, apenas o último valor se aplicará.

```
$ aws <command> <subcommand> [options and parameters]
```

Parâmetros pode levar vários tipos de valores de entrada, como números, sequências de caracteres, listas, mapas e estruturas de JSON. O que é compatível depende do comando e do subcomando que você especificar.

## Exemplos

### Amazon S3

O exemplo a seguir lista todos os seus buckets do Amazon S3.

```
$ aws s3 ls
2018-12-11 17:08:50 my-bucket
2018-12-14 14:55:44 my-bucket2
```

Para obter mais informações sobre os comandos do Amazon S3, consulte [aws s3](#) na Referência de comandos da AWS CLI.

### AWS CloudFormation

O exemplo de comando [create-change-set](#) a seguir altera o nome da pilha cloudformation para *my-change-set*.

```
$ aws cloudformation create-change-set --stack-name my-stack --change-set-name my-change-set
```

Para obter mais informações sobre os comandos AWS CloudFormation, consulte [aws cloudformation](#) na Referência de comandos da AWS CLI.

## Comandos de espera

Alguns serviços da AWS contam com comandos `wait`. Qualquer comando que usa `aws wait` normalmente espera até que um comando seja concluído antes de passar para a próxima etapa. Isso é especialmente útil para comandos em várias partes ou scripts, pois você pode usar um comando `wait` para impedir o avanço para etapas subsequentes se o comando `wait` falhar.

A AWS CLI usa uma estrutura em várias partes na linha de comando para o comando `wait` que deve ser especificada nesta ordem:

1. A chamada básica para o programa `aws`.
2. O comando de nível superior que normalmente corresponde a um serviço do AWS compatível com a AWS CLI.

3. O comando `wait`.
4. O subcomando que especifica a operação a ser realizada.
5. As opções gerais da CLI ou os parâmetros necessários para a operação. Você pode especificá-los em qualquer ordem, desde que siga as três primeiras partes. Se um parâmetro exclusivo for especificado várias vezes, apenas o último valor se aplicará.

```
$ aws <command> wait <subcommand> [options and parameters]
```

Parâmetros pode levar vários tipos de valores de entrada, como números, sequências de caracteres, listas, mapas e estruturas de JSON. O que é compatível depende do comando e do subcomando que você especificar.

#### Note

Nem todos os serviços da AWS comportam comandos `wait`. Consulte o [AWS CLIGuia de referência da](#) para ver se seu serviço comporta comandos `wait`.

## Exemplos

### AWS CloudFormation

Os exemplos de comando [wait change-set-create-complete](#) a seguir pausam e continuam somente depois que o comando pode confirmar que o conjunto de alterações `my-change-set` na pilha `my-stack` está pronto para ser executado.

```
$ aws cloudformation wait change-set-create-complete --stack-name my-stack --change-set-name my-change-set
```

Para obter mais informações sobre os comandos da AWS CloudFormation `wait`, consulte [wait](#) na Referência de comandos da AWS CLI.

### AWS CodeDeploy

Os exemplos de comando [wait deployment-successful](#) pausam até a implantação de `d-A1B2C3111` ser concluída com êxito.

```
$ aws deploy wait deployment-successful --deployment-id d-A1B2C3111
```

Para obter mais informações sobre os comandos da AWS CodeDeploy `wait`, consulte [wait](#) na Referência de comandos da AWS CLI.

## Especificar valores de parâmetro para a AWS CLI

Muitos parâmetros usados na AWS Command Line Interface (AWS CLI) são strings ou valores numéricos simples, como o nome do par de chaves `my-key-pair` no exemplo a seguir.

```
$ aws ec2 create-key-pair --key-name my-key-pair
```

A formatação entre os terminais pode variar. Por exemplo, a maioria dos terminais faz distinção entre letras maiúsculas e minúsculas, mas não é o caso do Powershell. Isso significa que os dois exemplos de comando a seguir produziram resultados diferentes para terminais com distinção entre letras maiúsculas e minúsculas, pois eles veem `MyFile*.txt` e `myfile*.txt` como parâmetros diferentes.

No entanto, o PowerShell processaria essas solicitações da mesma forma, já que vê `MyFile*.txt` e `myfile*.txt` como os mesmos parâmetros.

```
$ aws s3 cp . s3://my-bucket/path --include "MyFile*.txt"
$ aws s3 cp . s3://my-bucket/path --include "myfile*.txt"
```

Para obter mais informações sobre a indistinção de letras maiúsculas e minúsculas do PowerShell, consulte [about\\_Case-Sensitivity](#) na documentação do PowerShell.

Às vezes, é necessário usar aspas ou literais em strings que incluem caracteres especiais ou de espaço. As regras sobre essa formatação também podem variar entre os terminais. Para obter mais informações sobre como usar aspas em parâmetros complexos, consulte [Aspas com strings na AWS CLI](#).

### Tópicos de parâmetros

- [Tipos comuns de parâmetros da AWS CLI](#)
- [Aspas com strings na AWS CLI](#)
- [Carregar os parâmetros da AWS CLI de um arquivo](#)
- [Esqueletos e arquivos de entrada da AWS CLI](#)
- [Usar sintaxe simplificada com a AWS CLI](#)

## Tipos comuns de parâmetros da AWS CLI

Esta seção descreve alguns dos tipos de parâmetros comuns e o formato típico necessário.

Caso haja problemas com a formatação de um parâmetro para um comando específico, verifique a ajuda inserindo **help** após o nome do comando. A ajuda de cada subcomando inclui o nome e a descrição de uma opção. O tipo de parâmetro da opção está listado entre parênteses. Para obter mais informações sobre a exibição da ajuda, consulte [the section called “Obter ajuda”](#).

Os tipos de parâmetro incluem:

- [String](#)
- [a função de hora](#)
- [Listar](#)
- [Boolean](#)
- [Inteiro](#)
- [Binário/blob \(objeto grande binário\) e blob de streaming](#)
- [Mapa](#)
- [Documento](#)

### String

Os parâmetros de string podem conter caracteres alfanuméricos, símbolos e espaço em branco no conjunto de caracteres [ASCII](#). Strings com espaço em branco devem ser incluídas entre aspas. Recomendamos que você não use símbolos nem espaços em branco diferentes do caractere de espaço padrão e observe as [regras de aspas](#) para evitar resultados inesperados.

Alguns parâmetros de sequência de caracteres pode aceitar dados binários a partir de um arquivo. Consulte [Arquivos binários](#) para ver um exemplo.

### a função de hora

As marcas de tempo são formatadas de acordo com a norma [ISO 8601](#). São geralmente chamados de parâmetros “DateTime” ou “Date”.

```
$ aws ec2 describe-spot-price-history --start-time 2014-10-13T19:00:00Z
```

Os formatos aceitos incluem:

- *AAAA-MM-DDThh:mm:ss.sssTZD (UTC)*, por exemplo, 2014-10-01T20:30:00.000Z
- *AAAA-MM-DDThh:mm:ss.sssTZD (com deslocamento)*, por exemplo, 2014-10-01T12:30:00.000-08:00
- *AAAA-MM-DD*, por exemplo, 2014-10-01
- Tempo do Unix em segundos, por exemplo, 1412195400. Isso às vezes é conhecido como [Tempo epoch Unix](#) e representa o número de segundos desde a meia-noite, 1º de janeiro de 1970 UTC.

Você pode definir o formato do carimbo de data/hora usando a configuração do arquivo [cli\\_timestamp\\_format](#).

## Listar

Uma ou mais strings separadas por espaços. Se qualquer um dos itens de string contiver espaços, você deverá colocar esse item entre aspas. Observe as [regras de aspas](#) de seu terminal para evitar resultados inesperados

```
$ aws ec2 describe-spot-price-history --instance-types m1.xlarge m1.medium
```

## Boolean

Sinalizador binário que ativa ou desativa uma opção. Por exemplo, `ec2 describe-spot-price-history` tem um parâmetro booleano `--dry-run` que, quando especificado, valida a consulta com o serviço sem realmente executá-la.

```
$ aws ec2 describe-spot-price-history --dry-run
```

A saída indica se o comando foi bem formado. Esse comando também inclui uma versão `--no-dry-run` do parâmetro que você pode usar para indicar explicitamente que o comando deve ser executado normalmente. A inclusão não é necessária, pois esse é o comportamento padrão.

## Inteiro

Um número inteiro não assinado.

```
$ aws ec2 describe-spot-price-history --max-items 5
```

## Binário/blob (objeto grande binário) e blob de streaming

Na AWS CLI, é possível transmitir um valor binário como uma string diretamente na linha de comando. Há dois tipos de blobs:

- [Blob](#)
- [Blob de streaming](#)

### Blob

Para transmitir um valor para um parâmetro com o tipo blob, é necessário especificar um caminho para um arquivo local que contenha os dados binários usando o prefixo `fileb://`. Arquivos referenciados usando o prefixo `fileb://` são sempre tratados como um binário bruto não codificado. O caminho especificado é interpretado como sendo relativo ao diretório de trabalho atual. Por exemplo, o parâmetro `--plaintext` para `aws kms encrypt` é um blob.

```
$ aws kms encrypt \
  --key-id 1234abcd-12ab-34cd-56ef-1234567890ab \
  --plaintext fileb://ExamplePlaintextFile \
  --output text \
  --query CiphertextBlob | base64 \
  --decode > ExampleEncryptedFile
```

### Blob de streaming

Blobs de streaming como `aws cloudsearchdomain upload-documents` não usam prefixos. Em vez disso, os parâmetros de blob de streaming são formatados usando o caminho direto do arquivo. O seguinte exemplo usa o caminho direto do arquivo `document-batch.json` para o comando `aws cloudsearchdomain upload-documents`:

```
$ aws cloudsearchdomain upload-documents \
  --endpoint-url https://doc-my-domain.us-west-1.cloudsearch.amazonaws.com \
  --content-type application/json \
  --documents document-batch.json
```

## Mapa

Um conjunto de pares de chave-valor especificado em JSON ou com a [sintaxe abreviada](#) da CLI. O exemplo de JSON a seguir lê um item de uma tabela do Amazon DynamoDB chamada `my-table` com

um parâmetro de mapa, `--key`. O parâmetro especifica a chave primária chamada `id` com um valor numérico de 1 em uma estrutura JSON aninhada.

Para uso mais avançado de JSON em uma linha de comando, considere usar um processador JSON de linha de comando, como `jq`, para criar strings JSON. Para obter mais informações sobre o `jq`, consulte o [repositório do jq](#) no GitHub.

```
$ aws dynamodb get-item --table-name my-table --key '{"id": {"N": "1"}}'

{
  "Item": {
    "name": {
      "S": "John"
    },
    "id": {
      "N": "1"
    }
  }
}
```

## Documento

### Note

A [sintaxe simplificada](#) não é compatível com tipos de documento.

Os tipos de documento são usados para enviar dados sem a necessidade de incorporar JSON dentro de strings. O tipo de documento permite que os serviços forneçam esquemas arbitrários para você usar tipos de dados mais flexíveis.

Isso permite enviar dados JSON sem precisar usar caracteres escape nos valores. Por exemplo, em vez de usar a seguinte entrada JSON com escape:

```
{"document": "{\"key\": true}"}
```

Você pode usar o seguinte tipo de documento:

```
{"document": {"key": true}}
```



## Valores válidos para tipos de documento

Devido à natureza flexível dos tipos de documento, existem vários tipos de valor válidos. Entre os valores válidos estão os seguintes:

### String

```
--option "value"
```

### Número

```
--option 123  
--option 123.456
```

### Boolean

```
--option true
```

### Nulo

```
--option null
```

### Array

```
--option ["value1", "value2", "value3"]  
--option ["value", 1, true, null, ["key1", 2.34], {"key2": "value2"}]
```

### Objeto

```
--option {"key": "value"}  
--option {"key1": "value1", "key2": 123, "key3": true, "key4": null, "key5":  
["value3", "value4"], "key6": {"value5": "value6"}}
```

## Aspas com strings na AWS CLI

Há duas formas principais de usar aspas simples e duplas na AWS CLI.

- [Uso de aspas em torno de strings que contêm espaços em branco](#)
- [Uso de aspas dentro de strings](#)

## Uso de aspas em torno de strings que contêm espaços em branco

Os nomes dos parâmetros e valores são separados por espaços na linha de comando. Se um valor de string contiver um espaço incorporado, você deve fechar a string inteira com aspas para evitar que a AWS CLI interprete mal o espaço como um divisor entre o valor e o próximo nome de parâmetro. O tipo de aspas depende do sistema operacional em que você executa a AWS CLI.

### Linux and macOS

Uso de aspas simples ' '

```
$ aws ec2 create-key-pair --key-name 'my key pair'
```

Para obter mais informações sobre como usar aspas, consulte a documentação do usuário para o seu shell preferido.

### PowerShell

Aspas simples (recomendado)

As aspas simples ' ' são chamadas de strings verbatim. A string é passada ao comando exatamente como você a digita, o que significa que as variáveis do PowerShell não serão passadas.

```
PS C:\> aws ec2 create-key-pair --key-name 'my key pair'
```

Aspas duplas

As aspas duplas " " são chamadas de string expandable. As variáveis podem ser passadas em strings expansíveis.

```
PS C:\> aws ec2 create-key-pair --key-name "my key pair"
```

Para obter mais informações sobre como usar aspas, consulte [Sobre regras de aspas](#) na Documentação do Microsoft PowerShell.

### Windows command prompt

Uso de aspas duplas " ".

```
C:\> aws ec2 create-key-pair --key-name "my key pair"
```

Opcionalmente, você pode separar o nome de parâmetro do valor com um sinal de igual =, em vez de um espaço. Isso geralmente é necessário apenas se o valor do parâmetro começa com um hífen.

```
$ aws ec2 delete-key-pair --key-name=-mykey
```

## Uso de aspas dentro de strings

As strings podem conter aspas, e seu shell pode exigir aspas de escape para que funcionem corretamente. Um dos tipos de valor de parâmetro comuns é uma string JSON. Isso é complexo, pois inclui espaços e aspas duplas " " em torno de cada nome de elemento e valor na estrutura JSON. A maneira como insere os parâmetros formatados pelo JSON na linha de comando difere dependendo de seu sistema operacional.

Para uso mais avançado de JSON na linha de comando, considere usar um processador JSON de linha de comando, como `jq`, para criar strings JSON. Para obter mais informações sobre o `jq`, consulte o [repositório do jq](#) no GitHub.

## Linux and macOS

Para que o Linux e o macOS interpretem strings literalmente, use aspas simples ' ' para delimitar estrutura de dados JSON, como no exemplo a seguir. Você não precisa adicionar sequências de escape para as aspas duplas incorporadas na string JSON, pois elas são tratadas literalmente. Como o JSON é delimitado por aspas simples, quaisquer aspas simples na string precisarão utilizar sequências de escape. Isso geralmente é feito usando-se uma barra invertida antes da aspas simples \ '.

```
$ aws ec2 run-instances \
  --image-id ami-12345678 \
  --block-device-mappings '[{"DeviceName":"/dev/sdb","Ebs":
{"VolumeSize":20,"DeleteOnTermination":false,"VolumeType":"standard"}}]'
```

Para obter mais informações sobre como usar aspas, consulte a documentação do usuário para o seu shell preferido.

## PowerShell

Use aspas simples ' ' ou aspas duplas " ".

Aspas simples (recomendado)

As aspas simples ' ' são chamadas de strings verbatim. A string é passada ao comando exatamente como você a digita, o que significa que as variáveis do PowerShell não serão passadas.

Como as estruturas de dados JSON incluem aspas duplas, sugerimos utilizar aspas simples ' ' para delimitá-las. Se você usar aspas simples, não será necessário usar uma sequência de escape para as aspas duplas incorporadas na string JSON. No entanto, será necessário usar a sequência de escape para cada uma das aspas simples utilizando um apóstrofe ` dentro da estrutura JSON.

```
PS C:\> aws ec2 run-instances `
  --image-id ami-12345678 `
  --block-device-mappings '[{"DeviceName":"/dev/sdb","Ebs":
{"VolumeSize":20,"DeleteOnTermination":false,"VolumeType":"standard"}}]'
```

### Aspas duplas

As aspas duplas " " são chamadas de string expandable. As variáveis podem ser passadas em strings expansíveis.

Se você usar aspas duplas, não será necessário usar uma sequência de escape para as aspas simples incorporadas na string JSON. No entanto, será necessário usar a sequência de escape para cada uma das aspas duplas utilizando um apóstrofe ` dentro da estrutura JSON, conforme mostrado no exemplo a seguir.

```
PS C:\> aws ec2 run-instances `
  --image-id ami-12345678 `
  --block-device-mappings "[{\"DeviceName`\":`\"/dev/sdb`\",`\"Ebs`\":
{`\"VolumeSize`\":20,`\"DeleteOnTermination`\":false,`\"VolumeType`\":`\"standard`\"}}]"
```

Para obter mais informações sobre como usar aspas, consulte [Sobre regras de aspas](#) na Documentação do Microsoft PowerShell.

#### Warning

Antes que o PowerShell envie um comando para a AWS CLI, ele determina se seu comando é interpretado usando o PowerShell típico ou regras de aspas da CommandLineToArgvW. Quando o PowerShell usa CommandLineToArgvW no processamento, é necessário escapar dos caracteres com barra invertida \.

Para obter mais informações sobre CommandLineToArgvW no PowerShell, consulte [O que há com o estranho tratamento de aspas e barras invertidas por CommandLineToArgVW](#) no Blog de desenvolvedores da Microsoft, [Todos delimitam argumentos de linha de comando da maneira errada](#) no Blog de documentação da Microsoft e [Função CommandLineToArgVW](#) na Documentação da Microsoft.

#### Aspas simples

As aspas simples ' ' são chamadas de strings verbatim. A string é passada ao comando exatamente como você a digita, o que significa que as variáveis do PowerShell não serão passadas. Caracteres de escape com barra invertida \.

```
PS C:\> aws ec2 run-instances `
  --image-id ami-12345678 `
  --block-device-mappings '[{"DeviceName\":\"/dev/sdb\", \"Ebs\":
{\"VolumeSize\":20, \"DeleteOnTermination\":false, \"VolumeType\":\"standard\"}}]'
```

#### Aspas duplas

As aspas duplas " " são chamadas de string expandable. As variáveis podem ser passadas em strings expansíveis. Para strings entre aspas duplas, é necessário escapar duas vezes usando \" para cada aspa, em vez de usar um único backtick. O backtick escapa da barra invertida e, em seguida, a barra invertida é usada como um caractere de escape para o processo CommandLineToArgvW.

```
PS C:\> aws ec2 run-instances `
  --image-id ami-12345678 `
  --block-device-mappings "[{\"DeviceName\": \"\"/dev/sdb\", \"Ebs\":
{\"VolumeSize\":20, \"DeleteOnTermination\":false, \"VolumeType\": \"
\"standard\"}}]"
```

#### Blobs (recomendados)

Para ignorar as regras de aspas do PowerShell para entrada de dados JSON, use Blobs para passar seus dados JSON diretamente para a AWS CLI. Para obter mais informações sobre o Blobs, consulte [Blob](#).

## Windows command prompt

O prompt de comando do Windows exige aspas duplas " " para delimitar a estrutura de dados JSON. Além disso, para evitar que o processador de comando interprete mal as aspas duplas

incorporadas no JSON, você também deve usar um caractere de escape (preceder com um caractere de barra invertida \) cada uma das aspas duplas " dentro da própria estrutura de dados JSON, como no exemplo a seguir.

```
C:\> aws ec2 run-instances ^  
    --image-id ami-12345678 ^  
    --block-device-mappings "[{\"DeviceName\":\"/dev/sdb\", \"Ebs\":  
    {\"VolumeSize\":20, \"DeleteOnTermination\":false, \"VolumeType\":\"standard\"}}]"
```

Somente as aspas duplas mais externas não são de escape.

## Carregar os parâmetros da AWS CLI de um arquivo

Alguns parâmetros esperam nomes de arquivos como argumentos, dos quais a AWS CLI carrega os dados. Outros parâmetros permitem que você especifique o valor do parâmetro como texto digitado na linha de comando ou lido de um arquivo. Independentemente se um arquivo for obrigatório ou opcional, você deverá codificá-lo corretamente para que a AWS CLI possa entendê-lo. A codificação do arquivo deve corresponder à localidade padrão do sistema de leitura. É possível determinar isso usando o método `locale.getpreferredencoding()` do Python.

### Note

Por padrão, o Windows PowerShell gera texto como UTF-16, o que entra em conflito com a codificação UTF-8 usada por arquivos JSON e muitos sistemas Linux. Recomendamos que você use `-Encoding ascii` com os comandos `Out-File` do PowerShell para garantir que a AWS CLI possa ler o arquivo resultante.

## Tópicos

- [Como carregar os parâmetros de um arquivo](#)
- [Arquivos binários](#)
- [Arquivos remotos](#)

## Como carregar os parâmetros de um arquivo

Às vezes é conveniente carregar um valor de parâmetro de um arquivo, em vez de tentar digitá-lo como um valor de parâmetro de linha de comando, por exemplo, quando o parâmetro é uma string

JSON complexa. Para especificar um arquivo que contém o valor, especifique um URL de arquivo no formato a seguir.

```
file://complete/path/to/file
```

- Os dois primeiros caracteres de barra “/” fazem parte da especificação. Se o caminho exigido começar com “/”, o resultado será três caracteres de barra: `file:///folder/file`.
- O URL fornece o caminho para o arquivo que apresenta o conteúdo real do parâmetro.
- Ao usar arquivos com espaços ou caracteres especiais, siga as [regras de aspas e escape](#) para seu terminal.

#### Note

Esse comportamento é desativado automaticamente para parâmetros que já esperam um URL, como um parâmetro que identifica um URL de modelo do AWS CloudFormation. Também é possível desativar esse comportamento desabilitando a configuração [cli\\_follow\\_urlparam](#) em seu arquivo de configuração AWS CLI.

Os caminhos do arquivo no exemplo a seguir são interpretados como sendo relativos ao diretório de trabalho atual.

#### Linux or macOS

```
// Read from a file in the current directory
$ aws ec2 describe-instances --filters file://filter.json

// Read from a file in /tmp
$ aws ec2 describe-instances --filters file:///tmp/filter.json

// Read from a file with a filename with whitespaces
$ aws ec2 describe-instances --filters 'file://filter content.json'
```

#### Windows command prompt

```
// Read from a file in C:\temp
C:\> aws ec2 describe-instances --filters file://C:\temp\filter.json

// Read from a file with a filename with whitespaces
```

```
C:\> aws ec2 describe-instances --filters "file://C:\temp\filter content.json"
```

A opção de prefixo `file://` é compatível com as expansões de estilo Unix, inclusive `~/`, `./` e `../`. No Windows, a expressão `~/` se expande para o seu diretório de usuário, armazenado na variável de ambiente `%USERPROFILE%`. Por exemplo, no Windows 10, haveria normalmente um diretório de usuário em `C:\Users\UserName\`.

Ainda é necessário inserir um caractere de escape nos documentos JSON que são incorporados como o valor de outro documento JSON.

```
$ aws sqs create-queue --queue-name my-queue --attributes file://attributes.json
```

attributes.json

```
{
  "RedrivePolicy": "{\\"deadLetterTargetArn\\":\\"arn:aws:sqs:us-
west-2:0123456789012:deadletter\\", \\"maxReceiveCount\\":\\"5\\"}"
}
```

## Arquivos binários

Para comandos que incluem dados binários como um parâmetro, especifique que os dados são conteúdo binário usando o prefixo `fileb://`. Comandos que aceitam dados binários incluem:

- **aws ec2 run-instances**: parâmetro `--user-data`.
- **aws s3api put-object**: parâmetro `--sse-customer-key`.
- **aws kms decrypt**: parâmetro `--ciphertext-blob`.

O exemplo a seguir gera uma chave binária AES de 256 bits usando uma ferramenta de linha de comando do Linux e a fornece ao Amazon S3 para criptografar o arquivo enviado no lado do servidor.

```
$ dd if=/dev/urandom bs=1 count=32 > sse.key
32+0 records in
32+0 records out
32 bytes (32 B) copied, 0.000164441 s, 195 kB/s
$ aws s3api put-object \
  --bucket my-bucket \
```



```
--key test.txt \  
--body test.txt \  
--sse-customer-key fileb://sse.key \  
--sse-customer-algorithm AES256  
{  
  "SSECustomerKeyMD5": "iVg8oWa8sy714+FjtesrJg==",  
  "SSECustomerAlgorithm": "AES256",  
  "ETag": "\"a6118e84b76cf98bf04bbe14b6045c6c\""  
}
```

## Arquivos remotos

A AWS CLI também é compatível com parâmetros de carregamento de um arquivo hospedado na Internet com um URL `http://` ou `https://`. O exemplo a seguir faz referência a um arquivo armazenado em um bucket do Amazon S3. Isso permite que você acesse arquivos de parâmetros de qualquer computador, mas requer que o contêiner seja acessível publicamente.

```
$ aws ec2 run-instances \  
  --image-id ami-12345678 \  
  --block-device-mappings http://my-bucket.s3.amazonaws.com/filename.json
```

O exemplo anterior assume que o arquivo `filename.json` contém os seguintes dados JSON.

```
[  
  {  
    "DeviceName": "/dev/sdb",  
    "Ebs": {  
      "VolumeSize": 20,  
      "DeleteOnTermination": false,  
      "VolumeType": "standard"  
    }  
  }  
]
```

Para outro exemplo com referência a um arquivo que contém parâmetros formatados pelo JSON, consulte [Anexar uma política gerenciada do IAM a um usuário](#).

## Esqueletos e arquivos de entrada da AWS CLI

A maioria dos comandos da AWS CLI aceita todas as entradas de parâmetros de um arquivo. Esses modelos podem ser gerados usando a opção `generate-cli-skeleton`.

## Tópicos

- [Sobre esqueletos e arquivos de entrada da AWS CLI](#)
- [Gerar um esqueleto de comando](#)

## Sobre esqueletos e arquivos de entrada da AWS CLI

A maioria dos comandos da AWS Command Line Interface (AWS CLI) aceita todas as entradas de parâmetros usando os parâmetros `--cli-input-json`.

Felizmente, esses mesmos comandos fornecem o parâmetro `--generate-cli-skeleton` para gerar um arquivo JSON com todos os parâmetros que você pode editar e preencher. Depois, você pode executar o comando com o parâmetro relevante `--cli-input-json` e apontar para o arquivo preenchido.

### Important

Vários comandos da AWS CLI não são mapeados diretamente para as operações individuais de API da AWS, como os comandos [aws s3](#). Esses comandos não são compatíveis com os parâmetros `--generate-cli-skeleton` ou `--cli-input-json` descritos neste tópico. Se você souber se um comando específico é compatível com esses parâmetros, execute o comando a seguir, substituindo os nomes do *serviço* e do *comando* pelos quais você está interessado.

```
$ aws service command help
```

A saída inclui uma seção *Synopsis* que mostra os parâmetros compatíveis com o comando especificado.

```
$ aws iam list-users help
...
SYNOPSIS
    list-users
    ...
    [--cli-input-json]
    ...
    [--generate-cli-skeleton <value>]
...
```

O parâmetro `--generate-cli-skeleton` faz com que o comando não seja executado, mas, em vez disso, gere e exiba um modelo de parâmetro que você pode personalizar e usar como entrada em um comando posterior. O modelo gerado inclui todos os parâmetros compatíveis com o comando.

O parâmetro `--generate-cli-skeleton` aceita um dos seguintes valores:

- `input`: o modelo gerado inclui todos os parâmetros de entrada formatados como JSON. Este é o valor padrão.
- `output`: o modelo gerado inclui todos os parâmetros de saída formatados como JSON.

Como a AWS CLI é essencialmente um “wrapper” em torno da API de serviço, o arquivo de esqueleto espera que você faça referência a todos os parâmetros pelo nome do parâmetro da API subjacente. Isso provavelmente é diferente do nome do parâmetro da AWS CLI. Por exemplo, um parâmetro da AWS CLI chamado `user-name` pode ser mapeado para o parâmetro da API do serviço da AWS chamado `UserName` (observe a capitalização alterada e o traço ausente). Recomendamos usar a opção `--generate-cli-skeleton` para gerar o modelo com os nomes de parâmetro “corretos” a fim de evitar erros. Você também pode fazer referência ao Guia de referência da API do serviço para ver os nomes de parâmetro esperados. É possível excluir quaisquer parâmetros do modelo que não sejam necessários e para os quais você não deseja fornecer um valor.

Por exemplo, se você executar o seguinte comando, ele gerará o modelo de parâmetro para o comando `run-instances` do Amazon Elastic Compute Cloud (Amazon EC2).

## JSON

O exemplo a seguir mostra como gerar um modelo formatado em JSON usando o valor padrão (`input`) para o `--generate-cli-skeleton` parâmetro.

```
$ aws ec2 run-instances --generate-cli-skeleton
```

```
{
  "DryRun": true,
  "ImageId": "",
  "MinCount": 0,
  "MaxCount": 0,
  "KeyName": "",
  "SecurityGroups": [
```

```
    ""
  ],
  "SecurityGroupIds": [
    ""
  ],
  "UserData": "",
  "InstanceType": "",
  "Placement": {
    "AvailabilityZone": "",
    "GroupName": "",
    "Tenancy": ""
  },
  "KernelId": "",
  "RamdiskId": "",
  "BlockDeviceMappings": [
    {
      "VirtualName": "",
      "DeviceName": "",
      "Ebs": {
        "SnapshotId": "",
        "VolumeSize": 0,
        "DeleteOnTermination": true,
        "VolumeType": "",
        "Iops": 0,
        "Encrypted": true
      },
      "NoDevice": ""
    }
  ],
  "Monitoring": {
    "Enabled": true
  },
  "SubnetId": "",
  "DisableApiTermination": true,
  "InstanceInitiatedShutdownBehavior": "",
  "PrivateIpAddress": "",
  "ClientToken": "",
  "AdditionalInfo": "",
  "NetworkInterfaces": [
    {
      "NetworkInterfaceId": "",
      "DeviceIndex": 0,
      "SubnetId": "",
      "Description": "",
```

```

        "PrivateIpAddress": "",
        "Groups": [
            ""
        ],
        "DeleteOnTermination": true,
        "PrivateIpAddresses": [
            {
                "PrivateIpAddress": "",
                "Primary": true
            }
        ],
        "SecondaryPrivateIpAddressCount": 0,
        "AssociatePublicIpAddress": true
    }
],
"IamInstanceProfile": {
    "Arn": "",
    "Name": ""
},
"EbsOptimized": true
}

```

## Gerar um esqueleto de comando

Para gerar e usar um arquivo de esqueleto de parâmetro

1. Execute o comando com o parâmetro `--generate-cli-skeleton` para produzir JSON e direcionar a saída a um arquivo para salvá-lo.

JSON

```
$ aws ec2 run-instances --generate-cli-skeleton input > ec2runinst.json
```

2. Abra o arquivo de esqueleto do parâmetro em seu editor de texto e remova os parâmetros que não são mais necessários. Por exemplo, você pode reduzir o modelo ao indicado a seguir. Verifique se o arquivo ainda é JSON válido depois de remover os elementos que não são necessários.

JSON

```
{
```

```
"DryRun": true,
"ImageId": "",
"KeyName": "",
"SecurityGroups": [
    ""
],
"InstanceType": "",
"Monitoring": {
    "Enabled": true
}
}
```

Neste exemplo, deixamos o parâmetro `DryRun` definido como `true` para usar o recurso de simulação do Amazon EC2. Esse recurso permite que você teste com segurança o comando sem realmente criar ou modificar nenhum recursos.

3. Preencha os valores restantes com valores apropriados para seu cenário. Neste exemplo, fornecemos o tipo de instância, o nome da chave, o grupo de segurança e o identificador da imagem de máquina da Amazon (AMI) que devem ser usados. Este exemplo assume a região padrão da AWS. A AMI `ami-dfc39aef` é uma imagem de 64 bits do hospedada na região `us-west-2`. Se você usar uma região diferente, será necessário [encontrar o ID da AMI correta para usar](#).

## JSON

```
{
  "DryRun": true,
  "ImageId": "ami-dfc39aef",
  "KeyName": "mykey",
  "SecurityGroups": [
    "my-sg"
  ],
  "InstanceType": "t2.micro",
  "Monitoring": {
    "Enabled": true
  }
}
```

4. Execute o comando com os parâmetros concluídos passando o arquivo do modelo concluído o parâmetro `--cli-input-json` usando o prefixo `file://`. A AWS CLI interpreta o caminho a ser associado ao diretório de trabalho atual, de forma que o exemplo a seguir que exibe

somente o nome do arquivo sem nenhum caminho seja procurado pelo arquivo diretamente no diretório de trabalho atual.

## JSON

```
$ aws ec2 run-instances --cli-input-json file://ec2runinst.json
```

```
A client error (DryRunOperation) occurred when calling the RunInstances operation: Request would have succeeded, but DryRun flag is set.
```

O erro de simulação indica que JSON está formado corretamente e os valores de parâmetro são válidos. Se outros problemas forem relatados na saída, corrija-os e repita a etapa anterior até que a mensagem “Request would have succeeded” seja exibida.

5. Agora você pode definir o parâmetro `DryRun` como `false` para desativar a simulação.

## JSON

```
{
  "DryRun": false,
  "ImageId": "ami-dfc39aef",
  "KeyName": "mykey",
  "SecurityGroups": [
    "my-sg"
  ],
  "InstanceType": "t2.micro",
  "Monitoring": {
    "Enabled": true
  }
}
```

6. Execute o comando, e `run-instances` realmente iniciará uma instância do Amazon EC2 e exibirá os detalhes gerados pelo início bem-sucedido. O formato da saída é controlado pelo parâmetro `--output`, separadamente do formato do modelo de parâmetro de entrada.

## JSON

```
$ aws ec2 run-instances --cli-input-json file://ec2runinst.json --output json
```

```
{
```

```
"OwnerId": "123456789012",  
"ReservationId": "r-d94a2b1",  
"Groups": [],  
"Instances": [  
...  
]
```

## Usar sintaxe simplificada com a AWS CLI

A AWS Command Line Interface (AWS CLI) pode aceitar muitos de seus parâmetros de opção no formato JSON. No entanto, pode ser entediante inserir grandes estruturas ou listas de JSON na linha de comando. Para tornar isso mais fácil, a AWS CLI também oferece suporte a uma sintaxe abreviada mais simples que permite a representação de seus parâmetros de opção em vez de utilizar o formato JSON completo.

### Tópicos

- [Parâmetros de estrutura](#)
- [Usar sintaxe simplificada com a AWS Command Line Interface](#)

## Parâmetros de estrutura

A sintaxe abreviada no AWS CLI facilita a inserção de parâmetros simples de entrada pelos usuários (estruturas não aninhadas). O formato é uma lista de pares de chave/valor separados por vírgula. Use as regras de [aspas](#) e escape apropriadas para seu terminal, pois a sintaxe abreviada são strings.

### Linux or macOS

```
--option key1=value1,key2=value2,key3=value3
```

### PowerShell

```
--option "key1=value1,key2=value2,key3=value3"
```

Ambos são equivalentes ao exemplo a seguir formatado em JSON.

```
--option '{"key1":"value1","key2":"value2","key3":"value3"}'
```



Não pode haver nenhum espaço em branco entre cada par de chave/valor separado por vírgula. Aqui está um exemplo do comando `update-table` do Amazon DynamoDB com a opção `--provisioned-throughput` especificada no formato simplificado.

```
$ aws dynamodb update-table \  
  --provisioned-throughput ReadCapacityUnits=15,WriteCapacityUnits=10 \  
  --table-name MyDDBTable
```

Isso é equivalente ao exemplo a seguir formatado em JSON.

```
$ aws dynamodb update-table \  
  --provisioned-throughput '{"ReadCapacityUnits":15,"WriteCapacityUnits":10}' \  
  --table-name MyDDBTable
```

## Usar sintaxe simplificada com a AWS Command Line Interface

Você pode especificar os parâmetros de entrada em um formulário de lista de duas formas: JSON ou abreviada. A sintaxe abreviada da AWS CLI é projetada para facilitar a inserção de listas com número, sequência de caracteres, estruturas aninhadas ou não.

O formato básico é mostrada aqui, onde os valores na lista são separados por um único espaço.

```
--option value1 value2 value3
```

Isso é equivalente ao exemplo a seguir, formatado em JSON.

```
--option '[value1,value2,value3]'
```

Como mencionado anteriormente, é possível especificar uma lista de números, uma lista de strings ou uma lista de estruturas de dados não aninhadas em formato abreviado. Veja a seguir um exemplo do comando `stop-instances` do Amazon Elastic Compute Cloud (Amazon EC2), onde o parâmetro de entrada (lista de strings) para a opção `--instance-ids` é especificado no formato simplificado.

```
$ aws ec2 stop-instances \  
  --instance-ids i-1486157a i-1286157c i-ec3a7e87
```

Isso é equivalente ao exemplo a seguir formatado em JSON.

```
$ aws ec2 stop-instances \  
  --instance-ids '["i-1486157a","i-1286157c","i-ec3a7e87"]'
```

A seguir está um exemplo do comando `create-tags` do Amazon EC2, que leva uma lista de estruturas não aninhadas para a opção `--tags`. A opção `--resources` especifica o ID da instância a ser marcada.

```
$ aws ec2 create-tags \  
  --resources i-1286157c \  
  --tags Key=My1stTag,Value=Value1 Key=My2ndTag,Value=Value2  
        Key=My3rdTag,Value=Value3
```

Isso é equivalente ao exemplo a seguir, formatado em JSON. O parâmetro JSON é escrito em várias linhas para melhor leitura.

```
$ aws ec2 create-tags \  
  --resources i-1286157c \  
  --tags '[  
    {"Key": "My1stTag", "Value": "Value1"},  
    {"Key": "My2ndTag", "Value": "Value2"},  
    {"Key": "My3rdTag", "Value": "Value3"}  
  ]'
```

## Controlar a saída do comando da AWS CLI

Esta seção descreve as diferentes maneiras de controlar a saída da AWS Command Line Interface (AWS CLI). Personalizar a saída da AWS CLI em seu terminal pode melhorar a legibilidade, simplificar a automação de scripts e facilitar a navegação em conjuntos de dados maiores.

A AWS CLI é compatível com vários [formatos de saída](#), incluindo [json](#), [text](#), e `table`. Alguns serviços têm [paginação](#) do lado do servidor para seus dados.

Por último, a AWS CLI com [filtragem nos lados do servidor e do cliente](#) que você pode usar individualmente ou em conjunto para filtrar a saída da sua AWS CLI. A filtragem no lado do servidor é processada primeiro e retorna a saída para a filtragem no lado do cliente. A filtragem no lado do servidor é compatível com a API do serviço. A filtragem no lado do cliente é suportada pelo cliente da AWS CLI com o parâmetro `--query`.

### Opções de saída do lado do servidor e do lado do cliente

As opções de saída do lado do servidor são recursos diretamente compatíveis com a API do AWS service (Serviço da AWS). Os dados filtrados ou paginados não são enviados ao cliente, o que pode acelerar os tempos de resposta HTTP e melhorar a largura de banda para conjuntos de dados maiores.

As opções de saída do lado do cliente são recursos criados pela AWS CLI. Todos os dados são enviados ao cliente e, em seguida, a AWS CLI filtra ou realiza a paginação do conteúdo exibido. As operações do lado do cliente não economizam velocidade ou largura de banda para conjuntos de dados maiores.

Quando as opções do lado do servidor e do lado do cliente são usadas juntas, as operações do lado do servidor são concluídas primeiro e depois enviadas ao cliente para operações do lado do cliente. Essa ação usa a economia em potencial de velocidade e largura de banda das opções do lado do servidor, enquanto usa recursos adicionais da AWS CLI para obter a saída desejada.

## Tópicos

- [Definição do formato da saída da AWS CLI](#)
- [Uso de opções de paginação da AWS CLI](#)
- [Filtragem da saída da AWS CLI](#)

## Definição do formato da saída da AWS CLI

Este tópico descreve os diferentes formatos de saída para a AWS Command Line Interface (AWS CLI). A AWS CLI oferece suporte aos seguintes formatos de saída:

- **[json](#)**: a saída é formatada como uma string [JSON](#).
- **[text](#)**: a saída é formatada como várias linhas de valores de string separados por tabulação. Isso pode ser útil para passar a saída para um processador de texto, como `grep`, `sed` ou `awk`.
- **[table](#)**: a saída é formatada como uma tabela usando os caracteres `+` e `|` para formar as bordas da célula. Geralmente, a informação é apresentada em um formato "amigável", que é muito mais fácil de ler do que outros, mas não tão útil programaticamente.

## Como selecionar o formato de saída

Como explicado no tópico sobre [configuração](#), você pode especificar o formato de saída de três maneiras:

- Usando a opção **output** em um perfil nomeado no arquivo **config**: o exemplo a seguir ajusta o formato de saída padrão como `text`.

```
[default]
output=text
```

- Usando a variável de ambiente **AWS\_DEFAULT\_OUTPUT**: a saída a seguir ajusta o formato como `table` para os comandos nessa sessão de linhas de comandos até que a variável seja alterada ou a sessão encerrada. Usar essa variável de ambiente substitui qualquer valor definido no arquivo `config`.

```
$ export AWS_DEFAULT_OUTPUT="table"
```

- Usando a opção **--output** na linha de comando: o exemplo a seguir define a saída apenas deste comando como `json`. Usar essa opção no comando substitui qualquer variável de ambiente definida atualmente ou o valor no arquivo `config`.

```
$ aws swf list-domains --registration-status REGISTERED --output json
```

### Important

O tipo de saída especificado por você modifica a forma de operação da opção `--query`.

- Se você especificar `--output text`, a saída será paginada antes que o filtro `--query` seja aplicado, e a AWS CLI executará a consulta uma vez em cada página da saída. Consequentemente, a consulta inclui o primeiro elemento correspondente em cada página, o que pode resultar em uma saída adicional inesperada. Para filtrar ainda mais a saída, é possível usar outras ferramentas da linha de comando, como `head` ou `tail`.
- Se você especificar `--output json`, a saída será completamente processada como uma única estrutura JSON nativa antes que o filtro `--query` seja aplicado. A AWS CLI executa a consulta apenas uma vez com relação a toda a estrutura, produzindo um resultado filtrado que é, então, a saída.

## Formato de saída JSON

[JSON](#) é o formato de saída padrão da AWS CLI. A maioria das linguagens de programação pode facilmente decodificar strings JSON usando funções internas ou com bibliotecas disponíveis publicamente. É possível combinar a saída JSON com a [opção --query](#) de maneiras poderosas para filtrar e formatar a saída em formato JSON da AWS CLI.

Para filtragem mais avançada que talvez você não consiga fazer com `--query`, é possível considerar `jq`, um processador JSON da linha de comando. Baixe e localize o tutorial oficial em <http://stedolan.github.io/jq/>.

Veja a seguir um exemplo de saída JSON.

```
$ aws iam list-users --output json
```

```
{
  "Users": [
    {
      "Path": "/",
      "UserName": "Admin",
      "UserId": "AIDA1111111111EXAMPLE",
      "Arn": "arn:aws:iam::123456789012:user/Admin",
      "CreateDate": "2014-10-16T16:03:09+00:00",
      "PasswordLastUsed": "2016-06-03T18:37:29+00:00"
    },
    {
      "Path": "/backup/",
      "UserName": "backup-user",
      "UserId": "AIDA2222222222EXAMPLE",
      "Arn": "arn:aws:iam::123456789012:user/backup/backup-user",
      "CreateDate": "2019-09-17T19:30:40+00:00"
    },
    {
      "Path": "/",
      "UserName": "cli-user",
      "UserId": "AIDA3333333333EXAMPLE",
      "Arn": "arn:aws:iam::123456789012:user/cli-user",
      "CreateDate": "2019-09-17T19:11:39+00:00"
    }
  ]
}
```

## Formato de saída de texto

O formato `text` organiza a saída da AWS CLI em linhas delimitadas por tabulação. Isso funciona bem com ferramentas de texto Unix tradicionais, como `grep`, `sed` e `awk`, bem como o processamento de texto executado pelo PowerShell.

O formato de saída `text` segue a estrutura básica mostrada abaixo. As colunas são classificadas alfabeticamente por nomes de chaves correspondentes subjacentes do objeto JSON.

```
IDENTIFIER  sorted-column1 sorted-column2
IDENTIFIER2 sorted-column1 sorted-column2
```

A seguir, veja um exemplo de saída `text`. Cada campo é separado dos outros por tabulação, com uma guia adicional na qual há um campo vazio.

```
$ aws iam list-users --output text
```

```
USERS  arn:aws:iam::123456789012:user/Admin                2014-10-16T16:03:09+00:00
2016-06-03T18:37:29+00:00  /                AIDA1111111111EXAMPLE  Admin
USERS  arn:aws:iam::123456789012:user/backup/backup-user      2019-09-17T19:30:40+00:00
                        /backup/  AIDA2222222222EXAMPLE  backup-user
USERS  arn:aws:iam::123456789012:user/cli-user                  2019-09-17T19:11:39+00:00
                        /                AIDA3333333333EXAMPLE  cli-user
```

A quarta coluna é o campo `PasswordLastUsed` e está vazia nas duas últimas entradas porque esses usuários nunca fazem login no AWS Management Console.

### Important

É altamente recomendável que, se você especificar uma saída `text`, também use sempre a opção `--query` para garantir um comportamento consistente.

Isso ocorre porque o formato de texto ordena alfabeticamente as colunas da saída pelo nome da chave do objeto JSON subjacente retornado pelo serviço da AWS, e recursos similares podem não têm os mesmos nomes de chave. Por exemplo, a representação JSON de uma instância do Amazon EC2 baseada em Linux pode ter elementos que não estão presentes na representação JSON de uma instância baseada em Windows ou vice-versa. Além disso, os recursos podem ter elementos de valores de chave adicionados ou removidos em futuras atualizações, alterando a ordem das colunas. Este é o local em que `--query` expande a funcionalidade da saída `text` para fornecer total controle sobre o formato de saída.

No exemplo a seguir, o comando especifica quais elementos devem ser exibidos e define a ordem das colunas com a notação da lista [key1, key2, ...]. Isso oferece a você a segurança de que os valores de chave corretos sempre serão exibido na coluna esperada. Por fim, observe como a AWS CLI resulta em None como os valores para as chaves que não existem.

```
$ aws iam list-users --output text --query 'Users[*].
[UserName,Arn,CreateDate,PasswordLastUsed,UserId]'
```

```
Admin          arn:aws:iam::123456789012:user/Admin
2014-10-16T16:03:09+00:00  2016-06-03T18:37:29+00:00  AIDA111111111111EXAMPLE
backup-user    arn:aws:iam::123456789012:user/backup-user
2019-09-17T19:30:40+00:00  None                        AIDA222222222222EXAMPLE
cli-user       arn:aws:iam::123456789012:user/cli-backup
2019-09-17T19:11:39+00:00  None                        AIDA333333333333EXAMPLE
```

O exemplo a seguir mostra como você pode usar grep e awk com a saída text do comando `aws ec2 describe-instances`. O primeiro comando exibe a zona de disponibilidade, o estado atual e o ID de cada instância na saída text. O segundo comando processa essa saída para exibir somente os IDs de todas as instâncias em execução na Zona de disponibilidade `us-west-2a`.

```
$ aws ec2 describe-instances --query 'Reservations[*].Instances[*].
[Placement.AvailabilityZone, State.Name, InstanceId]' --output text
```

```
us-west-2a      running i-4b41a37c
us-west-2a      stopped i-a071c394
us-west-2b      stopped i-97a217a0
us-west-2a      running i-3045b007
us-west-2a      running i-6fc67758
```

```
$ aws ec2 describe-instances --query 'Reservations[*].Instances[*].
[Placement.AvailabilityZone, State.Name, InstanceId]' --output text | grep us-west-2a |
grep running | awk '{print $3}'
```

```
i-4b41a37c
i-3045b007
i-6fc67758
```

O exemplo a seguir vai mais longe e mostra não apenas como filtrar a saída, mas como usar essa saída para automatizar a alteração dos tipos de instância para cada instância interrompida.

```
$ aws ec2 describe-instances --query 'Reservations[*].Instances[*].[State.Name,
  InstanceId]' --output text |
> grep stopped |
> awk '{print $2}' |
> while read line;
> do aws ec2 modify-instance-attribute --instance-id $line --instance-type '{"Value":
  "m1.medium"}';
> done
```

A saída `text` também pode ser útil no PowerShell. Como as colunas na saída `text` são delimitadas por tabulação, é possível dividir facilmente a saída em uma matriz usando o delimitador ``t` do PowerShell. O comando a seguir exibe o valor da terceira coluna (`InstanceId`) se a primeira coluna (`AvailabilityZone`) corresponder à string `us-west-2a`.

```
PS C:\>aws ec2 describe-instances --query 'Reservations[*].Instances[*].
[Placement.AvailabilityZone, State.Name, InstanceId]' --output text |
%{if ($_.split("`t")[0] -match "us-west-2a") { $_.split("`t")[2]; } }
```

```
-4b41a37c
i-a071c394
i-3045b007
i-6fc67758
```

Embora o exemplo anterior mostre como usar o parâmetro `--query` para analisar os objetos JSON subjacentes e extrair a coluna desejada, o PowerShell tem capacidade própria para lidar com o JSON, se a compatibilidade entre as plataformas não for uma preocupação. Em vez de lidar com a saída como texto, o que é exigido pela maioria dos shells de comando, o PowerShell permite que você use o cmdlet `ConvertFrom-Json` para produzir um objeto estruturado hierarquicamente. Depois, é possível acessar diretamente o membro desejado usando esse objeto.

```
(aws ec2 describe-instances --output json | ConvertFrom-
Json).Reservations.Instances.InstanceId
```



**i** Tip

Se você enviar o texto e filtrar a saída para um único campo usando o parâmetro `--query`, a saída será uma única linha de valores separados por tabulação. Para obter cada valor em uma linha separada, você pode colocar o campo de saída entre colchetes, conforme mostrado nos exemplos a seguir.

Saída de linha única separada por tabulação:

```
$ aws iam list-groups-for-user --user-name susan --output text --query
"Groups[].GroupName"
```

```
HRDepartment    Developers      SpreadsheetUsers  LocalAdmins
```

Cada valor em sua própria linha, colocando `[GroupName]` entre colchetes:

```
$ aws iam list-groups-for-user --user-name susan --output text --query
"Groups[].[GroupName]"
```

```
HRDepartment
Developers
SpreadsheetUsers
LocalAdmins
```

## Formato de saída de tabela

O formato `table` produz representações legíveis da saída complexa da AWS CLI em um formato tabular.

```
$ aws iam list-users --output table
```

```
-----
|
| ListUsers                                     |
+-----+
+
||
| Users                                     ||
|
```

```

|+-----+-----+-----+-----+
+-----+-----+-----+-----+
||           Arn           |           CreateDate           |
PasswordLastUsed | Path |           UserId           |           Username           ||
|+-----+-----+-----+-----+
+-----+-----+-----+-----+
|| arn:aws:iam::123456789012:user/Admin           | 2014-10-16T16:03:09+00:00 |
2016-06-03T18:37:29+00:00 | /           | AIDA111111111111EXAMPLE | Admin           ||
|| arn:aws:iam::123456789012:user/backup/backup-user | 2019-09-17T19:30:40+00:00 |
           | /backup/ | AIDA222222222222EXAMPLE | backup-user ||
|| arn:aws:iam::123456789012:user/cli-user           | 2019-09-17T19:11:39+00:00 |
           | /           | AIDA333333333333EXAMPLE | cli-user           ||
+-----+-----+-----+-----+
+

```

É possível combinar a opção `--query` com o formato `table` para exibir um conjunto de elementos pré-selecionado na saída bruta. As diferenças da saída entre as notações de lista e dicionário: no primeiro exemplo, os nomes de coluna são ordenados alfabeticamente, e, no segundo, as colunas sem nome são ordenadas conforme definido pelo usuário. Para obter mais informações sobre a opção `--query`, consulte [Filtragem da saída da AWS CLI](#).

```

$ aws ec2 describe-volumes --query 'Volumes[*].
{ID:VolumeId,InstanceId:Attachments[0].InstanceId,AZ:AvailabilityZone,Size:Size}' --
output table

```

```

-----
|           DescribeVolumes           |
+-----+-----+-----+-----+
|      AZ      |      ID      | InstanceId | Size |
+-----+-----+-----+-----+
| us-west-2a | vol-e11a5288 | i-a071c394 | 30   |
| us-west-2a | vol-2e410a47 | i-4b41a37c | 8    |
+-----+-----+-----+-----+

```

```

$ aws ec2 describe-volumes --query 'Volumes[*].
[VolumeId,Attachments[0].InstanceId,AvailabilityZone,Size]' --output table

```

```

-----
|           DescribeVolumes           |
+-----+-----+-----+-----+

```

```
| vol-e11a5288| i-a071c394 | us-west-2a | 30 |
| vol-2e410a47| i-4b41a37c | us-west-2a | 8 |
+-----+-----+-----+-----+
```

## Uso de opções de paginação da AWS CLI

Este tópico descreve as diferentes maneiras de paginar a saída pela AWS CLI.

### Paginação do lado do servidor

Para comandos que podem retornar uma grande lista de itens, a AWS Command Line Interface (AWS CLI) oferece várias opções para controlar o número de itens incluídos na saída quando a AWS CLI chama a API de um serviço para preencher a lista.

As opções incluem o seguinte:

- [Como usar o parâmetro `--no-paginate`](#)
- [Como usar o parâmetro `--page-size`](#)
- [Como usar o parâmetro `--max-items`](#)
- [Como usar o parâmetro `--starting-token`](#)

Por padrão, a AWS CLI usa um tamanho de página determinado pelo serviço específico e recupera todos os itens disponíveis. Por exemplo, o Amazon S3 tem um tamanho de página padrão de mil. Se você executar `aws s3api list-objects` em um bucket do Amazon S3 que contém 3,5 mil objetos, a AWS CLI fará automaticamente quatro chamadas para o Amazon S3, lidando com a lógica de paginação específica do serviço para você em segundo plano e retornando todos os 3,5 mil objetos na saída final.

### Como usar o parâmetro `--no-paginate`

A opção `--no-paginate` desabilita os seguintes tokens de paginação no lado do cliente. Ao usar um comando, por padrão a AWS CLI faz várias chamadas automaticamente para retornar todos os resultados possíveis para criar a paginação. Uma chamada para cada página. Desabilitar a paginação faz com que a AWS CLI chame apenas uma vez pela primeira página de resultados do comando.

Por exemplo, se você executar `aws s3api list-objects` em um bucket do Amazon S3 que contém 3500 objetos, o AWS CLI fará apenas a primeira chamada para o Amazon S3, retornando apenas os primeiros 1000 objetos na saída final.

```
$ aws s3api list-objects \  
  --bucket my-bucket \  
  --no-paginate  
{  
  "Contents": [  
  ...
```

### Como usar o parâmetro `--page-size`

Caso haja problemas ao executar os comandos da lista em um grande número de recursos, o tamanho da página padrão poderá ser muito alto. Isso pode fazer com que as chamadas para os serviços do AWS excedam o tempo máximo permitido e gerar um erro de “expiração”. Você pode usar a opção `--page-size` para especificar que a AWS CLI solicite um número menor de itens de cada chamada para o serviço da AWS. A AWS CLI ainda recuperará a lista completa, mas executará um número maior de chamadas de API de serviço em segundo plano e recuperará um número menor de itens em cada chamada. Isso fornece às chamadas individuais chances melhores de sucesso sem um tempo limite. Alterar o tamanho da página não afeta a saída, afeta somente o número de chamadas da API que precisam ser feitas para gerar a saída.

```
$ aws s3api list-objects \  
  --bucket my-bucket \  
  --page-size 100  
{  
  "Contents": [  
  ...
```

### Como usar o parâmetro `--max-items`

Para incluir menos itens por vez na saída da AWS CLI, use a opção `--max-items`. A AWS CLI ainda lida com paginação com o serviço conforme descrito anteriormente, mas imprime apenas o número de itens que você especificar de cada vez.

```
$ aws s3api list-objects \  
  --bucket my-bucket \  
  --max-items 100  
{  
  "NextToken": "eyJNYXJrZXIiOiBudWxsLCAiYm90b190cnVuY2F0ZV9hbW91bnQiOiAxZjQ==",  
  "Contents": [  
  ...
```

## Como usar o parâmetro `--starting-token`

Se o número de saída de itens (`--max-items`) for menor do que o número total de itens retornados pelas chamadas de API subjacentes, a saída incluirá um `NextToken` que pode ser passado para um comando subsequente para recuperar o próximo conjunto de itens. O exemplo a seguir mostra como usar o valor `NextToken` retornado pelo exemplo anterior e permite que você recupere os segundos 100 itens.

### Note

O parâmetro `--starting-token` não pode ser nulo nem vazio. Se o comando anterior não retornar um valor `NextToken`, não haverá mais itens a serem retornados e você não precisará chamar o comando novamente.

```
$ aws s3api list-objects \  
  --bucket my-bucket \  
  --max-items 100 \  
  --starting-token eyJNYXJrZXIiOiBudWxsLCAiYm90b190cnVuY2F0ZV9hbW91bnQiOiAxfQ==\  
{  
  "Contents": [  
  ...
```

O serviço da AWS especificado pode não retornar itens na mesma ordem cada vez que você chamá-lo. Se você especificar valores diferentes para `--page-size` e `--max-items`, poderá obter resultados inesperados com itens ausentes ou duplicados. Para evitar que isso aconteça, use o mesmo número para `--page-size` e `--max-items` para sincronizar a paginação da AWS CLI com a paginação do serviço subjacente. Também é possível recuperar a lista completa e executar quaisquer operações necessárias de paginação localmente.

## Filtragem da saída da AWS CLI

A AWS Command Line Interface (AWS CLI) conta com filtragem nos lados do servidor e do cliente que você pode usar individualmente ou em conjunto para filtrar sua saída da AWS CLI. A filtragem no lado do servidor é processada primeiro e retorna a saída para a filtragem no lado do cliente.

- A filtragem no lado do servidor é suportada pela API, e você geralmente a implementa com um parâmetro `--filter`. O serviço retorna apenas resultados correspondentes que podem acelerar os tempos de resposta de HTTP para conjuntos de dados grandes.

- A filtragem no lado do cliente é suportada pelo cliente da AWS CLI com o parâmetro `--query`. Esse parâmetro oferece recursos que a filtragem do lado do servidor pode não ter.

## Tópicos

- [Filtragem no lado do servidor](#)
- [Filtragem no lado do cliente](#)
- [Combinação das filtragens no lado do servidor e no lado do cliente](#)
- [Recursos adicionais](#)

## Filtragem no lado do servidor

A filtragem no lado do servidor na AWS CLI é fornecida pela API do serviço AWS. O serviço da AWS retorna apenas os registros na resposta HTTP que correspondem ao seu filtro, o que pode acelerar os tempos de resposta de HTTP para conjuntos de dados grandes. Como a filtragem no lado do servidor é definida pela API do serviço, os nomes e funções dos parâmetros variam entre os serviços. Alguns nomes de parâmetros comuns usados na filtragem são:

- `--filter` como [ses](#) e [ce](#).
- `--filters` como [ec2](#), [autoescalabilidade](#) e [rds](#).
- Nomes que começam com a palavra `filter`, por exemplo `--filter-expression`, para o comando [aws dynamodb scan](#).

Para obter informações sobre se um comando específico oferece filtragem no lado do servidor e a respeito das regras de filtragem, consulte o [AWS CLIGuia de referência da](#) .

## Filtragem no lado do cliente

A AWS CLI fornece recursos de filtragem integrada baseados em JSON no lado do cliente com o parâmetro `--query`. O parâmetro `--query` é uma ferramenta poderosa que você pode usar para personalizar o conteúdo e estilo da sua saída. O parâmetro `--query` recebe a resposta HTTP retornada pelo servidor e filtra os resultados antes de exibi-los. Como toda a resposta HTTP é enviada para o cliente antes da filtragem, a filtragem no lado do cliente pode ser mais lenta do que a filtragem no lado do servidor para conjuntos de dados grandes.

A consulta usa a [sintaxe JMESPath](#) para criar expressões para filtrar sua saída. Para aprender a sintaxe JMESPath, consulte o [Tutorial](#) no site do JMESPath.

### Important

O tipo de saída especificado por você modifica a forma de operação da opção `--query`.

- Se você especificar `--output text`, a saída será paginada antes que o filtro `--query` seja aplicado, e a AWS CLI executará a consulta uma vez em cada página da saída. Consequentemente, a consulta inclui o primeiro elemento correspondente em cada página, o que pode resultar em uma saída adicional inesperada. Para filtrar ainda mais a saída, é possível usar outras ferramentas da linha de comando, como `head` ou `tail`.
- Se você especificar `--output json`, a saída será completamente processada como uma única estrutura JSON nativa antes que o filtro `--query` seja aplicado. A AWS CLI executa a consulta apenas uma vez com relação a toda a estrutura, produzindo um resultado filtrado que é, então, a saída.

## Tópicos de filtragem no lado do cliente

- [Antes de começar](#)
- [Identificadores](#)
- [Como selecionar em uma lista](#)
- [Como filtrar dados aninhados](#)
- [Resultados da simplificação](#)
- [Filtragem de valores específicos](#)
- [Encadeamento de expressões](#)
- [Filtragem por vários valores de identificador](#)
- [Adição de rótulos a valores de identificador](#)
- [Funções](#)
- [Exemplos avançados `--query`](#)

## Antes de começar

Ao usar expressões de filtro usadas nesses exemplos, certifique-se de usar as regras de aspas corretas para o shell do terminal. Para obter mais informações, consulte [the section called “Aspas com strings”](#).

A saída JSON a seguir mostra um exemplo do que o parâmetro `--query` pode produzir. A saída descreve três volumes do Amazon EBS anexados a instâncias separadas do Amazon EC2.

### Exemplo de saída

```
$ aws ec2 describe-volumes
{
  "Volumes": [
    {
      "AvailabilityZone": "us-west-2a",
      "Attachments": [
        {
          "AttachTime": "2013-09-17T00:55:03.000Z",
          "InstanceId": "i-a071c394",
          "VolumeId": "vol-e11a5288",
          "State": "attached",
          "DeleteOnTermination": true,
          "Device": "/dev/sda1"
        }
      ],
      "VolumeType": "standard",
      "VolumeId": "vol-e11a5288",
      "State": "in-use",
      "SnapshotId": "snap-f23ec1c8",
      "CreateTime": "2013-09-17T00:55:03.000Z",
      "Size": 30
    },
    {
      "AvailabilityZone": "us-west-2a",
      "Attachments": [
        {
          "AttachTime": "2013-09-18T20:26:16.000Z",
          "InstanceId": "i-4b41a37c",
          "VolumeId": "vol-2e410a47",
          "State": "attached",
          "DeleteOnTermination": true,
          "Device": "/dev/sda1"
        }
      ],
      "VolumeType": "standard",
      "VolumeId": "vol-2e410a47",
      "State": "in-use",
      "SnapshotId": "snap-708e8348",
      "CreateTime": "2013-09-18T20:26:15.000Z",
```



```

    "Size": 8
  },
  {
    "AvailabilityZone": "us-west-2a",
    "Attachments": [
      {
        "AttachTime": "2020-11-20T19:54:06.000Z",
        "InstanceId": "i-1jd73kv8",
        "VolumeId": "vol-a1b3c7nd",
        "State": "attached",
        "DeleteOnTermination": true,
        "Device": "/dev/sda1"
      }
    ],
    "VolumeType": "standard",
    "VolumeId": "vol-a1b3c7nd",
    "State": "in-use",
    "SnapshotId": "snap-234087fb",
    "CreateTime": "2020-11-20T19:54:05.000Z",
    "Size": 15
  }
]
}

```

## Identificadores

Identificadores são rótulos para valores de saída. Ao criar filtros, você usa identificadores para refinar os resultados da consulta. No exemplo de saída a seguir, todos os identificadores como Volumes, AvailabilityZone e AttachTime estão destacados.

```

$ aws ec2 describe-volumes
{
  "Volumes": [
    {
      "AvailabilityZone": "us-west-2a",
      "Attachments": [
        {
          "AttachTime": "2013-09-17T00:55:03.000Z",
          "InstanceId": "i-a071c394",
          "VolumeId": "vol-e11a5288",
          "State": "attached",
          "DeleteOnTermination": true,
          "Device": "/dev/sda1"
        }
      ]
    }
  ]
}

```

```

    }
  ],
  "VolumeType": "standard",
  "VolumeId": "vol-e11a5288",
  "State": "in-use",
  "SnapshotId": "snap-f23ec1c8",
  "CreateTime": "2013-09-17T00:55:03.000Z",
  "Size": 30
},
{
  "AvailabilityZone": "us-west-2a",
  "Attachments": [
    {
      "AttachTime": "2013-09-18T20:26:16.000Z",
      "InstanceId": "i-4b41a37c",
      "VolumeId": "vol-2e410a47",
      "State": "attached",
      "DeleteOnTermination": true,
      "Device": "/dev/sda1"
    }
  ],
  "VolumeType": "standard",
  "VolumeId": "vol-2e410a47",
  "State": "in-use",
  "SnapshotId": "snap-708e8348",
  "CreateTime": "2013-09-18T20:26:15.000Z",
  "Size": 8
},
{
  "AvailabilityZone": "us-west-2a",
  "Attachments": [
    {
      "AttachTime": "2020-11-20T19:54:06.000Z",
      "InstanceId": "i-1jd73kv8",
      "VolumeId": "vol-a1b3c7nd",
      "State": "attached",
      "DeleteOnTermination": true,
      "Device": "/dev/sda1"
    }
  ],
  "VolumeType": "standard",
  "VolumeId": "vol-a1b3c7nd",
  "State": "in-use",
  "SnapshotId": "snap-234087fb",

```

```
    "CreateTime": "2020-11-20T19:54:05.000Z",
    "Size": 15
  }
]
```

Para obter mais informações, consulte [Identificadores](#) no site do JMESPath.

Como selecionar em uma lista

Uma lista ou matriz é um identificador que é seguido por um colchete “[”, como Volumes e Attachments na [the section called “Antes de começar”](#).

Sintaxe

```
<listName>[ ]
```

Para filtrar toda a saída de uma matriz, você pode usar a notação curinga. Expressões [curinga](#) são expressões usadas para retornar elementos usando a notação \*.

O exemplo a seguir consulta todo o conteúdo Volumes.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[*]'
[
  {
    "AvailabilityZone": "us-west-2a",
    "Attachments": [
      {
        "AttachTime": "2013-09-17T00:55:03.000Z",
        "InstanceId": "i-a071c394",
        "VolumeId": "vol-e11a5288",
        "State": "attached",
        "DeleteOnTermination": true,
        "Device": "/dev/sda1"
      }
    ],
    "VolumeType": "standard",
    "VolumeId": "vol-e11a5288",
    "State": "in-use",
    "SnapshotId": "snap-f23ec1c8",
    "CreateTime": "2013-09-17T00:55:03.000Z",
    "Size": 30
  }
]
```

```

},
{
  "AvailabilityZone": "us-west-2a",
  "Attachments": [
    {
      "AttachTime": "2020-11-20T19:54:06.000Z",
      "InstanceId": "i-1jd73kv8",
      "VolumeId": "vol-a1b3c7nd",
      "State": "attached",
      "DeleteOnTermination": true,
      "Device": "/dev/sda1"
    }
  ],
  "VolumeType": "standard",
  "VolumeId": "vol-a1b3c7nd",
  "State": "in-use",
  "SnapshotId": "snap-234087fb",
  "CreateTime": "2020-11-20T19:54:05.000Z",
  "Size": 15
}
]

```

Para exibir um volume específico na matriz por índice, chame o índice da matriz. Por exemplo, o primeiro item na matriz Volumes tem um índice igual a 0, o que resulta na consulta `Volumes[0]`. Para obter mais informações sobre índices de matriz, consulte [Expressões de índice](#) no site do JMESPath.

```

$ aws ec2 describe-volumes \
  --query 'Volumes[0]'
{
  "AvailabilityZone": "us-west-2a",
  "Attachments": [
    {
      "AttachTime": "2013-09-17T00:55:03.000Z",
      "InstanceId": "i-a071c394",
      "VolumeId": "vol-e11a5288",
      "State": "attached",
      "DeleteOnTermination": true,
      "Device": "/dev/sda1"
    }
  ],
  "VolumeType": "standard",
  "VolumeId": "vol-e11a5288",

```

```
"State": "in-use",
"SnapshotId": "snap-f23ec1c8",
"CreateTime": "2013-09-17T00:55:03.000Z",
"Size": 30
}
```

Para exibir um intervalo específico de volumes por índice, use `slice` com a seguinte sintaxe, onde `start` é o índice da matriz inicial, `stop` é o índice onde o filtro interrompe o processamento e `step` é o intervalo de salto.

### Sintaxe

```
<arrayName>[<start>:<stop>:<step>]
```

Se algum deles for omitido da expressão `slice`, os seguintes valores padrão serão usados:

- Start: o primeiro índice na lista, 0.
- Stop: o último índice na lista.
- Step: sem pular etapas, onde o valor é 1.

Para retornar somente os dois primeiros volumes, use um valor de `start` igual a 0, um valor de `stop` igual 2 e um valor de `step` igual a 1, conforme mostrado no exemplo a seguir.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[0:2:1]'
[
  {
    "AvailabilityZone": "us-west-2a",
    "Attachments": [
      {
        "AttachTime": "2013-09-17T00:55:03.000Z",
        "InstanceId": "i-a071c394",
        "VolumeId": "vol-e11a5288",
        "State": "attached",
        "DeleteOnTermination": true,
        "Device": "/dev/sda1"
      }
    ],
    "VolumeType": "standard",
    "VolumeId": "vol-e11a5288",
    "State": "in-use",
```

```

    "SnapshotId": "snap-f23ec1c8",
    "CreateTime": "2013-09-17T00:55:03.000Z",
    "Size": 30
  },
  {
    "AvailabilityZone": "us-west-2a",
    "Attachments": [
      {
        "AttachTime": "2013-09-18T20:26:16.000Z",
        "InstanceId": "i-4b41a37c",
        "VolumeId": "vol-2e410a47",
        "State": "attached",
        "DeleteOnTermination": true,
        "Device": "/dev/sda1"
      }
    ],
    "VolumeType": "standard",
    "VolumeId": "vol-2e410a47",
    "State": "in-use",
    "SnapshotId": "snap-708e8348",
    "CreateTime": "2013-09-18T20:26:15.000Z",
    "Size": 8
  }
]

```

Como este exemplo contém valores padrão, você pode encurtar o slice de Volumes[0:2:1] para Volumes[:2].

O exemplo a seguir omite valores padrão e retorna cada dois volumes em toda a matriz.

```

$ aws ec2 describe-volumes \
  --query 'Volumes[:2]'
[
  {
    "AvailabilityZone": "us-west-2a",
    "Attachments": [
      {
        "AttachTime": "2013-09-17T00:55:03.000Z",
        "InstanceId": "i-a071c394",
        "VolumeId": "vol-e11a5288",
        "State": "attached",
        "DeleteOnTermination": true,
        "Device": "/dev/sda1"
      }
    ]
  }
]

```

```

    }
  ],
  "VolumeType": "standard",
  "VolumeId": "vol-e11a5288",
  "State": "in-use",
  "SnapshotId": "snap-f23ec1c8",
  "CreateTime": "2013-09-17T00:55:03.000Z",
  "Size": 30
},
{
  "AvailabilityZone": "us-west-2a",
  "Attachments": [
    {
      "AttachTime": "2020-11-20T19:54:06.000Z",
      "InstanceId": "i-1jd73kv8",
      "VolumeId": "vol-a1b3c7nd",
      "State": "attached",
      "DeleteOnTermination": true,
      "Device": "/dev/sda1"
    }
  ],
  "VolumeType": "standard",
  "VolumeId": "vol-a1b3c7nd",
  "State": "in-use",
  "SnapshotId": "snap-234087fb",
  "CreateTime": "2020-11-20T19:54:05.000Z",
  "Size": 15
}
]

```

Os valores de `step` também podem ser números negativos, o que resulta na filtragem em ordem inversa de uma matriz, conforme mostrado no exemplo a seguir.

```

$ aws ec2 describe-volumes \
  --query 'Volumes[::-2]'
[
  {
    "AvailabilityZone": "us-west-2a",
    "Attachments": [
      {
        "AttachTime": "2020-11-20T19:54:06.000Z",
        "InstanceId": "i-1jd73kv8",
        "VolumeId": "vol-a1b3c7nd",

```

```

        "State": "attached",
        "DeleteOnTermination": true,
        "Device": "/dev/sda1"
    },
    {
        "VolumeType": "standard",
        "VolumeId": "vol-a1b3c7nd",
        "State": "in-use",
        "SnapshotId": "snap-234087fb",
        "CreateTime": "2020-11-20T19:54:05.000Z",
        "Size": 15
    },
    {
        "AvailabilityZone": "us-west-2a",
        "Attachments": [
            {
                "AttachTime": "2013-09-17T00:55:03.000Z",
                "InstanceId": "i-a071c394",
                "VolumeId": "vol-e11a5288",
                "State": "attached",
                "DeleteOnTermination": true,
                "Device": "/dev/sda1"
            }
        ],
        "VolumeType": "standard",
        "VolumeId": "vol-e11a5288",
        "State": "in-use",
        "SnapshotId": "snap-f23ec1c8",
        "CreateTime": "2013-09-17T00:55:03.000Z",
        "Size": 30
    }
]

```

Para obter mais informações, consulte [Slices](#) no site do JMESPath.

### Como filtrar dados aninhados

Para refinar a filtragem de `Volumes[*]` para valores aninhados, use subexpressões anexando um ponto e seus critérios de filtragem.

### Sintaxe

```
<expression>.<expression>
```



O exemplo a seguir mostra todas as informações de Attachments para todos os volumes.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[*].Attachments'
[
  [
    {
      "AttachTime": "2013-09-17T00:55:03.000Z",
      "InstanceId": "i-a071c394",
      "VolumeId": "vol-e11a5288",
      "State": "attached",
      "DeleteOnTermination": true,
      "Device": "/dev/sda1"
    }
  ],
  [
    {
      "AttachTime": "2013-09-18T20:26:16.000Z",
      "InstanceId": "i-4b41a37c",
      "VolumeId": "vol-2e410a47",
      "State": "attached",
      "DeleteOnTermination": true,
      "Device": "/dev/sda1"
    }
  ],
  [
    {
      "AttachTime": "2020-11-20T19:54:06.000Z",
      "InstanceId": "i-1jd73kv8",
      "VolumeId": "vol-a1b3c7nd",
      "State": "attached",
      "DeleteOnTermination": true,
      "Device": "/dev/sda1"
    }
  ]
]
```

Para filtrar ainda mais os valores aninhados, acrescente a expressão para cada identificador aninhado. O exemplo a seguir lista o State para todos os Volumes.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[*].Attachments[*].State'
[
```

```
[
  "attached"
],
[
  "attached"
],
[
  "attached"
]
]
```

## Resultados da simplificação

Para obter mais informações, consulte [Subexpressões](#) no site do JMESPath.

Você pode simplificar os resultados para `Volumes[*].Attachments[*].State` removendo a notação curinga, o que resultará na consulta `Volumes[*].Attachments[].State`. A simplificação muitas vezes é útil para melhorar a legibilidade dos resultados.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[*].Attachments[].State'
[
  "attached",
  "attached",
  "attached"
]
```

Para obter mais informações, consulte [Simplificação](#) no site do JMESPath.

## Filtragem de valores específicos

Para filtrar valores específicos em uma lista, use uma expressão de filtro como mostrado na sintaxe a seguir.

### Sintaxe

```
? <expression> <comparator> <expression>]
```

Os comparadores de expressão incluem `==`, `!=`, `<`, `<=`, `>` e `>=`. O exemplo a seguir filtra `VolumeIds` para todos os `Volumes` em um `State` igual a `Attached`.

```
$ aws ec2 describe-volumes \
```

```
--query 'Volumes[*].Attachments[?State==`attached`].VolumeId'
[
  [
    "vol-e11a5288"
  ],
  [
    "vol-2e410a47"
  ],
  [
    "vol-a1b3c7nd"
  ]
]
```

Os resultados podem ser simplificados, o que resulta no exemplo a seguir.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[*].Attachments[?State==`attached`].VolumeId[]'
[
  "vol-e11a5288",
  "vol-2e410a47",
  "vol-a1b3c7nd"
]
```

O exemplo a seguir filtra VolumeIds para todos os Volumes com tamanho inferior a 20.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[?Size < `20`].VolumeId'
[
  "vol-2e410a47",
  "vol-a1b3c7nd"
]
```

Para obter mais informações, consulte [Expressões de filtragem](#) no site do JMESPath.

## Encadeamento de expressões

Você pode encadear resultados de um filtro para uma nova lista e, em seguida, filtrar o resultado com outra expressão usando a seguinte sintaxe:

## Sintaxe

```
<expression> | <expression>]
```

O exemplo a seguir leva os resultados do filtro da expressão `Volumes[*].Attachments[].InstanceId` e produz o primeiro resultado na matriz.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[*].Attachments[].InstanceId | [0]'
"i-a071c394"
```

Esse exemplo faz isso criando primeiro a matriz a partir da expressão a seguir.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[*].Attachments[].InstanceId'
"i-a071c394",
"i-4b41a37c",
"i-1jd73kv8"
```

Depois, devolva o primeiro elemento para essa matriz.

```
"i-a071c394"
```

Para obter mais informações, consulte [Expressões de encadeamento](#) no site do JMESPath.

### Filtragem por vários valores de identificador

Para filtrar por vários identificadores, use uma lista de seleção múltipla usando a seguinte sintaxe:

#### Sintaxe

```
<listName>[].[<expression>, <expression>]
```

No exemplo a seguir, `VolumeId` e `VolumeType` são filtrados na lista `Volumes`, o que resulta na expressão abaixo.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[].[VolumeId, VolumeType]'
[
  [
    "vol-e11a5288",
    "standard"
  ],
  [
```

```
    "vol-2e410a47",  
    "standard"  
  ],  
  [  
    "vol-a1b3c7nd",  
    "standard"  
  ]  
]
```

Para adicionar dados aninhados à lista, adicione outra lista de seleção múltipla. O exemplo a seguir expande o exemplo anterior filtrando também por InstanceId e State na lista aninhada Attachments. Isso resulta na seguinte expressão.

```
$ aws ec2 describe-volumes \  
  --query 'Volumes[][VolumeId, VolumeType, Attachments[][InstanceId, State]]'  
[  
  [  
    "vol-e11a5288",  
    "standard",  
    [  
      [  
        "i-a071c394",  
        "attached"  
      ]  
    ]  
  ],  
  [  
    "vol-2e410a47",  
    "standard",  
    [  
      [  
        "i-4b41a37c",  
        "attached"  
      ]  
    ]  
  ],  
  [  
    "vol-a1b3c7nd",  
    "standard",  
    [  
      [  
        "i-1jd73kv8",  
        "attached"  
      ]  
    ]  
  ]  
]
```

```

    ]
  ]
]
]

```

Para torná-la mais legível, simplifique a expressão conforme mostrado no exemplo a seguir.

```

$ aws ec2 describe-volumes \
  --query 'Volumes[][VolumeId, VolumeType, Attachments[][InstanceId, State]][]'
[
  "vol-e11a5288",
  "standard",
  [
    "i-a071c394",
    "attached"
  ],
  "vol-2e410a47",
  "standard",
  [
    "i-4b41a37c",
    "attached"
  ],
  "vol-a1b3c7nd",
  "standard",
  [
    "i-1jd73kv8",
    "attached"
  ]
]

```

Para obter mais informações, consulte [Lista de multisseleção](#) no site do JMESPath.

### Adição de rótulos a valores de identificador

Para tornar essa saída mais fácil de ler, use um hash de seleção múltipla com a sintaxe a seguir.

### Sintaxe

```
<listName>[].{<label>: <expression>, <label>: <expression>}
```

O rótulo do identificador não precisa ser o mesmo que o nome do identificador. O exemplo a seguir usa o rótulo `Type` para os valores `VolumeType`.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[].{VolumeType: VolumeType}'
[
  {
    "VolumeType": "standard",
  },
  {
    "VolumeType": "standard",
  },
  {
    "VolumeType": "standard",
  }
]
```

Para simplificar, o exemplo a seguir mantém os nomes dos identificadores para cada rótulo e exibe VolumeId, VolumeType, InstanceId e State para todos os volumes:

```
$ aws ec2 describe-volumes \
  --query 'Volumes[].{VolumeId: VolumeId, VolumeType: VolumeType, InstanceId:
  Attachments[0].InstanceId, State: Attachments[0].State}'
[
  {
    "VolumeId": "vol-e11a5288",
    "VolumeType": "standard",
    "InstanceId": "i-a071c394",
    "State": "attached"
  },
  {
    "VolumeId": "vol-2e410a47",
    "VolumeType": "standard",
    "InstanceId": "i-4b41a37c",
    "State": "attached"
  },
  {
    "VolumeId": "vol-a1b3c7nd",
    "VolumeType": "standard",
    "InstanceId": "i-1jd73kv8",
    "State": "attached"
  }
]
```

Para obter mais informações, consulte [Hash de multisseleção](#) no site do JMESPath.

## Funções

A sintaxe JMESPath contém muitas funções que você pode usar em suas consultas. Para obter informações sobre funções JMESPath, consulte [Funções integradas](#) no site do JMESPath.

Para demonstrar como você pode incorporar uma função em suas consultas, o exemplo a seguir usa a função `sort_by`. A função `sort_by` classifica uma matriz usando uma expressão como a chave de classificação com a seguinte sintaxe:

### Sintaxe

```
sort_by(<listName>, <sort expression>)[<expression>
```

O exemplo a seguir usa o [exemplo de hash de multisseleção](#) anterior e classifica a saída por `VolumeId`.

```
$ aws ec2 describe-volumes \
  --query 'sort_by(Volumes, &VolumeId)[].{VolumeId: VolumeId, VolumeType: VolumeType,
  InstanceId: Attachments[0].InstanceId, State: Attachments[0].State}'
[
  {
    "VolumeId": "vol-2e410a47",
    "VolumeType": "standard",
    "InstanceId": "i-4b41a37c",
    "State": "attached"
  },
  {
    "VolumeId": "vol-a1b3c7nd",
    "VolumeType": "standard",
    "InstanceId": "i-1jd73kv8",
    "State": "attached"
  },
  {
    "VolumeId": "vol-e11a5288",
    "VolumeType": "standard",
    "InstanceId": "i-a071c394",
    "State": "attached"
  }
]
```

Para obter mais informações, consulte [sort\\_by](#) no site do JMESPath.



## Exemplos avançados --query

Para extrair informações de um item específico

O exemplo a seguir usa o parâmetro `--query` para localizar um item específico em uma lista e extrai as informações desse item. O exemplo lista todas as `AvailabilityZones` associadas ao endpoint de serviço especificado. Ele extrai o item da lista `ServiceDetails` que tem o `ServiceName` especificado e gera o campo `AvailabilityZones` do item selecionado.

```
$ aws --region us-east-1 ec2 describe-vpc-endpoint-services \
    --query 'ServiceDetails[?ServiceName==`com.amazonaws.us-east-1.ecs`].AvailabilityZones'
[
  [
    "us-east-1a",
    "us-east-1b",
    "us-east-1c",
    "us-east-1d",
    "us-east-1e",
    "us-east-1f"
  ]
]
```

Para mostrar snapshots após a data de criação especificada

O exemplo a seguir mostra como listar todos os seus snapshots que foram criados após uma data especificada, incluindo apenas alguns dos campos disponíveis na saída.

```
$ aws ec2 describe-snapshots --owner self \
    --output json \
    --query 'Snapshots[?StartTime>=`2018-02-07`].{Id:SnapshotId,VId:VolumeId,Size:VolumeSize}'
[
  {
    "id": "snap-0effb42b7a1b2c3d4",
    "vid": "vol-0be9bb0bf12345678",
    "Size": 8
  }
]
```

Para mostrar as AMIs mais recentes

O exemplo a seguir lista as cinco imagens de máquina da Amazon (AMIs) mais recentes que você criou, classificadas das mais recentes para as mais antigas.

```
$ aws ec2 describe-images \
  --owners self \
  --query 'reverse(sort_by(Images,&CreationDate))[:5].{id:ImageId,date:CreationDate}'
[
  {
    "id": "ami-0a1b2c3d4e5f60001",
    "date": "2018-11-28T17:16:38.000Z"
  },
  {
    "id": "ami-0a1b2c3d4e5f60002",
    "date": "2018-09-15T13:51:22.000Z"
  },
  {
    "id": "ami-0a1b2c3d4e5f60003",
    "date": "2018-08-19T10:22:45.000Z"
  },
  {
    "id": "ami-0a1b2c3d4e5f60004",
    "date": "2018-05-03T12:04:02.000Z"
  },
  {
    "id": "ami-0a1b2c3d4e5f60005",
    "date": "2017-12-13T17:16:38.000Z"
  }
]
```

Para mostrar instâncias não íntegras do Auto Scaling

O exemplo a seguir mostra apenas o InstanceId para todas as instâncias com problemas de integridade no grupo de AutoScaling especificado.

```
$ aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-name My-AutoScaling-Group-Name \
  --output text \
  --query 'AutoScalingGroups[*].Instances[?HealthStatus==`Unhealthy`].InstanceId'
```

Para incluir volumes com a tag especificada

O exemplo a seguir descreve todas as instâncias com uma tag `test`. Contanto que haja outra etiqueta além de `test` anexada ao volume, o volume ainda é retornado nos resultados.

A expressão abaixo para retornar todas as tags com a tag `test` em uma matriz. Qualquer etiqueta que não é a etiqueta `test` contém um valor `null`.

```
$ aws ec2 describe-volumes \
  --query 'Volumes.Tags[?Value == `test`]'
```

Para excluir volumes com a etiqueta especificada

O exemplo a seguir descreve todas as instâncias sem uma etiqueta `test`. A utilização de uma expressão `?Value != `test`` simples não funciona para excluir um volume, pois os volumes podem ter várias etiquetas. Contanto que haja outra etiqueta além de `test` anexada ao volume, o volume ainda é retornado nos resultados.

Para excluir todos os volumes com a etiqueta `test`, comece com a expressão abaixo para retornar todas as etiquetas com a etiqueta `test` em um array. Qualquer etiqueta que não é a etiqueta `test` contém um valor `null`.

```
$ aws ec2 describe-volumes \
  --query 'Volumes.Tags[?Value == `test`]'
```

Em seguida, filtrar todos os resultados `test` positivos usando a função `not_null`.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[!not_null(Tags[?Value == `test`].Value)]'
```

Encadeie os resultados para simplificá-los, o que resultará na consulta a seguir.

```
$ aws ec2 describe-volumes \
  --query 'Volumes[!not_null(Tags[?Value == `test`].Value)] | []'
```

## Combinação das filtrações no lado do servidor e no lado do cliente

É possível usar as filtrações no lado do servidor e no lado do cliente ao mesmo tempo. A filtração no lado do servidor é concluída primeiro, o que envia os dados para o cliente que, em seguida, são filtrados pelo parâmetro `--query`. Se você estiver usando conjuntos de dados grandes, aplicar a filtração no lado do servidor primeiro pode reduzir a quantidade de dados enviados ao cliente para

cada chamada AWS CLI sem prejudicar a personalização poderosa proporcionada pela filtragem no lado do cliente.

O exemplo a seguir mostra volumes do Amazon EC2 usando filtragens no lado do servidor e no lado do cliente. O serviço filtra uma lista de todos os volumes anexados na zona de disponibilidade us-west-2a. O parâmetro `--query` limita ainda mais a saída apenas para aqueles volumes com um valor `Size` maior que 50 e mostra apenas os campos especificados com nomes definidos pelo usuário.

```
$ aws ec2 describe-volumes \
  --filters "Name=availability-zone,Values=us-west-2a" "Name=status,Values=attached" \
  --query 'Volumes[?Size > `50`].{Id:VolumeId,Size:Size,Type:VolumeType}'
[
  {
    "Id": "vol-0be9bb0bf12345678",
    "Size": 80,
    "VolumeType": "gp2"
  }
]
```

O exemplo a seguir recupera uma lista de imagens que atendem a vários critérios. Depois, usa o parâmetro `--query` para classificar a saída por `CreationDate`, selecionando apenas as mais recentes. Por fim, ele exibe o `ImageId` dessa imagem.

```
$ aws ec2 describe-images \
  --owners amazon \
  --filters "Name=name,Values=amzn*gp2" "Name=virtualization-type,Values=hvm" \
  "Name=root-device-type,Values=ebs" \
  --query "sort_by(Images, &CreationDate)[-1].ImageId" \
  --output text
ami-00ced3122871a4921
```

O exemplo a seguir exibe o número de volumes disponíveis acima de 1000 IOPS usando `length` para contar quantos há em uma lista.

```
$ aws ec2 describe-volumes \
  --filters "Name=status,Values=available" \
  --query 'length(Volumes[?Iops > `1000`])'
3
```

## Recursos adicionais

### JMESPath Terminal

O JMESPath Terminal é um terminal de comandos interativos que permitem experimentar expressões JMESPath usadas na filtragem no lado do cliente. Usando o comando `jpterm`, o terminal mostra os resultados imediatos da consulta enquanto você está digitando. Você pode encaminhar diretamente a saída da AWS CLI para o terminal, permitindo assim a experimentação avançada de consultas.

O exemplo a seguir encaminha a saída da `aws ec2 describe-volumes` diretamente para o JMESPath Terminal.

```
$ aws ec2 describe-volumes | jpterm
```

Para obter mais informações sobre o JMESPath Terminal e instruções de instalação, consulte [JMESPath Terminal](#) no GitHub.

### Utilitário jq

O `jq` fornece uma maneira de transformar sua saída no lado do cliente em um formato de saída desejado. Para obter mais informações sobre o `jq` e instruções de instalação, consulte [jq](#) no GitHub.

## Códigos de retorno da AWS CLI

O código de retorno geralmente é um código oculto enviado após a execução de um comando da AWS Command Line Interface (AWS CLI) que descreve o status do comando. Você pode usar o comando `echo` para exibir o código enviado do último comando AWS CLI e usar esses códigos para determinar se um comando foi bem-sucedido ou se falhou, e por que um comando pode ter apresentado um erro. Além dos códigos de retorno, você pode visualizar mais detalhes sobre uma falha executando seus comandos com a opção `--debug`. Essa opção produz um relatório detalhado das etapas que a AWS CLI usa para processar o comando, e o resultado que foi gerado por cada etapa.

Para determinar o código de retorno de um comando de AWS CLI, execute um dos seguintes comandos imediatamente após a execução do comando de CLI.

## Linux and macOS

```
$ echo $?  
0
```

## Windows PowerShell

```
PS> echo $lastexitcode  
0
```

## Windows Command Prompt

```
C:\> echo %errorlevel%  
0
```

Veja a seguir os valores de código de retorno que podem ser retornados ao final da execução de um comando de AWS Command Line Interface (AWS CLI).

| Códig | Significado                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0     | O serviço respondeu com um código de status de resposta HTTP de 200, o que indica que não houve erros gerados pela AWS CLI e pelo serviço da AWS para o qual a solicitação foi enviada.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 1     | Uma ou mais operações de transferência do Amazon S3 falhou. Limitado a comandos do S3.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 2     | O significado desse código de retorno depende do comando. <ul style="list-style-type: none"><li>• Aplicável a todos os comandos da AWS CLI : não foi possível analisar o comando inserido. Falhas de análise podem ser causadas, entre outros motivos, pela ausência de subcomandos ou argumentos necessários ou pelo uso de comandos ou argumentos desconhecidos.</li><li>• Limitado a comandos do S3: um ou mais arquivos marcados para transferência foram ignorados durante o processo de transferência. No entanto, todos os outros arquivos marcados para transferência foram transferidos com êxito. Os arquivos que são ignorados durante o processo de transferência incluem: arquivos inexistentes, arquivos</li></ul> |

| Códig | Significado                                                                                                                                                                     |
|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|       | que são dispositivos de caracteres especiais, dispositivo de bloqueio especial, filas FIFO ou soquetes, além de arquivos para os quais o usuário não tem permissões de leitura. |
| 130   | O comando foi interrompido por um SIGINT. Este é o sinal enviado por você para cancelar um comando com <code>Ctrl+C</code> .                                                    |
| 255   | Ocorreu uma falha no comando. Houve erros gerados pela AWS CLI ou pelo serviço da AWS para o qual a solicitação foi enviada.                                                    |

## Criar e usar aliases da AWS CLI

Os aliases são atalhos que você pode criar na AWS Command Line Interface (AWS CLI) para encurtar comandos ou scripts que utiliza com frequência. Os aliases são criados no arquivo `alias` localizado em sua pasta de configuração.

### Tópicos

- [Pré-requisitos](#)
- [Etapa 1: Criação do arquivo de alias](#)
- [Etapa 2: Criação de um alias](#)
- [Passo 3: Como chamar um alias](#)
- [Exemplos de repositório de alias](#)
- [Recursos](#)

## Pré-requisitos

Para usar comandos de alias, é necessário fazer o seguinte:

- Instale e configure a AWS CLI. Para obter mais informações, consulte [Instalar a AWS CLI](#) e [Autenticação e credenciais de acesso](#).
- Usar, no mínimo, a AWS CLI versão 1.11.24 ou 2.0.0.
- (Opcional) Para usar scripts bash em alias da AWS CLI, é necessário usar um terminal compatível com bash.

## Etapa 1: Criação do arquivo de alias

Para criar o arquivo `alias`, você pode utilizar a navegação de arquivos e um editor de texto ou utilizar o terminal preferido seguindo o procedimento passo a passo. Para criar rapidamente seu arquivo de alias, use o seguinte bloco de comandos.

### Linux and macOS

```
$ mkdir -p ~/.aws/cli  
$ echo '[toplevel]' > ~/.aws/cli/alias
```

### Windows

```
C:\> md %USERPROFILE%\aws\cli  
C:\> echo [toplevel] > %USERPROFILE%\aws\cli\alias
```

Para criar o arquivo de alias

1. Crie uma pasta chamada `cli` na pasta de configuração da AWS CLI. Por padrão, a pasta de configuração é `~/.aws/` no Linux ou no macOS e `%USERPROFILE%\aws\` no Windows. Você pode criar isso via sua navegação de arquivos ou usando o comando a seguir.

### Linux and macOS

```
$ mkdir -p ~/.aws/cli
```

### Windows

```
C:\> md %USERPROFILE%\aws\cli
```

O caminho padrão da pasta `cli` resultante é `~/.aws/cli/` no Linux ou no macOS e `%USERPROFILE%\aws\cli` no Windows.

2. Na pasta `cli`, crie um arquivo de texto chamado `alias` sem extensão e adicione `[toplevel]` à primeira linha. Você pode criar esse arquivo com seu editor de texto preferido ou usar o comando a seguir.



## Linux and macOS

```
$ echo '[toplevel]' > ~/.aws/cli/alias
```

## Windows

```
C:\> echo [toplevel] > %USERPROFILE%\aws/cli/alias
```

## Etapa 2: Criação de um alias

Você pode criar um alias usando comandos básicos ou scripts bash.

### Criação um alias de comando básico

Você pode criar seu alias ao adicionar um comando usando a seguinte sintaxe no arquivo `alias` criado na etapa anterior.

#### Sintaxe

```
aliasname = command [--options]
```

**aliasname** é o que você atribui ao seu alias. **command** é o comando que você deseja chamar, o qual pode incluir outros aliases. Você pode incluir opções ou parâmetros em seu alias ou adicioná-los ao chamar seu alias.

O exemplo a seguir cria um alias chamado `aws whoami` usando o comando [aws sts get-caller-identity](#). Como esse alias chama um comando da AWS CLI existente, você pode escrever o comando sem usar o prefixo `aws`.

```
whoami = sts get-caller-identity
```

O exemplo a seguir usa o exemplo `whoami` anterior e adiciona o filtro `Account` e as opções `output` de texto.

```
whoami2 = sts get-caller-identity --query Account --output text
```

## Criar um alias de subcomando

### Note

O recurso de alias do subcomando requer, no mínimo, a versão da AWS CLI 1.11.24 ou 2.0.0

É possível criar um alias para subcomandos adicionando um comando usando a seguinte sintaxe no arquivo `alias` criado na etapa anterior.

### Sintaxe

```
[command commandGroup]  
aliasname = command [--options]
```

O *commandGroup* é o namespace do comando, por exemplo, o comando `aws ec2 describe-regions` está sob o grupo do comando `ec2`. *aliasname* é o que você atribui ao seu alias.

*command* é o comando que você deseja chamar, o qual pode incluir outros aliases. Você pode incluir opções ou parâmetros em seu alias ou adicioná-los ao chamar seu alias.

O exemplo a seguir cria um alias chamado `aws ec2 regions` usando o comando [aws ec2 describe-regions](#). Como esse alias chama um comando da AWS CLI existente sob o namespace do comando `ec2`, é possível escrever o comando sem usar o prefixo `aws ec2`.

```
[command ec2]  
regions = describe-regions --query Regions[].RegionName
```

Para criar aliases de comandos fora do namespace do comando, use um ponto de exclamação como prefixo do comando completo. O exemplo a seguir cria um alias chamado `aws ec2 instance-profiles` usando o comando [aws iam list-instance-profiles](#).

```
[command ec2]  
instance-profiles = !aws iam list-instance-profiles
```

 Note

Os aliases usam apenas namespaces de comandos existentes e não é possível criar outros. Por exemplo, você não pode criar um alias com a seção `[command johnsmith]`, pois o namespace do comando `johnsmith` ainda não existe.

## Criação de um alias de script bash

 Warning

Para usar scripts bash em alias da AWS CLI, é necessário usar um terminal compatível com bash.

Você pode criar um alias usando scripts bash para processos mais avançados usando a sintaxe a seguir.

### Sintaxe

```
aliasname =  
    !f() {  
        script content  
    }; f
```

**aliasname** é o nome do seu alias e **script content** é o script que você deseja executar ao chamar o alias.

O exemplo a seguir usa `opendns` para mostrar seu endereço IP atual. Como você pode usar aliases em outros aliases, o alias `myip` a seguir é útil para permitir ou revogar o acesso ao seu endereço IP de dentro de outros aliases.

```
myip =  
    !f() {  
        dig +short myip.opendns.com @resolver1.opendns.com  
    }; f
```

Os exemplos de script a seguir chamam o alias `aws myip` anterior para autorizar seu endereço IP para ingresso em um grupo de segurança do Amazon EC2.

```
authorize-my-ip =  
  !f() {  
    ip=$(aws myip)  
    aws ec2 authorize-security-group-ingress --group-id ${1} --cidr $ip/32 --protocol  
    tcp --port 22  
  }; f
```

Quando você chama aliases que usam scripts bash, as variáveis são sempre passadas na ordem em que você as inseriu. Nos scripts bash, os nomes das variáveis não são levados em consideração, apenas a ordem em que aparecem. No exemplo de alias `textalert` a seguir, a variável para a opção `--message` é a primeira e para a opção `--phone-number` é a segunda.

```
textalert =  
  !f() {  
    aws sns publish --message "${1}" --phone-number ${2}  
  }; f
```

## Passo 3: Como chamar um alias

Para executar o alias que você criou em seu arquivo `alias`, use a sintaxe a seguir. Você pode adicionar opções extras ao chamar seu alias.

Sintaxe

```
$ aws aliasname
```

O exemplo a seguir usa o alias do comando `aws whoami`.

```
$ aws  
whoami  
{  
  "UserId": "A12BCD34E5FGHI6JKLM",  
  "Account": "1234567890987",  
  "Arn": "arn:aws:iam::1234567890987:user/userName"  
}
```

O exemplo a seguir usa o alias `aws whoami` com opções adicionais para retornar somente o número `Account` na saída de `text`.

```
$ aws whoami --query Account --output
text
1234567890987
```

O exemplo a seguir usa o [alias do subcomando](#) `aws ec2 regions`.

```
$ aws ec2
regions
[
  "ap-south-1",
  "eu-north-1",
  "eu-west-3",
  "eu-west-2",
  ...
```

## Como chamar um alias usando variáveis de script bash

Quando você chama aliases que usam scripts bash, as variáveis são passadas na ordem em que foram inseridas. Nos scripts bash, os nomes das variáveis não são levados em consideração, apenas a ordem em que aparecem. Por exemplo, no alias `textalert` a seguir, a variável para a opção `--message` é a primeira e para a opção `--phone-number` é a segunda.

```
textalert =
!f() {
  aws sns publish --message "${1}" --phone-number ${2}
}; f
```

Ao chamar o alias `textalert`, você precisa passar variáveis na mesma ordem em que elas são executadas no alias. No exemplo a seguir usamos as variáveis `$message` e `$phone`. A variável `$message` é passada como `${1}` para a opção `--message` e a variável `$phone` é passada como `${2}` para a opção `--phone-number`. Isso faz com que o alias `textalert` seja chamado com êxito para enviar uma mensagem.

```
$ aws textalert $message
$phone
{
  "MessageId": "1ab2cd3e4-fg56-7h89-i01j-2klmn34567"
}
```

No exemplo a seguir, a ordem é invertida quando o alias é chamado para `$phone` e `$message`. A variável `$phone` é passada como `${1}` para a opção `--message` e a variável `$message` é passada como `${2}` para a opção `--phone-number`. Como as variáveis estão fora de ordem, o alias passa as variáveis incorretamente. Isso causa um erro porque o conteúdo de `$message` não corresponde aos requisitos de formatação de número de telefone para a opção `--phone-number`.

```
$ aws textalert $phone
$message
usage: aws [options] <command> <subcommand> [<subcommand> ...] [parameters]
To see help text, you can run:

aws help
aws <command> help
aws <command> <subcommand> help

Unknown options: text
```

## Exemplos de repositório de alias

O [repositório de alias da AWS CLI](#) no GitHub contém exemplos de alias da AWS CLI criados pela equipe de desenvolvedores da AWS CLI e pela comunidade. Você pode usar todo o arquivo de exemplo de alias ou use aliases individuais para seu próprio uso.

### Warning

A execução dos comandos nesta seção exclui seu arquivo `alias`. Para evitar que o arquivo de alias existente seja sobrescrito, altere o local de download.

Para usar aliases do repositório

1. Instale o Git. Para obter instruções de instalação, consulte [Introdução: Instalação do Git](#) na Documentação do Git.
2. Instale o comando `jp`. O comando `jp` é usado no alias `tostring`. Para obter instruções de instalação, consulte [JMESPath \(jp\) Readme.md](#) no GitHub.
3. Instale o comando `jq`. O comando `jq` é usado no alias `tostring-with-jq`. Para obter instruções de instalação, consulte [Processador JSON \(jq\)](#) no GitHub.
4. Baixe o `alias` de uma das seguintes formas:

- Execute os seguintes comandos que são baixados do repositório e copiam o arquivo `alias` para sua pasta de configuração.

Linux and macOS

```
$ git clone https://github.com/awslabs/awscli-aliases.git
$ mkdir -p ~/.aws/cli
$ cp awscli-aliases/alias ~/.aws/cli/alias
```

Windows

```
C:\> git clone https://github.com/awslabs/awscli-aliases.git
C:\> md %USERPROFILE%\aws\cli
C:\> copy awscli-aliases\alias %USERPROFILE%\aws\cli
```

- Baixe diretamente o repositório e salve na pasta `cli` em sua pasta de configuração da AWS CLI. Por padrão, a pasta de configuração é `~/.aws/` no Linux ou no macOS e `%USERPROFILE%\aws\` no Windows.

5. Para verificar se os aliases estão funcionando, execute o `alias` a seguir.

```
$ aws whoami
```

Isso exibe a mesma resposta que o comando `aws sts get-caller-identity`:

```
{
  "Account": "012345678901",
  "UserId": "AIUAINBADX2VEG2TC6HD6",
  "Arn": "arn:aws:iam::012345678901:user/myuser"
}
```

## Recursos

- O [repositório de alias da AWS CLI](#) no GitHub contém exemplos de alias da AWS CLI criados pela equipe de desenvolvedores da AWS CLI e contribuições da comunidade da AWS CLI.
- The alias feature announcement from [AWS re:Invent 2016: The Effective AWS CLI User](#) no YouTube.
- [aws sts get-caller-identity](#)

- [aws ec2 describe-instances](#)
- [aws sns publish](#)



# Exemplos de código

Este capítulo fornece vários exemplos que mostram como usar a AWS Command Line Interface (AWS CLI) com Serviços da AWS.

Neste guia, há os seguintes tipos de exemplos da AWS CLI:

- [Exemplos de comando guiados](#): exemplos de comando guiados do Guia do usuário da AWS CLI sobre como usar a AWS CLI com alguns Serviços da AWS. Geralmente, esses exemplos são mais detalhados que os [do guia de referência da AWS CLI](#).
- [AWS CLI usando exemplos de código de script Bash](#): exemplos de script bash de código aberto. Os exemplos de script Bash estão hospedados no [Repositório de exemplos de código da AWS](#) no GitHub.

## Exemplo de feedback

Você não consegue encontrar o que precisa? Solicite um exemplo de comando usando o link Fornecer feedback na parte inferior desta página ou na respectiva página de comando [do guia de referência da AWS CLI](#).

Quer contribuir? Contribua com exemplos de comandos da AWS CLI no [Repositório de códigos de exemplos da AWS](#) no GitHub. Para obter mais informações sobre contribuição, consulte as [etapas rápidas de contribuição de códigos de exemplos da AWS CLI](#) nas páginas do GitHub.

## Exemplos de comando guiado da AWS CLI

Esta seção fornece exemplos que mostram como usar a AWS Command Line Interface (AWS CLI) para acessar vários serviços da AWS.

## Note

Para ter uma referência completa de todos os comandos disponíveis para cada serviço, consulte o [AWS CLIGuia de referência da e o](#) ou use a ajuda da linha de comando integrada. Para obter mais informações, consulte [Obter ajuda com a AWS CLI](#).

## Serviços

- [Uso do Amazon DynamoDB com a AWS CLI](#)
- [Usar o Amazon EC2 com a AWS CLI](#)
- [Usar o Amazon S3 Glacier com a AWS CLI](#)
- [Uso do AWS Identity and Access Management pela AWS CLI](#)
- [Uso do Amazon S3 com a AWS CLI](#)
- [Uso do Amazon SNS com a AWS CLI](#)

## Uso do Amazon DynamoDB com a AWS CLI

### Introdução ao Amazon DynamoDB

#### [O que é o Amazon DynamoDB?](#)

A AWS Command Line Interface (AWS CLI) oferece suporte a todos os serviços de banco de dados da AWS, incluindo o Amazon DynamoDB. Use a AWS CLI para operações imromptu, como a criação de uma tabela. Ela também pode ser usada para incorporar operações do DynamoDB em scripts utilitários.

Para obter mais informações sobre como usar a AWS CLI com o DynamoDB, consulte [dynamodb](#) na Referência de comandos da AWS CLI.

Para listar os comandos da AWS CLI para o DynamoDB, use o comando a seguir.

```
$ aws dynamodb help
```

## Tópicos

- [Pré-requisitos](#)
- [Criação e uso de tabelas do DynamoDB](#)
- [Uso do DynamoDB Local](#)
- [Recursos](#)

## Pré-requisitos

Para executar os comandos dynamodb, você precisa:

- Instale e configure a AWS CLI. Para obter mais informações, consulte [Instalar a AWS CLI](#) e [Autenticação e credenciais de acesso](#).

## Criação e uso de tabelas do DynamoDB

O formato da linha de comando consiste em um nome de comando do DynamoDB, seguido pelos parâmetros desse comando. A AWS CLI oferece suporte à [sintaxe simplificada](#) da CLI para os valores de parâmetro, assim como JSON completo.

O exemplo a seguir cria uma tabela chamada MusicCollection.

```
$ aws dynamodb create-table \  
  --table-name MusicCollection \  
  --attribute-definitions AttributeName=Artist,AttributeType=S  
  AttributeName=SongTitle,AttributeType=S \  
  --key-schema AttributeName=Artist,KeyType=HASH  
  AttributeName=SongTitle,KeyType=RANGE \  
  --provisioned-throughput ReadCapacityUnits=1,WriteCapacityUnits=1
```

Adicione novas linhas à tabela com comandos semelhantes aos mostrados no exemplo a seguir. Esses exemplos usam uma combinação de sintaxe abreviada e JSON.

```
$ aws dynamodb put-item \  
  --table-name MusicCollection \  
  --item '{  
    "Artist": {"S": "No One You Know"},  
    "SongTitle": {"S": "Call Me Today"} ,  
    "AlbumTitle": {"S": "Somewhat Famous"}  
  }' \  
  --return-consumed-capacity TOTAL  
{  
  "ConsumedCapacity": {  
    "CapacityUnits": 1.0,  
    "TableName": "MusicCollection"  
  }  
}
```

```
$ aws dynamodb put-item \
  --table-name MusicCollection \
  --item '{
    "Artist": {"S": "Acme Band"},
    "SongTitle": {"S": "Happy Day"} ,
    "AlbumTitle": {"S": "Songs About Life"}
  }' \
  --return-consumed-capacity TOTAL
{
  "ConsumedCapacity": {
    "CapacityUnits": 1.0,
    "TableName": "MusicCollection"
  }
}
```

Pode ser difícil compor um JSON válido em um comando com uma única linha. Para tornar isso mais fácil, a AWS CLI pode ler arquivos JSON. Por exemplo, considere o seguinte trecho de código JSON, que é armazenado em um arquivo chamado `expression-attributes.json`.

```
{
  ":v1": {"S": "No One You Know"},
  ":v2": {"S": "Call Me Today"}
}
```

Você pode usar esse arquivo para emitir uma solicitação query usando a AWS CLI. No exemplo a seguir, o conteúdo do arquivo `expression-attributes.json` é usado como o valor para o parâmetro `--expression-attribute-values`.

```
$ aws dynamodb query --table-name MusicCollection \
  --key-condition-expression "Artist = :v1 AND SongTitle = :v2" \
  --expression-attribute-values file://expression-attributes.json
{
  "Count": 1,
  "Items": [
    {
      "AlbumTitle": {
        "S": "Somewhat Famous"
      },
      "SongTitle": {
        "S": "Call Me Today"
      },
    },
  ],
}
```

```
        "Artist": {
            "S": "No One You Know"
        }
    ],
    "ScannedCount": 1,
    "ConsumedCapacity": null
}
```

## Uso do DynamoDB Local

Além do DynamoDB, é possível usar a AWS CLI com o DynamoDB local. O DynamoDB Local é um banco de dados e servidor pequeno no lado do cliente que copia o serviço do DynamoDB. O DynamoDB Local permite criar aplicativos que usam a API do DynamoDB sem de fato manipular tabelas ou dados no serviço da Web do DynamoDB. Em vez disso, todas as ações da API são roteadas para um banco de dados local. Isso economiza a taxa de transferência provisionada, o armazenamento de dados e as taxas de transferência de dados.

Para obter mais informações sobre o DynamoDB Local e como usá-lo com a AWS CLI, consulte as seguintes seções do [Guia do desenvolvedor do Amazon DynamoDB](#):

- [DynamoDB Local](#)
- [Como usar a AWS CLI com o DynamoDB Local](#)

## Recursos

Referência da AWS CLI:

- [aws dynamodb](#)
- [aws dynamodb create-table](#)
- [aws dynamodb put-item](#)
- [aws dynamodb query](#)

Referência do serviço:

- [DynamoDB Local](#) no Guia do desenvolvedor do Amazon DynamoDB
- [Como usar a AWS CLI com o DynamoDB Local](#) no Guia do desenvolvedor do Amazon DynamoDB

# Usar o Amazon EC2 com a AWS CLI

## Introdução ao Amazon Elastic Compute Cloud

### [Introdução ao Amazon EC2: servidor de nuvem elástico e hospedagem com a AWS](#)

Você pode acessar os recursos do Amazon Elastic Compute Cloud (Amazon EC2) usando a AWS Command Line Interface (AWS CLI). Para listar os comandos da AWS CLI para o Amazon EC2, use o comando a seguir.

```
aws ec2 help
```

Antes de executar quaisquer comandos, defina suas credenciais padrão. Para obter mais informações, consulte [Configurar o AWS CLI](#).

Este tópico mostra exemplos curtos de comandos da AWS CLI que executam tarefas comuns para o Amazon EC2.

Para exemplos de formas longas de comandos da AWS CLI, consulte [Repositório de exemplos de código da AWS CLI](#) no GitHub.

## Tópicos

- [Criar, exibir e excluir pares de chaves do Amazon EC2](#)
- [Criar, configurar e excluir grupos de segurança para o Amazon EC2](#)
- [Inicializar, listar e encerrar instâncias do Amazon EC2](#)
- [Alterar um tipo de instância do Amazon EC2 com um script bash](#)

## Criar, exibir e excluir pares de chaves do Amazon EC2

Você pode usar a AWS Command Line Interface (AWS CLI) para criar, exibir e excluir seus pares de chaves do Amazon Elastic Compute Cloud (Amazon EC2). Você usa pares de chaves para se conectar a uma instância do Amazon EC2.

Você deve fornecer o par de chaves para o AmazonEC2 ao criar a instância e, em seguida, usar esse par de chaves para autenticar ao se conectar à instância.

 Note

Para obter exemplos de outros comandos, consulte o [AWS CLI Guia de referência da](#) .

## Tópicos

- [Pré-requisitos](#)
- [Criar um par de chaves](#)
- [Exibir o par de chaves](#)
- [Excluir o par de chaves](#)
- [Referências](#)

## Pré-requisitos

Para executar os comandos `ec2`, você precisa:

- Instale e configure a AWS CLI. Para obter mais informações, consulte [Instalar a AWS CLI](#) e [Autenticação e credenciais de acesso](#).
- Defina suas permissões do IAM para permitir acesso ao Amazon EC2. Para obter mais informações sobre permissões do IAM para Amazon EC2, consulte [Políticas do IAM para Amazon EC2](#) no Manual do usuário do Amazon EC2 para instâncias do Linux.

## Criar um par de chaves

Para criar um par de chaves, use o comando `aws ec2 create-key-pair` com a opção `--query` e a opção `--output text` para canalizar sua chave privada diretamente para um arquivo.

```
$ aws ec2 create-key-pair --key-name MyKeyPair --query 'KeyMaterial' --output text  
> MyKeyPair.pem
```

Para o PowerShell, o padrão de redirecionamento `> file` assume a codificação UTF-8, que não pode ser usada com alguns clientes SSH. Portanto, você deve converter a saída redirecionando-a para o comando `out-file` e definir explicitamente a codificação para `ascii`.

```
PS C:\>aws ec2 create-key-pair --key-name MyKeyPair --query 'KeyMaterial' --output text  
| out-file -encoding ascii -filepath MyKeyPair.pem
```

O arquivo resultante `MyKeyPair.pem` é semelhante ao seguinte.

```
-----BEGIN RSA PRIVATE KEY-----
EXAMPLEKEYKCAQEAY7WZhaDsR1W3mRlQtvhwyORRX8gnxgDAfRt/gx42kWXsT4rXE/b5CpSgie/
vBoU7jLxx92pNHoFnByP+Dc21eyyz6CvjTmWA0JwfWiW5/akH7i05dSrvC7dQkW2duV5QuUdEQW
Z/aNxMniGQE6XAgfwlnXVBwrerrQo+ZWQeqiUwwMkuEbLeJFLhMCvYURpUMSC1oehm449ilx9X1F
G50TCFe0zf18dqqCP6GzbPaIjiU19xX/az0R9V+tpU0zEL+wmXnZt3/nHPQ5xvD20JH67km6SuPW
oPzev/D8V+x4+bHthfSjR9Y7DvQFjfbVwHXigBdtZcU2/wei8D/HYwIDAQABAoIBAGZ1kaEvnrrqu
/uler7vgIn5m7lN5LKw4hJLAIW6tUT/fzvtcHK0SkbQCQXuriHmQ2MQyJX/0kn2NfjLV/ufGxbL1
mb5qwMGUnEpJaZD6QSSs3kICLwWUYUiGfc0uiSbmJoap/GTLU0W5Mfcv36PaBUNy5p53V6G7hXb2
bahyWyJNfjLe4M86yd2YK3V2CmK+X/B0sShnJ36+hjrXPPWmV3N9zEmCdJjA+K15DYmhm/tJWSD9
81oGk9TopEp7CkIfatEATyyZiVqoRq6k64iuM9JkA30zdXzMqexXVJ1TLZVEH0E7bh1Y9d801ozR
oQs/FiZNAx2iijCWyv0lpjE73+kCgYEA9mZtyhkHkFDpwrSM1APaL8oNAbbjwEy7Z5Mqfq1+lIp1
YkriL0DbLXlvRAH+yHPRit2hH0jtUNZh4Axv+cpG09qbUI3+43eEy24B7G/Uh+GTfbjsXs0xQx/x
p9otyVwc7hsQ5TA5PZb+mvkJ50BEKzet9XcKw0NBVELGhnEPe7cCgYEA06Vgov6YHleHui9kHuws
ayav0elc5zkxjF9nfHFJRry21R1trw2Vdpn+9g481URipzWV0Eihvm+xTtmaZ1Sp//lkq75XDwnU
WA8gkn603QE3fq2yN98BURsAKdJfJ5RL1HvGQvTe10HLYXpJnEkHv+Unl2ajLivWUt5pbBrKbUC
gYBjb0+0Zk0sCcpZ29sbzjYjpIddErySIyRX5gV2uNQwAjlDp9PfN295yQ+BxMBXiIycWVQiw0bH
oMo7yykABY70zd5wQewBQ4AdSlWSX4nGDtsiFxiI5sKuAAe0CbTosy1s8w8fxoJ5Tz1sdoxNeGs
Arq6Wv/G16zQuAE9zK9vwwKBgF+09VI/1wJBirsDGz9whVwFFPrTkJNvJZzYt69qezx1sjgFKshy
WBhd4xHZtmCqpBP1AymEjr/T0lboxyARMXMnIOWIANXMGb4KGSyl1mzSVAoQ+fqr+cJ3d0dyP1lj
jjb0Ed/NY8fr1NDxAVHE8BSkdsx2f6ELEyBKJSRr9snRAoGAMrTwYneXzvTskF/S5Fyu0i0egLda
NWUH38v/nDCgEpIXD5Hn3qAEcju1IjmbwlvTW+nY2jVhv7UGd8MjwUTNGItbdb6nsYqM2asrnF3qS
VRkAKKKYegJkpUfVTTrW0YFjXkfcR/V+QFL50ndHAKJXjW7a4ejJLncTzmZSpYzwApc=
-----END RSA PRIVATE KEY-----
```

Sua chave privada não é armazenada na AWS e só pode ser recuperada quando é criada. Não será possível recuperá-la posteriormente. Ao invés disso, se você perder a chave privada, deverá criar um novo par de chaves.

Se você estiver se conectando à sua instância a partir de um computador com Linux, recomendamos que você use o seguinte comando para definir as permissões do arquivo de chave privada, de maneira que apenas você possa lê-lo.

```
$ chmod 400 MyKeyPair.pem
```

Exibir o par de chaves

A “impressão digital” é gerada a partir do par de chaves, e você pode usá-la para verificar se a chave privada presente em sua máquina local corresponde à chave pública armazenada na AWS.



A impressão digital é um hash SHA1 de uma cópia codificada DER da chave privada. Esse valor é capturado quando o par de chaves é criado e é armazenado na AWS com a chave pública. Você pode visualizar a impressão digital no console do Amazon EC2 ou executando o comando [aws ec2 describe-key-pairs](#) da AWS CLI.

O exemplo a seguir exibe a impressão digital para MyKeyPair.

```
$ aws ec2 describe-key-pairs --key-name MyKeyPair
{
  "KeyPairs": [
    {
      "KeyName": "MyKeyPair",
      "KeyFingerprint":
"1f:51:ae:28:bf:89:e9:d8:1f:25:5d:37:2d:7d:b8:ca:9f:f5:f1:6f"
    }
  ]
}
```

Para obter mais informações sobre chaves e impressões digitais, consulte [Pares de chaves do Amazon EC2](#) no Manual do usuário do Amazon EC2 para instâncias do Linux.

Excluir o par de chaves

Para excluir um par de chaves, execute o comando [aws ec2 delete-key-pair](#), substituindo *MyKeyPair* pelo nome do par a ser excluído.

```
$ aws ec2 delete-key-pair --key-name MyKeyPair
```

## Referências

Referência da AWS CLI:

- [aws ec2](#)
- [aws ec2 create-key-pair](#)
- [aws ec2 delete-key-pair](#)
- [aws ec2 describe-key-pairs](#)

Outra referência:

- [Documentação do Amazon Elastic Compute Cloud](#)

- Para visualizar e contribuir para o SDK da AWS e exemplos de código da AWS CLI, consulte o [Repositório de exemplos de código da AWS](#) no GitHub.

## Criar, configurar e excluir grupos de segurança para o Amazon EC2

Você pode criar um grupo de segurança para suas instâncias do Amazon Elastic Compute Cloud (Amazon EC2) que atua essencialmente como um firewall, com regras que determinam qual tráfego de rede pode entrar e sair.

É possível usar a AWS Command Line Interface (AWS CLI) para criar um grupo de segurança, adicionar regras a grupos de segurança existentes e excluir grupos de segurança.

### Note

Para obter exemplos de outros comandos, consulte o [AWS CLIGuia de referência da](#) .

### Tópicos

- [Pré-requisitos](#)
- [Crie um grupo de segurança](#)
- [Adicionar regras ao grupo de segurança](#)
- [Excluir o grupo de segurança](#)
- [Referências](#)

### Pré-requisitos

Para executar os comandos `ec2`, você precisa:

- Instale e configure a AWS CLI. Para obter mais informações, consulte [Instalar a AWS CLI](#) e [Autenticação e credenciais de acesso](#).
- Defina suas permissões do IAM para permitir acesso ao Amazon EC2. Para obter mais informações sobre permissões do IAM para Amazon EC2, consulte [Políticas do IAM para Amazon EC2](#) no Manual do usuário do Amazon EC2 para instâncias do Linux.

### Crie um grupo de segurança

É possível criar grupos de segurança associados a nuvens privadas virtuais (VPCs).

O exemplo do [aws ec2 create-security-group](#) a seguir mostra como criar um grupo de segurança para a VPC especificada.

```
$ aws ec2 create-security-group --group-name my-sg --description "My security group" --  
vpc-id vpc-1a2b3c4d  
{  
  "GroupId": "sg-903004f8"  
}
```

Para visualizar as informações iniciais para um grupo de segurança, execute o comando [aws ec2 describe-security-groups](#). Você pode fazer referência a um grupo de segurança do EC2-VPC somente por seu vpc-id, e não apenas por seu nome.

```
$ aws ec2 describe-security-groups --group-ids sg-903004f8  
{  
  "SecurityGroups": [  
    {  
      "IpPermissionsEgress": [  
        {  
          "IpProtocol": "-1",  
          "IpRanges": [  
            {  
              "CidrIp": "0.0.0.0/0"  
            }  
          ],  
          "UserIdGroupPairs": []  
        }  
      ],  
      "Description": "My security group"  
      "IpPermissions": [],  
      "GroupName": "my-sg",  
      "VpcId": "vpc-1a2b3c4d",  
      "OwnerId": "123456789012",  
      "GroupId": "sg-903004f8"  
    }  
  ]  
}
```

## Adicionar regras ao grupo de segurança

Quando você executa uma instância do Amazon EC2, é necessário ativar regras no grupo de segurança para permitir o tráfego de rede de entrada como meio de se conectar à imagem.

Por exemplo, se estiver iniciando uma instância do Windows, normalmente você adiciona uma regra para permitir que o tráfego de entrada na porta TCP 3389 ofereça suporte ao Protocolo de Desktop Remoto (RDP). Se estiver iniciando uma instância do Linux, geralmente você adiciona uma regra para permitir que o tráfego de entrada na porta 22 ofereça suporte às conexões SSH.

Use o comando [aws ec2 authorize-security-group-ingress](#) para adicionar uma regra a um grupo de segurança. Um parâmetro obrigatório deste comando é o endereço IP público de seu computador ou rede (na forma de um intervalo de endereços) ao qual seu computador está conectado na notação [CIDR](#).

#### Note

Fornecemos o serviço <https://checkip.amazonaws.com/> para permitir a você determinar seu endereço IP público. Para encontrar outros serviços que podem ajudar você a identificar o endereço IP, use o navegador para pesquisar “qual é meu endereço IP”. Se você se conectar por meio de um ISP ou protegido por um firewall usando um endereço IP dinâmico (por meio de um gateway NAT de uma rede privada), seu endereço poderá mudar periodicamente. Nesse caso, você deve descobrir o intervalo de endereços IP usados por computadores cliente.

O exemplo a seguir mostra como adicionar uma regra para RDP (porta TCP 3389) em um grupo de segurança do EC2-VPC com o ID `sg-903004f8` usando seu endereço IP.

Para começar, encontre seu endereço IP.

```
$ curl https://checkip.amazonaws.com
x.x.x.x
```

Em seguida, você pode adicionar o endereço IP ao seu grupo de segurança, executando o comando [aws ec2 authorize-security-group-ingress](#).

```
$ aws ec2 authorize-security-group-ingress --group-id sg-903004f8 --protocol tcp --port 3389 --cidr x.x.x.x/x
```

O comando a seguir adiciona uma regra para habilitar o SSH para instâncias no mesmo grupo de segurança.

```
$ aws ec2 authorize-security-group-ingress --group-id sg-903004f8 --protocol tcp --port 22 --cidr x.x.x.x/x
```

Para visualizar as alterações no grupo de segurança, execute o comando [aws ec2 describe-security-groups](#).

```
$ aws ec2 describe-security-groups --group-ids sg-903004f8
{
  "SecurityGroups": [
    {
      "IpPermissionsEgress": [
        {
          "IpProtocol": "-1",
          "IpRanges": [
            {
              "CidrIp": "0.0.0.0/0"
            }
          ],
          "UserIdGroupPairs": []
        }
      ],
      "Description": "My security group"
      "IpPermissions": [
        {
          "ToPort": 22,
          "IpProtocol": "tcp",
          "IpRanges": [
            {
              "CidrIp": "x.x.x.x/x"
            }
          ],
          "UserIdGroupPairs": [],
          "FromPort": 22
        }
      ],
      "GroupName": "my-sg",
      "OwnerId": "123456789012",
      "GroupId": "sg-903004f8"
    }
  ]
}
```

## Excluir o grupo de segurança

Para excluir um grupo de segurança, execute o comando [aws ec2 delete-security-group](#).

### Note

Não é possível excluir um grupo de segurança se ele estiver atualmente anexado a um ambiente.

O exemplo de comando a seguir exclui um grupo de segurança do EC2-VPC.

```
$ aws ec2 delete-security-group --group-id sg-903004f8
```

## Referências

Referência da AWS CLI:

- [aws ec2](#)
- [aws ec2 authorize-security-group-ingress](#)
- [aws ec2 create-security-group](#)
- [aws ec2 delete-security-group](#)
- [aws ec2 describe-security-groups](#)

Outra referência:

- [Documentação do Amazon Elastic Compute Cloud](#)
- Para visualizar e contribuir para o SDK da AWS e exemplos de código da AWS CLI, consulte o [Repositório de exemplos de código da AWS](#) no GitHub.

## Inicializar, listar e encerrar instâncias do Amazon EC2

Você pode usar a AWS Command Line Interface (AWS CLI) para iniciar, listar e encerrar instâncias do Amazon Elastic Compute Cloud (Amazon EC2). Se iniciar uma instância que não esteja no nível gratuito da AWS, você será cobrado depois de iniciar a instância e cobrado pelo tempo que a instância estiver em execução, mesmo que ela permaneça inativa.

 Note

Para obter exemplos de outros comandos, consulte o [AWS CLI Guia de referência da](#) .

## Tópicos

- [Pré-requisitos](#)
- [Executar sua instância](#)
- [Adicionar um dispositivo de blocos à instância](#)
- [Adicionar uma etiqueta à instância](#)
- [Conecte-se à sua instância](#)
- [Listar as instâncias](#)
- [Encerrar a instância](#)
- [Referências](#)

## Pré-requisitos

Para executar os comandos ec2 neste tópico, você precisa:

- Instale e configure a AWS CLI. Para obter mais informações, consulte [Instalar a AWS CLI](#) e [Autenticação e credenciais de acesso](#).
- Defina suas permissões do IAM para permitir acesso ao Amazon EC2. Para obter mais informações sobre permissões do IAM para Amazon EC2, consulte [Políticas do IAM para Amazon EC2](#) no Manual do usuário do Amazon EC2 para instâncias do Linux.
- Crie um [par de chaves](#) e um [grupo de segurança](#).
- Selecione uma Imagem de máquina da Amazon (AMI) e anote o ID da AMI. Para obter mais informações, consulte [Como encontrar uma AMI](#) no Manual do usuário do Amazon EC2 para instâncias do Linux.

## Executar sua instância

Para iniciar uma instância do Amazon EC2 usando a AMI selecionada, use o comando [aws ec2 run-instances](#). É possível iniciar a instância em uma nuvem privada virtual (VPC).

Inicialmente, a instância será exibida no estado `pending`, mas mudará para o estado `running` depois de alguns minutos.

O exemplo a seguir mostra como iniciar uma instância `t2.micro` na sub-rede especificada de uma VPC. Substitua os valores dos parâmetros *em itálico* pelos seus próprios.

```
$ aws ec2 run-instances --image-id ami-xxxxxxx --count 1 --instance-type t2.micro --
key-name MyKeyPair --security-group-ids sg-903004f8 --subnet-id subnet-6e7f829e
{
  "OwnerId": "123456789012",
  "ReservationId": "r-5875ca20",
  "Groups": [
    {
      "GroupName": "my-sg",
      "GroupId": "sg-903004f8"
    }
  ],
  "Instances": [
    {
      "Monitoring": {
        "State": "disabled"
      },
      "PublicDnsName": null,
      "Platform": "windows",
      "State": {
        "Code": 0,
        "Name": "pending"
      },
      "EbsOptimized": false,
      "LaunchTime": "2013-07-19T02:42:39.000Z",
      "PrivateIpAddress": "10.0.1.114",
      "ProductCodes": [],
      "VpcId": "vpc-1a2b3c4d",
      "InstanceId": "i-5203422c",
      "ImageId": "ami-173d747e",
      "PrivateDnsName": "ip-10-0-1-114.ec2.internal",
      "KeyName": "MyKeyPair",
      "SecurityGroups": [
        {
          "GroupName": "my-sg",
          "GroupId": "sg-903004f8"
        }
      ]
    }
  ],
}
```



```
"ClientToken": null,
"SubnetId": "subnet-6e7f829e",
"InstanceType": "t2.micro",
"NetworkInterfaces": [
  {
    "Status": "in-use",
    "SourceDestCheck": true,
    "VpcId": "vpc-1a2b3c4d",
    "Description": "Primary network interface",
    "NetworkInterfaceId": "eni-a7edb1c9",
    "PrivateIpAddresses": [
      {
        "PrivateDnsName": "ip-10-0-1-114.ec2.internal",
        "Primary": true,
        "PrivateIpAddress": "10.0.1.114"
      }
    ],
    "PrivateDnsName": "ip-10-0-1-114.ec2.internal",
    "Attachment": {
      "Status": "attached",
      "DeviceIndex": 0,
      "DeleteOnTermination": true,
      "AttachmentId": "eni-attach-52193138",
      "AttachTime": "2013-07-19T02:42:39.000Z"
    },
    "Groups": [
      {
        "GroupName": "my-sg",
        "GroupId": "sg-903004f8"
      }
    ],
    "SubnetId": "subnet-6e7f829e",
    "OwnerId": "123456789012",
    "PrivateIpAddress": "10.0.1.114"
  }
],
"SourceDestCheck": true,
"Placement": {
  "Tenancy": "default",
  "GroupName": null,
  "AvailabilityZone": "us-west-2b"
},
"Hypervisor": "xen",
"BlockDeviceMappings": [
```

```

        {
            "DeviceName": "/dev/sda1",
            "Ebs": {
                "Status": "attached",
                "DeleteOnTermination": true,
                "VolumeId": "vol-877166c8",
                "AttachTime": "2013-07-19T02:42:39.000Z"
            }
        },
        "Architecture": "x86_64",
        "StateReason": {
            "Message": "pending",
            "Code": "pending"
        },
        "RootDeviceName": "/dev/sda1",
        "VirtualizationType": "hvm",
        "RootDeviceType": "ebs",
        "Tags": [
            {
                "Value": "MyInstance",
                "Key": "Name"
            }
        ],
        "AmiLaunchIndex": 0
    }
]
}

```

## Adicionar um dispositivo de blocos à instância

Cada instância lançada tem um volume do dispositivo root associados. Use o mapeamento de dispositivos de blocos para especificar volumes adicionais do Amazon Elastic Block Store (Amazon EBS) ou volumes de armazenamento de instâncias para anexar a uma instância quando ela for iniciada.

Para adicionar um dispositivo de blocos em sua instância, especifique a opção `--block-device-mappings` ao usar `run-instances`.

O exemplo de parâmetro a seguir provisiona um volume do Amazon EBS padrão de 20 GB e o mapeia em sua instância usando o identificador `/dev/sdf`.

```
--block-device-mappings "[{\"DeviceName\":\"/dev/sdf\", \"Ebs\":{\"VolumeSize\":20, \"DeleteOnTermination\":false} }]"
```

O exemplo a seguir adiciona um volume do Amazon EBS mapeado em `/dev/sdf` com base em um snapshot existente. Um snapshot representa uma imagem que é carregada no volume para você. Ao especificar um snapshot, não é necessário especificar um tamanho de volume; ele será grande o suficiente para armazenar sua imagem. No entanto, se você especificar um tamanho, ele deverá ser maior que ou igual ao tamanho do snapshot.

```
--block-device-mappings "[{\"DeviceName\":\"/dev/sdf\", \"Ebs\":{\"SnapshotId\":\"snap-a1b2c3d4\"} }]"
```

O exemplo a seguir adiciona dois volumes à sua instância. O número de volumes disponíveis para sua instância depende do seu tipo de instância.

```
--block-device-mappings "[{\"DeviceName\":\"/dev/sdf\", \"VirtualName\":\"ephemeral0\"}, {\"DeviceName\":\"/dev/sdg\", \"VirtualName\":\"ephemeral1\"}]"
```

O exemplo a seguir cria o mapeamento (`/dev/sdj`), mas não provisiona um volume para a instância.

```
--block-device-mappings "[{\"DeviceName\":\"/dev/sdj\", \"NoDevice\":\"\"}]"
```

Para obter mais informações, consulte [Mapeamento de dispositivos de blocos](#) no Guia do usuário do Amazon EC2 para instâncias Linux.

## Adicionar uma etiqueta à instância

Uma etiqueta é um rótulo atribuído a um recurso da AWS. Permite adicionar metadados aos seus recursos que você pode usar para diversas finalidades. Para obter mais informações, consulte [Marcação de recursos](#) no Manual do usuário do Amazon EC2 para instâncias do Linux.

O exemplo a seguir mostra como adicionar uma etiqueta com o nome da chave “Name” e o valor “MyInstance” à instância especificada com o comando [aws ec2 create-tags](#).

```
$ aws ec2 create-tags --resources i-5203422c --tags Key=Name,Value=MyInstance
```

## Conecte-se à sua instância

Durante a execução da instância, é possível se conectar a ela e usá-la da mesma forma que você usaria um computador. Para obter mais informações, consulte [Conectar à sua instância do Amazon EC2](#) no Guia do usuário do Amazon EC2 para instâncias do Linux.

## Listar as instâncias

Use o AWS CLI para listar suas instâncias e visualizar informações sobre elas. Liste todas as suas instâncias, ou filtre os resultados de acordo com as instâncias de interesse.

Os exemplos a seguir mostram como usar o comando [aws ec2 describe-instances](#).

O comando a seguir relaciona todas as suas instâncias.

```
$ aws ec2 describe-instances
```

O comando a seguir filtra a lista apenas para suas instâncias t2.micro e mostra apenas os valores InstanceId para cada correspondência.

```
$ aws ec2 describe-instances --filters "Name=instance-type,Values=t2.micro" --query  
"Reservations[].Instances[].InstanceId"  
[  
    "i-05e998023d9c69f9a"  
]
```

O comando a seguir lista todas as suas instâncias que tem a etiqueta Name=MyInstance.

```
$ aws ec2 describe-instances --filters "Name=tag:Name,Values=MyInstance"
```

O comando a seguir relaciona as instâncias que foram executadas usando qualquer uma das seguintes AMIs: ami-x0123456, ami-y0123456 e ami-z0123456.

```
$ aws ec2 describe-instances --filters "Name=image-id,Values=ami-x0123456,ami-  
y0123456,ami-z0123456"
```

## Encerrar a instância

O encerramento de uma instância significa excluí-la. Você não pode se reconectar com uma instância depois de tê-la encerrado.

Assim que o estado da instância de mudar para shutting-down ou para terminated, não há mais custos para essa instância. Se você desejar se reconectar a ela mais tarde, use [stop-instances](#) em vez de `terminate-instances`. Para obter mais informações, consulte [Terminar a instância](#) no Guia do usuário do Amazon EC2 para instâncias do Linux.

Para excluir uma instância, use o comando [aws ec2 terminate-instances](#).

```
$ aws ec2 terminate-instances --instance-ids i-5203422c
{
  "TerminatingInstances": [
    {
      "InstanceId": "i-5203422c",
      "CurrentState": {
        "Code": 32,
        "Name": "shutting-down"
      },
      "PreviousState": {
        "Code": 16,
        "Name": "running"
      }
    }
  ]
}
```

## Referências

Referência da AWS CLI:

- [aws ec2](#)
- [aws ec2 create-tags](#)
- [aws ec2 describe-instances](#)
- [aws ec2 run-instances](#)
- [aws ec2 terminate-instances](#)

Outra referência:

- [Documentação do Amazon Elastic Compute Cloud](#)
- Para visualizar e contribuir para o SDK da AWS e exemplos de código da AWS CLI, consulte o [Repositório de exemplos de código da AWS](#) no GitHub.

## Alterar um tipo de instância do Amazon EC2 com um script bash

Este exemplo de script bash para Amazon EC2 altera o tipo de instância para uma instância do Amazon EC2 usando a AWS Command Line Interface (AWS CLI). Ele interrompe a instância se ela estiver em execução, altera o tipo de instância e, em seguida, se solicitado, a reinicia. Scripts shell são programas desenvolvidos para ser executados em uma interface de linha de comando.

### Note

Para obter exemplos de outros comandos, consulte o [AWS CLI Guia de referência da](#) .

### Tópicos

- [Antes de começar](#)
- [Sobre este exemplo](#)
- [Parâmetros](#)
- [Arquivos](#)
- [Referências](#)

### Antes de começar

Antes que você possa executar qualquer um dos exemplos abaixo, as seguintes tarefas deverão ser concluídas.

- Instale e configure a AWS CLI. Para obter mais informações, consulte [Instalar a AWS CLI](#) e [Autenticação e credenciais de acesso](#).
- O perfil utilizado deve ter permissões que garantem as operações da AWS realizadas pelos exemplos.
- Uma instância do Amazon EC2 em execução na conta para a qual você tem permissão para interromper e modificar. Se você executar o script de teste, ele iniciará uma instância para você, testará alterando o tipo e encerrará a instância.
- Como prática recomendada da AWS, conceda a esse código privilégio mínimo ou apenas as permissões necessárias para executar uma tarefa. Para obter mais informações, consulte [Conceder privilégio mínimo](#) no Manual do usuário do AWS Identity and Access Management (IAM).

- Este código não foi testado em todas as regiões da AWS. Alguns serviços da AWS só estão disponíveis em regiões específicas. Para obter mais informações, consulte [Endpoints de serviço e cotas](#) no Guia de referência geral da AWS.
- Executar este código pode resultar em cobranças em sua conta da AWS. É sua responsabilidade garantir que todos os recursos criados por este script sejam removidos após você terminar de usá-los.

## Sobre este exemplo

Este exemplo é escrito como uma função no arquivo de script shell `change_ec2_instance_type.sh` que você pode usar como `source` a partir de outro script ou da linha de comando. Cada arquivo de script contém comentários descrevendo cada uma das funções. Uma vez que a função esteja na memória, você poderá chamá-la via linha de comando. Por exemplo, os seguintes comandos alteram o tipo de instância especificada para `t2.nano`:

```
$ source ./change_ec2_instance_type.sh
$ ./change_ec2_instance_type -i *instance-id* -t new-type
```

Para obter o exemplo completo e arquivos de script para download, consulte [Alterar tipo de instância do Amazon EC2](#) no Repositório de exemplos de código da AWS no GitHub.

## Parâmetros

`-i:` (string) especifica o ID da instância a ser modificada.

`-t:` (string) especifica o tipo de instância do Amazon EC2 de destino.

`-r:` (opção) por padrão, não está definida. Se `-r` estiver definido, reiniciará a instância após a alteração do tipo.

`-f:` (opção) por padrão, o script solicita ao usuário que confirme o encerramento da instância antes de fazer a mudança. Se `-f` estiver definida, a função não avisará o usuário antes de encerrar a instância para realizar a alteração de tipo.

`-v:` (opção) por padrão, o script opera silenciosamente e exibe a saída somente em caso de erro. Se `-v` estiver definida, a função exibirá o status ao longo de sua operação.

## Arquivos

### **change\_ec2\_instance\_type.sh**

O arquivo de script principal contém a função `change_ec2_instance_type()` que executa as seguintes tarefas:

- Verifica se a instância do Amazon EC2 especificada existe.
- A menos que a opção `-f` esteja selecionada, avisará o usuário antes de interromper a instância.
- Altera o tipo de instância
- Se você definir `-r`, reiniciará a instância e confirmará que a instância está sendo executada

Visualize o código para [change\\_ec2\\_instance\\_type.sh](#) no GitHub.

### **test\_change\_ec2\_instance\_type.sh**

O script do arquivo `change_ec2_instance_type_test.sh` testa os vários caminhos de código para a função `change_ec2_instance_type`. Se todas as etapas no script de teste funcionarem corretamente, ele removerá todos os recursos que criou.

Você pode executar o script de teste com os seguintes parâmetros:

- `-v`: (opção) cada teste mostra um status de aprovação/falha conforme eles são executados. Por padrão, os testes são executados silenciosamente e a saída inclui apenas o status final de aprovação/falha.
- `-i`: (opção) o script faz uma pausa após cada teste para permitir que você navegue pelos resultados intermediários de cada etapa. Isso permite examinar o status atual da instância usando o console do Amazon EC2. O script avançará para a próxima etapa após você pressionar ENTER no prompt.

Visualize o código para [test\\_change\\_ec2\\_instance\\_type.sh](#) no GitHub.

### **awsdocs\_general.sh**

O arquivo de script `awsdocs_general.sh` contém funções de uso geral usadas em exemplos avançados para a AWS CLI.

Visualize o código para [awsdocs\\_general.sh](#) no GitHub.



## Referências

### Referência da AWS CLI:

- [aws ec2](#)
- [aws ec2 describe-instances](#)
- [aws ec2 modify-instance-attribute](#)
- [aws ec2 start-instances](#)
- [aws ec2 stop-instances](#)
- [aws ec2 wait instance-running](#)
- [aws ec2 wait instance-stopped](#)

### Outra referência:

- [Documentação do Amazon Elastic Compute Cloud](#)
- Para visualizar e contribuir para o SDK da AWS e exemplos de código da AWS CLI, consulte o [Repositório de exemplos de código da AWS](#) no GitHub.

## Usar o Amazon S3 Glacier com a AWS CLI

### Introdução ao Amazon S3 Glacier

#### [Introdução ao Amazon S3 Glacier](#)

Este tópico mostra exemplos de comandos da AWS CLI que executam tarefas comuns para o S3 Glacier. Os exemplos demonstram como usar a AWS CLI para fazer upload de um arquivo grande para o S3 Glacier dividindo-o em partes menores e fazendo upload a partir da linha de comando.

Você pode acessar os recursos do Amazon S3 Glacier usando a AWS Command Line Interface (AWS CLI). Para listar os comandos da AWS CLI para o S3 Glacier, use o comando a seguir.

```
aws glacier help
```

 Note

Para obter referência de comandos e exemplos adicionais, consulte [aws glacier](#) na Referência de comandos da AWS CLI.

## Tópicos

- [Pré-requisitos](#)
- [Criar um cofre do Amazon S3 Glacier](#)
- [Preparar um arquivo para upload](#)
- [Iniciar um multipart upload e fazer upload de arquivos](#)
- [Concluir o upload](#)
- [Recursos](#)

## Pré-requisitos

Para executar os comandos `glacier`, você precisa:

- Instale e configure a AWS CLI. Para obter mais informações, consulte [Instalar a AWS CLI](#) e [Autenticação e credenciais de acesso](#).
- Este tutorial usa várias ferramentas de linha de comando que normalmente vêm pré-instaladas em sistemas operacionais Unix, incluindo Linux e macOS. Os usuários do Windows podem usar as mesmas ferramentas instalando o [Cygwin](#) e executando os comandos do terminal Cygwin. São observados comandos originários do Windows e utilitários que executam as mesmas funções onde disponível.

## Criar um cofre do Amazon S3 Glacier

Crie um cofre com o [create-vault](#).

```
$ aws glacier create-vault --account-id - --vault-name myvault
{
  "location": "/123456789012/vaults/myvault"
}
```

 Note

Todos os comandos do S3 Glacier exigem um parâmetro de ID de conta. Use o caractere de hífen (`--account-id -`) para usar a conta atual.

## Preparar um arquivo para upload

Crie um arquivo para o upload de teste. Os comandos a seguir criam um arquivo com o nome *largefile* com exatamente 3 MiB de dados aleatórios.

Linux ou macOS

```
$ dd if=/dev/urandom of=largefile bs=3145728 count=1
1+0 records in
1+0 records out
3145728 bytes (3.1 MB) copied, 0.205813 s, 15.3 MB/s
```

dd é um utilitário que copia um número de bytes a partir de um arquivo de entrada para um arquivo de saída. O exemplo anterior usa o arquivo de dispositivo do sistema `/dev/urandom` como uma fonte de dados aleatórios. O `fsutil` executa uma função semelhante no Windows.

Windows

```
C:\> fsutil file createnew largefile 3145728
File C:\temp\largefile is created
```

Em seguida, divida o arquivo em blocos de 1 MiB (1.048.576 bytes).

```
$ split -b 1048576 --verbose largefile chunk
creating file `chunkaa'
creating file `chunkab'
creating file `chunkac'
```

 Note

[HJ-Split](#) é um divisor de arquivos gratuito para Windows e muitas outras plataformas.

## Iniciar um multipart upload e fazer upload de arquivos

Crie um upload fracionado no Amazon S3 Glacier usando o comando [initiate-multipart-upload](#).

```
$ aws glacier initiate-multipart-upload --account-id - --archive-description "multipart
upload test" --part-size 1048576 --vault-name myvault
{
  "uploadId": "19gaRezEXAMPLES6Ry5YYdqthHOC_kGRCT03L9yetr220UmPtBYKk-
0ssZtLqyFu7sY1_1R7vgFuJV6NtcV5zpsJ",
  "location": "/123456789012/vaults/myvault/multipart-
uploads/19gaRezEXAMPLES6Ry5YYdqthHOC_kGRCT03L9yetr220UmPtBYKk-
0ssZtLqyFu7sY1_1R7vgFuJV6NtcV5zpsJ"
}
```

O S3 Glacier exige o tamanho de cada parte em bytes (1 MiB neste exemplo), o nome do cofre e uma ID de conta para configurar o upload fracionado. A AWS CLI gera um ID de upload quando a operação é concluída. Salve o ID de upload para um shell variável para uso posterior.

Linux ou macOS

```
$ UPLOADID="19gaRezEXAMPLES6Ry5YYdqthHOC_kGRCT03L9yetr220UmPtBYKk-
0ssZtLqyFu7sY1_1R7vgFuJV6NtcV5zpsJ"
```

Windows

```
C:\> set UPLOADID="19gaRezEXAMPLES6Ry5YYdqthHOC_kGRCT03L9yetr220UmPtBYKk-
0ssZtLqyFu7sY1_1R7vgFuJV6NtcV5zpsJ"
```

Depois, use o comando [upload-multipart-part](#) para fazer upload das três partes.

```
$ aws glacier upload-multipart-part --upload-id $UPLOADID --body chunkaa --range 'bytes
0-1048575/*' --account-id - --vault-name myvault
{
  "checksum": "e1f2a7cd6e047fa606fe2f0280350f69b9f8cfa602097a9a026360a7edc1f553"
}
$ aws glacier upload-multipart-part --upload-id $UPLOADID --body chunkab --range 'bytes
1048576-2097151/*' --account-id - --vault-name myvault
{
  "checksum": "e1f2a7cd6e047fa606fe2f0280350f69b9f8cfa602097a9a026360a7edc1f553"
```

```
}  
$ aws glacier upload-multipart-part --upload-id $UPLOADID --body chunkac --range 'bytes  
2097152-3145727/*' --account-id - --vault-name myvault  
{  
  "checksum": "e1f2a7cd6e047fa606fe2f0280350f69b9f8cfa602097a9a026360a7edc1f553"  
}
```

#### Note

O exemplo anterior usa o sinal de dólar (\$) para fazer referência ao conteúdo da variável de shell UPLOADID no Linux. Na linha de comando do Windows, use um sinal de porcentagem (%) nos dois lados do nome da variável (por exemplo, %UPLOADID%).

Especifique o intervalo de bytes de cada parte ao fazer upload para que o S3 Glacier possa remontá-lo na ordem correta. Cada parte é 1.048.576 bytes, portanto, a primeira parte ocupa 0-1048575 bytes, a segunda 1048576-2097151 e a terceira 2097152-3145727.

## Concluir o upload

O Amazon S3 Glacier exige uma árvore hash do arquivo original para confirmar que todas as partes carregadas chegaram à AWS intactas.

Para calcular uma árvore hash, você deve dividir o arquivo em partes de 1 MiB e calcular um binário SHA-256 hash de cada item. Em seguida, divida a lista de hashes em pares, combine os dois binários hashes em cada par e execute hashes dos resultados. Repita esse processo até que haja apenas um hash à esquerda. Se houver um número ímpar de hashes em qualquer nível, envie para o próximo nível sem modificá-lo.

A chave para calcular uma árvore hash corretamente ao usar os utilitários de linha de comando é armazenar cada hash em formato binário e converter apenas para hexadecimal na última etapa. A combinação ou hash de qualquer versão hexadecimal hash em árvore gerará um resultado incorreto.

#### Note

Os usuários do Windows podem usar o comando type em vez do cat. OpenSSL está disponível para Windows em [OpenSSL.org](https://www.openssl.org).

## Para calcular uma árvore hash

1. Se ainda não fez isso, divida o arquivo original em partes de 1 MiB.

```
$ split --bytes=1048576 --verbose largefile chunk
creating file `chunkaa'
creating file `chunkab'
creating file `chunkac'
```

2. Calcule e armazene o hash SHA-256 binário de cada fragmento.

```
$ openssl dgst -sha256 -binary chunkaa > hash1
$ openssl dgst -sha256 -binary chunkab > hash2
$ openssl dgst -sha256 -binary chunkac > hash3
```

3. Combine os primeiros dois hashes e execute o hash binário do resultado.

```
$ cat hash1 hash2 > hash12
$ openssl dgst -sha256 -binary hash12 > hash12hash
```

4. Combine o pai de partes de hash aa e ab com o hash de bloco ac e o resultado do hash, desta vez exibindo hexadecimal. Armazene o resultado em um shell variável.

```
$ cat hash12hash hash3 > hash123
$ openssl dgst -sha256 hash123
SHA256(hash123)= 9628195fcdcbbbe76cdde932d4646fa7de5f219fb39823836d81f0cc0e18aa67
$ TREEHASH=9628195fcdcbbbe76cdde932d4646fa7de5f219fb39823836d81f0cc0e18aa67
```

Por fim, preencha o upload com o comando [complete-multipart-upload](#). Este comando usa o tamanho do arquivo original em bytes, o valor de hash em árvore final em hexadecimal, e a ID da conta e nome do cofre.

```
$ aws glacier complete-multipart-upload --checksum $TREEHASH --archive-size 3145728 --
upload-id $UPLOADID --account-id - --vault-name myvault
{
  "archiveId": "d3AbWhE0YE1m6f_fI1jPG82F8xzbMEEZmrAllGAA0NJAzo5QdP-
N83MKqd96Unspoa5H5lItWX-sK8-QS0ZhwsyGiu9-R-
kwWUyS1dSB1mgPPWkEbeFfqDSav053rU7FvVLHfRc6hg",
  "checksum": "9628195fcdcbbbe76cdde932d4646fa7de5f219fb39823836d81f0cc0e18aa67",
```

```
"location": "/123456789012/vaults/myvault/archives/
d3AbWhE0YE1m6f_fI1jPG82F8xzbMEEZmrAllGAA0NJAzo5QdP-N83MKqd96Unspoa5H5lItWX-sK8-
QS0ZhwsyGiu9-R-kwUyS1dSB1mgPPWkEbeFfqDSav053rU7FvVLHfRc6hg"
}
```

Também é possível verificar o status do cofre usando o comando [describe-vault](#).

```
$ aws glacier describe-vault --account-id - --vault-name myvault
{
  "SizeInBytes": 3178496,
  "VaultARN": "arn:aws:glacier:us-west-2:123456789012:vaults/myvault",
  "LastInventoryDate": "2018-12-07T00:26:19.028Z",
  "NumberOfArchives": 1,
  "CreationDate": "2018-12-06T21:23:45.708Z",
  "VaultName": "myvault"
}
```

#### Note

O status do cofre é atualizado cerca de uma vez por dia. Consulte [Como trabalhar com cofres](#) para obter mais informações.

Já é seguro remover o bloco e os arquivos de hash que você criou.

```
$ rm chunk* hash*
```

Para obter mais informações sobre uploads fracionados, consulte [Upload de arquivos grandes em partes](#) e [Como computar somas de verificação](#) no Guia do desenvolvedor do Amazon S3 Glacier.

## Recursos

Referência da AWS CLI:

- [aws glacier](#)
- [aws glacier complete-multipart-upload](#)
- [aws glacier create-vault](#)
- [aws glacier describe-vault](#)
- [aws glacier initiate-multipart-upload](#)

## Referência do serviço:

- [Guia do desenvolvedor do Amazon S3 Glacier](#)
- [Carregar arquivos grandes em partes](#) no Guia do desenvolvedor do Amazon S3 Glacier
- [Calcular somas de verificação](#) no Guia do desenvolvedor do Amazon S3 Glacier
- [Trabalhar com vaults](#) no Guia do desenvolvedor do Amazon S3 Glacier

## Uso do AWS Identity and Access Management pela AWS CLI

Uma introdução à AWS Identity and Access Management.

### [Introdução à AWS Identity and Access Management](#)

Acesse os recursos do AWS Identity and Access Management (IAM) usando a AWS Command Line Interface (AWS CLI). Para listar os comandos da AWS CLI para o IAM use o comando a seguir.

```
aws iam help
```

Este tópico mostra exemplos de comandos da AWS CLI que executam tarefas comuns para o IAM.

Antes de executar quaisquer comandos, defina suas credenciais padrão. Para obter mais informações, consulte [Configurar o AWS CLI](#).

Para obter mais informações sobre o serviço do IAM, consulte o [Guia do usuário do AWS Identity and Access Management](#).

### Tópicos

- [Criar usuários e grupos do IAM](#)
- [Anexar uma política gerenciada do IAM a um usuário](#)
- [Definir uma senha inicial para um usuário do IAM](#)
- [Criar uma chave de acesso para um usuário do IAM](#)

## Criar usuários e grupos do IAM

Este tópico descreve como usar os comandos da AWS Command Line Interface (AWS CLI) para criar um grupo do AWS Identity and Access Management (IAM) e um usuário, depois adicionar o



usuário ao grupo. Para obter mais informações sobre o serviço do IAM, consulte o [Guia do usuário do AWS Identity and Access Management](#).

Antes de executar quaisquer comandos, defina suas credenciais padrão. Para obter mais informações, consulte [Configurar o AWS CLI](#).

Como criar um grupo e adicionar um novo usuário a ele

1. Use o comando [create-group](#) para criar o grupo.

```
$ aws iam create-group --group-name MyIamGroup
{
  "Group": {
    "GroupName": "MyIamGroup",
    "CreateDate": "2018-12-14T03:03:52.834Z",
    "GroupId": "AGPAJNUJ2W4IJVEXAMPLE",
    "Arn": "arn:aws:iam::123456789012:group/MyIamGroup",
    "Path": "/"
  }
}
```

2. Use o comando [create-user](#) para criar o usuário.

```
$ aws iam create-user --user-name MyUser
{
  "User": {
    "UserName": "MyUser",
    "Path": "/",
    "CreateDate": "2018-12-14T03:13:02.581Z",
    "UserId": "AIDAJY2PE5XUZ4EXAMPLE",
    "Arn": "arn:aws:iam::123456789012:user/MyUser"
  }
}
```

3. Use o comando [add-user-to-group](#) para adicionar o usuário ao grupo.

```
$ aws iam add-user-to-group --user-name MyUser --group-name MyIamGroup
```

4. Para verificar se o grupo MyIamGroup contém MyUser, use o comando [get-group](#).

```
$ aws iam get-group --group-name MyIamGroup
{
  "Group": {
```

```

    "GroupName": "MyIamGroup",
    "CreateDate": "2018-12-14T03:03:52Z",
    "GroupId": "AGPAJNUJ2W4IJVEXAMPLE",
    "Arn": "arn:aws:iam::123456789012:group/MyIamGroup",
    "Path": "/"
  },
  "Users": [
    {
      "UserName": "MyUser",
      "Path": "/",
      "CreateDate": "2018-12-14T03:13:02Z",
      "UserId": "AIDAJY2PE5XUZ4EXAMPLE",
      "Arn": "arn:aws:iam::123456789012:user/MyUser"
    }
  ],
  "IsTruncated": "false"
}

```

## Anexar uma política gerenciada do IAM a um usuário

Este tópico descreve como usar os comandos da AWS Command Line Interface (AWS CLI) para associar uma política do AWS Identity and Access Management (IAM) a um usuário. Neste exemplo, a política fornece ao usuário “Acesso de Usuário Power”. Para obter mais informações sobre o serviço do IAM, consulte o [Guia do usuário do AWS Identity and Access Management](#).

Antes de executar quaisquer comandos, defina suas credenciais padrão. Para obter mais informações, consulte [Configurar o AWS CLI](#).

Como associar uma política gerenciada do IAM a um usuário

1. Determine o nome de recurso da Amazon (ARN) da política a ser anexada. O comando a seguir usa `list-policies` para encontrar o ARN da política com o nome `PowerUserAccess`. Depois, ele armazena o ARN em uma variável de ambiente.

```

$ export POLICYARN=$(aws iam list-policies --query 'Policies[?
PolicyName==`PowerUserAccess`].{ARN:Arn}' --output text) ~
$ echo $POLICYARN
arn:aws:iam::aws:policy/PowerUserAccess

```

2. Para anexar a política, use o comando [attach-user-policyattach-user-policy](#) e faça referência à variável de ambiente que contém o ARN da política.

```
$ aws iam attach-user-policy --user-name MyUser --policy-arn $POLICYARN
```

3. Verifique se a política foi anexada ao usuário executando o comando [list-attached-user-policies](#).

```
$ aws iam list-attached-user-policies --user-name MyUser
{
  "AttachedPolicies": [
    {
      "PolicyName": "PowerUserAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/PowerUserAccess"
    }
  ]
}
```

Para obter mais informações, consulte [Recursos de gerenciamento de acesso](#). Este tópico fornece links para uma visão geral das permissões e políticas, bem como links para exemplos de políticas de acesso ao Amazon S3, Amazon EC2 e outros serviços.

## Definir uma senha inicial para um usuário do IAM

Este tópico descreve como usar os comandos da AWS Command Line Interface (AWS CLI) para definir uma senha inicial para um usuário do AWS Identity and Access Management (IAM). Para obter mais informações sobre o serviço do IAM, consulte o [Guia do usuário do AWS Identity and Access Management](#).

Antes de executar quaisquer comandos, defina suas credenciais padrão. Para obter mais informações, consulte [Configurar o AWS CLI](#).

O comando a seguir usa [create-login-profile](#) para definir uma senha inicial para o usuário especificado. Quando o usuário faz login pela primeira vez, ele precisa alterar a senha para algo que apenas ele sabe.

```
$ aws iam create-login-profile --user-name MyUser --password My!User1Login8P@ssword --password-reset-required
{
  "LoginProfile": {
    "UserName": "MyUser",
    "CreateDate": "2018-12-14T17:27:18Z",
```

```
    "PasswordResetRequired": true
  }
}
```

Você pode usar o comando `update-login-profile` para alterar a senha para um usuário.

```
$ aws iam update-login-profile --user-name MyUser --password My!User1ADifferentP@ssword
```

## Criar uma chave de acesso para um usuário do IAM

Este tópico descreve como usar os comandos da AWS Command Line Interface (AWS CLI) para criar um conjunto de chaves de acesso para um usuário do AWS Identity and Access Management (IAM). Para obter mais informações sobre o serviço do IAM, consulte o [Guia do usuário do AWS Identity and Access Management](#).

Antes de executar quaisquer comandos, defina suas credenciais padrão. Para obter mais informações, consulte [Configurar o AWS CLI](#).

Você pode usar o comando [create-access-key](#) para criar uma chave de acesso para um usuário. Uma chave de acesso é um conjunto de credenciais de segurança que consiste em um ID de chave de acesso e uma chave secreta.

Um usuário pode criar apenas duas chaves de acesso ao mesmo tempo. Se você tentar criar um terceiro conjunto, o comando retornará um erro `LimitExceeded`.

```
$ aws iam create-access-key --user-name MyUser
{
  "AccessKey": {
    "UserName": "MyUser",
    "AccessKeyId": "AKIAIOSFODNN7EXAMPLE",
    "Status": "Active",
    "SecretAccessKey": "wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY",
    "CreateDate": "2018-12-14T17:34:16Z"
  }
}
```

Use o comando [delete-access-key](#) para excluir uma chave de acesso para um usuário. Especifique qual chave de acesso deve ser excluída usando o ID de chave de acesso.

```
$ aws iam delete-access-key --user-name MyUser --access-key-id AKIAIOSFODNN7EXAMPLE
```

# Uso do Amazon S3 com a AWS CLI

## Introdução ao Amazon Simple Storage Service (Amazon S3)

### [Introdução ao Amazon Simple Storage Service \(Amazon S3\): armazenamento na nuvem na AWS](#)

É possível acessar os recursos do Amazon Simple Storage Service (Amazon S3) usando a AWS Command Line Interface (AWS CLI). A AWS CLI oferece dois níveis de comandos para acessar o Amazon S3:

- `s3`: comandos de alto nível que simplificam a execução de tarefas comuns, como criar, manipular e excluir objetos e buckets.
- `s3api`: expõe o acesso direto a todas as operações da API do Amazon S3, o que permite realizar operações avançadas.

Tópicos neste guia:

- [Uso de comandos de alto nível \(s3\) com a AWS CLI](#)
- [Uso de comandos em nível de API \(s3api\) com a AWS CLI](#)
- [Exemplo de script de operações de ciclo de vida de bucket Amazon S3](#)

## Uso de comandos de alto nível (s3) com a AWS CLI

Este tópico descreve alguns dos comandos que você pode utilizar para gerenciar buckets e objetos do Amazon S3 usando os comandos [aws s3](#) na AWS CLI. Para comandos não abordados neste tópico e exemplos de comandos adicionais, consulte os comandos [aws s3](#) na Referência da AWS CLI.

Os comandos `aws s3` de alto nível simplificam o gerenciamento de objetos do Amazon S3. Esses comandos permitem a você gerenciar o conteúdo do Amazon S3 dentro dele mesmo e com diretórios locais.

Tópicos

- [Pré-requisitos](#)
- [Antes de começar](#)
- [Criar um bucket](#)

- [Listar buckets e objetos](#)
- [Excluir buckets](#)
- [Excluir objetos](#)
- [Mover objetos](#)
- [Copiar objetos](#)
- [Sincronizar objetos](#)
- [Opções usadas com frequência para comandos s3](#)
- [Recursos](#)

## Pré-requisitos

Para executar os comandos s3, você precisa:

- Instale e configure a AWS CLI. Para obter mais informações, consulte [Instalar a AWS CLI](#) e [Autenticação e credenciais de acesso](#).
- O perfil utilizado deve ter permissões que garantem as operações da AWS realizadas pelos exemplos.
- Entenda estes termos do Amazon S3:
  - Bucket: uma pasta de nível superior do Amazon S3.
  - Prefixo: uma pasta do Amazon S3 em um bucket.
  - Objeto: qualquer item hospedado em um bucket do Amazon S3.

## Antes de começar

Esta seção descreve algumas coisas a serem observadas antes de usar comandos da aws s3.

## Carregamentos de objetos grandes

Quando você usa comandos da aws s3 para fazer upload de objetos grandes em um bucket do Amazon S3, a AWS CLI executa automaticamente um upload fracionado. Quando esses comandos da aws s3 são usados, não é possível continuar uploads que falharam.

Se o upload fracionado falhar devido a um tempo limite excedido, ou se você o cancelou manualmente na AWS CLI, a AWS CLI interromperá o upload e limpará todos os arquivos que foram criados. Esse processo pode levar alguns minutos.

Se o processo de upload fracionado ou limpeza for cancelado por um comando kill ou uma falha do sistema, os arquivos criados permanecerão no bucket do Amazon S3. Para limpar o carregamento fracionado, use o comando [s3api abort-multipart-upload](#)

## Propriedades e tags de arquivos em cópias com várias partes

Quando você usa a versão da AWS CLI versão 1 dos comandos no namespace do `aws s3` para copiar um arquivo de um local de bucket do Amazon S3 para outro local de bucket do Amazon S3 e essa operação usa a [cópia fracionada](#), nenhuma propriedade de arquivo do objeto de origem é copiada para o objeto de destino.

## Criar um bucket

Use o comando [s3 mb](#) para criar um bucket. Os nomes de bucket devem ser globalmente exclusivos (únicos em todo o Amazon S3) e devem ser compatíveis com o DNS.

Nomes de bucket podem conter letras minúsculas, números, hífen e pontos. Os nomes de bucket podem iniciar e terminar apenas com uma letra ou número e não podem conter um ponto ao lado de um hífen ou outro ponto.

## Sintaxe

```
$ aws s3 mb <target> [--options]
```

## Exemplos de s3 mb

O exemplo a seguir cria o bucket `s3://bucket-name`.

```
$ aws s3 mb s3://bucket-name
```

## Listar buckets e objetos

Para listar seus buckets, pastas ou objetos, use o comando [s3 ls](#). O uso do comando sem um destino ou opções lista todos os buckets.

## Sintaxe

```
$ aws s3 ls <target> [--options]
```

Para ver algumas opções comuns que podem ser usadas com este comando e exemplos, consulte [Opções usadas com frequência para comandos s3](#). Para obter uma lista completa de opções disponíveis, consulte [s3 ls](#) na Referência de comandos da AWS CLI.

## Exemplos de s3 ls

O exemplo a seguir lista todos os seus buckets do Amazon S3.

```
$ aws s3 ls
2018-12-11 17:08:50 my-bucket
2018-12-14 14:55:44 my-bucket2
```

O comando a seguir lista todos os objetos e prefixos em um bucket. Neste exemplo de saída, o prefixo `example/` tem um arquivo chamado `MyFile1.txt`.

```
$ aws s3 ls s3://bucket-name
                                PRE example/
2018-12-04 19:05:48              3 MyFile1.txt
```

Você pode filtrar a saída para um prefixo específico, incluindo-o no comando. O comando a seguir relaciona os objetos em `bucket-name/example` (ou seja, objetos em `bucket-name` filtrados pelo prefixo `example/`).

```
$ aws s3 ls s3://bucket-name/example/
2018-12-06 18:59:32              3 MyFile1.txt
```

## Excluir buckets

Para excluir um bucket, use o comando [s3 rb](#).

## Sintaxe

```
$ aws s3 rb <target> [--options]
```

## Exemplos de s3 rb

O exemplo a seguir remove o bucket `s3://bucket-name`.

```
$ aws s3 rb s3://bucket-name
```



Por padrão, o bucket deve estar vazio para a operação ser bem-sucedida. Para remover um bucket que não está vazio, é necessário incluir a opção `--force`. Se você estiver usando um bucket com versionamento que contém objetos anteriormente excluídos, mas ainda retidos, esse comando não permitirá que você remova o bucket. Você deve primeiro remover todo o conteúdo.

O exemplo de comando a seguir exclui todos os objetos no bucket e, em seguida, exclui o bucket.

```
$ aws s3 rb s3://bucket-name --force
```

## Excluir objetos

Para excluir objetos em um bucket ou seu diretório local, use o comando [s3 rm](#).

## Sintaxe

```
$ aws s3 rm <target> [--options]
```

Para ver algumas opções comuns que podem ser usadas com este comando e exemplos, consulte [Opções usadas com frequência para comandos s3](#). Para obter uma lista completa de opções, consulte [s3 rm](#) na Referência de comandos da AWS CLI.

## Exemplos de s3 rm

O exemplo a seguir exclui `filename.txt` de `s3://bucket-name/example`.

```
$ aws s3 rm s3://bucket-name/example/filename.txt
```

O exemplo a seguir exclui todos os objetos de `s3://bucket-name/example` usando a opção `--recursive`.

```
$ aws s3 rm s3://bucket-name/example --recursive
```

## Mover objetos

Use o comando [s3 mv](#) para mover objetos de um bucket ou diretório local.

## Sintaxe

```
$ aws s3 mv <source> <target> [--options]
```

Para ver algumas opções comuns que podem ser usadas com este comando e exemplos, consulte [Opções usadas com frequência para comandos s3](#). Para obter uma lista completa de opções disponíveis, consulte [s3 mv](#) na Referência de comandos da AWS CLI.

### Exemplos de s3 mv

O exemplo a seguir move todos os objetos de `s3://bucket-name/example` para `s3://my-bucket/`.

```
$ aws s3 mv s3://bucket-name/example s3://my-bucket/
```

O exemplo a seguir move um arquivo local do diretório de trabalho atual para o bucket do Amazon S3 com o comando `s3 mv`.

```
$ aws s3 mv filename.txt s3://bucket-name
```

O exemplo a seguir move um arquivo do bucket do Amazon S3 para o diretório de trabalho atual, onde `./` especifica o diretório de trabalho atual.

```
$ aws s3 mv s3://bucket-name/filename.txt ./
```

### Copiar objetos

Use o comando [s3 cp](#) para copiar objetos de um bucket ou diretório local.

### Sintaxe

```
$ aws s3 cp <source> <target> [--options]
```

Você pode usar o parâmetro dash para transmitir arquivos para entrada padrão (stdin) ou saída padrão (stdout).

#### Warning

Se estiver usando o PowerShell, o shell poderá alterar a codificação de um CRLF ou adicionar um CRLF à entrada ou saída encadeada ou à saída redirecionada.

O comando `s3 cp` usa a sintaxe a seguir para fazer upload de um fluxo de arquivos do stdin para um bucket especificado.

## Sintaxe

```
$ aws s3 cp - <target> [--options]
```

O comando `s3 cp` usa a sintaxe a seguir para baixar um fluxo de arquivos do Amazon S3 para `stdout`.

## Sintaxe

```
$ aws s3 cp <target> [--options] -
```

Para ver algumas opções comuns que podem ser usadas com este comando e exemplos, consulte [Opções usadas com frequência para comandos s3](#). Para obter a lista completa de opções, consulte [s3 cp](#) na Referência de comandos da AWS CLI.

## Exemplos do `s3 cp`

O exemplo a seguir copia todos os objetos de `s3://bucket-name/example` para `s3://my-bucket/`.

```
$ aws s3 cp s3://bucket-name/example s3://my-bucket/
```

O exemplo a seguir copia um arquivo local do diretório de trabalho atual para o bucket do Amazon S3 com o comando `s3 cp`.

```
$ aws s3 cp filename.txt s3://bucket-name
```

O exemplo a seguir copia um arquivo do bucket do Amazon S3 para o diretório de trabalho atual, onde `./` especifica o diretório de trabalho atual.

```
$ aws s3 cp s3://bucket-name/filename.txt ./
```

O exemplo a seguir usa `echo` para transmitir o texto “hello world” para o arquivo `s3://bucket-name/filename.txt`.

```
$ echo "hello world" | aws s3 cp - s3://bucket-name/filename.txt
```

O exemplo a seguir transmite o arquivo `s3://bucket-name/filename.txt` para `stdout` e imprime o conteúdo no console.

```
$ aws s3 cp s3://bucket-name/filename.txt -  
hello world
```

O exemplo a seguir transmite o conteúdo de `s3://bucket-name/pre` para `stdout`, usa o comando `bzip2` para comprimir os arquivos e faz upload do novo arquivo compactado chamado `key.bz2` para `s3://bucket-name`.

```
$ aws s3 cp s3://bucket-name/pre - | bzip2 --best | aws s3 cp - s3://bucket-name/  
key.bz2
```

## Sincronizar objetos

O comando [s3 sync](#) sincroniza o conteúdo de um bucket e um diretório ou o conteúdo de dois buckets. Normalmente, `s3 sync` copia arquivos que estão faltando ou desatualizadas ou objetos entre a origem e o destino. No entanto, você também pode fornecer a opção `--delete` para remover arquivos ou objetos do destino que não estão presentes na origem.

## Sintaxe

```
$ aws s3 sync <source> <target> [--options]
```

Para ver algumas opções comuns que podem ser usadas com este comando e exemplos, consulte [Opções usadas com frequência para comandos s3](#). Para obter uma lista completa de opções, consulte [s3 sync](#) na Referência de comandos da AWS CLI.

## Exemplos de s3 sync

O exemplo a seguir sincroniza o conteúdo de um prefixo do Amazon S3 chamado `path` no bucket chamado `my-bucket` com o diretório de trabalho atual.

`s3 sync` atualiza todos os arquivos que têm tamanho ou hora de modificação diferentes dos arquivos com o mesmo nome no destino. A saída exibe operações específicas executadas durante a sincronização. Observe que a operação sincroniza recursivamente o subdiretório `MySubdirectory` e seu conteúdo com `s3://my-bucket/path/MySubdirectory`.

```
$ aws s3 sync . s3://my-bucket/path  
upload: MySubdirectory\MyFile3.txt to s3://my-bucket/path/MySubdirectory/MyFile3.txt  
upload: MyFile2.txt to s3://my-bucket/path/MyFile2.txt
```

```
upload: MyFile1.txt to s3://my-bucket/path/MyFile1.txt
```

O exemplo a seguir, uma extensão do exemplo anterior, mostra como usar a opção `--delete`.

```
// Delete local file
$ rm ./MyFile1.txt

// Attempt sync without --delete option - nothing happens
$ aws s3 sync . s3://my-bucket/path

// Sync with deletion - object is deleted from bucket
$ aws s3 sync . s3://my-bucket/path --delete
delete: s3://my-bucket/path/MyFile1.txt

// Delete object from bucket
$ aws s3 rm s3://my-bucket/path/MySubdirectory/MyFile3.txt
delete: s3://my-bucket/path/MySubdirectory/MyFile3.txt

// Sync with deletion - local file is deleted
$ aws s3 sync s3://my-bucket/path . --delete
delete: MySubdirectory\MyFile3.txt

// Sync with Infrequent Access storage class
$ aws s3 sync . s3://my-bucket/path --storage-class STANDARD_IA
```

Quando a opção `--delete` é usada, as opções `--exclude` e `--include` podem filtrar arquivos ou objetos a serem excluídos durante uma operação `s3 sync`. Nesse caso, a sequência de parâmetro deve especificar os arquivos a serem excluídos ou incluídos, a exclusão no contexto do diretório de destino ou bucket. Por exemplo:

```
Assume local directory and s3://my-bucket/path currently in sync and each contains 3
files:
MyFile1.txt
MyFile2.rtf
MyFile88.txt
...

// Sync with delete, excluding files that match a pattern. MyFile88.txt is deleted,
while remote MyFile1.txt is not.
$ aws s3 sync . s3://my-bucket/path --delete --exclude "path/MyFile?.txt"
delete: s3://my-bucket/path/MyFile88.txt
...
```

```
// Sync with delete, excluding MyFile2.rtf - local file is NOT deleted
$ aws s3 sync s3://my-bucket/path . --delete --exclude "./MyFile2.rtf"
download: s3://my-bucket/path/MyFile1.txt to MyFile1.txt
...

// Sync with delete, local copy of MyFile2.rtf is deleted
$ aws s3 sync s3://my-bucket/path . --delete
delete: MyFile2.rtf
```

## Opções usadas com frequência para comandos s3

As opções a seguir são usadas frequentemente para os comandos descritos neste tópico. Para obter uma lista completa de opções que você pode usar em um comando, consulte o comando específico no [AWS CLI Guia de referência da](#).

### acl

s3 sync e s3 cp podem usar a opção --acl. Isso permite definir as permissões de acesso para arquivos copiados para o Amazon S3. A opção --acl aceita os valores private, public-read, public-read-write. Para obter mais informações, consulte [ACL pré-configurada](#) no Manual do usuário do Amazon Simple Storage Service.

```
$ aws s3 sync . s3://my-bucket/path --acl public-read
```

### exclude

Ao usar o comando s3 cp, s3 mv, s3 sync ou s3 rm, você pode filtrar os resultados usando a opção --exclude ou --include. A opção --exclude define regras para excluir apenas objetos do comando, e as opções se aplicam na ordem especificada. Isso é mostrado no exemplo a seguir.

```
Local directory contains 3 files:
MyFile1.txt
MyFile2.rtf
MyFile88.txt

// Exclude all .txt files, resulting in only MyFile2.rtf being copied
$ aws s3 cp . s3://my-bucket/path --exclude "*.txt"

// Exclude all .txt files but include all files with the "MyFile*.txt" format,
resulting in, MyFile1.txt, MyFile2.rtf, MyFile88.txt being copied
```

```
$ aws s3 cp . s3://my-bucket/path --exclude "*.txt" --include "MyFile*.txt"

// Exclude all .txt files, but include all files with the "MyFile*.txt" format,
but exclude all files with the "MyFile?.txt" format resulting in, MyFile2.rtf and
MyFile88.txt being copied
$ aws s3 cp . s3://my-bucket/path --exclude "*.txt" --include "MyFile*.txt" --
exclude "MyFile?.txt"
```

## include

Ao usar o comando `s3 cp`, `s3 mv`, `s3 sync` ou `s3 rm`, você pode filtrar os resultados usando a opção `--exclude` ou `--include`. A opção `--include` define regras para incluir apenas objetos especificados para o comando, e as opções se aplicam na ordem especificada. Isso é mostrado no exemplo a seguir.

```
Local directory contains 3 files:
MyFile1.txt
MyFile2.rtf
MyFile88.txt

// Include all .txt files, resulting in MyFile1.txt and MyFile88.txt being copied
$ aws s3 cp . s3://my-bucket/path --include "*.txt"

// Include all .txt files but exclude all files with the "MyFile*.txt" format,
resulting in no files being copied
$ aws s3 cp . s3://my-bucket/path --include "*.txt" --exclude "MyFile*.txt"

// Include all .txt files, but exclude all files with the "MyFile*.txt" format, but
include all files with the "MyFile?.txt" format resulting in MyFile1.txt being
copied

$ aws s3 cp . s3://my-bucket/path --include "*.txt" --exclude "MyFile*.txt" --
include "MyFile?.txt"
```

## grant (conceder)

Os comandos `s3 cp`, `s3 mv` e `s3 sync` incluem uma opção `--grants` que pode ser usada para conceder permissões referentes a esse objeto para usuários ou grupos específicos. Defina a opção `--grants` como uma lista de permissões usando a sintaxe a seguir. Substitua `Permission`, `Grantee_Type` e `Grantee_ID` pelos seus próprios valores.

### Sintaxe

```
--grants Permission=Grantee_Type=Grantee_ID  
[Permission=Grantee_Type=Grantee_ID ...]
```

Cada valor contém os seguintes elementos:

- *Permission* (Permissão): especifica as permissões concedidas. Pode ser definida como `read`, `readacl`, `writeacl` ou `full`.
- *Grantee\_Type* (Tipo de favorecido): especifica como identificar o favorecido. Pode ser definido como `uri`, `emailaddress` ou `id`.
- *Grantee\_ID* – Especifica o favorecido com base em *Grantee\_Type*.
  - `uri`: o URI do grupo. Para obter mais informações, consulte [Quem é o favorecido?](#)
  - `emailaddress` - O endereço de e-mail da conta.
  - `id` - O ID canônico da conta

Para obter mais informações sobre controle de acesso do Amazon S3, consulte [Controle de acesso](#).

O exemplo a seguir copia um objeto em um bucket. Ele concede permissões `read` sobre o objeto a todos os usuários e permissões `full` (`read`, `readacl` e `writeacl`) à conta associada a `user@example.com`.

```
$ aws s3 cp file.txt s3://my-bucket/ --grants read=uri=http://acs.amazonaws.com/groups/global/AllUsers full=emailaddress=user@example.com
```

Também será possível especificar uma classe de armazenamento não padrão (`REDUCED_REDUNDANCY` ou `STANDARD_IA`) para objetos cujo upload você fizer no Amazon S3. Para fazer isso, use a opção `--storage-class`.

```
$ aws s3 cp file.txt s3://my-bucket/ --storage-class REDUCED_REDUNDANCY
```

## recursive

Quando você usa essa opção, o comando é executado em todos os arquivos ou objetos sob o diretório ou prefixo especificado. O exemplo a seguir exclui `s3://my-bucket/path` e todo o seu conteúdo.

```
$ aws s3 rm s3://my-bucket/path --recursive
```



## Recursos

### Referência da AWS CLI:

- [aws s3](#)
- [aws s3 cp](#)
- [aws s3 mb](#)
- [aws s3 mv](#)
- [aws s3 ls](#)
- [aws s3 rb](#)
- [aws s3 rm](#)
- [aws s3 sync](#)

### Referência do serviço:

- [Como trabalhar com buckets do Amazon S3](#) no Manual do usuário do Amazon Simple Storage Service
- [Como trabalhar com objetos do Amazon S3](#) no Manual do usuário do Amazon Simple Storage Service
- [Como listar chaves hierarquicamente usando um prefixo e delimitador](#) no Manual do usuário do Amazon Simple Storage Service
- [Anular carregamentos fracionados para um bucket do S3 usando o AWS SDK for .NET \(nível baixo\)](#) no Manual do usuário do Amazon Simple Storage Service

## Uso de comandos em nível de API (s3api) com a AWS CLI

Os comandos no nível de API (contidos no conjunto de comandos `s3api`) fornecem acesso direto às APIs do Amazon Simple Storage Service (Amazon S3) e ativam algumas operações que não são expostas em comandos de alto nível do `s3`. Esses comandos são equivalentes aos outros serviços da AWS que fornecem acesso em nível de API para a funcionalidade de serviços. Para obter mais informações sobre os comando do `s3`, consulte [Uso de comandos de alto nível \(s3\) com a AWS CLI](#)

Este tópico fornece exemplos que demonstram como usar os comandos de nível inferior que são mapeados em APIs do Amazon S3. Além disso, você pode encontrar exemplos para cada comando da API do S3 na seção `s3api` do [AWS CLIGuia de referência da](#) .

## Tópicos

- [Pré-requisitos](#)
- [Aplicar uma ACL personalizada](#)
- [Configurar uma política de registro em log](#)
- [Recursos](#)

## Pré-requisitos

Para executar os comandos `s3api`, você precisa:

- Instale e configure a AWS CLI. Para obter mais informações, consulte [Instalar a AWS CLI](#) e [Autenticação e credenciais de acesso](#).
- O perfil utilizado deve ter permissões que garantem as operações da AWS realizadas pelos exemplos.
- Entenda estes termos do Amazon S3:
  - Bucket: uma pasta de nível superior do Amazon S3.
  - Prefixo: uma pasta do Amazon S3 em um bucket.
  - Objeto: qualquer item hospedado em um bucket do Amazon S3.

## Aplicar uma ACL personalizada

Com comandos de alto nível, é possível usar a opção `--acl` para aplicar listas de controle de acesso (ACLs) predefinidas a objetos do Amazon S3. No entanto, não é possível usar esse comando para definir ACLs em todo o bucket. No entanto, você pode fazer isso usando o comando em nível de API [put-bucket-acl](#).

O exemplo a seguir mostra como conceder o controle total a dois usuários do AWS (`user1@example.com` e `user2@example.com`) e a permissão de leitura a todos. O identificador para “todos” vem de um URI especial que você transmite como um parâmetro.

```
$ aws s3api put-bucket-acl --bucket MyBucket --grant-full-control  
'emailaddress="user1@example.com",emailaddress="user2@example.com"' --grant-read  
'uri="http://acs.amazonaws.com/groups/global/AllUsers"'
```

Para obter detalhes sobre como construir as ACLs, consulte [PUT Bucket acl](#) na Referência da API do Amazon Simple Storage Service. Os comandos de ACL s3api na CLI, como `put-bucket-acl`, usam a mesma [notação de argumento abreviada](#).

## Configurar uma política de registro em log

O comando da API `put-bucket-logging` configura uma política de registro de buckets.

No exemplo a seguir, ao usuário da AWS `usuario@exemplo.com` recebe controle total sobre os arquivos de log, e todos os usuários terão acesso de leitura a eles. Observe que o comando `put-bucket-acl` também deve conceder ao sistema de entrega de log do Amazon S3 (especificado por um URI) as permissões necessárias para ler e gravar os logs no bucket.

```
$ aws s3api put-bucket-acl --bucket MyBucket --grant-read-acp 'URI="http://acs.amazonaws.com/groups/s3/LogDelivery"' --grant-write 'URI="http://acs.amazonaws.com/groups/s3/LogDelivery"'
$ aws s3api put-bucket-logging --bucket MyBucket --bucket-logging-status file://logging.json
```

O arquivo `logging.json` no comando anterior tem o seguinte conteúdo.

```
{
  "LoggingEnabled": {
    "TargetBucket": "MyBucket",
    "TargetPrefix": "MyBucketLogs/",
    "TargetGrants": [
      {
        "Grantee": {
          "Type": "AmazonCustomerByEmail",
          "EmailAddress": "user@example.com"
        },
        "Permission": "FULL_CONTROL"
      },
      {
        "Grantee": {
          "Type": "Group",
          "URI": "http://acs.amazonaws.com/groups/global/AllUsers"
        },
        "Permission": "READ"
      }
    ]
  }
}
```

```
}
```

## Recursos

### Referência da AWS CLI:

- [aws s3api](#)
- [aws s3api put-bucket-acl](#)
- [aws s3api put-bucket-logging](#)

### Referência do serviço:

- [Como trabalhar com buckets do Amazon S3](#) no Manual do usuário do Amazon Simple Storage Service
- [Como trabalhar com objetos do Amazon S3](#) no Manual do usuário do Amazon Simple Storage Service
- [Como listar chaves hierarquicamente usando um prefixo e delimitador](#) no Manual do usuário do Amazon Simple Storage Service
- [Anular carregamentos fracionados para um bucket do S3 usando o AWS SDK for .NET \(nível baixo\)](#) no Manual do usuário do Amazon Simple Storage Service

## Exemplo de script de operações de ciclo de vida de bucket Amazon S3

Este tópico usa um exemplo de script bash para operações de ciclo de vida do bucket do Amazon S3 usando a AWS Command Line Interface (AWS CLI). Este exemplo de desenvolvimento de scripts usa o conjunto de comandos [aws s3api](#). Scripts shell são programas desenvolvidos para ser executados em uma interface de linha de comando.

### Tópicos

- [Antes de começar](#)
- [Sobre este exemplo](#)
- [Arquivos](#)
- [Referências](#)

## Antes de começar

Antes que você possa executar qualquer um dos exemplos abaixo, as seguintes tarefas deverão ser concluídas.

- Instale e configure a AWS CLI. Para obter mais informações, consulte [Instalar a AWS CLI](#) e [Autenticação e credenciais de acesso](#).
- O perfil utilizado deve ter permissões que garantem as operações da AWS realizadas pelos exemplos.
- Como prática recomendada da AWS, conceda a esse código privilégio mínimo ou apenas as permissões necessárias para executar uma tarefa. Para obter mais informações, consulte [Conceder privilégio mínimo](#) no Guia do usuário do IAM.
- Este código não foi testado em todas as regiões da AWS. Alguns serviços da AWS só estão disponíveis em regiões específicas. Para obter mais informações, consulte [Endpoints de serviço e cotas](#) no Guia de referência geral da AWS.
- Executar este código pode resultar em cobranças em sua conta da AWS. É sua responsabilidade garantir que todos os recursos criados por este script sejam removidos após você terminar de usá-los.

O serviço Amazon S3 usa os seguintes termos:

- Bucket: uma pasta de nível superior do Amazon S3.
- Prefixo: uma pasta do Amazon S3 em um bucket.
- Objeto: qualquer item hospedado em um bucket do Amazon S3.

## Sobre este exemplo

Este exemplo mostra como interagir com algumas das operações básicas do Amazon S3 usando um conjunto de funções em arquivos de script de shell. As funções estão localizadas no arquivo de script shell chamado `bucket-operations.sh`. Você pode chamar essas funções em outro arquivo. Cada arquivo de script contém comentários descrevendo cada uma das funções.

Para ver os resultados intermediários de cada etapa, execute o script com um parâmetro `-i`. É possível exibir o status atual do bucket ou do conteúdo usando o console do Amazon S3. O script avançará para a próxima etapa somente após você pressionar ENTER no prompt.

Para obter o exemplo completo e os arquivos de script para download, consulte [Operações de ciclo de vida de bucket do Amazon S3](#) no Repositório de exemplos de código da AWS no GitHub.

## Arquivos

O exemplo contém os seguintes arquivos:

### bucket-operations.sh

Esse arquivo de script principal pode ser originado de outro arquivo. Ele inclui funções que executam as seguintes tarefas:

- Criar um bucket e verificar se ele existe
- Copiar um arquivo do computador local para um bucket
- Copiar um arquivo de um local de bucket para um local de bucket diferente
- Listar o conteúdo de um bucket
- Excluir um arquivo de um bucket
- Excluir um bucket

Visualize o código para [bucket-operations.sh](#) no GitHub.

### test-bucket-operations.sh

O arquivo de script de shell `test-bucket-operations.sh` demonstra como chamar as funções utilizando o arquivo `bucket-operations.sh` e chamando cada uma das funções. Após chamar funções, o script de teste remove todos os recursos que ele criou.

Visualize o código para [test-bucket-operations.sh](#) no GitHub.

### awsdocs-general.sh

O arquivo de script `awsdocs-general.sh` contém funções de uso geral usadas em exemplos avançados de código para a AWS CLI.

Visualize o código para [awsdocs-general.sh](#) no GitHub.

## Referências

Referência da AWS CLI:

- [aws s3api](#)
- [aws s3api create-bucket](#)
- [aws s3api copy-object](#)
- [aws s3api delete-bucket](#)
- [aws s3api delete-object](#)
- [aws s3api head-bucket](#)
- [aws s3api list-objects](#)
- [aws s3api put-object](#)

Outra referência:

- [Como trabalhar com buckets do Amazon S3](#) no Manual do usuário do Amazon Simple Storage Service
- [Como trabalhar com objetos do Amazon S3](#) no Manual do usuário do Amazon Simple Storage Service
- Para visualizar e contribuir para o SDK da AWS e exemplos de código da AWS CLI, consulte o [Repositório de exemplos de código da AWS](#) no GitHub.

## Uso do Amazon SNS com a AWS CLI

É possível acessar os recursos do Amazon Simple Notification Service (Amazon SNS) usando a AWS Command Line Interface (AWS CLI). Para listar os comandos da AWS CLI para o Amazon SNS, use o comando a seguir.

```
aws sns help
```

Antes de executar quaisquer comandos, defina suas credenciais padrão. Para obter mais informações, consulte [Configurar o AWS CLI](#).

Este tópico mostra exemplos de comandos da AWS CLI que executam tarefas comuns para o Amazon SNS.

### Tópicos

- [Criar um tópico](#)

- [Assinar um tópico](#)
- [Publicar em um tópico](#)
- [Cancelar a assinatura de um tópico](#)
- [Excluir um tópico](#)

## Criar um tópico

Para criar um tópico, use o comando [sns create-topic](#) e especifique o nome a ser atribuído a ele.

```
$ aws sns create-topic --name my-topic
{
  "TopicArn": "arn:aws:sns:us-west-2:123456789012:my-topic"
}
```

Anote o TopicArn da resposta que você usará mais tarde para publicar uma mensagem.

## Assinar um tópico

Para assinar um tópico, use o comando [sns subscribe](#).

O exemplo a seguir especifica o protocolo email e um endereço de e-mail para o notification-endpoint.

```
$ aws sns subscribe --topic-arn arn:aws:sns:us-west-2:123456789012:my-topic --
protocol email --notification-endpoint saanvi@example.com
{
  "SubscriptionArn": "pending confirmation"
}
```

O AWS envia imediatamente uma mensagem de confirmação para o endereço de e-mail especificado no comando subscribe. A mensagem de e-mail tem o seguinte texto.

```
You have chosen to subscribe to the topic:
arn:aws:sns:us-west-2:123456789012:my-topic
To confirm this subscription, click or visit the following link (If this was in error
no action is necessary):
Confirm subscription
```



Assim que o destinatário clicar no link [Confirm subscription](#) (Confirmar assinatura), o navegador do destinatário exibirá uma mensagem de notificação com informações semelhantes à seguinte.

```
Subscription confirmed!
```

```
You have subscribed saanvi@example.com to the topic:my-topic.
```

```
Your subscription's id is:
```

```
arn:aws:sns:us-west-2:123456789012:my-topic:1328f057-de93-4c15-512e-8bb22EXAMPLE
```

```
If it was not your intention to subscribe, click here to unsubscribe.
```

## Publicar em um tópico

Para enviar uma mensagem a todos os assinantes de um tópico, use o comando [sns publish](#).

O exemplo a seguir envia a mensagem “Hello World!” para todos os assinantes do tópico especificado.

```
$ aws sns publish --topic-arn arn:aws:sns:us-west-2:123456789012:my-topic --  
message "Hello World!"  
{  
    "MessageId": "4e41661d-5eec-5ddf-8dab-2c867EXAMPLE"  
}
```

Neste exemplo, a AWS envia uma mensagem de e-mail com o texto “Hello World!” para saanvi@example.com.

## Cancelar a assinatura de um tópico

Para cancelar a assinatura de um tópico e parar de receber as mensagens publicadas nesse tópico, use o comando [sns unsubscribe](#) e especifique o ARN do tópico do qual você deseja cancelar a assinatura.

```
$ aws sns unsubscribe --subscription-arn arn:aws:sns:us-west-2:123456789012:my-  
topic:1328f057-de93-4c15-512e-8bb22EXAMPLE
```

Para verificar se a assinatura foi cancelada com êxito, use o comando [sns list-subscriptions](#) para confirmar que o ARN não aparece mais na lista.

```
$ aws sns list-subscriptions
```

## Excluir um tópico

Para excluir um tópico, execute o comando [sns delete-topic](#).

```
$ aws sns delete-topic --topic-arn arn:aws:sns:us-west-2:123456789012:my-topic
```

Para verificar se a AWS excluiu o tópico com êxito, use o comando [sns list-topics](#) para confirmar que o tópico não aparece mais na lista.

```
$ aws sns list-topics
```

## AWS CLI com exemplos de código de script Bash

Os exemplos de código neste tópico mostram como usar a AWS Command Line Interface com script Bash com a AWS.

Ações são trechos de código de programas maiores e devem ser executadas em contexto. Embora as ações mostrem como chamar funções de serviço específicas, é possível ver as ações contextualizadas em seus devidos cenários e exemplos entre serviços.

Cenários são exemplos de código que mostram como realizar uma tarefa específica chamando várias funções dentro do mesmo serviço.

Exemplos entre serviços são amostras de aplicações que funcionam em vários Serviços da AWS.

### Exemplos

- [Ações e cenários de uso da AWS CLI com script Bash](#)

## Ações e cenários de uso da AWS CLI com script Bash

Os exemplos de código a seguir mostram como realizar ações e implementar cenários comuns usando a AWS Command Line Interface com script Bash por meio de Serviços da AWS.

Ações são trechos de código de programas maiores e devem ser executadas em contexto. Embora as ações mostrem como chamar funções de serviço específicas, é possível ver as ações contextualizadas em seus devidos cenários e exemplos entre serviços.

Cenários são exemplos de código que mostram como realizar uma tarefa específica chamando várias funções dentro do mesmo serviço.

## Serviços

- [Exemplos do DynamoDB de uso da AWS CLI com script Bash](#)
- [Exemplos do HealthImaging de uso da AWS CLI com script Bash](#)
- [Exemplos do IAM que usam a AWS CLI com script Bash](#)
- [Exemplos do Amazon S3 de uso da AWS CLI com script Bash](#)
- [Exemplos do AWS STS de uso da AWS CLI com script Bash](#)

## Exemplos do DynamoDB de uso da AWS CLI com script Bash

Os exemplos de código a seguir mostram como realizar ações e implementar cenários comuns usando a AWS Command Line Interface com script Bash por meio do DynamoDB.

Ações são trechos de código de programas maiores e devem ser executadas em contexto. Embora as ações mostrem como chamar funções de serviço específicas, é possível ver as ações contextualizadas em seus devidos cenários e exemplos entre serviços.

Cenários são exemplos de código que mostram como realizar uma tarefa específica chamando várias funções dentro do mesmo serviço.

Cada exemplo inclui um link para o GitHub, onde você pode encontrar instruções sobre como configurar e executar o código no contexto.

## Tópicos

- [Ações](#)
- [Cenários](#)

## Ações

### Criar uma tabela

O exemplo de código a seguir mostra como criar uma tabela do DynamoDB.

### AWS CLI com script Bash

#### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function dynamodb_create_table
#
# This function creates an Amazon DynamoDB table.
#
# Parameters:
#     -n table_name -- The name of the table to create.
#     -a attribute_definitions -- JSON file path of a list of attributes and their
types.
#     -k key_schema -- JSON file path of a list of attributes and their key types.
#     -p provisioned_throughput -- Provisioned throughput settings for the table.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_create_table() {
    local table_name attribute_definitions key_schema provisioned_throughput response
    local option OPTARG # Required to use getopt command in a function.

    #####
    # Function usage explanation
    #####
    function usage() {
        echo "function dynamodb_create_table"
        echo "Creates an Amazon DynamoDB table."
        echo " -n table_name -- The name of the table to create."
        echo " -a attribute_definitions -- JSON file path of a list of attributes and
their types."
        echo " -k key_schema -- JSON file path of a list of attributes and their key
types."
        echo " -p provisioned_throughput -- Provisioned throughput settings for the
table."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "n:a:k:p:h" option; do
        case "${option}" in
            n) table_name="${OPTARG}" ;;
            a) attribute_definitions="${OPTARG}" ;;
            k) key_schema="${OPTARG}" ;;
            p) provisioned_throughput="${OPTARG}" ;;
        esac
    done
}
```

```
h)
    usage
    return 0
;;
\?)
    echo "Invalid parameter"
    usage
    return 1
;;
esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$attribute_definitions" ]]; then
    errecho "ERROR: You must provide an attribute definitions json file path the -a
parameter."
    usage
    return 1
fi

if [[ -z "$key_schema" ]]; then
    errecho "ERROR: You must provide a key schema json file path the -k parameter."
    usage
    return 1
fi

if [[ -z "$provisioned_throughput" ]]; then
    errecho "ERROR: You must provide a provisioned throughput json file path the -p
parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    table_name:    $table_name"
iecho "    attribute_definitions:  $attribute_definitions"
iecho "    key_schema:    $key_schema"
iecho "    provisioned_throughput:  $provisioned_throughput"
```

```
iecho ""

response=$(aws dynamodb create-table \
  --table-name "$table_name" \
  --attribute-definitions file://"${attribute_definitions}" \
  --key-schema file://"${key_schema}" \
  --provisioned-throughput "$provisioned_throughput")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
  aws_cli_error_log $error_code
  errecho "ERROR: AWS reports create-table operation failed.$response"
  return 1
fi

return 0
}
```

As funções utilitárias usadas neste exemplo.

```
#####
# function iecho
#
# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.
#####
function iecho() {
  if [[ $VERBOSE == true ]]; then
    echo "$@"
  fi
}

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
  printf "%s\n" "$*" 1>&2
}
```

```
#####
# function aws_cli_error_log()
#
# This function is used to log the error messages from the AWS CLI.
#
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-
return-codes.
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:
#     0: - Success.
#
#####
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    elif [ "$err_code" == 252 ]; then
        errecho " Command syntax invalid."
    elif [ "$err_code" == 253 ]; then
        errecho " The system environment or configuration was invalid."
    elif [ "$err_code" == 254 ]; then
        errecho " The service returned an error."
    elif [ "$err_code" == 255 ]; then
        errecho " 255 is a catch-all error."
    fi

    return 0
}
```

- Para obter detalhes da API, consulte [CreateTable](#) na Referência de comandos da AWS CLI.

## Excluir uma tabela

O exemplo de código a seguir mostra como excluir uma tabela do DynamoDB.

## AWS CLI com script Bash

 Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function dynamodb_delete_table
#
# This function deletes a DynamoDB table.
#
# Parameters:
#     -n table_name  -- The name of the table to delete.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_delete_table() {
    local table_name response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function dynamodb_delete_table"
        echo "Deletes an Amazon DynamoDB table."
        echo " -n table_name  -- The name of the table to delete."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "n:h" option; do
        case "${option}" in
            n) table_name="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
```



```

        usage
        return 1
    ;;
esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    table_name:  $table_name"
iecho ""

response=$(aws dynamodb delete-table \
    --table-name "$table_name")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports delete-table operation failed.$response"
    return 1
fi

return 0
}

```

As funções utilitárias usadas neste exemplo.

```

#####
# function iecho
#
# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.
#####
function iecho() {
    if [[ $VERBOSE == true ]]; then
        echo "$@"
    fi
}

```

```

fi
}

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function aws_cli_error_log()
#
# This function is used to log the error messages from the AWS CLI.
#
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-return-codes.
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:
#     0: - Success.
#
#####
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    elif [ "$err_code" == 252 ]; then
        errecho " Command syntax invalid."
    elif [ "$err_code" == 253 ]; then
        errecho " The system environment or configuration was invalid."
    elif [ "$err_code" == 254 ]; then
        errecho " The service returned an error."
    elif [ "$err_code" == 255 ]; then
        errecho " 255 is a catch-all error."
    fi
}

```

```
fi

return 0
}
```

- Para obter detalhes da API, consulte [DeleteTable](#) na Referência de comandos da AWS CLI.

## Excluir um item de uma tabela

O exemplo de código a seguir mostra como excluir um item de uma tabela do DynamoDB.

## AWS CLI com script Bash

### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function dynamodb_delete_item
#
# This function deletes an item from a DynamoDB table.
#
# Parameters:
#     -n table_name  -- The name of the table.
#     -k keys       -- Path to json file containing the keys that identify the item to
#                      delete.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_delete_item() {
    local table_name keys response
    local option OPTARG # Required to use getopt command in a function.

    # #####
    # Function usage explanation
    #####
    function usage() {
```

```

    echo "function dynamodb_delete_item"
    echo "Delete an item from a DynamoDB table."
    echo " -n table_name  -- The name of the table."
    echo " -k keys    -- Path to json file containing the keys that identify the item
to delete."
    echo ""
}
while getopts "n:k:h" option; do
    case "${option}" in
        n) table_name="${OPTARG}" ;;
        k) keys="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$keys" ]]; then
    errecho "ERROR: You must provide a keys json file path the -k parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    table_name:  $table_name"
iecho "    keys:      $keys"
iecho ""

response=$(aws dynamodb delete-item \
    --table-name "$table_name" \
    --key file://"${keys}")

```

```

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports delete-item operation failed.$response"
    return 1
fi

return 0
}

```

As funções utilitárias usadas neste exemplo.

```

#####
# function iecho
#
# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.
#####
function iecho() {
    if [[ $VERBOSE == true ]]; then
        echo "$@"
    fi
}

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function aws_cli_error_log()
#
# This function is used to log the error messages from the AWS CLI.
#

```

```
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-
return-codes.
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:
#     0: - Success.
#
#####
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    elif [ "$err_code" == 252 ]; then
        errecho " Command syntax invalid."
    elif [ "$err_code" == 253 ]; then
        errecho " The system environment or configuration was invalid."
    elif [ "$err_code" == 254 ]; then
        errecho " The service returned an error."
    elif [ "$err_code" == 255 ]; then
        errecho " 255 is a catch-all error."
    fi

    return 0
}
```

- Para obter detalhes da API, consulte [Deleteltem](#) na Referência de comandos da AWS CLI.

## Obter um lote de itens

O exemplo de código a seguir mostra como obter um lote de itens do DynamoDB.

## AWS CLI com script Bash

 Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function dynamodb_batch_get_item
#
# This function gets a batch of items from a DynamoDB table.
#
# Parameters:
#     -i item  -- Path to json file containing the keys of the items to get.
#
# Returns:
#     The items as json output.
#
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_batch_get_item() {
    local item response
    local option OPTARG # Required to use getopt command in a function.

    #####
    # Function usage explanation
    #####
    function usage() {
        echo "function dynamodb_batch_get_item"
        echo "Get a batch of items from a DynamoDB table."
        echo " -i item  -- Path to json file containing the keys of the items to get."
        echo ""
    }

    while getopt "i:h" option; do
        case "${option}" in
            i) item="${OPTARG}" ;;
            h)
                usage
                return 0
        esac
    done
}
```

```

        ;;
    \?)
        echo "Invalid parameter"
        usage
        return 1
    ;;
esac
done
export OPTIND=1

if [[ -z "$item" ]]; then
    errecho "ERROR: You must provide an item with the -i parameter."
    usage
    return 1
fi

response=$(aws dynamodb batch-get-item \
    --request-items file://"$item")
local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports batch-get-item operation failed.$response"
    return 1
fi

echo "$response"

return 0
}

```

As funções utilitárias usadas neste exemplo.

```

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

```



```
#####
# function aws_cli_error_log()
#
# This function is used to log the error messages from the AWS CLI.
#
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-
return-codes.
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:
#     0: - Success.
#
#####
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    elif [ "$err_code" == 252 ]; then
        errecho " Command syntax invalid."
    elif [ "$err_code" == 253 ]; then
        errecho " The system environment or configuration was invalid."
    elif [ "$err_code" == 254 ]; then
        errecho " The service returned an error."
    elif [ "$err_code" == 255 ]; then
        errecho " 255 is a catch-all error."
    fi

    return 0
}
```

- Para obter detalhes da API, consulte [BatchGetItem](#) na Referência de comandos da AWS CLI.

Obter um item de uma tabela

O exemplo de código a seguir mostra como obter um item de uma tabela do DynamoDB.

## AWS CLI com script Bash

 Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function dynamodb_get_item
#
# This function gets an item from a DynamoDB table.
#
# Parameters:
#     -n table_name  -- The name of the table.
#     -k keys        -- Path to json file containing the keys that identify the item to
#                       get.
#     [-q query]    -- Optional JMESPath query expression.
#
# Returns:
#     The item as text output.
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_get_item() {
    local table_name keys query response
    local option OPTARG # Required to use getopt command in a function.

    # #####
    # Function usage explanation
    #####
    function usage() {
        echo "function dynamodb_get_item"
        echo "Get an item from a DynamoDB table."
        echo " -n table_name  -- The name of the table."
        echo " -k keys        -- Path to json file containing the keys that identify the item
to get."
        echo " [-q query]    -- Optional JMESPath query expression."
        echo ""
    }
    query=""
```

```
while getopts "n:k:q:h" option; do
  case "${option}" in
    n) table_name="${OPTARG}" ;;
    k) keys="${OPTARG}" ;;
    q) query="${OPTARG}" ;;
    h)
      usage
      return 0
      ;;
    \?)
      echo "Invalid parameter"
      usage
      return 1
      ;;
  esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
  errecho "ERROR: You must provide a table name with the -n parameter."
  usage
  return 1
fi

if [[ -z "$keys" ]]; then
  errecho "ERROR: You must provide a keys json file path the -k parameter."
  usage
  return 1
fi

if [[ -n "$query" ]]; then
  response=$(aws dynamodb get-item \
    --table-name "$table_name" \
    --key file://"${keys}" \
    --output text \
    --query "$query")
else
  response=$(
    aws dynamodb get-item \
      --table-name "$table_name" \
      --key file://"${keys}" \
      --output text
  )
fi
```

```

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports get-item operation failed.$response"
    return 1
fi

if [[ -n "$query" ]]; then
    echo "$response" | sed "/^\t/s/\t//1" # Remove initial tab that the JMSEPath
query inserts on some strings.
else
    echo "$response"
fi

return 0
}

```

As funções utilitárias usadas neste exemplo.

```

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function aws_cli_error_log()
#
# This function is used to log the error messages from the AWS CLI.
#
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-return-codes.
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:

```

```
#          0: - Success.
#
#####
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    elif [ "$err_code" == 252 ]; then
        errecho " Command syntax invalid."
    elif [ "$err_code" == 253 ]; then
        errecho " The system environment or configuration was invalid."
    elif [ "$err_code" == 254 ]; then
        errecho " The service returned an error."
    elif [ "$err_code" == 255 ]; then
        errecho " 255 is a catch-all error."
    fi

    return 0
}
```

- Para obter detalhes da API, consulte [GetItem](#) na Referência de comandos da AWS CLI.

## Obter informações sobre uma tabela

O exemplo de código a seguir mostra como obter informações sobre uma tabela do DynamoDB.

## AWS CLI com script Bash

### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function dynamodb_describe_table
```

```

#
# This function returns the status of a DynamoDB table.
#
# Parameters:
#     -n table_name  -- The name of the table.
#
# Response:
#     - TableStatus:
#     And:
#     0 - Table is active.
#     1 - If it fails.
#####
function dynamodb_describe_table {
    local table_name
    local option OPTARG # Required to use getopt command in a function.

    #####
    # Function usage explanation
    #####
    function usage() {
        echo "function dynamodb_describe_table"
        echo "Describe the status of a DynamoDB table."
        echo "  -n table_name  -- The name of the table."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "n:h" option; do
        case "${option}" in
            n) table_name="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
    export OPTIND=1

    if [[ -z "$table_name" ]]; then

```

```

    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

local table_status
table_status=$(
    aws dynamodb describe-table \
        --table-name "$table_name" \
        --output text \
        --query 'Table.TableStatus'
)

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log "$error_code"
    errecho "ERROR: AWS reports describe-table operation failed.$table_status"
    return 1
fi

echo "$table_status"

return 0
}

```

As funções utilitárias usadas neste exemplo.

```

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function aws_cli_error_log()
#
# This function is used to log the error messages from the AWS CLI.
#

```

```
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-
return-codes.
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:
#     0: - Success.
#
#####
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    elif [ "$err_code" == 252 ]; then
        errecho " Command syntax invalid."
    elif [ "$err_code" == 253 ]; then
        errecho " The system environment or configuration was invalid."
    elif [ "$err_code" == 254 ]; then
        errecho " The service returned an error."
    elif [ "$err_code" == 255 ]; then
        errecho " 255 is a catch-all error."
    fi

    return 0
}
```

- Para obter detalhes da API, consulte [DescribeTable](#) na Referência de comandos da AWS CLI.

## Listar tabelas

O exemplo de código a seguir mostra como listar tabelas do DynamoDB.



## AWS CLI com script Bash

 Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function dynamodb_list_tables
#
# This function lists all the tables in a DynamoDB.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_list_tables() {
    response=$(aws dynamodb list-tables \
        --output text \
        --query "TableNames")

    local error_code=${?}

    if [[ $error_code -ne 0 ]]; then
        aws_cli_error_log $error_code
        errecho "ERROR: AWS reports batch-write-item operation failed.$response"
        return 1
    fi

    echo "$response" | tr -s "[:space:]" "\n"

    return 0
}
```

As funções utilitárias usadas neste exemplo.

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
```

```
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function aws_cli_error_log()
#
# This function is used to log the error messages from the AWS CLI.
#
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-
return-codes.
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:
#     0: - Success.
#
#####
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    elif [ "$err_code" == 252 ]; then
        errecho " Command syntax invalid."
    elif [ "$err_code" == 253 ]; then
        errecho " The system environment or configuration was invalid."
    elif [ "$err_code" == 254 ]; then
        errecho " The service returned an error."
    elif [ "$err_code" == 255 ]; then
        errecho " 255 is a catch-all error."
    fi

    return 0
}

```

- Para obter detalhes da API, consulte [ListTable](#) na Referência de comandos da AWS CLI.

## Colocar um item em uma tabela

O exemplo de código a seguir mostra como colocar um item em uma tabela do DynamoDB.

### AWS CLI com script Bash

#### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function dynamodb_put_item
#
# This function puts an item into a DynamoDB table.
#
# Parameters:
#     -n table_name -- The name of the table.
#     -i item -- Path to json file containing the item values.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_put_item() {
    local table_name item response
    local option OPTARG # Required to use getopt command in a function.

    #####
    # Function usage explanation
    #####
    function usage() {
        echo "function dynamodb_put_item"
        echo "Put an item into a DynamoDB table."
        echo " -n table_name -- The name of the table."
        echo " -i item -- Path to json file containing the item values."
        echo ""
    }

    while getopts "n:i:h" option; do
        case "${option}" in
            n) table_name="${OPTARG}" ;;
```

```
i) item="${OPTARG}" ;;
h)
    usage
    return 0
    ;;
\?)
    echo "Invalid parameter"
    usage
    return 1
    ;;
esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$item" ]]; then
    errecho "ERROR: You must provide an item with the -i parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    table_name:  $table_name"
iecho "    item:       $item"
iecho ""
iecho ""

response=$(aws dynamodb put-item \
    --table-name "$table_name" \
    --item file://"${item}")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports put-item operation failed.$response"
    return 1
fi
```

```

    return 0

}

```

As funções utilitárias usadas neste exemplo.

```

#####
# function iecho
#
# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.
#####
function iecho() {
    if [[ $VERBOSE == true ]]; then
        echo "$@"
    fi
}

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function aws_cli_error_log()
#
# This function is used to log the error messages from the AWS CLI.
#
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-
return-codes.
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:
#     0: - Success.
#
#####

```

```
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    elif [ "$err_code" == 252 ]; then
        errecho " Command syntax invalid."
    elif [ "$err_code" == 253 ]; then
        errecho " The system environment or configuration was invalid."
    elif [ "$err_code" == 254 ]; then
        errecho " The service returned an error."
    elif [ "$err_code" == 255 ]; then
        errecho " 255 is a catch-all error."
    fi

    return 0
}
```

- Para obter detalhes da API, consulte [PutItem](#) na Referência de comandos da AWS CLI.

## Consultar uma tabela

O exemplo de código a seguir mostra como consultar uma tabela do DynamoDB.

## AWS CLI com script Bash

### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function dynamodb_query
#
# This function queries a DynamoDB table.
#
```

```

# Parameters:
#     -n table_name -- The name of the table.
#     -k key_condition_expression -- The key condition expression.
#     -a attribute_names -- Path to JSON file containing the attribute names.
#     -v attribute_values -- Path to JSON file containing the attribute values.
#     [-p projection_expression] -- Optional projection expression.
#
# Returns:
#     The items as json output.
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_query() {
    local table_name key_condition_expression attribute_names attribute_values
    projection_expression response
    local option OPTARG # Required to use getopt command in a function.

    # #####
    # Function usage explanation
    #####
    function usage() {
        echo "function dynamodb_query"
        echo "Query a DynamoDB table."
        echo " -n table_name -- The name of the table."
        echo " -k key_condition_expression -- The key condition expression."
        echo " -a attribute_names -- Path to JSON file containing the attribute names."
        echo " -v attribute_values -- Path to JSON file containing the attribute
values."
        echo " [-p projection_expression] -- Optional projection expression."
        echo ""
    }

    while getopt "n:k:a:v:p:h" option; do
        case "${option}" in
            n) table_name="${OPTARG}" ;;
            k) key_condition_expression="${OPTARG}" ;;
            a) attribute_names="${OPTARG}" ;;
            v) attribute_values="${OPTARG}" ;;
            p) projection_expression="${OPTARG}" ;;
            h)
                usage
                return 0
            ;;
        esac
    done
}

```

```
\?)
    echo "Invalid parameter"
    usage
    return 1
    ;;
esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$key_condition_expression" ]]; then
    errecho "ERROR: You must provide a key condition expression with the -k
parameter."
    usage
    return 1
fi

if [[ -z "$attribute_names" ]]; then
    errecho "ERROR: You must provide a attribute names with the -a parameter."
    usage
    return 1
fi

if [[ -z "$attribute_values" ]]; then
    errecho "ERROR: You must provide a attribute values with the -v parameter."
    usage
    return 1
fi

if [[ -z "$projection_expression" ]]; then
    response=$(aws dynamodb query \
        --table-name "$table_name" \
        --key-condition-expression "$key_condition_expression" \
        --expression-attribute-names file://"${attribute_names}" \
        --expression-attribute-values file://"${attribute_values}")
else
    response=$(aws dynamodb query \
        --table-name "$table_name" \
        --key-condition-expression "$key_condition_expression" \
```



```

        --expression-attribute-names file://"${attribute_names}" \
        --expression-attribute-values file://"${attribute_values}" \
        --projection-expression "${projection_expression}")
    fi

    local error_code=${?}

    if [[ $error_code -ne 0 ]]; then
        aws_cli_error_log $error_code
        errecho "ERROR: AWS reports query operation failed.$response"
        return 1
    fi

    echo "$response"

    return 0
}

```

As funções utilitárias usadas neste exemplo.

```

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function aws_cli_error_log()
#
# This function is used to log the error messages from the AWS CLI.
#
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-return-codes.
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:
#     0: - Success.

```

```
#
#####
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    elif [ "$err_code" == 252 ]; then
        errecho " Command syntax invalid."
    elif [ "$err_code" == 253 ]; then
        errecho " The system environment or configuration was invalid."
    elif [ "$err_code" == 254 ]; then
        errecho " The service returned an error."
    elif [ "$err_code" == 255 ]; then
        errecho " 255 is a catch-all error."
    fi

    return 0
}
```

- Para obter detalhes da API, consulte [Query](#) na Referência de comandos da AWS CLI.

## Verificar uma tabela

O exemplo de código a seguir mostra como verificar uma tabela do DynamoDB.

## AWS CLI com script Bash

### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function dynamodb_scan
#
```

```

# This function scans a DynamoDB table.
#
# Parameters:
#     -n table_name -- The name of the table.
#     -f filter_expression -- The filter expression.
#     -a expression_attribute_names -- Path to JSON file containing the expression
#     attribute names.
#     -v expression_attribute_values -- Path to JSON file containing the
#     expression attribute values.
#     [-p projection_expression] -- Optional projection expression.
#
# Returns:
#     The items as json output.
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_scan() {
    local table_name filter_expression expression_attribute_names
    expression_attribute_values projection_expression response
    local option OPTARG # Required to use getopt command in a function.

    # #####
    # Function usage explanation
    #####
    function usage() {
        echo "function dynamodb_scan"
        echo "Scan a DynamoDB table."
        echo " -n table_name -- The name of the table."
        echo " -f filter_expression -- The filter expression."
        echo " -a expression_attribute_names -- Path to JSON file containing the
expression attribute names."
        echo " -v expression_attribute_values -- Path to JSON file containing the
expression attribute values."
        echo " [-p projection_expression] -- Optional projection expression."
        echo ""
    }

    while getopt "n:f:a:v:p:h" option; do
        case "${option}" in
            n) table_name="${OPTARG}" ;;
            f) filter_expression="${OPTARG}" ;;
            a) expression_attribute_names="${OPTARG}" ;;
            v) expression_attribute_values="${OPTARG}" ;;

```

```
p) projection_expression="${OPTARG}" ;;
h)
    usage
    return 0
    ;;
\?)
    echo "Invalid parameter"
    usage
    return 1
    ;;
esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$filter_expression" ]]; then
    errecho "ERROR: You must provide a filter expression with the -f parameter."
    usage
    return 1
fi

if [[ -z "$expression_attribute_names" ]]; then
    errecho "ERROR: You must provide expression attribute names with the -a
parameter."
    usage
    return 1
fi

if [[ -z "$expression_attribute_values" ]]; then
    errecho "ERROR: You must provide expression attribute values with the -v
parameter."
    usage
    return 1
fi

if [[ -z "$projection_expression" ]]; then
    response=$(aws dynamodb scan \
        --table-name "$table_name" \
        --filter-expression "$filter_expression" \
```

```

        --expression-attribute-names file://"${expression_attribute_names}" \
        --expression-attribute-values file://"${expression_attribute_values}")
else
    response=$(aws dynamodb scan \
        --table-name "${table_name}" \
        --filter-expression "${filter_expression}" \
        --expression-attribute-names file://"${expression_attribute_names}" \
        --expression-attribute-values file://"${expression_attribute_values}" \
        --projection-expression "${projection_expression}")
fi

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports scan operation failed.$response"
    return 1
fi

echo "$response"

return 0
}

```

As funções utilitárias usadas neste exemplo.

```

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function aws_cli_error_log()
#
# This function is used to log the error messages from the AWS CLI.
#
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-
return-codes.

```

```
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:
#     0: - Success.
#
#####
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    elif [ "$err_code" == 252 ]; then
        errecho " Command syntax invalid."
    elif [ "$err_code" == 253 ]; then
        errecho " The system environment or configuration was invalid."
    elif [ "$err_code" == 254 ]; then
        errecho " The service returned an error."
    elif [ "$err_code" == 255 ]; then
        errecho " 255 is a catch-all error."
    fi

    return 0
}
```

- Para obter detalhes da API, consulte [Scan](#) na Referência de comandos da AWS CLI.

## Atualizar um item em uma tabela

O exemplo de código a seguir mostra como atualizar um item em uma tabela do DynamoDB.

## AWS CLI com script Bash

 Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function dynamodb_update_item
#
# This function updates an item in a DynamoDB table.
#
#
# Parameters:
#     -n table_name  -- The name of the table.
#     -k keys       -- Path to json file containing the keys that identify the item to
#                     update.
#     -e update expression  -- An expression that defines one or more attributes
#                     to be updated.
#     -v values     -- Path to json file containing the update values.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_update_item() {
    local table_name keys update_expression values response
    local option OPTARG # Required to use getopt command in a function.

    #####
    # Function usage explanation
    #####
    function usage() {
        echo "function dynamodb_update_item"
        echo "Update an item in a DynamoDB table."
        echo "  -n table_name  -- The name of the table."
        echo "  -k keys       -- Path to json file containing the keys that identify the item
to update."
        echo "  -e update expression  -- An expression that defines one or more
attributes to be updated."
        echo "  -v values     -- Path to json file containing the update values."
    }
}
```

```
    echo ""
}

while getopts "n:k:e:v:h" option; do
    case "${option}" in
        n) table_name="${OPTARG}" ;;
        k) keys="${OPTARG}" ;;
        e) update_expression="${OPTARG}" ;;
        v) values="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$keys" ]]; then
    errecho "ERROR: You must provide a keys json file path the -k parameter."
    usage
    return 1
fi

if [[ -z "$update_expression" ]]; then
    errecho "ERROR: You must provide an update expression with the -e parameter."
    usage
    return 1
fi

if [[ -z "$values" ]]; then
    errecho "ERROR: You must provide a values json file path the -v parameter."
    usage
    return 1
fi
```



```

iecho "Parameters:\n"
iecho "    table_name:  $table_name"
iecho "    keys:    $keys"
iecho "    update_expression:  $update_expression"
iecho "    values:    $values"

response=$(aws dynamodb update-item \
  --table-name "$table_name" \
  --key file://" $keys" \
  --update-expression "$update_expression" \
  --expression-attribute-values file://" $values")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
  aws_cli_error_log $error_code
  errecho "ERROR: AWS reports update-item operation failed.$response"
  return 1
fi

return 0
}

```

As funções utilitárias usadas neste exemplo.

```

#####
# function iecho
#
# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.
#####
function iecho() {
  if [[ $VERBOSE == true ]]; then
    echo "$@"
  fi
}

#####
# function errecho
#

```

```

# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function aws_cli_error_log()
#
# This function is used to log the error messages from the AWS CLI.
#
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-return-codes.
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:
#     0: - Success.
#
#####
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    elif [ "$err_code" == 252 ]; then
        errecho " Command syntax invalid."
    elif [ "$err_code" == 253 ]; then
        errecho " The system environment or configuration was invalid."
    elif [ "$err_code" == 254 ]; then
        errecho " The service returned an error."
    elif [ "$err_code" == 255 ]; then
        errecho " 255 is a catch-all error."
    fi

    return 0
}

```

- Para obter detalhes da API, consulte [UpdateItem](#) na Referência de comandos da AWS CLI.

## Gravar um lote de itens

O exemplo de código a seguir mostra como gravar um lote de itens do DynamoDB.

## AWS CLI com script Bash

### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function dynamodb_batch_write_item
#
# This function writes a batch of items into a DynamoDB table.
#
# Parameters:
#     -i item  -- Path to json file containing the items to write.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_batch_write_item() {
    local item response
    local option OPTARG # Required to use getopt command in a function.

    #####
    # Function usage explanation
    #####
    function usage() {
        echo "function dynamodb_batch_write_item"
        echo "Write a batch of items into a DynamoDB table."
        echo " -i item  -- Path to json file containing the items to write."
        echo ""
    }
    while getopt "i:h" option; do
        case "${option}" in
            i) item="${OPTARG}" ;;

```

```

    h)
        usage
        return 0
        ;;
    \?)
        echo "Invalid parameter"
        usage
        return 1
        ;;
    esac
done
export OPTIND=1

if [[ -z "$item" ]]; then
    errecho "ERROR: You must provide an item with the -i parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    table_name:  $table_name"
iecho "    item:       $item"
iecho ""

response=$(aws dynamodb batch-write-item \
    --request-items file://"${item}")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports batch-write-item operation failed.$response"
    return 1
fi

return 0
}

```

As funções utilitárias usadas neste exemplo.

```

#####
# function iecho

```

```

#
# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.
#####
function iecho() {
    if [[ $VERBOSE == true ]]; then
        echo "$@"
    fi
}

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function aws_cli_error_log()
#
# This function is used to log the error messages from the AWS CLI.
#
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-return-codes.
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:
#     0: - Success.
#
#####
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    elif [ "$err_code" == 252 ]; then

```

```

    errecho " Command syntax invalid."
elif [ "$err_code" == 253 ]; then
    errecho " The system environment or configuration was invalid."
elif [ "$err_code" == 254 ]; then
    errecho " The service returned an error."
elif [ "$err_code" == 255 ]; then
    errecho " 255 is a catch-all error."
fi

return 0
}

```

- Para obter detalhes da API, consulte [BatchWriteItem](#) na Referência de comandos da AWS CLI.

## Cenários

### Conceitos básicos de tabelas, itens e consultas

O código de exemplo a seguir mostra como:

- Criar uma tabela que possa conter dados de filmes.
- Colocar, obter e atualizar um único filme na tabela.
- Grave dados de filmes na tabela usando um arquivo JSON de exemplo.
- Consultar filmes que foram lançados em determinado ano.
- Verificar filmes que foram lançados em um intervalo de anos.
- Exclua um filme da tabela e, depois, exclua a tabela.

### AWS CLI com script Bash

#### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

O cenário de conceitos básicos do DynamoDB.

```
#####
```

```
# function dynamodb_getting_started_movies
#
# Scenario to create an Amazon DynamoDB table and perform a series of operations on
# the table.
#
# Returns:
#     0 - If successful.
#     1 - If an error occurred.
#####
function dynamodb_getting_started_movies() {

    source ./dynamodb_operations.sh

    key_schema_json_file="dynamodb_key_schema.json"
    attribute_definitions_json_file="dynamodb_attr_def.json"
    item_json_file="movie_item.json"
    key_json_file="movie_key.json"
    batch_json_file="batch.json"
    attribute_names_json_file="attribute_names.json"
    attributes_values_json_file="attribute_values.json"

    echo_repeat "*" 88
    echo
    echo "Welcome to the Amazon DynamoDB getting started demo."
    echo
    echo_repeat "*" 88
    echo

    local table_name
    echo -n "Enter a name for a new DynamoDB table: "
    get_input
    table_name=$get_input_result

    local provisioned_throughput="ReadCapacityUnits=5,WriteCapacityUnits=5"

    echo '[
{"AttributeName": "year", "KeyType": "HASH"},
{"AttributeName": "title", "KeyType": "RANGE"}
]' >"$key_schema_json_file"

    echo '[
{"AttributeName": "year", "AttributeType": "N"},
{"AttributeName": "title", "AttributeType": "S"}
]' >"$attribute_definitions_json_file"
```

```
if dynamodb_create_table -n "$table_name" -a "$attribute_definitions_json_file" \
-k "$key_schema_json_file" -p "$provisioned_throughput" 1>/dev/null; then
    echo "Created a DynamoDB table named $table_name"
else
    errecho "The table failed to create. This demo will exit."
    clean_up
    return 1
fi

echo "Waiting for the table to become active...."

if dynamodb_wait_table_active -n "$table_name"; then
    echo "The table is now active."
else
    errecho "The table failed to become active. This demo will exit."
    cleanup "$table_name"
    return 1
fi

echo
echo_repeat "*" 88
echo

echo -n "Enter the title of a movie you want to add to the table: "
get_input
local added_title
added_title=$get_input_result

local added_year
get_int_input "What year was it released? "
added_year=$get_input_result

local rating
get_float_input "On a scale of 1 - 10, how do you rate it? " "1" "10"
rating=$get_input_result

local plot
echo -n "Summarize the plot for me: "
get_input
plot=$get_input_result

echo '{
    "year": {"N" : "'"$added_year"'"},
```



```

    "title": {"S" : ""$added_title""},
    "info": {"M" : {"plot": {"S" : ""$plot""}, "rating": {"N" : ""$rating""} } }
  }' >"$item_json_file"

if dynamodb_put_item -n "$table_name" -i "$item_json_file"; then
  echo "The movie '$added_title' was successfully added to the table
'$table_name'."
else
  errecho "Put item failed. This demo will exit."
  clean_up "$table_name"
  return 1
fi

echo
echo_repeat "*" 88
echo

echo "Let's update your movie '$added_title'."
get_float_input "You rated it $rating, what new rating would you give it? " "1"
"10"
rating=$get_input_result

echo -n "You summarized the plot as '$plot'."
echo "What would you say now? "
get_input
plot=$get_input_result

echo '{
  "year": {"N" : ""$added_year""},
  "title": {"S" : ""$added_title""}
}' >"$key_json_file"

echo '{
  ":r": {"N" : ""$rating""},
  ":p": {"S" : ""$plot""}
}' >"$item_json_file"

local update_expression="SET info.rating = :r, info.plot = :p"

if dynamodb_update_item -n "$table_name" -k "$key_json_file" -e
"$update_expression" -v "$item_json_file"; then
  echo "Updated '$added_title' with new attributes."
else
  errecho "Update item failed. This demo will exit."

```

```
    clean_up "$table_name"
    return 1
fi

echo
echo_repeat "*" 88
echo

echo "We will now use batch write to upload 150 movie entries into the table."

local batch_json
for batch_json in movie_files/movies_*.json; do
    echo "{ \"$table_name\" : $(<"$batch_json") }" >"$batch_json_file"
    if dynamodb_batch_write_item -i "$batch_json_file" 1>/dev/null; then
        echo "Entries in $batch_json added to table."
    else
        errecho "Batch write failed. This demo will exit."
        clean_up "$table_name"
        return 1
    fi
done

local title="The Lord of the Rings: The Fellowship of the Ring"
local year="2001"

if get_yes_no_input "Let's move on...do you want to get info about '$title'? (y/n)"; then
    echo '{
"year": {"N" : "'$year'"},
"title": {"S" : "'$title'"}
}' >"$key_json_file"
    local info
    info=$(dynamodb_get_item -n "$table_name" -k "$key_json_file")

    # shellcheck disable=SC2181
    if [[ ${?} -ne 0 ]]; then
        errecho "Get item failed. This demo will exit."
        clean_up "$table_name"
        return 1
    fi

    echo "Here is what I found:"
    echo "$info"
fi
```

```

local ask_for_year=true
while [[ "$ask_for_year" == true ]]; do
    echo "Let's get a list of movies released in a given year."
    get_int_input "Enter a year between 1972 and 2018: " "1972" "2018"
    year=$get_input_result
    echo '{
"#n": "year"
}' >"$attribute_names_json_file"

    echo '{
":v": {"N" :""$year""}
}' >"$attributes_values_json_file"

    response=$(dynamodb_query -n "$table_name" -k "#n=:v" -a
"$attribute_names_json_file" -v "$attributes_values_json_file")

    # shellcheck disable=SC2181
    if [[ ${?} -ne 0 ]]; then
        errecho "Query table failed. This demo will exit."
        clean_up "$table_name"
        return 1
    fi

    echo "Here is what I found:"
    echo "$response"

    if ! get_yes_no_input "Try another year? (y/n) "; then
        ask_for_year=false
    fi
done

echo "Now let's scan for movies released in a range of years. Enter a year: "
get_int_input "Enter a year between 1972 and 2018: " "1972" "2018"
local start=$get_input_result

get_int_input "Enter another year: " "1972" "2018"
local end=$get_input_result

echo '{
"#n": "year"
}' >"$attribute_names_json_file"

echo '{

```

```

    ":v1": {"N" : ""$start""},
    ":v2": {"N" : ""$end""}
  }' >"$attributes_values_json_file"

response=$(dynamodb_scan -n "$table_name" -f "#n BETWEEN :v1 AND :v2" -a
"$attribute_names_json_file" -v "$attributes_values_json_file")

# shellcheck disable=SC2181
if [[ ${?} -ne 0 ]]; then
    errecho "Scan table failed. This demo will exit."
    clean_up "$table_name"
    return 1
fi

echo "Here is what I found:"
echo "$response"

echo
echo_repeat "*" 88
echo

echo "Let's remove your movie '$added_title' from the table."

if get_yes_no_input "Do you want to remove '$added_title'? (y/n) "; then
    echo '{
"year": {"N" : ""$added_year""},
"title": {"S" : ""$added_title""}
}' >"$key_json_file"

    if ! dynamodb_delete_item -n "$table_name" -k "$key_json_file"; then
        errecho "Delete item failed. This demo will exit."
        clean_up "$table_name"
        return 1
    fi
fi

if get_yes_no_input "Do you want to delete the table '$table_name'? (y/n) "; then
    if ! clean_up "$table_name"; then
        return 1
    fi
else
    if ! clean_up; then
        return 1
    fi
fi

```

```

fi

return 0
}

```

As funções do DynamoDB usadas nesse cenário.

```

#####
# function dynamodb_create_table
#
# This function creates an Amazon DynamoDB table.
#
# Parameters:
#     -n table_name -- The name of the table to create.
#     -a attribute_definitions -- JSON file path of a list of attributes and their
types.
#     -k key_schema -- JSON file path of a list of attributes and their key types.
#     -p provisioned_throughput -- Provisioned throughput settings for the table.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_create_table() {
    local table_name attribute_definitions key_schema provisioned_throughput response
    local option OPTARG # Required to use getopt command in a function.

    #####
    # Function usage explanation
    #####
    function usage() {
        echo "function dynamodb_create_table"
        echo "Creates an Amazon DynamoDB table."
        echo " -n table_name -- The name of the table to create."
        echo " -a attribute_definitions -- JSON file path of a list of attributes and
their types."
        echo " -k key_schema -- JSON file path of a list of attributes and their key
types."
        echo " -p provisioned_throughput -- Provisioned throughput settings for the
table."
        echo ""
    }
}

```

```
# Retrieve the calling parameters.
while getopts "n:a:k:p:h" option; do
    case "${option}" in
        n) table_name="${OPTARG}" ;;
        a) attribute_definitions="${OPTARG}" ;;
        k) key_schema="${OPTARG}" ;;
        p) provisioned_throughput="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$attribute_definitions" ]]; then
    errecho "ERROR: You must provide an attribute definitions json file path the -a
parameter."
    usage
    return 1
fi

if [[ -z "$key_schema" ]]; then
    errecho "ERROR: You must provide a key schema json file path the -k parameter."
    usage
    return 1
fi

if [[ -z "$provisioned_throughput" ]]; then
    errecho "ERROR: You must provide a provisioned throughput json file path the -p
parameter."
    usage
```

```

    return 1
fi

iecho "Parameters:\n"
iecho "    table_name:    $table_name"
iecho "    attribute_definitions:  $attribute_definitions"
iecho "    key_schema:    $key_schema"
iecho "    provisioned_throughput:  $provisioned_throughput"
iecho ""

response=$(aws dynamodb create-table \
    --table-name "$table_name" \
    --attribute-definitions file://"${attribute_definitions}" \
    --key-schema file://"${key_schema}" \
    --provisioned-throughput "$provisioned_throughput")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports create-table operation failed.$response"
    return 1
fi

return 0
}

#####
# function dynamodb_describe_table
#
# This function returns the status of a DynamoDB table.
#
# Parameters:
#     -n table_name  -- The name of the table.
#
# Response:
#     - TableStatus:
#     And:
#     0 - Table is active.
#     1 - If it fails.
#####
function dynamodb_describe_table {
    local table_name
    local option OPTARG # Required to use getopt command in a function.

```

```
#####
# Function usage explanation
#####
function usage() {
    echo "function dynamodb_describe_table"
    echo "Describe the status of a DynamoDB table."
    echo "  -n table_name  -- The name of the table."
    echo ""
}

# Retrieve the calling parameters.
while getopts "n:h" option; do
    case "${option}" in
        n) table_name="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

local table_status
table_status=$(
    aws dynamodb describe-table \
        --table-name "$table_name" \
        --output text \
        --query 'Table.TableStatus'
)

local error_code=${?}
```



```

    if [[ $error_code -ne 0 ]]; then
        aws_cli_error_log "$error_code"
        errecho "ERROR: AWS reports describe-table operation failed.$table_status"
        return 1
    fi

    echo "$table_status"

    return 0
}

#####
# function dynamodb_put_item
#
# This function puts an item into a DynamoDB table.
#
# Parameters:
#     -n table_name -- The name of the table.
#     -i item -- Path to json file containing the item values.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_put_item() {
    local table_name item response
    local option OPTARG # Required to use getopt command in a function.

    #####
    # Function usage explanation
    #####
    function usage() {
        echo "function dynamodb_put_item"
        echo "Put an item into a DynamoDB table."
        echo " -n table_name -- The name of the table."
        echo " -i item -- Path to json file containing the item values."
        echo ""
    }

    while getopt "n:i:h" option; do
        case "${option}" in
            n) table_name="${OPTARG}" ;;
            i) item="${OPTARG}" ;;
            h)

```

```
        usage
        return 0
        ;;
    \?)
        echo "Invalid parameter"
        usage
        return 1
        ;;
    esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$item" ]]; then
    errecho "ERROR: You must provide an item with the -i parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    table_name:  $table_name"
iecho "    item:       $item"
iecho ""
iecho ""

response=$(aws dynamodb put-item \
    --table-name "$table_name" \
    --item file://"${item}")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports put-item operation failed.$response"
    return 1
fi

return 0
```

```

}

#####
# function dynamodb_update_item
#
# This function updates an item in a DynamoDB table.
#
#
# Parameters:
#     -n table_name  -- The name of the table.
#     -k keys       -- Path to json file containing the keys that identify the item to
#                     update.
#     -e update expression  -- An expression that defines one or more attributes
#                     to be updated.
#     -v values     -- Path to json file containing the update values.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_update_item() {
    local table_name keys update_expression values response
    local option OPTARG # Required to use getopt command in a function.

    #####
    # Function usage explanation
    #####
    function usage() {
        echo "function dynamodb_update_item"
        echo "Update an item in a DynamoDB table."
        echo " -n table_name  -- The name of the table."
        echo " -k keys       -- Path to json file containing the keys that identify the item
to update."
        echo " -e update expression  -- An expression that defines one or more
attributes to be updated."
        echo " -v values     -- Path to json file containing the update values."
        echo ""
    }

    while getopts "n:k:e:v:h" option; do
        case "${option}" in
            n) table_name="${OPTARG}" ;;
            k) keys="${OPTARG}" ;;
            e) update_expression="${OPTARG}" ;;

```

```
v) values="${OPTARG}" ;;
h)
    usage
    return 0
    ;;
\?)
    echo "Invalid parameter"
    usage
    return 1
    ;;
esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$keys" ]]; then
    errecho "ERROR: You must provide a keys json file path the -k parameter."
    usage
    return 1
fi

if [[ -z "$update_expression" ]]; then
    errecho "ERROR: You must provide an update expression with the -e parameter."
    usage
    return 1
fi

if [[ -z "$values" ]]; then
    errecho "ERROR: You must provide a values json file path the -v parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    table_name:  $table_name"
iecho "    keys:       $keys"
iecho "    update_expression:  $update_expression"
iecho "    values:     $values"

response=$(aws dynamodb update-item \
```

```

--table-name "$table_name" \
--key file://" $keys" \
--update-expression "$update_expression" \
--expression-attribute-values file://" $values")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports update-item operation failed.$response"
    return 1
fi

return 0
}

#####
# function dynamodb_batch_write_item
#
# This function writes a batch of items into a DynamoDB table.
#
# Parameters:
#     -i item -- Path to json file containing the items to write.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_batch_write_item() {
    local item response
    local option OPTARG # Required to use getopt command in a function.

    #####
    # Function usage explanation
    #####
    function usage() {
        echo "function dynamodb_batch_write_item"
        echo "Write a batch of items into a DynamoDB table."
        echo " -i item -- Path to json file containing the items to write."
        echo ""
    }
    while getopt "i:h" option; do
        case "${option}" in

```

```

        i) item="${OPTARG}" ;;
    h)
        usage
        return 0
        ;;
    \?)
        echo "Invalid parameter"
        usage
        return 1
        ;;
    esac
done
export OPTIND=1

if [[ -z "$item" ]]; then
    errecho "ERROR: You must provide an item with the -i parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    table_name:  $table_name"
iecho "    item:        $item"
iecho ""

response=$(aws dynamodb batch-write-item \
    --request-items file://"$item")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports batch-write-item operation failed.$response"
    return 1
fi

return 0
}

#####
# function dynamodb_get_item
#
# This function gets an item from a DynamoDB table.
#

```

```

# Parameters:
#     -n table_name  -- The name of the table.
#     -k keys       -- Path to json file containing the keys that identify the item to
# get.
#     [-q query]    -- Optional JMESPath query expression.
#
# Returns:
#     The item as text output.
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_get_item() {
    local table_name keys query response
    local option OPTARG # Required to use getopt command in a function.

    # #####
    # Function usage explanation
    #####
    function usage() {
        echo "function dynamodb_get_item"
        echo "Get an item from a DynamoDB table."
        echo " -n table_name  -- The name of the table."
        echo " -k keys       -- Path to json file containing the keys that identify the item
to get."
        echo " [-q query]  -- Optional JMESPath query expression."
        echo ""
    }
    query=""
    while getopt "n:k:q:h" option; do
        case "${option}" in
            n) table_name="${OPTARG}" ;;
            k) keys="${OPTARG}" ;;
            q) query="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
}

```

```
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$keys" ]]; then
    errecho "ERROR: You must provide a keys json file path the -k parameter."
    usage
    return 1
fi

if [[ -n "$query" ]]; then
    response=$(aws dynamodb get-item \
        --table-name "$table_name" \
        --key file://"${keys}" \
        --output text \
        --query "$query")
else
    response=$(
        aws dynamodb get-item \
            --table-name "$table_name" \
            --key file://"${keys}" \
            --output text
    )
fi

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports get-item operation failed.$response"
    return 1
fi

if [[ -n "$query" ]]; then
    echo "$response" | sed "/^\t/s/\t//1" # Remove initial tab that the JMSEPath
query inserts on some strings.
else
    echo "$response"
fi
```



```

    return 0
}

#####
# function dynamodb_query
#
# This function queries a DynamoDB table.
#
# Parameters:
#     -n table_name -- The name of the table.
#     -k key_condition_expression -- The key condition expression.
#     -a attribute_names -- Path to JSON file containing the attribute names.
#     -v attribute_values -- Path to JSON file containing the attribute values.
#     [-p projection_expression] -- Optional projection expression.
#
# Returns:
#     The items as json output.
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_query() {
    local table_name key_condition_expression attribute_names attribute_values
    projection_expression response
    local option OPTARG # Required to use getopt command in a function.

    # #####
    # Function usage explanation
    #####
    function usage() {
        echo "function dynamodb_query"
        echo "Query a DynamoDB table."
        echo " -n table_name -- The name of the table."
        echo " -k key_condition_expression -- The key condition expression."
        echo " -a attribute_names -- Path to JSON file containing the attribute names."
        echo " -v attribute_values -- Path to JSON file containing the attribute
values."
        echo " [-p projection_expression] -- Optional projection expression."
        echo ""
    }

    while getopt "n:k:a:v:p:h" option; do
        case "${option}" in

```

```
n) table_name="${OPTARG}" ;;
k) key_condition_expression="${OPTARG}" ;;
a) attribute_names="${OPTARG}" ;;
v) attribute_values="${OPTARG}" ;;
p) projection_expression="${OPTARG}" ;;
h)
    usage
    return 0
    ;;
\?)
    echo "Invalid parameter"
    usage
    return 1
    ;;
esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$key_condition_expression" ]]; then
    errecho "ERROR: You must provide a key condition expression with the -k
parameter."
    usage
    return 1
fi

if [[ -z "$attribute_names" ]]; then
    errecho "ERROR: You must provide a attribute names with the -a parameter."
    usage
    return 1
fi

if [[ -z "$attribute_values" ]]; then
    errecho "ERROR: You must provide a attribute values with the -v parameter."
    usage
    return 1
fi

if [[ -z "$projection_expression" ]]; then
```

```

    response=$(aws dynamodb query \
        --table-name "$table_name" \
        --key-condition-expression "$key_condition_expression" \
        --expression-attribute-names file://"$attribute_names" \
        --expression-attribute-values file://"$attribute_values")
else
    response=$(aws dynamodb query \
        --table-name "$table_name" \
        --key-condition-expression "$key_condition_expression" \
        --expression-attribute-names file://"$attribute_names" \
        --expression-attribute-values file://"$attribute_values" \
        --projection-expression "$projection_expression")
fi

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports query operation failed.$response"
    return 1
fi

echo "$response"

return 0
}

#####
# function dynamodb_scan
#
# This function scans a DynamoDB table.
#
# Parameters:
#     -n table_name -- The name of the table.
#     -f filter_expression -- The filter expression.
#     -a expression_attribute_names -- Path to JSON file containing the expression
#     attribute names.
#     -v expression_attribute_values -- Path to JSON file containing the
#     expression attribute values.
#     [-p projection_expression] -- Optional projection expression.
#
# Returns:
#     The items as json output.
#
# And:

```

```

#      0 - If successful.
#      1 - If it fails.
#####
function dynamodb_scan() {
    local table_name filter_expression expression_attribute_names
    expression_attribute_values projection_expression response
    local option OPTARG # Required to use getopt command in a function.

    # #####
    # Function usage explanation
    #####
    function usage() {
        echo "function dynamodb_scan"
        echo "Scan a DynamoDB table."
        echo " -n table_name -- The name of the table."
        echo " -f filter_expression -- The filter expression."
        echo " -a expression_attribute_names -- Path to JSON file containing the
expression attribute names."
        echo " -v expression_attribute_values -- Path to JSON file containing the
expression attribute values."
        echo " [-p projection_expression] -- Optional projection expression."
        echo ""
    }

    while getopt "n:f:a:v:p:h" option; do
        case "${option}" in
            n) table_name="${OPTARG}" ;;
            f) filter_expression="${OPTARG}" ;;
            a) expression_attribute_names="${OPTARG}" ;;
            v) expression_attribute_values="${OPTARG}" ;;
            p) projection_expression="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
    export OPTIND=1

```

```
if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$filter_expression" ]]; then
    errecho "ERROR: You must provide a filter expression with the -f parameter."
    usage
    return 1
fi

if [[ -z "$expression_attribute_names" ]]; then
    errecho "ERROR: You must provide expression attribute names with the -a
parameter."
    usage
    return 1
fi

if [[ -z "$expression_attribute_values" ]]; then
    errecho "ERROR: You must provide expression attribute values with the -v
parameter."
    usage
    return 1
fi

if [[ -z "$projection_expression" ]]; then
    response=$(aws dynamodb scan \
        --table-name "$table_name" \
        --filter-expression "$filter_expression" \
        --expression-attribute-names file://"${expression_attribute_names}" \
        --expression-attribute-values file://"${expression_attribute_values}")
else
    response=$(aws dynamodb scan \
        --table-name "$table_name" \
        --filter-expression "$filter_expression" \
        --expression-attribute-names file://"${expression_attribute_names}" \
        --expression-attribute-values file://"${expression_attribute_values}" \
        --projection-expression "$projection_expression")
fi

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
```

```

    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports scan operation failed.$response"
    return 1
fi

echo "$response"

return 0
}

#####
# function dynamodb_delete_item
#
# This function deletes an item from a DynamoDB table.
#
# Parameters:
#     -n table_name  -- The name of the table.
#     -k keys        -- Path to json file containing the keys that identify the item to
#                       delete.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_delete_item() {
    local table_name keys response
    local option OPTARG # Required to use getopt command in a function.

    # #####
    # Function usage explanation
    #####
    function usage() {
        echo "function dynamodb_delete_item"
        echo "Delete an item from a DynamoDB table."
        echo " -n table_name  -- The name of the table."
        echo " -k keys        -- Path to json file containing the keys that identify the item
to delete."
        echo ""
    }
    while getopt "n:k:h" option; do
        case "${option}" in
            n) table_name="${OPTARG}" ;;
            k) keys="${OPTARG}" ;;
            h)

```

```
        usage
        return 0
        ;;
    \?)
        echo "Invalid parameter"
        usage
        return 1
        ;;
    esac
done
export OPTIND=1

if [[ -z "$table_name" ]]; then
    errecho "ERROR: You must provide a table name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$keys" ]]; then
    errecho "ERROR: You must provide a keys json file path the -k parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    table_name:  $table_name"
iecho "    keys:       $keys"
iecho ""

response=$(aws dynamodb delete-item \
    --table-name "$table_name" \
    --key file://"${keys}")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports delete-item operation failed.$response"
    return 1
fi

return 0
}
```

```
#####
# function dynamodb_delete_table
#
# This function deletes a DynamoDB table.
#
# Parameters:
#     -n table_name  -- The name of the table to delete.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function dynamodb_delete_table() {
    local table_name response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function dynamodb_delete_table"
        echo "Deletes an Amazon DynamoDB table."
        echo " -n table_name  -- The name of the table to delete."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "n:h" option; do
        case "${option}" in
            n) table_name="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
    export OPTIND=1

    if [[ -z "$table_name" ]]; then
        errecho "ERROR: You must provide a table name with the -n parameter."
    fi
}
```



```

        usage
        return 1
    fi

    iecho "Parameters:\n"
    iecho "    table_name:  $table_name"
    iecho ""

    response=$(aws dynamodb delete-table \
        --table-name "$table_name")

    local error_code=${?}

    if [[ $error_code -ne 0 ]]; then
        aws_cli_error_log $error_code
        errecho "ERROR: AWS reports delete-table operation failed.$response"
        return 1
    fi

    return 0
}

```

As funções utilitárias usadas nesse cenário.

```

#####
# function iecho
#
# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.
#####
function iecho() {
    if [[ $VERBOSE == true ]]; then
        echo "$@"
    fi
}

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {

```

```

    printf "%s\n" "$*" 1>&2
}

#####
# function aws_cli_error_log()
#
# This function is used to log the error messages from the AWS CLI.
#
# See https://docs.aws.amazon.com/cli/latest/topic/return-codes.html#cli-aws-help-return-codes.
#
# The function expects the following argument:
#     $1 - The error code returned by the AWS CLI.
#
# Returns:
#     0: - Success.
#
#####
function aws_cli_error_log() {
    local err_code=$1
    errecho "Error code : $err_code"
    if [ "$err_code" == 1 ]; then
        errecho " One or more S3 transfers failed."
    elif [ "$err_code" == 2 ]; then
        errecho " Command line failed to parse."
    elif [ "$err_code" == 130 ]; then
        errecho " Process received SIGINT."
    elif [ "$err_code" == 252 ]; then
        errecho " Command syntax invalid."
    elif [ "$err_code" == 253 ]; then
        errecho " The system environment or configuration was invalid."
    elif [ "$err_code" == 254 ]; then
        errecho " The service returned an error."
    elif [ "$err_code" == 255 ]; then
        errecho " 255 is a catch-all error."
    fi

    return 0
}

```

- Para obter detalhes da API, consulte os tópicos a seguir na Referência de comandos da AWS CLI.

- [BatchWriteItem](#)
- [CreateTable](#)
- [DeleteItem](#)
- [DeleteTable](#)
- [DescribeTable](#)
- [GetItem](#)
- [PutItem](#)
- [Query](#)
- [Scan](#)
- [UpdateItem](#)

## Exemplos do HealthImaging de uso da AWS CLI com script Bash

Os exemplos de código a seguir mostram como realizar ações e implementar cenários comuns usando a AWS Command Line Interface com script Bash por meio do HealthImaging.

Ações são trechos de código de programas maiores e devem ser executadas em contexto. Embora as ações mostrem como chamar funções de serviço específicas, é possível ver as ações contextualizadas em seus devidos cenários e exemplos entre serviços.

Cenários são exemplos de código que mostram como realizar uma tarefa específica chamando várias funções dentro do mesmo serviço.

Cada exemplo inclui um link para o GitHub, onde você pode encontrar instruções sobre como configurar e executar o código no contexto.

### Tópicos

- [Ações](#)

### Ações

#### Criar um datastore

O exemplo de código a seguir mostra como criar um datastore do HealthImaging.

## AWS CLI com script Bash

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function imaging_create_datastore
#
# This function creates an AWS HealthImaging data store for importing DICOM P10
# files.
#
# Parameters:
#     -n data_store_name - The name of the data store.
#
# Returns:
#     The datastore ID.
#     And:
#     0 - If successful.
#     1 - If it fails.
#####
function imaging_create_datastore() {
    local datastore_name response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function imaging_create_datastore"
        echo "Creates an AWS HealthImaging data store for importing DICOM P10 files."
        echo "  -n data_store_name - The name of the data store."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "n:h" option; do
        case "${option}" in
            n) datastore_name="${OPTARG}" ;;
            h)
                usage

```

```
        return 0
        ;;
    \?)
        echo "Invalid parameter"
        usage
        return 1
        ;;
    esac
done
export OPTIND=1

if [[ -z "$datastore_name" ]]; then
    errecho "ERROR: You must provide a data store name with the -n parameter."
    usage
    return 1
fi

response=$(aws medical-imaging create-datastore \
    --datastore-name "$datastore_name" \
    --output text \
    --query 'datastoreId')

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports medical-imaging create-datastore operation failed.
$response"
    return 1
fi

echo "$response"

return 0
}
```

- Para obter detalhes da API, consulte [CreateDatastore](#) na Referência de comandos da AWS CLI.

**Note**

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

## Excluir um datastore

O exemplo de código a seguir mostra como excluir um datastore do HealthImaging.

### AWS CLI com script Bash

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function imaging_delete_datastore
#
# This function deletes an AWS HealthImaging data store.
#
# Parameters:
#     -i datastore_id - The ID of the data store.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function imaging_delete_datastore() {
    local datastore_id response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function imaging_delete_datastore"
        echo "Deletes an AWS HealthImaging data store."
        echo "  -i datastore_id - The ID of the data store."
        echo ""
    }
```

```

}

# Retrieve the calling parameters.
while getopts "i:h" option; do
    case "${option}" in
        i) datastore_id="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$datastore_id" ]]; then
    errecho "ERROR: You must provide a data store ID with the -i parameter."
    usage
    return 1
fi

response=$(aws medical-imaging delete-datastore \
    --datastore-id "$datastore_id")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports medical-imaging delete-datastore operation failed.
$response"
    return 1
fi

return 0
}

```

- Para obter detalhes da API, consulte [DeleteDatastore](#) na Referência de comandos da AWS CLI.

**Note**

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

## Obter propriedades do datastore

O exemplo de código a seguir mostra como obter propriedades do datastore do HealthImaging.

### AWS CLI com script Bash

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function imaging_get_datastore
#
# Get a data store's properties.
#
# Parameters:
#     -i data_store_id - The ID of the data store.
#
# Returns:
#     [datastore_name, datastore_id, datastore_status, datastore_arn,  created_at,
#     updated_at]
#     And:
#     0 - If successful.
#     1 - If it fails.
#####
function imaging_get_datastore() {
    local datastore_id option OPTARG # Required to use getopt command in a function.
    local error_code
    # bashsupport disable=BP5008
    function usage() {
        echo "function imaging_get_datastore"
        echo "Gets a data store's properties."
    }
}
```



```
    echo "  -i datastore_id - The ID of the data store."
    echo ""
}

# Retrieve the calling parameters.
while getopts "i:h" option; do
    case "${option}" in
        i) datastore_id="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$datastore_id" ]]; then
    errecho "ERROR: You must provide a data store ID with the -i parameter."
    usage
    return 1
fi

local response

response=$(
    aws medical-imaging get-datastore \
        --datastore-id "$datastore_id" \
        --output text \
        --query "[ datastoreProperties.datastoreName,
datastoreProperties.datastoreId, datastoreProperties.datastoreStatus,
datastoreProperties.datastoreArn,  datastoreProperties.createdAt,
datastoreProperties.updatedAt]"
)
error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports list-datastores operation failed.$response"
    return 1
fi
```

```
fi

echo "$response"

return 0
}
```

- Para obter detalhes da API, consulte [GetDatastore](#) na Referência de comandos da AWS CLI.

#### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

## Listar datastores

O exemplo de código a seguir mostra como listar datastores do HealthImaging.

### AWS CLI com script Bash

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function imaging_list_datastores
#
# List the HealthImaging data stores in the account.
#
# Returns:
#     [[datastore_name, datastore_id, datastore_status]]
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function imaging_list_datastores() {
```

```
local option OPTARG # Required to use getopt command in a function.
local error_code
# bashsupport disable=BP5008
function usage() {
    echo "function imaging_list_datastores"
    echo "Lists the AWS HealthImaging data stores in the account."
    echo ""
}

# Retrieve the calling parameters.
while getopt "h" option; do
    case "${option}" in
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

local response
response=$(aws medical-imaging list-datastores \
    --output text \
    --query "datastoreSummaries[*][datastoreName, datastoreId, datastoreStatus]")
error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports list-datastores operation failed.$response"
    return 1
fi

echo "$response"

return 0
}
```

- Para obter detalhes da API, consulte [ListDatastores](#) na Referência de comandos da AWS CLI.

 Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

## Exemplos do IAM que usam a AWS CLI com script Bash

Os exemplos de código a seguir mostram como realizar ações e implementar cenários comuns usando a AWS Command Line Interface com script Bash por meio do IAM.

Ações são trechos de código de programas maiores e devem ser executadas em contexto. Embora as ações mostrem como chamar funções de serviço específicas, é possível ver as ações contextualizadas em seus devidos cenários e exemplos entre serviços.

Cenários são exemplos de código que mostram como realizar uma tarefa específica chamando várias funções dentro do mesmo serviço.

Cada exemplo inclui um link para o GitHub, onde você pode encontrar instruções sobre como configurar e executar o código no contexto.

### Tópicos

- [Ações](#)
- [Cenários](#)

### Ações

#### Anexar uma política a uma função

O exemplo de código a seguir mostra como anexar uma política do IAM a um perfil.

#### AWS CLI com script Bash

 Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function iam_attach_role_policy
#
# This function attaches an IAM policy to a role.
#
# Parameters:
#     -n role_name -- The name of the IAM role.
#     -p policy_ARN -- The IAM policy document ARN..
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_attach_role_policy() {
    local role_name policy_arn response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_attach_role_policy"
        echo "Attaches an AWS Identity and Access Management (IAM) policy to an IAM
role."
        echo "  -n role_name    The name of the IAM role."
        echo "  -p policy_ARN -- The IAM policy document ARN."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "n:p:h" option; do
        case "${option}" in
            n) role_name="${OPTARG}" ;;
            p) policy_arn="${OPTARG}" ;;
            h)
                usage

```

```
        return 0
        ;;
    \?)
        echo "Invalid parameter"
        usage
        return 1
        ;;
    esac
done
export OPTIND=1

if [[ -z "$role_name" ]]; then
    errecho "ERROR: You must provide a role name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$policy_arn" ]]; then
    errecho "ERROR: You must provide a policy ARN with the -p parameter."
    usage
    return 1
fi

response=$(aws iam attach-role-policy \
    --role-name "$role_name" \
    --policy-arn "$policy_arn")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports attach-role-policy operation failed.\n$response"
    return 1
fi

echo "$response"

return 0
}
```

- Para obter detalhes da API, consulte [AttachRolePolicy](#) na Referência de comandos da AWS CLI.

## Criar uma política

O exemplo de código a seguir mostra como criar uma política do IAM.

### AWS CLI com script Bash

#### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function iam_create_policy
#
# This function creates an IAM policy.
#
# Parameters:
#     -n policy_name -- The name of the IAM policy.
#     -p policy_json -- The policy document.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_create_policy() {
    local policy_name policy_document response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_create_policy"
        echo "Creates an AWS Identity and Access Management (IAM) policy."
        echo "  -n policy_name    The name of the IAM policy."
    }
}
```

```
    echo "  -p policy_json -- The policy document."
    echo ""
}

# Retrieve the calling parameters.
while getopts "n:p:h" option; do
    case "${option}" in
        n) policy_name="${OPTARG}" ;;
        p) policy_document="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$policy_name" ]]; then
    errecho "ERROR: You must provide a policy name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$policy_document" ]]; then
    errecho "ERROR: You must provide a policy document with the -p parameter."
    usage
    return 1
fi

response=$(aws iam create-policy \
    --policy-name "$policy_name" \
    --policy-document "$policy_document" \
    --output text \
    --query Policy.Arn)

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
```



```

    errecho "ERROR: AWS reports create-policy operation failed.\n$response"
    return 1
fi

echo "$response"
}

```

- Para obter detalhes da API, consulte [CreatePolicy](#) na Referência de comandos da AWS CLI.

## Criar um perfil

O exemplo de código a seguir mostra como criar um perfil do IAM.

### AWS CLI com script Bash

#### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function iam_create_role
#
# This function creates an IAM role.
#
# Parameters:
#     -n role_name -- The name of the IAM role.
#     -p policy_json -- The assume role policy document.
#
# Returns:
#     The ARN of the role.

```

```

#      And:
#      0 - If successful.
#      1 - If it fails.
#####
function iam_create_role() {
    local role_name policy_document response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_create_user_access_key"
        echo "Creates an AWS Identity and Access Management (IAM) role."
        echo "  -n role_name    The name of the IAM role."
        echo "  -p policy_json -- The assume role policy document."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "n:p:h" option; do
        case "${option}" in
            n) role_name="${OPTARG}" ;;
            p) policy_document="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
    export OPTIND=1

    if [[ -z "$role_name" ]]; then
        errecho "ERROR: You must provide a role name with the -n parameter."
        usage
        return 1
    fi

    if [[ -z "$policy_document" ]]; then
        errecho "ERROR: You must provide a policy document with the -p parameter."
        usage
    fi
}

```

```
    return 1
fi

response=$(aws iam create-role \
  --role-name "$role_name" \
  --assume-role-policy-document "$policy_document" \
  --output text \
  --query Role.Arn)

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
  aws_cli_error_log $error_code
  errecho "ERROR: AWS reports create-role operation failed.\n$response"
  return 1
fi

echo "$response"

return 0
}
```

- Para obter detalhes da API, consulte [CreateRole](#) na Referência de comandos da AWS CLI.

## Criar um usuário

O exemplo de código a seguir mostra como criar um usuário do IAM.

### Warning

Para evitar riscos de segurança, não use usuários do IAM para autenticação ao desenvolver software com propósito específico ou trabalhar com dados reais. Em vez disso, use federação com um provedor de identidade, como [AWS IAM Identity Center](#).

## AWS CLI com script Bash

 Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function iecho
#
# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.
#####
function iecho() {
    if [[ $VERBOSE == true ]]; then
        echo "$@"
    fi
}

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function iam_create_user
#
# This function creates the specified IAM user, unless
# it already exists.
#
# Parameters:
#     -u user_name  -- The name of the user to create.
#
# Returns:
#     The ARN of the user.
#     And:
#     0 - If successful.
```

```

#      1 - If it fails.
#####
function iam_create_user() {
    local user_name response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_create_user"
        echo "Creates an WS Identity and Access Management (IAM) user. You must supply a
username:"
        echo "  -u user_name    The name of the user. It must be unique within the
account."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "u:h" option; do
        case "${option}" in
            u) user_name="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
    export OPTIND=1

    if [[ -z "$user_name" ]]; then
        errecho "ERROR: You must provide a username with the -u parameter."
        usage
        return 1
    fi

    iecho "Parameters:\n"
    iecho "    User name:  $user_name"
    iecho ""

    # If the user already exists, we don't want to try to create it.

```

```
if (iam_user_exists "$user_name"); then
    errecho "ERROR: A user with that name already exists in the account."
    return 1
fi

response=$(aws iam create-user --user-name "$user_name" \
    --output text \
    --query 'User.Arn')

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports create-user operation failed.$response"
    return 1
fi

echo "$response"

return 0
}
```

- Para obter detalhes da API, consulte [CreateUser](#) na Referência de comandos da AWS CLI.

## Criar uma chave de acesso

O exemplo de código a seguir mostra como criar uma chave de acesso do IAM.

### Warning

Para evitar riscos de segurança, não use usuários do IAM para autenticação ao desenvolver software com propósito específico ou trabalhar com dados reais. Em vez disso, use federação com um provedor de identidade, como [AWS IAM Identity Center](#).

## AWS CLI com script Bash

 Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function iam_create_user_access_key
#
# This function creates an IAM access key for the specified user.
#
# Parameters:
#     -u user_name -- The name of the IAM user.
#     [-f file_name] -- The optional file name for the access key output.
#
# Returns:
#     [access_key_id access_key_secret]
#     And:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_create_user_access_key() {
    local user_name file_name response
    local option OPTARG # Required to use getopt command in a function.

# bashsupport disable=BP5008
function usage() {
    echo "function iam_create_user_access_key"
    echo "Creates an AWS Identity and Access Management (IAM) key pair."
    echo "  -u user_name    The name of the IAM user."
    echo "  [-f file_name]  Optional file name for the access key output."
}
```

```
    echo ""
}

# Retrieve the calling parameters.
while getopts "u:f:h" option; do
    case "${option}" in
        u) user_name="${OPTARG}" ;;
        f) file_name="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$user_name" ]]; then
    errecho "ERROR: You must provide a username with the -u parameter."
    usage
    return 1
fi

response=$(aws iam create-access-key \
    --user-name "$user_name" \
    --output text)

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports create-access-key operation failed.$response"
    return 1
fi

if [[ -n "$file_name" ]]; then
    echo "$response" >"$file_name"
fi

local key_id key_secret
```



```
# shellcheck disable=SC2086
key_id=$(echo $response | cut -f 2 -d ' ')
# shellcheck disable=SC2086
key_secret=$(echo $response | cut -f 4 -d ' ')

echo "$key_id $key_secret"

return 0
}
```

- Para obter detalhes da API, consulte [CreateAccessKey](#) na Referência de comandos da AWS CLI.

## Excluir uma política

O exemplo de código a seguir mostra como excluir uma política do IAM.

### AWS CLI com script Bash

#### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function iecho
#
# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.
#####
function iecho() {
    if [[ $VERBOSE == true ]]; then
        echo "$@"
    fi
}

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
```

```
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function iam_delete_policy
#
# This function deletes an IAM policy.
#
# Parameters:
#     -n policy_arn -- The name of the IAM policy arn.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_delete_policy() {
    local policy_arn response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_delete_policy"
        echo "Deletes an WS Identity and Access Management (IAM) policy"
        echo "  -n policy_arn -- The name of the IAM policy arn."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "n:h" option; do
        case "${option}" in
            n) policy_arn="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
done
```

```
export OPTIND=1

if [[ -z "$policy_arn" ]]; then
    errecho "ERROR: You must provide a policy arn with the -n parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    Policy arn: $policy_arn"
iecho ""

response=$(aws iam delete-policy \
    --policy-arn "$policy_arn")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports delete-policy operation failed.\n$response"
    return 1
fi

iecho "delete-policy response:$response"
iecho

return 0
}
```

- Para obter detalhes da API, consulte [DeletePolicy](#) na Referência de comandos da AWS CLI.

## Excluir uma função

O exemplo de código a seguir mostra como excluir um perfil do IAM.

## AWS CLI com script Bash

### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function iecho
#
# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.
#####
function iecho() {
    if [[ $VERBOSE == true ]]; then
        echo "$@"
    fi
}

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function iam_delete_role
#
# This function deletes an IAM role.
#
# Parameters:
#     -n role_name -- The name of the IAM role.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_delete_role() {
    local role_name response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_delete_role"
        echo "Deletes an WS Identity and Access Management (IAM) role"
        echo "  -n role_name -- The name of the IAM role."
        echo ""
    }
}
```

```
}

# Retrieve the calling parameters.
while getopts "n:h" option; do
    case "${option}" in
        n) role_name="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

echo "role_name:$role_name"
if [[ -z "$role_name" ]]; then
    errecho "ERROR: You must provide a role name with the -n parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    Role name:  $role_name"
iecho ""

response=$(aws iam delete-role \
    --role-name "$role_name")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports delete-role operation failed.\n$response"
    return 1
fi

iecho "delete-role response:$response"
iecho
```

```
    return 0
}
```

- Para obter detalhes da API, consulte [DeleteRole](#) na Referência de comandos da AWS CLI.

## Excluir um usuário

O exemplo de código a seguir mostra como excluir um usuário do IAM.

### Warning

Para evitar riscos de segurança, não use usuários do IAM para autenticação ao desenvolver software com propósito específico ou trabalhar com dados reais. Em vez disso, use federação com um provedor de identidade, como [AWS IAM Identity Center](#).

## AWS CLI com script Bash

### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function iecho
#
# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.
#####
function iecho() {
    if [[ $VERBOSE == true ]]; then
        echo "$@"
    fi
}

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
```

```
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function iam_delete_user
#
# This function deletes the specified IAM user.
#
# Parameters:
#     -u user_name  -- The name of the user to create.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_delete_user() {
    local user_name response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_delete_user"
        echo "Deletes an WS Identity and Access Management (IAM) user. You must supply a
username:"
        echo "  -u user_name    The name of the user."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "u:h" option; do
        case "${option}" in
            u) user_name="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
}
```

```
done
export OPTIND=1

if [[ -z "$user_name" ]]; then
    errecho "ERROR: You must provide a username with the -u parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    User name:    $user_name"
iecho ""

# If the user does not exist, we don't want to try to delete it.
if (! iam_user_exists "$user_name"); then
    errecho "ERROR: A user with that name does not exist in the account."
    return 1
fi

response=$(aws iam delete-user \
    --user-name "$user_name")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports delete-user operation failed.$response"
    return 1
fi

iecho "delete-user response:$response"
iecho

return 0
}
```

- Para obter detalhes da API, consulte [DeleteUser](#) na Referência de comandos da AWS CLI.

## Excluir uma chave de acesso

O exemplo de código a seguir mostra como excluir uma chave de acesso do IAM.



**⚠ Warning**

Para evitar riscos de segurança, não use usuários do IAM para autenticação ao desenvolver software com propósito específico ou trabalhar com dados reais. Em vez disso, use federação com um provedor de identidade, como [AWS IAM Identity Center](#).

## AWS CLI com script Bash

**i Note**

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function iam_delete_access_key
#
# This function deletes an IAM access key for the specified IAM user.
#
# Parameters:
#     -u user_name -- The name of the user.
#     -k access_key -- The access key to delete.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_delete_access_key() {
    local user_name access_key response
    local option OPTARG # Required to use getopt command in a function.
```

```
# bashsupport disable=BP5008
function usage() {
    echo "function iam_delete_access_key"
    echo "Deletes an WS Identity and Access Management (IAM) access key for the
specified IAM user"
    echo "  -u user_name    The name of the user."
    echo "  -k access_key    The access key to delete."
    echo ""
}

# Retrieve the calling parameters.
while getopts "u:k:h" option; do
    case "${option}" in
        u) user_name="${OPTARG}" ;;
        k) access_key="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$user_name" ]]; then
    errecho "ERROR: You must provide a username with the -u parameter."
    usage
    return 1
fi

if [[ -z "$access_key" ]]; then
    errecho "ERROR: You must provide an access key with the -k parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "  Username:  $user_name"
iecho "  Access key: $access_key"
iecho ""
```

```

response=$(aws iam delete-access-key \
  --user-name "$user_name" \
  --access-key-id "$access_key")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
  aws_cli_error_log $error_code
  errecho "ERROR: AWS reports delete-access-key operation failed.\n$response"
  return 1
fi

iecho "delete-access-key response:$response"
iecho

return 0
}

```

- Para obter detalhes da API, consulte [DeleteAccessKey](#) na Referência de comandos da AWS CLI.

## Desanexar uma política de uma função

O exemplo de código a seguir mostra como desanexar uma política do IAM.

### AWS CLI com script Bash

#### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
  printf "%s\n" "$*" 1>&2
}

```

```

}

#####
# function iam_detach_role_policy
#
# This function detaches an IAM policy to a role.
#
# Parameters:
#     -n role_name -- The name of the IAM role.
#     -p policy_ARN -- The IAM policy document ARN..
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_detach_role_policy() {
    local role_name policy_arn response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_detach_role_policy"
        echo "Detaches an AWS Identity and Access Management (IAM) policy to an IAM
role."
        echo "  -n role_name    The name of the IAM role."
        echo "  -p policy_ARN -- The IAM policy document ARN."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "n:p:h" option; do
        case "${option}" in
            n) role_name="${OPTARG}" ;;
            p) policy_arn="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
}

```

```
done
export OPTIND=1

if [[ -z "$role_name" ]]; then
    errecho "ERROR: You must provide a role name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$policy_arn" ]]; then
    errecho "ERROR: You must provide a policy ARN with the -p parameter."
    usage
    return 1
fi

response=$(aws iam detach-role-policy \
    --role-name "$role_name" \
    --policy-arn "$policy_arn")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports detach-role-policy operation failed.\n$response"
    return 1
fi

echo "$response"

return 0
}
```

- Para obter detalhes da API, consulte [DetachRolePolicy](#) na Referência de comandos da AWS CLI.

## Obter um usuário

O exemplo de código a seguir mostra como obter um usuário do IAM.

**⚠ Warning**

Para evitar riscos de segurança, não use usuários do IAM para autenticação ao desenvolver software com propósito específico ou trabalhar com dados reais. Em vez disso, use federação com um provedor de identidade, como [AWS IAM Identity Center](#).

## AWS CLI com script Bash

**i Note**

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function iam_user_exists
#
# This function checks to see if the specified AWS Identity and Access Management
# (IAM) user already exists.
#
# Parameters:
#     $1 - The name of the IAM user to check.
#
# Returns:
#     0 - If the user already exists.
#     1 - If the user doesn't exist.
#####
function iam_user_exists() {
    local user_name
    user_name=$1
```

```
# Check whether the IAM user already exists.
# We suppress all output - we're interested only in the return code.

local errors
errors=$(aws iam get-user \
  --user-name "$user_name" 2>&1 >/dev/null)

local error_code=${?}

if [[ $error_code -eq 0 ]]; then
  return 0 # 0 in Bash script means true.
else
  if [[ $errors != *"error"*(NoSuchEntity)* ]]; then
    aws_cli_error_log $error_code
    errecho "Error calling iam get-user $errors"
  fi

  return 1 # 1 in Bash script means false.
fi
}
```

- Para obter detalhes da API, consulte [GetUser](#) na Referência de comandos da AWS CLI.

## Listar as chaves de acesso de um usuário

O exemplo de código a seguir mostra como listar as chaves de acesso do IAM de um usuário.

### Warning

Para evitar riscos de segurança, não use usuários do IAM para autenticação ao desenvolver software com propósito específico ou trabalhar com dados reais. Em vez disso, use federação com um provedor de identidade, como [AWS IAM Identity Center](#).

## AWS CLI com script Bash

### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function iam_list_access_keys
#
# This function lists the access keys for the specified user.
#
# Parameters:
#     -u user_name -- The name of the IAM user.
#
# Returns:
#     access_key_ids
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_list_access_keys() {

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_list_access_keys"
        echo "Lists the AWS Identity and Access Management (IAM) access key IDs for the
specified user."
        echo "  -u user_name    The name of the IAM user."
        echo ""
    }

    local user_name response
    local option OPTARG # Required to use getopt command in a function.
    # Retrieve the calling parameters.
    while getopt "u:h" option; do
        case "${option}" in
            u) user_name="${OPTARG}" ;;
            h)
                usage
                return 0
        esac
    done
}
```



```
;;
\?)
    echo "Invalid parameter"
    usage
    return 1
;;
esac
done
export OPTIND=1

if [[ -z "$user_name" ]]; then
    errecho "ERROR: You must provide a username with the -u parameter."
    usage
    return 1
fi

response=$(aws iam list-access-keys \
    --user-name "$user_name" \
    --output text \
    --query 'AccessKeyMetadata[].AccessKeyId')

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports list-access-keys operation failed.$response"
    return 1
fi

echo "$response"

return 0
}
```

- Para obter detalhes da API, consulte [ListAccessKeys](#) na Referência de comandos da AWS CLI.

## Listar usuários

O exemplo de código a seguir mostra como listar usuários do IAM.

**⚠ Warning**

Para evitar riscos de segurança, não use usuários do IAM para autenticação ao desenvolver software com propósito específico ou trabalhar com dados reais. Em vez disso, use federação com um provedor de identidade, como [AWS IAM Identity Center](#).

## AWS CLI com script Bash

**i Note**

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function iam_list_users
#
# List the IAM users in the account.
#
# Returns:
#     The list of users names
#     And:
#     0 - If the user already exists.
#     1 - If the user doesn't exist.
#####
function iam_list_users() {
    local option OPTARG # Required to use getopt command in a function.
    local error_code
    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_list_users"
```

```
    echo "Lists the AWS Identity and Access Management (IAM) user in the account."
    echo ""
}

# Retrieve the calling parameters.
while getopts "h" option; do
    case "${option}" in
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

local response

response=$(aws iam list-users \
    --output text \
    --query "Users[].UserName")
error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports list-users operation failed.$response"
    return 1
fi

echo "$response"

return 0
}
```

- Para obter detalhes da API, consulte [ListUsers](#) na Referência de comandos da AWS CLI.

## Cenários

### Criar um usuário e assumir uma função

O exemplo de código a seguir mostra como criar um usuário e assumir um perfil.

#### Warning

Para evitar riscos de segurança, não use usuários do IAM para autenticação ao desenvolver software com propósito específico ou trabalhar com dados reais. Em vez disso, use federação com um provedor de identidade, como [AWS IAM Identity Center](#).

- Crie um usuário sem permissões.
- Crie uma função que conceda permissão para listar os buckets do Amazon S3 para a conta.
- Adicione uma política para permitir que o usuário assuma a função.
- Assuma o perfil e liste buckets do S3 usando credenciais temporárias, depois limpe os recursos.

### AWS CLI com script Bash

#### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function iam_create_user_assume_role
#
# Scenario to create an IAM user, create an IAM role, and apply the role to the
# user.
#
# "IAM access" permissions are needed to run this code.
# "STS assume role" permissions are needed to run this code. (Note: It might be
# necessary to
# create a custom policy).
#
# Returns:
# 0 - If successful.
# 1 - If an error occurred.
```

```
#####
function iam_create_user_assume_role() {
{
    if [ "$IAM_OPERATIONS_SOURCED" != "True" ]; then

        source ./iam_operations.sh
    fi
}

echo_repeat "*" 88
echo "Welcome to the IAM create user and assume role demo."
echo
echo "This demo will create an IAM user, create an IAM role, and apply the role to
the user."
echo_repeat "*" 88
echo

echo -n "Enter a name for a new IAM user: "
get_input
user_name=$get_input_result

local user_arn
user_arn=$(iam_create_user -u "$user_name")

# shellcheck disable=SC2181
if [[ ${?} == 0 ]]; then
    echo "Created demo IAM user named $user_name"
else
    errecho "$user_arn"
    errecho "The user failed to create. This demo will exit."
    return 1
fi

local access_key_response
access_key_response=$(iam_create_user_access_key -u "$user_name")
# shellcheck disable=SC2181
if [[ ${?} != 0 ]]; then
    errecho "The access key failed to create. This demo will exit."
    clean_up "$user_name"
    return 1
fi

IFS=$'\t ' read -r -a access_key_values <<<"$access_key_response"
local key_name=${access_key_values[0]}
```

```

local key_secret=${access_key_values[1]}

echo "Created access key named $key_name"

echo "Wait 10 seconds for the user to be ready."
sleep 10
echo_repeat "*" 88
echo

local iam_role_name
iam_role_name=$(generate_random_name "test-role")
echo "Creating a role named $iam_role_name with user $user_name as the principal."

local assume_role_policy_document="{
  \"Version\": \"2012-10-17\",
  \"Statement\": [{
    \"Effect\": \"Allow\",
    \"Principal\": {\"AWS\": \"$user_arn\"},
    \"Action\": \"sts:AssumeRole\"
  }]
}"

local role_arn
role_arn=$(iam_create_role -n "$iam_role_name" -p "$assume_role_policy_document")

# shellcheck disable=SC2181
if [ ${?} == 0 ]; then
  echo "Created IAM role named $iam_role_name"
else
  errecho "The role failed to create. This demo will exit."
  clean_up "$user_name" "$key_name"
  return 1
fi

local policy_name
policy_name=$(generate_random_name "test-policy")
local policy_document="{
  \"Version\": \"2012-10-17\",
  \"Statement\": [{
    \"Effect\": \"Allow\",
    \"Action\": \"s3:ListAllMyBuckets\",
    \"Resource\": \"arn:aws:s3:::*\"}]}"

local policy_arn

```

```

policy_arn=$(iam_create_policy -n "$policy_name" -p "$policy_document")
# shellcheck disable=SC2181
if [[ ${?} == 0 ]]; then
    echo "Created IAM policy named $policy_name"
else
    errecho "The policy failed to create."
    clean_up "$user_name" "$key_name" "$iam_role_name"
    return 1
fi

if (iam_attach_role_policy -n "$iam_role_name" -p "$policy_arn"); then
    echo "Attached policy $policy_arn to role $iam_role_name"
else
    errecho "The policy failed to attach."
    clean_up "$user_name" "$key_name" "$iam_role_name" "$policy_arn"
    return 1
fi

local assume_role_policy_document="{
    \"Version\": \"2012-10-17\",
    \"Statement\": [{
        \"Effect\": \"Allow\",
        \"Action\": \"sts:AssumeRole\",
        \"Resource\": \"$role_arn\"}]}"

local assume_role_policy_name
assume_role_policy_name=$(generate_random_name "test-assume-role-")

# shellcheck disable=SC2181
local assume_role_policy_arn
assume_role_policy_arn=$(iam_create_policy -n "$assume_role_policy_name" -p
"$assume_role_policy_document")
# shellcheck disable=SC2181
if [ ${?} == 0 ]; then
    echo "Created IAM policy named $assume_role_policy_name for sts assume role"
else
    errecho "The policy failed to create."
    clean_up "$user_name" "$key_name" "$iam_role_name" "$policy_arn" "$policy_arn"
    return 1
fi

echo "Wait 10 seconds to give AWS time to propagate these new resources and
connections."
sleep 10

```

```
echo_repeat "*" 88
echo

echo "Try to list buckets without the new user assuming the role."
echo_repeat "*" 88
echo

# Set the environment variables for the created user.
# bashsupport disable=BP2001
export AWS_ACCESS_KEY_ID=$key_name
# bashsupport disable=BP2001
export AWS_SECRET_ACCESS_KEY=$key_secret

local buckets
buckets=$(s3_list_buckets)

# shellcheck disable=SC2181
if [ ${?} == 0 ]; then
    local bucket_count
    bucket_count=$(echo "$buckets" | wc -w | xargs)
    echo "There are $bucket_count buckets in the account. This should not have
happened."
else
    errecho "Because the role with permissions has not been assumed, listing buckets
failed."
fi

echo
echo_repeat "*" 88
echo "Now assume the role $iam_role_name and list the buckets."
echo_repeat "*" 88
echo

local credentials

credentials=$(sts_assume_role -r "$role_arn" -n "AssumeRoleDemoSession")
# shellcheck disable=SC2181
if [ ${?} == 0 ]; then
    echo "Assumed role $iam_role_name"
else
    errecho "Failed to assume role."
    export AWS_ACCESS_KEY_ID=""
    export AWS_SECRET_ACCESS_KEY=""
```



```

    clean_up "$user_name" "$key_name" "$iam_role_name" "$policy_arn" "$policy_arn"
"$assume_role_policy_arn"
    return 1
fi

IFS=$'\t ' read -r -a credentials <<<"$credentials"

export AWS_ACCESS_KEY_ID=${credentials[0]}
export AWS_SECRET_ACCESS_KEY=${credentials[1]}
# bashsupport disable=BP2001
export AWS_SESSION_TOKEN=${credentials[2]}

buckets=$(s3_list_buckets)

# shellcheck disable=SC2181
if [ ${?} == 0 ]; then
    local bucket_count
    bucket_count=$(echo "$buckets" | wc -w | xargs)
    echo "There are $bucket_count buckets in the account. Listing buckets succeeded
because of "
    echo "the assumed role."
else
    errecho "Failed to list buckets. This should not happen."
    export AWS_ACCESS_KEY_ID=""
    export AWS_SECRET_ACCESS_KEY=""
    export AWS_SESSION_TOKEN=""
    clean_up "$user_name" "$key_name" "$iam_role_name" "$policy_arn" "$policy_arn"
"$assume_role_policy_arn"
    return 1
fi

local result=0
export AWS_ACCESS_KEY_ID=""
export AWS_SECRET_ACCESS_KEY=""

echo
echo_repeat "*" 88
echo "The created resources will now be deleted."
echo_repeat "*" 88
echo

clean_up "$user_name" "$key_name" "$iam_role_name" "$policy_arn" "$policy_arn"
"$assume_role_policy_arn"

```

```
# shellcheck disable=SC2181
if [[ ${?} -ne 0 ]]; then
    result=1
fi

return $result
}
```

As funções do IAM usadas neste cenário.

```
#####
# function iam_user_exists
#
# This function checks to see if the specified AWS Identity and Access Management
# (IAM) user already exists.
#
# Parameters:
#     $1 - The name of the IAM user to check.
#
# Returns:
#     0 - If the user already exists.
#     1 - If the user doesn't exist.
#####
function iam_user_exists() {
    local user_name
    user_name=$1

    # Check whether the IAM user already exists.
    # We suppress all output - we're interested only in the return code.

    local errors
    errors=$(aws iam get-user \
        --user-name "$user_name" 2>&1 >/dev/null)

    local error_code=${?}

    if [[ $error_code -eq 0 ]]; then
        return 0 # 0 in Bash script means true.
    else
        if [[ $errors != *"error"*(NoSuchEntity)* ]]; then
            aws_cli_error_log $error_code
            errecho "Error calling iam get-user $errors"
        fi
    fi
}
```

```

    fi

    return 1 # 1 in Bash script means false.
fi
}

#####
# function iam_create_user
#
# This function creates the specified IAM user, unless
# it already exists.
#
# Parameters:
#     -u user_name  -- The name of the user to create.
#
# Returns:
#     The ARN of the user.
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_create_user() {
    local user_name response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_create_user"
        echo "Creates an WS Identity and Access Management (IAM) user. You must supply a
username:"
        echo "  -u user_name    The name of the user. It must be unique within the
account."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "u:h" option; do
        case "${option}" in
            u) user_name="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)

```

```

        echo "Invalid parameter"
        usage
        return 1
    ;;
esac
done
export OPTIND=1

if [[ -z "$user_name" ]]; then
    errecho "ERROR: You must provide a username with the -u parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    User name:  $user_name"
iecho ""

# If the user already exists, we don't want to try to create it.
if (iam_user_exists "$user_name"); then
    errecho "ERROR: A user with that name already exists in the account."
    return 1
fi

response=$(aws iam create-user --user-name "$user_name" \
    --output text \
    --query 'User.Arn')

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports create-user operation failed.$response"
    return 1
fi

echo "$response"

return 0
}

#####
# function iam_create_user_access_key
#

```

```
# This function creates an IAM access key for the specified user.
#
# Parameters:
#     -u user_name -- The name of the IAM user.
#     [-f file_name] -- The optional file name for the access key output.
#
# Returns:
#     [access_key_id access_key_secret]
#     And:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_create_user_access_key() {
    local user_name file_name response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_create_user_access_key"
        echo "Creates an AWS Identity and Access Management (IAM) key pair."
        echo "  -u user_name    The name of the IAM user."
        echo "  [-f file_name]  Optional file name for the access key output."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "u:f:h" option; do
        case "${option}" in
            u) user_name="${OPTARG}" ;;
            f) file_name="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
    export OPTIND=1

    if [[ -z "$user_name" ]]; then
```

```

    errecho "ERROR: You must provide a username with the -u parameter."
    usage
    return 1
fi

response=$(aws iam create-access-key \
    --user-name "$user_name" \
    --output text)

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports create-access-key operation failed.$response"
    return 1
fi

if [[ -n "$file_name" ]]; then
    echo "$response" >"$file_name"
fi

local key_id key_secret
# shellcheck disable=SC2086
key_id=$(echo $response | cut -f 2 -d ' ')
# shellcheck disable=SC2086
key_secret=$(echo $response | cut -f 4 -d ' ')

echo "$key_id $key_secret"

return 0
}

#####
# function iam_create_role
#
# This function creates an IAM role.
#
# Parameters:
#     -n role_name -- The name of the IAM role.
#     -p policy_json -- The assume role policy document.
#
# Returns:
#     The ARN of the role.
#     And:

```

```

#      0 - If successful.
#      1 - If it fails.
#####
function iam_create_role() {
    local role_name policy_document response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_create_user_access_key"
        echo "Creates an AWS Identity and Access Management (IAM) role."
        echo "  -n role_name    The name of the IAM role."
        echo "  -p policy_json -- The assume role policy document."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "n:p:h" option; do
        case "${option}" in
            n) role_name="${OPTARG}" ;;
            p) policy_document="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
    export OPTIND=1

    if [[ -z "$role_name" ]]; then
        errecho "ERROR: You must provide a role name with the -n parameter."
        usage
        return 1
    fi

    if [[ -z "$policy_document" ]]; then
        errecho "ERROR: You must provide a policy document with the -p parameter."
        usage
        return 1
    fi
}

```

```

fi

response=$(aws iam create-role \
  --role-name "$role_name" \
  --assume-role-policy-document "$policy_document" \
  --output text \
  --query Role.Arn)

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
  aws_cli_error_log $error_code
  errecho "ERROR: AWS reports create-role operation failed.\n$response"
  return 1
fi

echo "$response"

return 0
}

#####
# function iam_create_policy
#
# This function creates an IAM policy.
#
# Parameters:
#   -n policy_name -- The name of the IAM policy.
#   -p policy_json -- The policy document.
#
# Returns:
#   0 - If successful.
#   1 - If it fails.
#####
function iam_create_policy() {
  local policy_name policy_document response
  local option OPTARG # Required to use getopt command in a function.

# bashsupport disable=BP5008
function usage() {
  echo "function iam_create_policy"
  echo "Creates an AWS Identity and Access Management (IAM) policy."
  echo "  -n policy_name    The name of the IAM policy."
  echo "  -p policy_json -- The policy document."

```



```
    echo ""
}

# Retrieve the calling parameters.
while getopts "n:p:h" option; do
    case "${option}" in
        n) policy_name="${OPTARG}" ;;
        p) policy_document="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$policy_name" ]]; then
    errecho "ERROR: You must provide a policy name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$policy_document" ]]; then
    errecho "ERROR: You must provide a policy document with the -p parameter."
    usage
    return 1
fi

response=$(aws iam create-policy \
    --policy-name "$policy_name" \
    --policy-document "$policy_document" \
    --output text \
    --query Policy.Arn)

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports create-policy operation failed.\n$response"
```

```

    return 1
fi

echo "$response"
}

#####
# function iam_attach_role_policy
#
# This function attaches an IAM policy to a role.
#
# Parameters:
#     -n role_name -- The name of the IAM role.
#     -p policy_ARN -- The IAM policy document ARN..
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_attach_role_policy() {
    local role_name policy_arn response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_attach_role_policy"
        echo "Attaches an AWS Identity and Access Management (IAM) policy to an IAM
role."
        echo "  -n role_name    The name of the IAM role."
        echo "  -p policy_ARN -- The IAM policy document ARN."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "n:p:h" option; do
        case "${option}" in
            n) role_name="${OPTARG}" ;;
            p) policy_arn="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"

```

```

        usage
        return 1
    ;;
esac
done
export OPTIND=1

if [[ -z "$role_name" ]]; then
    errecho "ERROR: You must provide a role name with the -n parameter."
    usage
    return 1
fi

if [[ -z "$policy_arn" ]]; then
    errecho "ERROR: You must provide a policy ARN with the -p parameter."
    usage
    return 1
fi

response=$(aws iam attach-role-policy \
    --role-name "$role_name" \
    --policy-arn "$policy_arn")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports attach-role-policy operation failed.\n$response"
    return 1
fi

echo "$response"

return 0
}

#####
# function iam_detach_role_policy
#
# This function detaches an IAM policy to a role.
#
# Parameters:
#     -n role_name -- The name of the IAM role.
#     -p policy_ARN -- The IAM policy document ARN..

```

```

#
# Returns:
#      0 - If successful.
#      1 - If it fails.
#####
function iam_detach_role_policy() {
    local role_name policy_arn response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_detach_role_policy"
        echo "Detaches an AWS Identity and Access Management (IAM) policy to an IAM
role."
        echo "  -n role_name    The name of the IAM role."
        echo "  -p policy_ARN -- The IAM policy document ARN."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "n:p:h" option; do
        case "${option}" in
            n) role_name="${OPTARG}" ;;
            p) policy_arn="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
    export OPTIND=1

    if [[ -z "$role_name" ]]; then
        errecho "ERROR: You must provide a role name with the -n parameter."
        usage
        return 1
    fi

    if [[ -z "$policy_arn" ]]; then

```

```

    errecho "ERROR: You must provide a policy ARN with the -p parameter."
    usage
    return 1
fi

response=$(aws iam detach-role-policy \
    --role-name "$role_name" \
    --policy-arn "$policy_arn")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports detach-role-policy operation failed.\n$response"
    return 1
fi

echo "$response"

return 0
}

#####
# function iam_delete_policy
#
# This function deletes an IAM policy.
#
# Parameters:
#     -n policy_arn -- The name of the IAM policy arn.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_delete_policy() {
    local policy_arn response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_delete_policy"
        echo "Deletes an WS Identity and Access Management (IAM) policy"
        echo "  -n policy_arn -- The name of the IAM policy arn."
        echo ""
    }

```

```
}

# Retrieve the calling parameters.
while getopts "n:h" option; do
    case "${option}" in
        n) policy_arn="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$policy_arn" ]]; then
    errecho "ERROR: You must provide a policy arn with the -n parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    Policy arn: $policy_arn"
iecho ""

response=$(aws iam delete-policy \
    --policy-arn "$policy_arn")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports delete-policy operation failed.\n$response"
    return 1
fi

iecho "delete-policy response:$response"
iecho

return 0
```

```

}

#####
# function iam_delete_role
#
# This function deletes an IAM role.
#
# Parameters:
#     -n role_name -- The name of the IAM role.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_delete_role() {
    local role_name response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function iam_delete_role"
        echo "Deletes an WS Identity and Access Management (IAM) role"
        echo "  -n role_name -- The name of the IAM role."
        echo ""
    }

    # Retrieve the calling parameters.
    while getopt "n:h" option; do
        case "${option}" in
            n) role_name="${OPTARG}" ;;
            h)
                usage
                return 0
                ;;
            \?)
                echo "Invalid parameter"
                usage
                return 1
                ;;
        esac
    done
    export OPTIND=1

    echo "role_name:$role_name"
}

```

```

if [[ -z "$role_name" ]]; then
    errecho "ERROR: You must provide a role name with the -n parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "    Role name: $role_name"
iecho ""

response=$(aws iam delete-role \
    --role-name "$role_name")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports delete-role operation failed.\n$response"
    return 1
fi

iecho "delete-role response:$response"
iecho

return 0
}

#####
# function iam_delete_access_key
#
# This function deletes an IAM access key for the specified IAM user.
#
# Parameters:
#     -u user_name -- The name of the user.
#     -k access_key -- The access key to delete.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_delete_access_key() {
    local user_name access_key response
    local option OPTARG # Required to use getopt command in a function.

```



```
# bashsupport disable=BP5008
function usage() {
    echo "function iam_delete_access_key"
    echo "Deletes an WS Identity and Access Management (IAM) access key for the
specified IAM user"
    echo "  -u user_name    The name of the user."
    echo "  -k access_key    The access key to delete."
    echo ""
}

# Retrieve the calling parameters.
while getopts "u:k:h" option; do
    case "${option}" in
        u) user_name="${OPTARG}" ;;
        k) access_key="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done
export OPTIND=1

if [[ -z "$user_name" ]]; then
    errecho "ERROR: You must provide a username with the -u parameter."
    usage
    return 1
fi

if [[ -z "$access_key" ]]; then
    errecho "ERROR: You must provide an access key with the -k parameter."
    usage
    return 1
fi

iecho "Parameters:\n"
iecho "  Username:  $user_name"
iecho "  Access key: $access_key"
iecho ""
```

```

response=$(aws iam delete-access-key \
  --user-name "$user_name" \
  --access-key-id "$access_key")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
  aws_cli_error_log $error_code
  errecho "ERROR: AWS reports delete-access-key operation failed.\n$response"
  return 1
fi

iecho "delete-access-key response:$response"
iecho

return 0
}

#####
# function iam_delete_user
#
# This function deletes the specified IAM user.
#
# Parameters:
#     -u user_name  -- The name of the user to create.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function iam_delete_user() {
  local user_name response
  local option OPTARG # Required to use getopt command in a function.

  # bashsupport disable=BP5008
  function usage() {
    echo "function iam_delete_user"
    echo "Deletes an WS Identity and Access Management (IAM) user. You must supply a
username:"
    echo "  -u user_name    The name of the user."
    echo ""
  }
}

```

```
# Retrieve the calling parameters.
while getopts "u:h" option; do
  case "${option}" in
    u) user_name="${OPTARG}" ;;
    h)
      usage
      return 0
      ;;
    \?)
      echo "Invalid parameter"
      usage
      return 1
      ;;
  esac
done
export OPTIND=1

if [[ -z "$user_name" ]]; then
  errecho "ERROR: You must provide a username with the -u parameter."
  usage
  return 1
fi

iecho "Parameters:\n"
iecho "    User name:  $user_name"
iecho ""

# If the user does not exist, we don't want to try to delete it.
if (! iam_user_exists "$user_name"); then
  errecho "ERROR: A user with that name does not exist in the account."
  return 1
fi

response=$(aws iam delete-user \
  --user-name "$user_name")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
  aws_cli_error_log $error_code
  errecho "ERROR: AWS reports delete-user operation failed.$response"
  return 1
fi
```

```
iecho "delete-user response:$response"
iecho

return 0
}
```

- Para obter detalhes da API, consulte os tópicos a seguir na Referência de comandos da AWS CLI.
  - [AttachRolePolicy](#)
  - [CreateAccessKey](#)
  - [CreatePolicy](#)
  - [CreateRole](#)
  - [CreateUser](#)
  - [DeleteAccessKey](#)
  - [DeletePolicy](#)
  - [DeleteRole](#)
  - [DeleteUser](#)
  - [DeleteUserPolicy](#)
  - [DetachRolePolicy](#)
  - [PutUserPolicy](#)

## Exemplos do Amazon S3 de uso da AWS CLI com script Bash

Os exemplos de código a seguir mostram como realizar ações e implementar cenários comuns usando a AWS Command Line Interface com script Bash por meio do Amazon S3.

Ações são trechos de código de programas maiores e devem ser executadas em contexto. Embora as ações mostrem como chamar funções de serviço específicas, é possível ver as ações contextualizadas em seus devidos cenários e exemplos entre serviços.

Cenários são exemplos de código que mostram como realizar uma tarefa específica chamando várias funções dentro do mesmo serviço.

Cada exemplo inclui um link para o GitHub, onde você pode encontrar instruções sobre como configurar e executar o código no contexto.

## Tópicos

- [Ações](#)
- [Cenários](#)

## Ações

Copiar um objeto de um bucket para outro

O exemplo de código a seguir mostra como copiar um objeto do S3 de um bucket para outro.

AWS CLI com script Bash

### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function copy_item_in_bucket
#
# This function creates a copy of the specified file in the same bucket.
#
# Parameters:
#     $1 - The name of the bucket to copy the file from and to.
#     $2 - The key of the source file to copy.
#     $3 - The key of the destination file.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
```

```
function copy_item_in_bucket() {
    local bucket_name=$1
    local source_key=$2
    local destination_key=$3
    local response

    response=$(aws s3api copy-object \
        --bucket "$bucket_name" \
        --copy-source "$bucket_name/$source_key" \
        --key "$destination_key")

    # shellcheck disable=SC2181
    if [[ $? -ne 0 ]]; then
        errecho "ERROR: AWS reports s3api copy-object operation failed.\n$response"
        return 1
    fi
}
```

- Para obter detalhes da API, consulte [CopyObject](#) na Referência de comandos da AWS CLI.

## Criar um bucket

O exemplo de código a seguir mostra como criar um bucket do S3.

## AWS CLI com script Bash

### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function iecho
#
# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.
#####
function iecho() {
    if [[ $VERBOSE == true ]]; then
        echo "$@"
    fi
}
```

```

    fi
}

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function create-bucket
#
# This function creates the specified bucket in the specified AWS Region, unless
# it already exists.
#
# Parameters:
#     -b bucket_name  -- The name of the bucket to create.
#     -r region_code  -- The code for an AWS Region in which to
#                        create the bucket.
#
# Returns:
#     The URL of the bucket that was created.
#
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function create_bucket() {
    local bucket_name region_code response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function create_bucket"
        echo "Creates an Amazon S3 bucket. You must supply a bucket name:"
        echo "  -b bucket_name    The name of the bucket. It must be globally unique."
        echo "  [-r region_code]  The code for an AWS Region in which the bucket is
created."
        echo ""
    }

    # Retrieve the calling parameters.

```

```

while getopts "b:r:h" option; do
    case "${option}" in
        b) bucket_name="${OPTARG}" ;;
        r) region_code="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done

if [[ -z "$bucket_name" ]]; then
    errecho "ERROR: You must provide a bucket name with the -b parameter."
    usage
    return 1
fi

local bucket_config_arg
# A location constraint for "us-east-1" returns an error.
if [[ -n "$region_code" ]] && [[ "$region_code" != "us-east-1" ]]; then
    bucket_config_arg="--create-bucket-configuration LocationConstraint=
$region_code"
fi

iecho "Parameters:\n"
iecho "    Bucket name:    $bucket_name"
iecho "    Region code:    $region_code"
iecho ""

# If the bucket already exists, we don't want to try to create it.
if (bucket_exists "$bucket_name"); then
    errecho "ERROR: A bucket with that name already exists. Try again."
    return 1
fi

# shellcheck disable=SC2086
response=$(aws s3api create-bucket \
    --bucket "$bucket_name" \
    $bucket_config_arg)

```



```
# shellcheck disable=SC2181
if [[ ${?} -ne 0 ]]; then
    errecho "ERROR: AWS reports create-bucket operation failed.\n$response"
    return 1
fi
}
```

- Para obter detalhes da API, consulte [CreateBucket](#) na Referência de comandos da AWS CLI.

## Excluir um bucket vazio

O exemplo de código a seguir mostra como excluir um bucket vazio do S3.

## AWS CLI com script Bash

### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function delete_bucket
#
# This function deletes the specified bucket.
#
# Parameters:
#     $1 - The name of the bucket.
#
# Returns:
#     0 - If successful.
```

```
#      1 - If it fails.
#####
function delete_bucket() {
    local bucket_name=$1
    local response

    response=$(aws s3api delete-bucket \
        --bucket "$bucket_name")

    # shellcheck disable=SC2181
    if [[ $? -ne 0 ]]; then
        errecho "ERROR: AWS reports s3api delete-bucket failed.\n$response"
        return 1
    fi
}
```

- Para obter detalhes da API, consulte [DeleteBucket](#) na Referência de comandos da AWS CLI.

## Excluir um objeto

O exemplo de código a seguir mostra como excluir um objeto do S3.

### AWS CLI com script Bash

#### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
```

```
# function delete_item_in_bucket
#
# This function deletes the specified file from the specified bucket.
#
# Parameters:
#     $1 - The name of the bucket.
#     $2 - The key (file name) in the bucket to delete.

# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function delete_item_in_bucket() {
    local bucket_name=$1
    local key=$2
    local response

    response=$(aws s3api delete-object \
        --bucket "$bucket_name" \
        --key "$key")

    # shellcheck disable=SC2181
    if [[ $? -ne 0 ]]; then
        errecho "ERROR: AWS reports s3api delete-object operation failed.\n$response"
        return 1
    fi
}
```

- Para obter detalhes da API, consulte [DeleteObject](#) na Referência de comandos da AWS CLI.

## Excluir vários objetos

O exemplo de código a seguir mostra como excluir vários objetos de um bucket do S3.

## AWS CLI com script Bash

### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function delete_items_in_bucket
#
# This function deletes the specified list of keys from the specified bucket.
#
# Parameters:
#     $1 - The name of the bucket.
#     $2 - A list of keys in the bucket to delete.

# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function delete_items_in_bucket() {
    local bucket_name=$1
    local keys=$2
    local response

    # Create the JSON for the items to delete.
    local delete_items
    delete_items="{\"Objects\":["
    for key in $keys; do
        delete_items="$delete_items{\"Key\": \"$key\"},"
    done
    delete_items=${delete_items%?} # Remove the final comma.
    delete_items="$delete_items]"

    response=$(aws s3api delete-objects \
        --bucket "$bucket_name" \
        --delete "$delete_items")

    # shellcheck disable=SC2181
    if [[ $? -ne 0 ]]; then
        errecho "ERROR: AWS reports s3api delete-object operation failed.\n$response"
    fi
}
```

```

    return 1
  fi
}

```

- Para obter detalhes da API, consulte [DeleteObjects](#) na Referência de comandos da AWS CLI.

## Determinar a existência de um bucket

O exemplo de código a seguir mostra como determinar a existência de um bucket do S3.

### AWS CLI com script Bash

#### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```

#####
# function bucket_exists
#
# This function checks to see if the specified bucket already exists.
#
# Parameters:
#     $1 - The name of the bucket to check.
#
# Returns:
#     0 - If the bucket already exists.
#     1 - If the bucket doesn't exist.
#####
function bucket_exists() {
    local bucket_name
    bucket_name=$1

    # Check whether the bucket already exists.
    # We suppress all output - we're interested only in the return code.

    if aws s3api head-bucket \
        --bucket "$bucket_name" \
        >/dev/null 2>&1; then

```

```

    return 0 # 0 in Bash script means true.
else
    return 1 # 1 in Bash script means false.
fi
}

```

- Para obter detalhes da API, consulte [HeadBucket](#) na Referência de comandos da AWS CLI.

## Obter um objeto de um bucket

O exemplo de código a seguir mostra como ler dados de um objeto em um bucket do S3.

### AWS CLI com script Bash

#### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function download_object_from_bucket
#
# This function downloads an object in a bucket to a file.
#
# Parameters:
#     $1 - The name of the bucket to download the object from.
#     $2 - The path and file name to store the downloaded bucket.
#     $3 - The key (name) of the object in the bucket.
#
# Returns:

```

```
#      0 - If successful.
#      1 - If it fails.
#####
function download_object_from_bucket() {
    local bucket_name=$1
    local destination_file_name=$2
    local object_name=$3
    local response

    response=$(aws s3api get-object \
        --bucket "$bucket_name" \
        --key "$object_name" \
        "$destination_file_name")

    # shellcheck disable=SC2181
    if [[ ${?} -ne 0 ]]; then
        errecho "ERROR: AWS reports put-object operation failed.\n$response"
        return 1
    fi
}
```

- Para obter detalhes da API, consulte [GetObject](#) na Referência de comandos da AWS CLI.

## Listar objetos em um bucket

O exemplo de código a seguir mostra como listar objetos em um bucket do S3.

## AWS CLI com script Bash

### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
```

```

    printf "%s\n" "$*" 1>&2
}

#####
# function list_items_in_bucket
#
# This function displays a list of the files in the bucket with each file's
# size. The function uses the --query parameter to retrieve only the key and
# size fields from the Contents collection.
#
# Parameters:
#     $1 - The name of the bucket.
#
# Returns:
#     The list of files in text format.
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function list_items_in_bucket() {
    local bucket_name=$1
    local response

    response=$(aws s3api list-objects \
        --bucket "$bucket_name" \
        --output text \
        --query 'Contents[].{Key: Key, Size: Size}')

    # shellcheck disable=SC2181
    if [[ ${?} -eq 0 ]]; then
        echo "$response"
    else
        errecho "ERROR: AWS reports s3api list-objects operation failed.\n$response"
        return 1
    fi
}

```

- Para obter detalhes da API, consulte [ListObjectsV2](#) na Referência de comandos da AWS CLI.

## Carregar um objeto em um bucket

O exemplo de código a seguir mostra como carregar um objeto em um bucket do S3.



## AWS CLI com script Bash

 Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function copy_file_to_bucket
#
# This function creates a file in the specified bucket.
#
# Parameters:
#     $1 - The name of the bucket to copy the file to.
#     $2 - The path and file name of the local file to copy to the bucket.
#     $3 - The key (name) to call the copy of the file in the bucket.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function copy_file_to_bucket() {
    local response bucket_name source_file destination_file_name
    bucket_name=$1
    source_file=$2
    destination_file_name=$3

    response=$(aws s3api put-object \
        --bucket "$bucket_name" \
        --body "$source_file" \
        --key "$destination_file_name")
```

```
# shellcheck disable=SC2181
if [[ ${?} -ne 0 ]]; then
    errecho "ERROR: AWS reports put-object operation failed.\n$response"
    return 1
fi
}
```

- Para obter detalhes da API, consulte [PutObject](#) na Referência de comandos da AWS CLI.

## Cenários

### Conceitos básicos de buckets e objetos

O código de exemplo a seguir mostra como:

- Criar um bucket e fazer upload de um arquivo para ele.
- Baixar um objeto de um bucket.
- Copiar um objeto em uma subpasta em um bucket.
- Listar os objetos em um bucket.
- Excluir os objetos do bucket e o bucket.

### AWS CLI com script Bash

#### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####
# function s3_getting_started
#
# This function creates, copies, and deletes S3 buckets and objects.
#
# Returns:
#     0 - If successful.
#     1 - If an error occurred.
#####
```

```
function s3_getting_started() {
{
  if [ "$BUCKET_OPERATIONS_SOURCED" != "True" ]; then
    cd bucket-lifecycle-operations || exit

    source ./bucket_operations.sh
    cd ..
  fi
}

echo_repeat "*" 88
echo "Welcome to the Amazon S3 getting started demo."
echo_repeat "*" 88

local bucket_name
bucket_name=$(generate_random_name "doc-example-bucket")

local region_code
region_code=$(aws configure get region)

if create_bucket -b "$bucket_name" -r "$region_code"; then
  echo "Created demo bucket named $bucket_name"
else
  errecho "The bucket failed to create. This demo will exit."
  return 1
fi

local file_name
while [ -z "$file_name" ]; do
  echo -n "Enter a file you want to upload to your bucket: "
  get_input
  file_name=$get_input_result

  if [ ! -f "$file_name" ]; then
    echo "Could not find file $file_name. Are you sure it exists?"
    file_name=""
  fi
done

local key
key="$(basename "$file_name")"

local result=0
if copy_file_to_bucket "$bucket_name" "$file_name" "$key"; then
```

```
    echo "Uploaded file $file_name into bucket $bucket_name with key $key."
else
    result=1
fi

local destination_file
destination_file="$file_name.download"
if yes_no_input "Would you like to download $key to the file $destination_file?
(y/n) "; then
    if download_object_from_bucket "$bucket_name" "$destination_file" "$key"; then
        echo "Downloaded $key in the bucket $bucket_name to the file
$destination_file."
    else
        result=1
    fi
fi

if yes_no_input "Would you like to copy $key a new object key in your bucket? (y/
n) "; then
    local to_key
    to_key="demo/$key"
    if copy_item_in_bucket "$bucket_name" "$key" "$to_key"; then
        echo "Copied $key in the bucket $bucket_name to the $to_key."
    else
        result=1
    fi
fi

local bucket_items
bucket_items=$(list_items_in_bucket "$bucket_name")

# shellcheck disable=SC2181
if [[ $? -ne 0 ]]; then
    result=1
fi

echo "Your bucket contains the following items."
echo -e "Name\t\tSize"
echo "$bucket_items"

if yes_no_input "Delete the bucket, $bucket_name, as well as the objects in it?
(y/n) "; then
    bucket_items=$(echo "$bucket_items" | cut -f 1)
```

```

    if delete_items_in_bucket "$bucket_name" "$bucket_items"; then
        echo "The following items were deleted from the bucket $bucket_name"
        echo "$bucket_items"
    else
        result=1
    fi

    if delete_bucket "$bucket_name"; then
        echo "Deleted the bucket $bucket_name"
    else
        result=1
    fi
fi

return $result
}

```

As funções do Amazon S3 usadas neste cenário.

```

#####
# function create-bucket
#
# This function creates the specified bucket in the specified AWS Region, unless
# it already exists.
#
# Parameters:
#     -b bucket_name  -- The name of the bucket to create.
#     -r region_code  -- The code for an AWS Region in which to
#                        create the bucket.
#
# Returns:
#     The URL of the bucket that was created.
#
# And:
#     0 - If successful.
#     1 - If it fails.
#####
function create_bucket() {
    local bucket_name region_code response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {

```

```

    echo "function create_bucket"
    echo "Creates an Amazon S3 bucket. You must supply a bucket name:"
    echo "  -b bucket_name    The name of the bucket. It must be globally unique."
    echo "  [-r region_code]    The code for an AWS Region in which the bucket is
created."
    echo ""
}

# Retrieve the calling parameters.
while getopts "b:r:h" option; do
    case "${option}" in
        b) bucket_name="${OPTARG}" ;;
        r) region_code="${OPTARG}" ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done

if [[ -z "$bucket_name" ]]; then
    errecho "ERROR: You must provide a bucket name with the -b parameter."
    usage
    return 1
fi

local bucket_config_arg
# A location constraint for "us-east-1" returns an error.
if [[ -n "$region_code" ]] && [[ "$region_code" != "us-east-1" ]]; then
    bucket_config_arg="--create-bucket-configuration LocationConstraint=
$region_code"
fi

iecho "Parameters:\n"
iecho "    Bucket name:    $bucket_name"
iecho "    Region code:    $region_code"
iecho ""

# If the bucket already exists, we don't want to try to create it.

```

```

if (bucket_exists "$bucket_name"); then
    errecho "ERROR: A bucket with that name already exists. Try again."
    return 1
fi

# shellcheck disable=SC2086
response=$(aws s3api create-bucket \
    --bucket "$bucket_name" \
    $bucket_config_arg)

# shellcheck disable=SC2181
if [[ ${?} -ne 0 ]]; then
    errecho "ERROR: AWS reports create-bucket operation failed.\n$response"
    return 1
fi
}

#####
# function copy_file_to_bucket
#
# This function creates a file in the specified bucket.
#
# Parameters:
#     $1 - The name of the bucket to copy the file to.
#     $2 - The path and file name of the local file to copy to the bucket.
#     $3 - The key (name) to call the copy of the file in the bucket.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function copy_file_to_bucket() {
    local response bucket_name source_file destination_file_name
    bucket_name=$1
    source_file=$2
    destination_file_name=$3

    response=$(aws s3api put-object \
        --bucket "$bucket_name" \
        --body "$source_file" \
        --key "$destination_file_name")

    # shellcheck disable=SC2181
    if [[ ${?} -ne 0 ]]; then

```

```

        errecho "ERROR: AWS reports put-object operation failed.\n$response"
    return 1
fi
}

#####
# function download_object_from_bucket
#
# This function downloads an object in a bucket to a file.
#
# Parameters:
#     $1 - The name of the bucket to download the object from.
#     $2 - The path and file name to store the downloaded bucket.
#     $3 - The key (name) of the object in the bucket.
#
# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function download_object_from_bucket() {
    local bucket_name=$1
    local destination_file_name=$2
    local object_name=$3
    local response

    response=$(aws s3api get-object \
        --bucket "$bucket_name" \
        --key "$object_name" \
        "$destination_file_name")

    # shellcheck disable=SC2181
    if [[ ${?} -ne 0 ]]; then
        errecho "ERROR: AWS reports put-object operation failed.\n$response"
        return 1
    fi
}

#####
# function copy_item_in_bucket
#
# This function creates a copy of the specified file in the same bucket.
#
# Parameters:
#     $1 - The name of the bucket to copy the file from and to.

```



```

#      $2 - The key of the source file to copy.
#      $3 - The key of the destination file.
#
# Returns:
#      0 - If successful.
#      1 - If it fails.
#####
function copy_item_in_bucket() {
    local bucket_name=$1
    local source_key=$2
    local destination_key=$3
    local response

    response=$(aws s3api copy-object \
        --bucket "$bucket_name" \
        --copy-source "$bucket_name/$source_key" \
        --key "$destination_key")

    # shellcheck disable=SC2181
    if [[ $? -ne 0 ]]; then
        errecho "ERROR: AWS reports s3api copy-object operation failed.\n$response"
        return 1
    fi
}

#####
# function list_items_in_bucket
#
# This function displays a list of the files in the bucket with each file's
# size. The function uses the --query parameter to retrieve only the key and
# size fields from the Contents collection.
#
# Parameters:
#      $1 - The name of the bucket.
#
# Returns:
#      The list of files in text format.
#      And:
#      0 - If successful.
#      1 - If it fails.
#####
function list_items_in_bucket() {
    local bucket_name=$1
    local response

```

```

response=$(aws s3api list-objects \
  --bucket "$bucket_name" \
  --output text \
  --query 'Contents[].{Key: Key, Size: Size}')

# shellcheck disable=SC2181
if [[ ${?} -eq 0 ]]; then
  echo "$response"
else
  errecho "ERROR: AWS reports s3api list-objects operation failed.\n$response"
  return 1
fi
}

#####
# function delete_items_in_bucket
#
# This function deletes the specified list of keys from the specified bucket.
#
# Parameters:
#   $1 - The name of the bucket.
#   $2 - A list of keys in the bucket to delete.
#
# Returns:
#   0 - If successful.
#   1 - If it fails.
#####
function delete_items_in_bucket() {
  local bucket_name=$1
  local keys=$2
  local response

  # Create the JSON for the items to delete.
  local delete_items
  delete_items="{\"Objects\":["
  for key in $keys; do
    delete_items="$delete_items{\"Key\": \"$key\"},"
  done
  delete_items=${delete_items%?} # Remove the final comma.
  delete_items="$delete_items]"

  response=$(aws s3api delete-objects \
    --bucket "$bucket_name" \

```

```

    --delete "$delete_items")

# shellcheck disable=SC2181
if [[ $? -ne 0 ]]; then
    errecho "ERROR: AWS reports s3api delete-object operation failed.\n$response"
    return 1
fi
}

#####
# function delete_bucket
#
# This function deletes the specified bucket.
#
# Parameters:
#     $1 - The name of the bucket.

# Returns:
#     0 - If successful.
#     1 - If it fails.
#####
function delete_bucket() {
    local bucket_name=$1
    local response

    response=$(aws s3api delete-bucket \
        --bucket "$bucket_name")

# shellcheck disable=SC2181
if [[ $? -ne 0 ]]; then
    errecho "ERROR: AWS reports s3api delete-bucket failed.\n$response"
    return 1
fi
}

```

- Para obter detalhes da API, consulte os tópicos a seguir na Referência de comandos da AWS CLI.
  - [CopyObject](#)
  - [CreateBucket](#)
  - [DeleteBucket](#)
  - [DeleteObjects](#)

- [GetObject](#)
- [ListObjectsV2](#)
- [PutObject](#)

## Exemplos do AWS STS de uso da AWS CLI com script Bash

Os exemplos de código a seguir mostram como realizar ações e implementar cenários comuns usando a AWS Command Line Interface com script Bash por meio do AWS STS.

Ações são trechos de código de programas maiores e devem ser executadas em contexto. Embora as ações mostrem como chamar funções de serviço específicas, é possível ver as ações contextualizadas em seus devidos cenários e exemplos entre serviços.

Cenários são exemplos de código que mostram como realizar uma tarefa específica chamando várias funções dentro do mesmo serviço.

Cada exemplo inclui um link para o GitHub, onde você pode encontrar instruções sobre como configurar e executar o código no contexto.

### Tópicos

- [Ações](#)

### Ações

#### Assumir uma função

O exemplo de código a seguir mostra como assumir um perfil com o AWS STS.

#### AWS CLI com script Bash

##### Note

Há mais no GitHub. Encontre o exemplo completo e saiba como configurar e executar no [AWSCode Examples Repository](#).

```
#####  
# function iecho  
#
```

```

# This function enables the script to display the specified text only if
# the global variable $VERBOSE is set to true.
#####
function iecho() {
    if [[ $VERBOSE == true ]]; then
        echo "$@"
    fi
}

#####
# function errecho
#
# This function outputs everything sent to it to STDERR (standard error output).
#####
function errecho() {
    printf "%s\n" "$*" 1>&2
}

#####
# function sts_assume_role
#
# This function assumes a role in the AWS account and returns the temporary
# credentials.
#
# Parameters:
#     -n role_session_name -- The name of the session.
#     -r role_arn -- The ARN of the role to assume.
#
# Returns:
#     [access_key_id, secret_access_key, session_token]
#     And:
#     0 - If successful.
#     1 - If an error occurred.
#####
function sts_assume_role() {
    local role_session_name role_arn response
    local option OPTARG # Required to use getopt command in a function.

    # bashsupport disable=BP5008
    function usage() {
        echo "function sts_assume_role"
        echo "Assumes a role in the AWS account and returns the temporary credentials:"
        echo "  -n role_session_name -- The name of the session."
        echo "  -r role_arn -- The ARN of the role to assume."
    }

```

```

    echo ""
}

while getopts n:r:h option; do
    case "${option}" in
        n) role_session_name=${OPTARG} ;;
        r) role_arn=${OPTARG} ;;
        h)
            usage
            return 0
            ;;
        \?)
            echo "Invalid parameter"
            usage
            return 1
            ;;
    esac
done

response=$(aws sts assume-role \
    --role-session-name "$role_session_name" \
    --role-arn "$role_arn" \
    --output text \
    --query "Credentials.[AccessKeyId, SecretAccessKey, SessionToken]")

local error_code=${?}

if [[ $error_code -ne 0 ]]; then
    aws_cli_error_log $error_code
    errecho "ERROR: AWS reports create-role operation failed.\n$response"
    return 1
fi

echo "$response"

return 0
}

```

- Para obter detalhes da API, consulte [AssumeRole](#) na Referência de comandos da AWS CLI.

# Segurança na AWS Command Line Interface

A segurança na nuvem na AWS é a nossa maior prioridade. Como cliente da AWS, você contará com um datacenter e uma arquitetura de rede criados para atender aos requisitos das organizações com as maiores exigências de segurança.

A segurança é uma responsabilidade compartilhada entre a AWS e você. O [modelo de responsabilidade compartilhada](#) descreve isso como segurança da nuvem e segurança na nuvem:

- Segurança da nuvem — a AWS é responsável pela proteção da infraestrutura que executa serviços AWS na Nuvem AWS. A AWS também fornece serviços que podem ser usados com segurança. Auditores de terceiros testam e verificam regularmente a eficácia da nossa segurança como parte dos [AWS Programas de Conformidade](#). Para saber mais sobre os programas de conformidade que se aplicam ao AWS Command Line Interface, consulte [AWS Serviços da em escopo por programa de conformidade](#).
- Segurança na nuvem: sua responsabilidade é determinada pelo serviço da AWS que você usa. Você também é responsável por outros fatores, incluindo a confidencialidade de seus dados, os requisitos da empresa e as leis e regulamentos aplicáveis.

Esta documentação ajuda você a entender como aplicar o modelo de responsabilidade compartilhada ao usar a AWS Command Line Interface (AWS CLI). Os tópicos a seguir mostram como configurar a AWS CLI para atender aos seus objetivos de segurança e conformidade. Você também aprende como usar a AWS CLI para ajudar a monitorar e a proteger os recursos da AWS.

## Tópicos

- [Proteção de dados no AWS CLI](#)
- [Identity and Access Management](#)
- [Validação de conformidade para este produto ou serviço da AWS](#)
- [Resiliência para este produto ou serviço da AWS](#)
- [Segurança da infraestrutura para esse produto ou serviço da AWS](#)
- [Impor uma versão mínima do TLS](#)

# Proteção de dados no AWS CLI

O AWS [modelo de responsabilidade compartilhada](#) se aplica à proteção de dados no AWS Command Line Interface. Conforme descrito nesse modelo, a AWS é responsável por proteger a infraestrutura global que executa toda a Nuvem AWS. Você é responsável por manter o controle sobre seu conteúdo hospedado nessa infraestrutura. Você também é responsável pelas tarefas de configuração e gerenciamento de segurança dos Serviços da AWS que usa. Para obter mais informações sobre a privacidade de dados, consulte as [Perguntas frequentes sobre privacidade de dados](#). Para mais informações sobre a proteção de dados na Europa, consulte o artigo [AWS Shared Responsibility Model and GDPR](#) no Blog de segurança da AWS.

Para fins de proteção de dados, recomendamos que você proteja as credenciais da Conta da AWS e configure as contas de usuário individuais com o AWS IAM Identity Center ou o AWS Identity and Access Management (IAM). Dessa maneira, cada usuário receberá apenas as permissões necessárias para cumprir suas obrigações de trabalho. Recomendamos também que você proteja seus dados das seguintes formas:

- Use uma autenticação multifator (MFA [multi-factor authentication]) com cada conta.
- Use SSL/TLS para se comunicar com os atributos da AWS. Exigimos TLS 1.2 e recomendamos TLS 1.3.
- Configure o registro em log das atividades da API e do usuário com o AWS CloudTrail
- Use as soluções de criptografia da AWS, juntamente com todos os controles de segurança padrão dos Serviços da AWS.
- Use serviços gerenciados de segurança avançada, como o Amazon Macie, que ajuda a localizar e proteger dados sigilosos armazenados no Amazon S3.
- Se você precisar de módulos criptográficos validados pelo FIPS 140-2 ao acessar a AWS por meio de uma interface de linha de comandos ou uma API, use um endpoint do FIPS. Para ter mais informações sobre endpoints do FIPS, consulte [Federal Information Processing Standard \(FIPS\) 140-2](#).

É altamente recomendável que nunca sejam colocadas informações de identificação confidenciais, como endereços de e-mail dos seus clientes, em marcações ou campos de formato livre, como um campo Nome. Isso inclui trabalhar com a AWS CLI ou outros Serviços da AWS usando o console, a API, a AWS CLI ou os AWS SDKs. Quaisquer dados inseridos em tags ou campos de texto de formato livre usados para nomes podem ser usados para logs de faturamento ou de diagnóstico. Se



você fornecer um URL para um servidor externo, recomendamos fortemente que não sejam incluídas informações de credenciais no URL para validar a solicitação a esse servidor.

## Criptografia de dados

Um atributo fundamental de qualquer serviço seguro é que as informações sejam criptografadas quando não estão sendo usadas ativamente.

### Criptografia inativa

A própria AWS CLI não armazena nenhum dado do cliente além das credenciais de que precisa para interagir com os serviços da AWS em nome do usuário.

Se você usar a AWS CLI para invocar um serviço da AWS que transmita dados do cliente ao computador local para armazenamento, consulte o capítulo Segurança e conformidade no Guia do usuário desse serviço para obter informações sobre como esses dados são armazenados, protegidos e criptografados.

### Criptografia em trânsito

Por padrão, todos os dados transmitidos do computador cliente que executa os endpoints de serviço da AWS CLI e da AWS são criptografados enviando tudo por meio de uma conexão HTTPS/TLS.

Você não precisa fazer nada para ativar o uso do HTTPS/TLS. Ele está sempre ativado, a menos que você o desative explicitamente para um comando individual usando a opção de linha de comando `--no-verify-ssl`.

## Identity and Access Management

O AWS Identity and Access Management (IAM) é um AWS service (Serviço da AWS) que ajuda a controlar o acesso aos recursos da AWS de forma segura. Os administradores do IAM controlam quem pode ser autenticado (conectado) e autorizado (ter permissões) a usar os recursos do AWS. O IAM é um AWS service (Serviço da AWS) que pode ser usado sem custo adicional.

### Tópicos

- [Público](#)
- [Autenticando com identidades](#)
- [Gerenciamento do acesso usando políticas](#)
- [Como Serviços da AWS funcionam com o IAM](#)

- [Solução de problemas de identidade e acesso do AWS](#)

## Público

O uso do AWS Identity and Access Management (IAM) varia dependendo do trabalho que for realizado no AWS.

**Usuário do serviço:** se você usar Serviços da AWS para fazer seu trabalho, o administrador fornecerá as credenciais e as permissões necessárias. À medida que usar mais recursos do AWS para fazer seu trabalho, você poderá precisar de permissões adicionais. Entender como o acesso é gerenciado pode ajudá-lo a solicitar as permissões corretas ao seu administrador. Se você não conseguir acessar um atributo na AWS, consulte [Solução de problemas de identidade e acesso do AWS](#) ou o guia do usuário do AWS service (Serviço da AWS) que você está usando.

**Administrador do serviço** – Se você for o responsável pelos recursos do AWS na empresa, provavelmente terá acesso total ao AWS. Cabe a você determinar quais funcionalidades e recursos do AWS os usuários do serviço devem acessar. Assim, você deve enviar solicitações ao administrador do IAM para alterar as permissões dos usuários de seu serviço. Revise as informações nesta página para entender os Introdução ao IAM. Para saber mais sobre como sua empresa pode usar o IAM com a AWS, consulte o guia do usuário do AWS service (Serviço da AWS) que você está usando.

**Administrador do IAM** – Se você for um administrador do IAM, talvez queira saber detalhes sobre como pode gravar políticas para gerenciar o acesso ao AWS. Para visualizar exemplos de políticas baseadas em identidade da AWS que podem ser usadas no IAM, consulte o guia do usuário do AWS service (Serviço da AWS) que você está usando.

## Autenticando com identidades

A autenticação é a forma como você faz login na AWS usando suas credenciais de identidade. É necessário ser autenticado (fazer login na AWS) como o usuário raiz da Usuário raiz da conta da AWS, como usuário do IAM ou assumindo um perfil do IAM.

Você pode fazer login na AWS como uma identidade federada usando credenciais fornecidas por uma fonte de identidades. AWS IAM Identity Center Os usuários do IAM Identity Center, a autenticação única da empresa e as suas credenciais do Google ou do Facebook são exemplos de identidades federadas. Quando você faz login como uma identidade federada, o administrador já configurou anteriormente a federação de identidades usando perfis do IAM. Quando você acessa a AWS usando a federação, está indiretamente assumindo um perfil.

É possível fazer login no AWS Management Console ou no portal de acesso da AWS dependendo do tipo de usuário que você é. Para obter mais informações sobre como fazer login na AWS, consulte [Como fazer login na conta da Conta da AWS](#) no Guia do usuário do Início de Sessão da AWS.

Se você acessar a AWS programaticamente, a AWS fornecerá um kit de desenvolvimento de software (SDK) e uma interface da linha de comando (CLI) para você assinar criptograficamente as solicitações usando as suas credenciais. Se você não utilizar as ferramentas da AWS, deverá assinar as solicitações por conta própria. Para obter mais informações sobre como usar o método recomendado para assinar solicitações por conta própria, consulte [Assinar solicitações de API da AWS](#) no Guia do usuário do IAM.

Independentemente do método de autenticação usado, também pode ser exigido que você forneça mais informações de segurança. Por exemplo, a AWS recomenda o uso da autenticação multifator (MFA) para aumentar a segurança de sua conta. Para saber mais, consulte [Autenticação multifator](#) no Guia AWS IAM Identity Center do usuário. [Usar a autenticação multifator \(MFA\) na AWS](#) no Guia do usuário do IAM.

## Usuário root da Conta da AWS

Ao criar uma Conta da AWS, você começa com uma identidade de login que tem acesso completo a todos os recursos e Serviços da AWS na conta. Essa identidade, denominada usuário raiz da Conta da AWS, e é acessada por login com o endereço de e-mail e a senha que você usou para criar a conta. É altamente recomendável não usar o usuário raiz para tarefas diárias. Proteja as credenciais do usuário raiz e use-as para executar as tarefas que somente ele pode executar. Para obter a lista completa das tarefas que exigem login como usuário raiz, consulte [Tarefas que exigem credenciais de usuário raiz](#) no Guia do usuário do IAM.

## Identidade federada

Como prática recomendada, exija que os usuários, inclusive os que precisam de acesso de administrador, usem a federação com um provedor de identidades para acessar Serviços da AWS usando credenciais temporárias.

Identidade federada é um usuário de seu diretório de usuários corporativos, um provedor de identidades da web, o AWS Directory Service, o diretório do Centro de Identidade ou qualquer usuário que acesse os Serviços da AWS usando credenciais fornecidas por meio de uma fonte de identidade. Quando as identidades federadas acessam Contas da AWS, elas assumem funções que fornecem credenciais temporárias.

Para o gerenciamento de acesso centralizado, recomendamos usar o AWS IAM Identity Center. Você pode criar usuários e grupos no Centro de Identidade do IAM ou se conectar e sincronizar com um conjunto de usuários e grupos em sua própria fonte de identidade para uso em todas as suas Contas da AWS e aplicações. Para obter mais informações sobre o Centro de Identidade do IAM, consulte [O que é o Centro de Identidade do IAM?](#) no Guia do usuário do AWS IAM Identity Center.

## Grupos e usuários do IAM

Um [usuário do IAM](#) é uma identidade dentro da Conta da AWS que tem permissões específicas para uma única pessoa ou aplicação. Sempre que possível, recomendamos depender de credenciais temporárias em vez de criar usuários do IAM com credenciais de longo prazo, como senhas e chaves de acesso. No entanto, se você tiver casos de uso específicos que exijam credenciais de longo prazo com usuários do IAM, recomendamos alternar as chaves de acesso. Para obter mais informações, consulte [Altere as chaves de acesso regularmente para casos de uso que exijam credenciais de longo prazo](#) no Guia do usuário do IAM.

Um [grupo do IAM](#) é uma identidade que especifica uma coleção de usuários do IAM. Não é possível fazer login como um grupo. É possível usar grupos para especificar permissões para vários usuários de uma vez. Os grupos facilitam o gerenciamento de permissões para grandes conjuntos de usuários. Por exemplo, você pode ter um grupo chamado IAMAdmins e atribuir a esse grupo permissões para administrar recursos do IAM.

Usuários são diferentes de funções. Um usuário é exclusivamente associado a uma pessoa ou a uma aplicação, mas uma função pode ser assumida por qualquer pessoa que precisar dela. Os usuários têm credenciais permanentes de longo prazo, mas as funções fornecem credenciais temporárias. Para saber mais, consulte [Quando criar um usuário do IAM \(em vez de uma função\)](#) no Guia do usuário do IAM.

## Funções do IAM

Um [perfil do IAM](#) é uma identidade dentro da Conta da AWS que tem permissões específicas. Ela é semelhante a um usuário do IAM, mas não está associada a uma pessoa específica. É possível assumir temporariamente um perfil do IAM no AWS Management Console [alternando perfis](#). É possível assumir um perfil chamando uma operação de API da AWS CLI ou da AWS, ou usando um URL personalizado. Para obter mais informações sobre métodos para o uso de perfis, consulte [Usar perfis do IAM](#) no Guia do usuário do IAM.

Os perfis do IAM com credenciais temporárias são úteis nas seguintes situações:

- **Acesso de usuário federado:** para atribuir permissões a identidades federadas, você pode criar um perfil e definir permissões para ele. Quando uma identidade federada é autenticada, essa identidade é associada ao perfil e recebe as permissões definidas pelo mesmo. Para obter mais informações sobre perfis para federação, consulte [Criar um perfil para um provedor de identidades de terceiros](#) no Guia do usuário do IAM. Se você usar o IAM Identity Center, configure um conjunto de permissões. Para controlar o que suas identidades podem acessar após a autenticação, o IAM Identity Center correlaciona o conjunto de permissões a um perfil no IAM. Para obter informações sobre conjuntos de permissões, consulte [Conjuntos de permissões](#) no Guia do usuário do AWS IAM Identity Center.
- **Permissões temporárias para usuários do IAM:** um usuário ou um perfil do IAM pode assumir um perfil do IAM para obter temporariamente permissões diferentes para uma tarefa específica.
- **Acesso entre contas:** é possível usar um perfil do IAM para permitir que alguém (uma entidade principal confiável) em outra conta acesse recursos em sua conta. As funções são a principal forma de conceder acesso entre contas. No entanto, alguns Serviços da AWS permitem que você anexe uma política diretamente a um recurso (em vez de usar uma função como proxy). Para saber a diferença entre funções e políticas baseadas em recurso para acesso entre contas, consulte [Como as funções do IAM diferem das políticas baseadas em recurso](#) no Guia do usuário do IAM.
- **Acesso entre serviços:** alguns Serviços da AWS usam recursos em outros Serviços da AWS. Por exemplo, quando você faz uma chamada em um serviço, é comum que esse serviço execute aplicações no Amazon EC2 ou armazene objetos no Amazon S3. Um serviço pode fazer isso usando as permissões do principal de chamada, usando um perfil de serviço ou um perfil vinculado ao serviço.
- **Encaminhamento de sessões de acesso (FAS):** qualquer pessoa que utilizar uma função ou usuário do IAM para realizar ações na AWS é considerada uma entidade principal. Ao usar alguns serviços, você pode executar uma ação que inicia outra ação em um serviço diferente. O recurso FAS utiliza as permissões da entidade principal que chama um AWS service (Serviço da AWS), combinadas às permissões do AWS service (Serviço da AWS) solicitante, para realizar solicitações para serviços downstream. As solicitações de FAS só são feitas quando um serviço recebe uma solicitação que exige interações com outros Serviços da AWS ou com recursos para serem concluídas. Nesse caso, você precisa ter permissões para executar ambas as ações. Para obter detalhes da política ao fazer solicitações de FAS, consulte [Encaminhar sessões de acesso](#).
- **Perfil de serviço:** um perfil de serviço é um [perfil do IAM](#) que um serviço assume para realizar ações em seu nome. Um administrador do IAM pode criar, modificar e excluir um perfil de

serviço do IAM. Para obter mais informações, consulte [Criar um perfil para delegar permissões a um AWS service \(Serviço da AWS\)](#) no Guia do usuário do IAM.

- Perfil vinculado ao serviço: um perfil vinculado ao serviço é um tipo de perfil de serviço vinculado a um AWS service (Serviço da AWS). O serviço pode assumir o perfil para executar uma ação em seu nome. Os perfis vinculados ao serviço aparecem em sua Conta da AWS e são de propriedade do serviço. Um administrador do IAM pode visualizar, mas não pode editar as permissões para perfis vinculados a serviço.
- Aplicações em execução no Amazon EC2: é possível usar um perfil do IAM para gerenciar credenciais temporárias para aplicações em execução em uma instância do EC2 e fazer solicitações da AWS CLI ou da AWS API. É preferível fazer isso e armazenar chaves de acesso na instância do EC2. Para atribuir uma função da AWS a uma instância do EC2 e disponibilizá-la para todas as suas aplicações, crie um perfil de instância que esteja anexado a ela. Um perfil de instância contém a função e permite que os programas em execução na instância do EC2 obtenham credenciais temporárias. Para mais informações, consulte [Usar um perfil do IAM para conceder permissões a aplicações em execução nas instâncias do Amazon EC2](#) no Guia do usuário do IAM.

Para saber se deseja usar os perfis do IAM, consulte [Quando criar um perfil do IAM \(em vez de um usuário\)](#) no Guia do usuário do IAM.

## Gerenciamento do acesso usando políticas

Você controla o acesso na AWS criando políticas e anexando-as a identidades ou recursos da AWS. Uma política é um objeto na AWS que, quando associado a uma identidade ou recurso, define suas permissões. A AWS avalia essas políticas quando uma entidade principal (usuário, usuário raiz ou sessão de função) faz uma solicitação. As permissões nas políticas determinam se a solicitação será permitida ou negada. A maioria das políticas são armazenadas na AWS como documentos JSON. Para obter mais informações sobre a estrutura e o conteúdo de documentos de políticas JSON, consulte [Visão geral das políticas JSON](#) no Guia do usuário do IAM.

Os administradores podem usar AWS as políticas JSON para especificar quem tem acesso a quê. Ou seja, qual entidade principal pode executar ações em quais recursos e em que condições.

Por padrão, usuários e funções não têm permissões. Para conceder aos usuários permissão para executar ações nos recursos de que eles precisam, um administrador do IAM pode criar políticas do IAM. O administrador pode então adicionar as políticas do IAM a perfis, e os usuários podem assumir os perfis.

As políticas do IAM definem permissões para uma ação, independentemente do método usado para executar a operação. Por exemplo, suponha que você tenha uma política que permite a ação `iam:GetRole`. Um usuário com essa política pode obter informações de perfis do AWS Management Console, da AWS CLI ou da API da AWS.

## Políticas baseadas em identidade

As políticas baseadas em identidade são documentos de políticas de permissões JSON que você pode anexar a uma identidade, como usuário, grupo de usuários ou perfil do IAM. Essas políticas controlam quais ações os usuários e funções podem realizar, em quais recursos e em que condições. Para saber como criar uma política baseada em identidade, consulte [Criar políticas do IAM](#) no Guia do usuário do IAM.

As políticas baseadas em identidade podem ser categorizadas ainda mais como políticas em linha ou políticas gerenciadas. As políticas em linha são anexadas diretamente a um único usuário, grupo ou função. As políticas gerenciadas são políticas independentes que podem ser anexadas a vários usuários, grupos e funções na Conta da AWS. As políticas gerenciadas incluem políticas gerenciadas pela AWS e políticas gerenciadas pelo cliente. Para saber como escolher entre uma política gerenciada ou uma política em linha, consulte [Escolher entre políticas gerenciadas e políticas em linha](#) no Guia do usuário do IAM.

## Políticas baseadas em recurso

Políticas baseadas em recurso são documentos de políticas JSON que você anexa a um recurso. São exemplos de políticas baseadas em recursos as políticas de confiança de perfil do IAM e as políticas de bucket do Amazon S3. Em serviços compatíveis com políticas baseadas em recursos, os administradores de serviço podem usá-las para controlar o acesso a um recurso específico. Para o recurso ao qual a política está anexada, a política define quais ações um principal especificado pode executar nesse recurso e em que condições. Você deve [especificar um principal](#) em uma política baseada em recursos. As entidades principais podem incluir contas, usuários, perfis, usuários federados ou Serviços da AWS.

Políticas baseadas em recursos são políticas em linha que estão localizadas nesse serviço. Não é possível usar as políticas gerenciadas da AWS do IAM em uma política baseada em recursos.

## Listas de controle de acesso (ACLs)

As listas de controle de acesso (ACLs) controlam quais entidades principais (membros, usuários ou funções da conta) têm permissões para acessar um recurso. As ACLs são semelhantes às políticas baseadas em recursos, embora não usem o formato de documento de política JSON.



Amazon S3, AWS WAF e Amazon VPC são exemplos de serviços que oferecem suporte a ACLs. Para saber mais sobre ACLs, consulte [Visão geral da lista de controle de acesso \(ACL\)](#) no Guia do desenvolvedor do Amazon Simple Storage Service.

## Outros tipos de política

A AWS aceita tipos de política menos comuns. Esses tipos de política podem definir o máximo de permissões concedidas a você pelos tipos de política mais comuns.

- **Limites de permissões:** um limite de permissões é um recurso avançado no qual você define o máximo de permissões que uma política baseada em identidade pode conceder a uma entidade do IAM (usuário ou perfil do IAM). É possível definir um limite de permissões para uma entidade. As permissões resultantes são a interseção das políticas baseadas em identidade de uma entidade e dos seus limites de permissões. As políticas baseadas em recurso que especificam o usuário ou a função no campo `Principal` não são limitadas pelo limite de permissões. Uma negação explícita em qualquer uma dessas políticas substitui a permissão. Para obter mais informações sobre limites de permissões, consulte [Limites de permissões para identidades do IAM](#) no Guia do usuário do IAM.
- **Políticas de controle de serviço (SCPs):** SCPs são políticas JSON que especificam as permissões máximas para uma organização ou unidade organizacional (UO) no AWS Organizations. O AWS Organizations é um serviço para agrupar e gerenciar centralmente várias Contas da AWS pertencentes à sua empresa. Se você habilitar todos os recursos em uma organização, poderá aplicar políticas de controle de serviço (SCPs) a qualquer uma ou a todas as contas. O SCP limita as permissões para entidades em contas-membro, incluindo cada Usuário raiz da conta da AWS. Para obter mais informações sobre o Organizations e SCPs, consulte [Como os SCPs funcionam](#) no Guia do usuário do AWS Organizations.
- **Políticas de sessão:** são políticas avançadas que você transmite como um parâmetro quando cria de forma programática uma sessão temporária para uma função ou um usuário federado. As permissões da sessão resultante são a interseção das políticas baseadas em identidade do usuário ou da função e das políticas de sessão. As permissões também podem ser provenientes de uma política baseada em recurso. Uma negação explícita em qualquer uma dessas políticas substitui a permissão. Para obter mais informações, consulte [Políticas de sessão](#) no Guia do usuário do IAM.



## Vários tipos de política

Quando vários tipos de política são aplicáveis a uma solicitação, é mais complicado compreender as permissões resultantes. Para saber como a AWS determina se deve permitir uma solicitação quando há vários tipos de política envolvidos, consulte [Lógica da avaliação](#) de políticas no Guia do usuário do IAM.

## Como Serviços da AWS funcionam com o IAM

Para obter uma visão geral de como Serviços da AWS funcionam com a maioria dos atributos do IAM, consulte [Serviços da AWS compatíveis com o IAM](#) no Guia do usuário do IAM.

Para saber como usar um AWS service (Serviço da AWS) específico com o IAM, consulte a seção de segurança do Guia do usuário do serviço relevante.

## Solução de problemas de identidade e acesso do AWS

Use as seguintes informações para ajudar a diagnosticar e corrigir problemas comuns que podem ser encontrados ao trabalhar com o e o IAM.AWS

### Tópicos

- [Não tenho autorização para executar uma ação no AWS](#)
- [Não estou autorizado a executar iam:PassRole](#)
- [Quero permitir que pessoas fora da minha Conta da AWS acessem meus recursos AWS](#)

## Não tenho autorização para executar uma ação no AWS

Se você receber uma mensagem de erro informando que não tem autorização para executar uma ação, suas políticas deverão ser atualizadas para permitir que você realize a ação.

O erro do exemplo a seguir ocorre quando o usuário do IAM mateojackson tenta usar o console para visualizar detalhes sobre um atributo *my-example-widget* fictício, mas não tem as permissões *awes:GetWidget* fictícias.

```
User: arn:aws:iam::123456789012:user/mateojackson is not authorized to perform:
awes:GetWidget on resource: my-example-widget
```

Nesse caso, a política do usuário mateojackson deve ser atualizada para permitir o acesso ao atributo *my-example-widget* usando a ação *awes:GetWidget*.

Se você precisar de ajuda, entre em contato com seu administrador da AWS. Seu administrador é a pessoa que forneceu suas credenciais de login.

## Não estou autorizado a executar iam:PassRole

Se você receber uma mensagem de erro informando que não está autorizado a executar a ação `iam:PassRole`, as suas políticas devem ser atualizadas para permitir que você passe uma função para o AWS.

Alguns Serviços da AWS permitem que você passe uma função existente para o serviço, em vez de criar uma nova função de serviço ou função vinculada ao serviço. Para fazê-lo, você deve ter permissões para passar o perfil para o serviço.

O exemplo de erro a seguir ocorre quando uma usuária do IAM chamada `marymajor` tenta utilizar o console para executar uma ação no AWS. No entanto, a ação exige que o serviço tenha permissões concedidas por um perfil de serviço. Mary não tem permissões para passar a função para o serviço.

```
User: arn:aws:iam::123456789012:user/marymajor is not authorized to perform:
iam:PassRole
```

Nesse caso, as políticas de Mary devem ser atualizadas para permitir que ela realize a ação `iam:PassRole`.

Se você precisar de ajuda, entre em contato com seu administrador da AWS. Seu administrador é a pessoa que forneceu suas credenciais de login.

## Quero permitir que pessoas fora da minha Conta da AWS acessem meus recursos AWS

Você pode criar uma função que os usuários de outras contas ou pessoas fora da organização podem usar para acessar seus recursos. Você pode especificar quem é confiável para assumir a função. Para serviços que oferecem suporte a políticas baseadas em recursos ou listas de controle de acesso (ACLs), você pode usar essas políticas para conceder às pessoas acesso aos seus recursos.

Saiba mais consultando o seguinte:

- Para saber se o AWS suporta esses recursos, consulte [Como Serviços da AWS funcionam com o IAM](#).

- Saiba como conceder acesso a seus recursos em todos os Contas da AWS pertencentes a você, consulte [Fornecendo Acesso a um Usuário do IAM em Outra Conta da AWS Pertencente a Você](#) no Guia de Usuário do IAM.
- Para saber como conceder acesso a seus recursos para terceiros Contas da AWS, consulte [Fornecimento de acesso a Contas da AWS pertencentes a terceiros](#) no Guia do usuário do IAM.
- Para saber como conceder acesso por meio da federação de identidades, consulte [Conceder acesso a usuários autenticados externamente \(federação de identidades\)](#) no Guia do usuário do IAM.
- Para saber a diferença entre usar perfis e políticas baseadas em recursos para acesso entre contas, consulte [Como os perfis do IAM diferem de políticas baseadas em recursos](#) no Guia do usuário do IAM.

## Validação de conformidade para este produto ou serviço da AWS

Para saber se um AWS service (Serviço da AWS) está no escopo de programas de conformidade específicos, consulte [Serviços da AWS no escopo por programa de conformidade](#) em que você está interessado. Para obter informações gerais, consulte [Programas de conformidade da AWS](#).

É possível fazer download de relatórios de auditoria de terceiros usando o AWS Artifact. Para obter mais informações, consulte [Downloading Reports in AWS Artifact](#).

Sua responsabilidade de conformidade ao usar o Serviços da AWS é determinada pela confidencialidade dos seus dados, pelos objetivos de conformidade da sua empresa e pelos regulamentos e leis aplicáveis. A AWS fornece os seguintes recursos para ajudar com a conformidade:

- [Guias de referência rápida de conformidade e segurança](#) - estes guias de implantação discutem considerações sobre arquitetura e fornecem as etapas para a implantação de ambientes de linha de base focados em segurança e conformidade na AWS.
- [Arquitetura para segurança e conformidade com HIPAA no Amazon Web Services](#): esse whitepaper descreve como as empresas podem usar a AWS para criar aplicações adequadas aos padrões HIPAA.

 Note

Nem todos os Serviços da AWS estão qualificados pela HIPAA. Para mais informações, consulte a [Referência dos serviços qualificados pela HIPAA](#).

- [Recursos de conformidade da AWS](#): essa coleção de manuais e guias pode ser aplicada a seu setor e local.
- [Guias de conformidade do cliente da AWS](#): entenda o modelo de responsabilidade compartilhada sob a ótica da conformidade. Os guias resumem as práticas recomendadas para proteção de Serviços da AWS e mapeiam as diretrizes para controles de segurança em várias estruturas (incluindo o Instituto Nacional de Padrões e Tecnologia (NIST), o Conselho de Padrões de Segurança do Setor de Cartões de Pagamento (PCI) e a Organização Internacional de Padronização (ISO)).
- [Avaliar recursos com regras](#) no Guia do desenvolvedor do AWS Config: o serviço AWS Config avalia como as configurações de recursos estão em conformidade com práticas internas, diretrizes do setor e regulamentos.
- [AWS Security Hub](#): este AWS service (Serviço da AWS) fornece uma visão abrangente do seu estado de segurança na AWS. O Security Hub usa controles de segurança para avaliar os recursos da AWS e verificar a conformidade com os padrões e as práticas recomendadas do setor de segurança. Para obter uma lista dos serviços e controles aceitos, consulte a [Referência de controles do Security Hub](#).
- [AWS Audit Manager](#): esse AWS service (Serviço da AWS) ajuda a auditar continuamente seu uso da AWS para simplificar a forma como você gerencia os riscos e a conformidade com regulamentos e padrões do setor.

Esse produto ou serviço da AWS segue o [modelo de responsabilidade compartilhada](#) por meio dos serviços específicos da Amazon Web Services (AWS) compatíveis. Para obter informações de segurança sobre o serviço da AWS, consulte a [página de documentação de segurança do serviço da AWS](#) e [Serviços da AWS que estão no escopo dos esforços de conformidade da AWS por programa de conformidade](#).

## Resiliência para este produto ou serviço da AWS

A infraestrutura global da AWS é criada com base em Regiões da AWS e zonas de disponibilidade.

As Regiões da AWS fornecem várias zonas de disponibilidade separadas e isoladas fisicamente, que são conectadas com baixa latência, throughputs elevadas e redes altamente redundantes.

Com as zonas de disponibilidade, é possível projetar e operar aplicações e bancos de dados que automaticamente executam o failover entre as zonas sem interrupção. As zonas de disponibilidade são mais altamente disponíveis, tolerantes a falhas e escaláveis que uma ou várias infraestruturas de datacenter tradicionais.

Para obter mais informações sobre AWS regiões e zonas de disponibilidade, consulte [AWS Infraestrutura Global](#).

Esse produto ou serviço da AWS segue o [modelo de responsabilidade compartilhada](#) por meio dos serviços específicos da Amazon Web Services (AWS) compatíveis. Para obter informações de segurança sobre o serviço da AWS, consulte a [página de documentação de segurança do serviço da AWS](#) e [Serviços da AWS que estão no escopo dos esforços de conformidade da AWS por programa de conformidade](#).

## Segurança da infraestrutura para esse produto ou serviço da AWS

Esse produto ou serviço da AWS usa serviços gerenciados e, portanto, é protegido pela segurança de rede global da AWS. Para obter informações sobre serviços de segurança da AWS e como a AWS protege a infraestrutura, consulte [Segurança na nuvem da AWS](#). Para projetar o ambiente da AWS utilizando as práticas recomendadas de segurança de infraestrutura, consulte [Proteção de infraestrutura](#) em Pilar segurança: AWS Well-Architected Framework.

Você usa chamadas de API publicadas pela AWS para acessar esse produto ou serviço da AWS pela rede. Os clientes precisam oferecer suporte para:

- Transport Layer Security (TLS). Exigimos TLS 1.2 e recomendamos TLS 1.3.
- Conjuntos de criptografia com sigilo de encaminhamento perfeito (perfect forward secrecy, ou PFS) como DHE (Ephemeral Diffie-Hellman, ou Efêmero Diffie-Hellman) ou ECDHE (Ephemeral Elliptic Curve Diffie-Hellman, ou Curva elíptica efêmera Diffie-Hellman). A maioria dos sistemas modernos, como Java 7 e versões posteriores, comporta esses modos.

Além disso, as solicitações devem ser assinadas utilizando um ID da chave de acesso e uma chave de acesso secreta associada a uma entidade principal do IAM. Ou é possível usar o [AWS Security Token Service](#) (AWS STS) para gerar credenciais de segurança temporárias para assinar solicitações.

Esse produto ou serviço da AWS segue o [modelo de responsabilidade compartilhada](#) por meio dos serviços específicos da Amazon Web Services (AWS) compatíveis. Para obter informações de segurança sobre o serviço da AWS, consulte a [página de documentação de segurança do serviço da AWS](#) e [Serviços da AWS que estão no escopo dos esforços de conformidade da AWS por programa de conformidade](#).

## Impor uma versão mínima do TLS

Para aumentar a segurança ao se comunicar com serviços da AWS, você deve usar o TLS 1.2 ou posterior. Quando você usa a AWS CLI, o Python é usado para definir a versão do TLS.

Para garantir que a AWS CLI versão 1 não use uma versão do TLS anterior ao TLS 1.2, talvez seja necessário recompilar o OpenSSL para impor esse mínimo e recompilar o Python para usar o OpenSSL recém-criado.

### Tópicos

- [Determinar os protocolos atualmente compatíveis](#)
- [Compilar OpenSSL e Python](#)

## Determinar os protocolos atualmente compatíveis

Primeiro, crie um certificado autoassinado a ser usado para o servidor de teste e o Python SDK usando OpenSSL.

```
$ openssl req -subj '/CN=localhost' -x509 -newkey rsa:4096 -nodes -keyout key.pem -out cert.pem -days 365
```

Depois, crie um servidor de teste usando OpenSSL.

```
$ openssl s_server -key key.pem -cert cert.pem -www
```

Em uma nova janela do terminal, crie um ambiente virtual e instale o SDK for Python.

```
$ python3 -m venv test-env
source test-env/bin/activate
pip install botocore
```

Crie um script Python chamado `check.py` que usa a biblioteca HTTP subjacente do SDK.

```
$ import urllib3
URL = 'https://localhost:4433/'

http = urllib3.PoolManager(
    ca_certs='cert.pem',
    cert_reqs='CERT_REQUIRED',
)
r = http.request('GET', URL)
print(r.data.decode('utf-8'))
```

Execute o novo script.

```
$ python check.py
```

Ele exibe detalhes sobre a conexão feita. Procure “Protocol : ” na saída. Se a saída for “TLSv1.2” ou posterior, o SDK definirá como padrão TLS v1.2 ou posterior. Se for uma versão anterior, é necessário recompilar o OpenSSL e recompilar o Python.

No entanto, mesmo que a instalação do Python defina como padrão TLS v1.2 ou posterior, ainda será possível que o Python renegocie para uma versão anterior ao TLS v1.2 se o servidor não for compatível com TLS v1.2 ou posterior. Para verificar se o Python não renegocia automaticamente para versões anteriores, reinicie o servidor de teste com o seguinte.

```
$ openssl s_server -key key.pem -cert cert.pem -no_tls1_3 -no_tls1_2 -www
```

Se estiver usando uma versão anterior do OpenSSL, talvez você não tenha o sinalizador `-no_tls_3` disponível. Se esse for o caso, remova o sinalizador porque a versão do OpenSSL que você está usando não é compatível com TLS v1.3. Execute novamente o script Python.

```
$ python check.py
```

Se a instalação do Python não renegocia para versões anteriores ao TLS 1.2 corretamente, você deve receber um erro SSL.

```
$ urllib3.exceptions.MaxRetryError: HTTPSConnectionPool(host='localhost',
port=4433): Max retries exceeded with url: / (Caused by SSLError(SSLError(1, '[SSL:
UNSUPPORTED_PROTOCOL] unsupported protocol (_ssl.c:1108)')))
```

Se você conseguir fazer uma conexão, será necessário recompilar o OpenSSL e o Python para desabilitar a negociação de protocolos anteriores ao TLS v1.2.

## Compilar OpenSSL e Python

Para garantir que o SDK ou a AWS CLI não negocie nada anterior ao TLS 1.2, é necessário recompilar o OpenSSL e o Python. Para fazer isso, copie o seguinte conteúdo para criar um script e executá-lo.

```
#!/usr/bin/env bash
set -e

OPENSSL_VERSION="1.1.1d"
OPENSSL_PREFIX="/opt/openssl-with-min-tls1_2"
PYTHON_VERSION="3.8.1"
PYTHON_PREFIX="/opt/python-with-min-tls1_2"

curl -O "https://www.openssl.org/source/openssl-$OPENSSL_VERSION.tar.gz"
tar -xzf "openssl-$OPENSSL_VERSION.tar.gz"
cd openssl-$OPENSSL_VERSION
./config --prefix=$OPENSSL_PREFIX no-ssl3 no-tls1 no-tls1_1 no-shared
make > /dev/null
sudo make install_sw > /dev/null

cd /tmp
curl -O "https://www.python.org/ftp/python/$PYTHON_VERSION/Python-$PYTHON_VERSION.tgz"
tar -xzf "Python-$PYTHON_VERSION.tgz"
cd Python-$PYTHON_VERSION
./configure --prefix=$PYTHON_PREFIX --with-openssl=$OPENSSL_PREFIX --disable-shared > /dev/null
make > /dev/null
sudo make install > /dev/null
```

Isso compila uma versão do Python que tem um OpenSSL vinculado estaticamente que não negocia automaticamente nada anterior ao TLS 1.2. Isso também instala o OpenSSL no diretório `/opt/openssl-with-min-tls1_2` e instala o Python no diretório `/opt/python-with-min-tls1_2`. Depois de executar esse script, verifique a instalação da nova versão do Python.

```
$ /opt/python-with-min-tls1_2/bin/python3 --version
```



Isso deve imprimir o seguinte.

```
$ Python 3.8.1
```

Para verificar se essa nova versão do Python não negocia uma versão anterior ao TLS 1.2, execute novamente as etapas em [Determinar os protocolos atualmente compatíveis](#) usando a versão do Python recém-instalada (ou seja, `/opt/python-with-min-tls1_2/bin/python3`).

# Solucionar erros da AWS CLI

Esta seção aborda erros comuns e os passos para a solução de problemas a serem seguidas para resolver o problema. Sugerimos seguir a [Solução geral de problemas](#) primeiro.

## Sumário

- [Solução geral de problemas para tentar primeiro](#)
  - [Verificar a formatação do comando AWS CLI](#)
  - [Confirme se você está executando uma versão recente da AWS CLI](#)
  - [Como usar a opção --debug](#)
  - [Habilitar e analisar logs de histórico de comandos da AWS CLI](#)
  - [Confirmar se a AWS CLI está configurada](#)
- [Erros de comando não encontrado](#)
- [O comando “aws --version” retorna uma versão diferente da que você instalou](#)
- [O comando “aws --version” retorna uma versão após a desinstalação da AWS CLI](#)
- [A AWS CLI processou um comando com um nome de parâmetro incompleto](#)
- [Erros de acesso negado](#)
- [Credenciais inválidas e erros de chave](#)
- [Assinatura não corresponde aos erros](#)
- [Erros de nenhum console do Windows encontrado](#)
- [Erros de certificado SSL](#)
- [Erros JSON inválidos](#)
- [Recursos adicionais](#)

## Solução geral de problemas para tentar primeiro

Se você receber um erro ou encontrar um problema com a AWS CLI, sugerimos as dicas gerais a seguir para ajudar na solução de problemas.

[Voltar ao início](#)

## Verificar a formatação do comando AWS CLI

Se você receber um erro indicando que um comando não existe ou que ele não reconhece um parâmetro que a documentação indica estar disponível, é provável que o comando esteja formatado incorretamente. Sugerimos verificar o seguinte:

- Verifique se há erros de ortografia e de formatação no comando.
- Confirme se todos os [escapes e citações apropriados para o seu terminal](#) estão corretos em seu comando.
- Gere um [esqueleto de AWS CLI](#) para confirmar a estrutura do comando.
- Para JSON, consulte a [solução adicional de problemas referente a valores JSON](#). Se você estiver tendo problemas com a formatação JSON de processamento de terminal, sugerimos ignorar as regras de cotação do terminal usando [Blobs para passar dados JSON diretamente para a AWS CLI](#).

Para obter mais informações sobre como um comando específico deve ser estruturado, consulte o [AWS CLIGuia de referência da](#).

[Voltar ao início](#)

## Confirme se você está executando uma versão recente da AWS CLI

Se você receber um erro indicando que um comando não existe ou que ele não reconhece um parâmetro que o [AWS CLIGuia de referência da](#) diz que está disponível, primeiro confirme se o comando está formatado corretamente. Se a formatação estiver correta, recomendamos atualizar para a versão mais recente da AWS CLI. Versões atualizadas da AWS CLI são lançadas quase todos os dias úteis. Novos serviços da AWS, recursos e parâmetros são apresentados nessas novas versões da AWS CLI. A única maneira de obter acesso a esses novos serviços, recursos ou parâmetros é atualizar para uma versão que foi lançada depois que esse elemento foi apresentado pela primeira vez.

A maneira como você atualiza sua versão da AWS CLI depende de como ela foi instalada originalmente, conforme descrito em [Instalar a AWS CLI](#).

Se você tiver usado um dos instaladores empacotados, talvez precise remover a instalação existente antes de baixar e instalar a versão mais recente para seu sistema operacional.

[Voltar ao início](#)

## Como usar a opção **--debug**

Quando a AWS CLI informar um erro que você não compreende imediatamente ou produzir resultados inesperados, você poderá obter mais detalhes sobre o erro executando o comando novamente com a opção **--debug**. Com esta opção, a AWS CLI gera detalhes sobre cada passo necessário para processar seu comando. Os detalhes na saída podem ajudar a determinar quando o erro ocorreu e fornecem dicas sobre o que o acionou o erro.

É possível enviar a saída para um arquivo de texto para análise posterior ou enviá-lo para AWS Support quando solicitado.

Quando você inclui a opção **--debug**, os detalhes incluem:

- Procurar credenciais
- Analisar os parâmetros fornecidos
- Criar a solicitação enviada aos servidores da AWS
- O conteúdo da solicitação enviada para a AWS
- O conteúdo da resposta não formatada
- A saída formatada

Veja a seguir um exemplo de um comando executado com e sem a opção **--debug**.

```
$ aws iam list-groups --profile MyTestProfile
{
  "Groups": [
    {
      "Path": "/",
      "GroupName": "MyTestGroup",
      "GroupId": "AGPA0123456789EXAMPLE",
      "Arn": "arn:aws:iam::123456789012:group/MyTestGroup",
      "CreateDate": "2019-08-12T19:34:04Z"
    }
  ]
}
```

```
$ aws iam list-groups --profile MyTestProfile --debug
2019-08-12 12:36:18,305 - MainThread - awscli.clidriver - DEBUG - CLI version: aws-
cli/1.16.215 Python/3.7.3 Linux/4.14.133-113.105.amzn2.x86_64 botocore/1.12.205
```

```
2019-08-12 12:36:18,305 - MainThread - awscli.clidriver - DEBUG - Arguments entered to
  CLI: ['iam', 'list-groups', '--debug']
2019-08-12 12:36:18,305 - MainThread - botocore.hooks - DEBUG - Event session-
  initialized: calling handler <function add_scalar_parsers at 0x7fdf173161e0>
2019-08-12 12:36:18,305 - MainThread - botocore.hooks - DEBUG - Event session-
  initialized: calling handler <function register_uri_param_handler at 0x7fdf17dec400>
2019-08-12 12:36:18,305 - MainThread - botocore.hooks - DEBUG - Event session-
  initialized: calling handler <function inject_assume_role_provider_cache at
  0x7fdf17da9378>
2019-08-12 12:36:18,307 - MainThread - botocore.credentials - DEBUG - Skipping
  environment variable credential check because profile name was explicitly set.
2019-08-12 12:36:18,307 - MainThread - botocore.hooks - DEBUG - Event session-
  initialized: calling handler <function attach_history_handler at 0x7fdf173ed9d8>
2019-08-12 12:36:18,308 - MainThread - botocore.loaders - DEBUG - Loading JSON
  file: /home/ec2-user/venv/lib/python3.7/site-packages/botocore/data/iam/2010-05-08/
  service-2.json
2019-08-12 12:36:18,317 - MainThread - botocore.hooks - DEBUG - Event building-command-
  table.iam: calling handler <function add_waiters at 0x7fdf1731a840>
2019-08-12 12:36:18,320 - MainThread - botocore.loaders - DEBUG - Loading JSON
  file: /home/ec2-user/venv/lib/python3.7/site-packages/botocore/data/iam/2010-05-08/
  waiters-2.json
2019-08-12 12:36:18,321 - MainThread - awscli.clidriver - DEBUG - OrderedDict([('path-
  prefix', <awscli.arguments.CLIArgument object at 0x7fdf171ac780>), ('marker',
  <awscli.arguments.CLIArgument object at 0x7fdf171b09e8>), ('max-items',
  <awscli.arguments.CLIArgument object at 0x7fdf171b09b0>)])
2019-08-12 12:36:18,322 - MainThread - botocore.hooks - DEBUG - Event building-
  argument-table.iam.list-groups: calling handler <function add_streaming_output_arg at
  0x7fdf17316510>
2019-08-12 12:36:18,322 - MainThread - botocore.hooks - DEBUG - Event building-
  argument-table.iam.list-groups: calling handler <function add_cli_input_json at
  0x7fdf17da9d90>
2019-08-12 12:36:18,322 - MainThread - botocore.hooks - DEBUG - Event building-
  argument-table.iam.list-groups: calling handler <function unify_paging_params at
  0x7fdf17328048>
2019-08-12 12:36:18,326 - MainThread - botocore.loaders - DEBUG - Loading JSON
  file: /home/ec2-user/venv/lib/python3.7/site-packages/botocore/data/iam/2010-05-08/
  paginators-1.json
2019-08-12 12:36:18,326 - MainThread - awscli.customizations.paginate - DEBUG -
  Modifying paging parameters for operation: ListGroups
2019-08-12 12:36:18,326 - MainThread - botocore.hooks - DEBUG - Event building-
  argument-table.iam.list-groups: calling handler <function add_generate_skeleton at
  0x7fdf1737eae8>
2019-08-12 12:36:18,326 - MainThread - botocore.hooks - DEBUG - Event
  before-building-argument-table-parser.iam.list-groups: calling handler
```

```

<bound method OverrideRequiredArgsArgument.override_required_args of
<awscli.customizations.cliinputjson.CliInputJSONArgument object at 0x7fdf171b0a58>>
2019-08-12 12:36:18,327 - MainThread - botocore.hooks - DEBUG - Event
before-building-argument-table-parser.iam.list-groups: calling handler
<bound method GenerateCliSkeletonArgument.override_required_args of
<awscli.customizations.generatecliskeleton.GenerateCliSkeletonArgument object at
0x7fdf171c5978>>
2019-08-12 12:36:18,327 - MainThread - botocore.hooks - DEBUG - Event operation-
args-parsed.iam.list-groups: calling handler functools.partial(<function
check_should_enable_pagination at 0x7fdf17328158>, ['marker', 'max-items'], {'max-
items': <awscli.arguments.CLIArgument object at 0x7fdf171b09b0>}, OrderedDict([('path-
prefix', <awscli.arguments.CLIArgument object at 0x7fdf171ac780>), ('marker',
<awscli.arguments.CLIArgument object at 0x7fdf171b09e8>), ('max-items',
<awscli.customizations.paginate.PageArgument object at 0x7fdf171c58d0>), ('cli-
input-json', <awscli.customizations.cliinputjson.CliInputJSONArgument object at
0x7fdf171b0a58>), ('starting-token', <awscli.customizations.paginate.PageArgument
object at 0x7fdf171b0a20>), ('page-size', <awscli.customizations.paginate.PageArgument
object at 0x7fdf171c5828>), ('generate-cli-skeleton',
<awscli.customizations.generatecliskeleton.GenerateCliSkeletonArgument object at
0x7fdf171c5978>)]))
2019-08-12 12:36:18,328 - MainThread - botocore.hooks - DEBUG - Event load-cli-
arg.iam.list-groups.path-prefix: calling handler <awscli.paramfile.URIArgumentHandler
object at 0x7fdf1725c978>
2019-08-12 12:36:18,328 - MainThread - botocore.hooks - DEBUG - Event load-cli-
arg.iam.list-groups.marker: calling handler <awscli.paramfile.URIArgumentHandler object
at 0x7fdf1725c978>
2019-08-12 12:36:18,328 - MainThread - botocore.hooks - DEBUG - Event load-cli-
arg.iam.list-groups.max-items: calling handler <awscli.paramfile.URIArgumentHandler
object at 0x7fdf1725c978>
2019-08-12 12:36:18,328 - MainThread - botocore.hooks - DEBUG -
Event load-cli-arg.iam.list-groups.cli-input-json: calling handler
<awscli.paramfile.URIArgumentHandler object at 0x7fdf1725c978>
2019-08-12 12:36:18,328 - MainThread - botocore.hooks - DEBUG -
Event load-cli-arg.iam.list-groups.starting-token: calling handler
<awscli.paramfile.URIArgumentHandler object at 0x7fdf1725c978>
2019-08-12 12:36:18,328 - MainThread - botocore.hooks - DEBUG - Event load-cli-
arg.iam.list-groups.page-size: calling handler <awscli.paramfile.URIArgumentHandler
object at 0x7fdf1725c978>
2019-08-12 12:36:18,328 - MainThread - botocore.hooks - DEBUG - Event
load-cli-arg.iam.list-groups.generate-cli-skeleton: calling handler
<awscli.paramfile.URIArgumentHandler object at 0x7fdf1725c978>
2019-08-12 12:36:18,329 - MainThread - botocore.hooks - DEBUG
- Event calling-command.iam.list-groups: calling handler

```

```

<bound method CliInputJSONArgument.add_to_call_parameters of
<awscli.customizations.cliinputjson.CliInputJSONArgument object at 0x7fdf171b0a58>>
2019-08-12 12:36:18,329 - MainThread - botocore.hooks - DEBUG -
Event calling-command.iam.list-groups: calling handler <bound
method GenerateCliSkeletonArgument.generate_json_skeleton of
<awscli.customizations.generatecliskeleton.GenerateCliSkeletonArgument object at
0x7fdf171c5978>>
2019-08-12 12:36:18,329 - MainThread - botocore.credentials - DEBUG - Looking for
credentials via: assume-role
2019-08-12 12:36:18,329 - MainThread - botocore.credentials - DEBUG - Looking for
credentials via: assume-role-with-web-identity
2019-08-12 12:36:18,329 - MainThread - botocore.credentials - DEBUG - Looking for
credentials via: shared-credentials-file
2019-08-12 12:36:18,329 - MainThread - botocore.credentials - INFO - Found credentials
in shared credentials file: ~/.aws/credentials
2019-08-12 12:36:18,330 - MainThread - botocore.loaders - DEBUG - Loading JSON file: /
home/ec2-user/venv/lib/python3.7/site-packages/botocore/data/endpoints.json
2019-08-12 12:36:18,334 - MainThread - botocore.hooks - DEBUG - Event choose-service-
name: calling handler <function handle_service_name_alias at 0x7fdf1898eb70>
2019-08-12 12:36:18,337 - MainThread - botocore.hooks - DEBUG - Event creating-client-
class.iam: calling handler <function add_generate_presigned_url at 0x7fdf18a028c8>
2019-08-12 12:36:18,337 - MainThread - botocore.regions - DEBUG - Using partition
endpoint for iam, us-west-2: aws-global
2019-08-12 12:36:18,337 - MainThread - botocore.args - DEBUG - The s3 config key is not
a dictionary type, ignoring its value of: None
2019-08-12 12:36:18,340 - MainThread - botocore.endpoint - DEBUG - Setting iam timeout
as (60, 60)
2019-08-12 12:36:18,341 - MainThread - botocore.loaders - DEBUG - Loading JSON file: /
home/ec2-user/venv/lib/python3.7/site-packages/botocore/data/_retry.json
2019-08-12 12:36:18,341 - MainThread - botocore.client - DEBUG - Registering retry
handlers for service: iam
2019-08-12 12:36:18,342 - MainThread - botocore.hooks - DEBUG - Event before-
parameter-build.iam.ListGroups: calling handler <function generate_idempotent_uuid at
0x7fdf189b10d0>
2019-08-12 12:36:18,342 - MainThread - botocore.hooks - DEBUG - Event before-
call.iam.ListGroups: calling handler <function inject_api_version_header_if_needed at
0x7fdf189b2a60>
2019-08-12 12:36:18,343 - MainThread - botocore.endpoint - DEBUG - Making
request for OperationModel(name=ListGroups) with params: {'url_path': '/',
'query_string': '', 'method': 'POST', 'headers': {'Content-Type': 'application/x-
www-form-urlencoded; charset=utf-8', 'User-Agent': 'aws-cli/1.16.215 Python/3.7.3
Linux/4.14.133-113.105.amzn2.x86_64 botocore/1.12.205'}, 'body': {'Action':
'ListGroups', 'Version': '2010-05-08'}, 'url': 'https://iam.amazonaws.com/',

```

```

'context': {'client_region': 'aws-global', 'client_config': <botocore.config.Config
object at 0x7fdf16e9a4a8>, 'has_streaming_input': False, 'auth_type': None}}
2019-08-12 12:36:18,343 - MainThread - botocore.hooks - DEBUG - Event request-
created.iam.ListGroups: calling handler <bound method RequestSigner.handler of
<botocore.signers.RequestSigner object at 0x7fdf16e9a470>>
2019-08-12 12:36:18,343 - MainThread - botocore.hooks - DEBUG - Event choose-
signer.iam.ListGroups: calling handler <function set_operation_specific_signer at
0x7fdf18996f28>
2019-08-12 12:36:18,343 - MainThread - botocore.auth - DEBUG - Calculating signature
using v4 auth.
2019-08-12 12:36:18,343 - MainThread - botocore.auth - DEBUG - CanonicalRequest:
POST
/

content-type:application/x-www-form-urlencoded; charset=utf-8
host:iam.amazonaws.com
x-amz-date:20190812T193618Z

content-type;host;x-amz-date
5f776d91EXAMPLE9b8cb5eb5d6d4a787a33ae41c8cd6eEXAMPLEEca69080e1e1f
2019-08-12 12:36:18,344 - MainThread - botocore.auth - DEBUG - StringToSign:
AWS4-HMAC-SHA256
20190812T193618Z
20190812/us-east-1/iam/aws4_request
ab7e367eEXAMPLE2769f178ea509978cf8bfa054874b3EXAMPLE8d043fab6cc9
2019-08-12 12:36:18,344 - MainThread - botocore.auth - DEBUG - Signature:
d85a0EXAMPLEb40164f2f539cdc76d4f294fe822EXAMPLE18ad1ddf58a1a3ce7
2019-08-12 12:36:18,344 - MainThread - botocore.endpoint - DEBUG - Sending
http request: <AWSPreparedRequest stream_output=False, method=POST,
url=https://iam.amazonaws.com/, headers={'Content-Type': b'application/
x-www-form-urlencoded; charset=utf-8', 'User-Agent': b'aws-cli/1.16.215
Python/3.7.3 Linux/4.14.133-113.105.amzn2.x86_64 botocore/1.12.205',
'X-Amz-Date': b'20190812T193618Z', 'Authorization': b'AWS4-HMAC-SHA256
Credential=AKIA01234567890EXAMPLE-east-1/iam/aws4_request, SignedHeaders=content-
type;host;x-amz-date, Signature=d85a07692aceb401EXAMPLEa1b18ad1ddf58a1a3ce7EXAMPLE',
'Content-Length': '36'}>
2019-08-12 12:36:18,344 - MainThread - urllib3.util.retry - DEBUG - Converted retries
value: False -> Retry(total=False, connect=None, read=None, redirect=0, status=None)
2019-08-12 12:36:18,344 - MainThread - urllib3.connectionpool - DEBUG - Starting new
HTTPS connection (1): iam.amazonaws.com:443
2019-08-12 12:36:18,664 - MainThread - urllib3.connectionpool - DEBUG - https://
iam.amazonaws.com:443 "POST / HTTP/1.1" 200 570

```



```

2019-08-12 12:36:18,664 - MainThread - botocore.parsers - DEBUG - Response headers:
{'x-amzn-RequestId': '74c11606-bd38-11e9-9c82-559da0adb349', 'Content-Type': 'text/
xml', 'Content-Length': '570', 'Date': 'Mon, 12 Aug 2019 19:36:18 GMT'}
2019-08-12 12:36:18,664 - MainThread - botocore.parsers - DEBUG - Response body:
b'<ListGroupResponse xmlns="https://iam.amazonaws.com/doc/2010-05-08/">\n
  <ListGroupResult>\n    <IsTruncated>false</IsTruncated>\n    <Groups>\n
    <member>\n      <Path>/</Path>\n      <GroupName>MyTestGroup</GroupName>
\n      <Arn>arn:aws:iam::123456789012:group/MyTestGroup</Arn>\n
    <GroupId>AGPA1234567890EXAMPLE</GroupId>\n      <CreateDate>2019-08-12T19:34:04Z</
CreateDate>\n    </member>\n    </Groups>\n  </ListGroupResult>\n
  <ResponseMetadata>\n    <RequestId>74c11606-bd38-11e9-9c82-559da0adb349</RequestId>\n
  </ResponseMetadata>\n</ListGroupResponse>\n'
2019-08-12 12:36:18,665 - MainThread - botocore.hooks - DEBUG - Event needs-
retry.iam.ListGroups: calling handler <botocore.retryhandler.RetryHandler object at
0x7fdf16e9a780>
2019-08-12 12:36:18,665 - MainThread - botocore.retryhandler - DEBUG - No retry needed.
2019-08-12 12:36:18,665 - MainThread - botocore.hooks - DEBUG - Event after-
call.iam.ListGroups: calling handler <function json_decode_policies at 0x7fdf189b1d90>
{
  "Groups": [
    {
      "Path": "/",
      "GroupName": "MyTestGroup",
      "GroupId": "AGPA123456789012EXAMPLE",
      "Arn": "arn:aws:iam::123456789012:group/MyTestGroup",
      "CreateDate": "2019-08-12T19:34:04Z"
    }
  ]
}

```

[Voltar ao início](#)

## Habilitar e analisar logs de histórico de comandos da AWS CLI

É possível habilitar os logs do histórico de comandos da AWS CLI usando a configuração de arquivo [cli\\_history](#). Após habilitar essa configuração, a AWS CLI registra o histórico de comandos da aws.

Você pode listar seu histórico usando o comando `aws history list` e usar os `command_ids` resultantes no comando `aws history show` para obter detalhes. Para obter mais informações, consulte [aws history](#) no guia de referência da AWS CLI.

Quando você inclui a opção `--debug`, os detalhes incluem:

- Chamadas de API feitas para o botocore
- Códigos de status
- Respostas HTTP
- Cabeçalhos
- Códigos de retorno

Você pode usar essas informações para confirmar que os dados do parâmetro e as chamadas de API estão se comportando da maneira esperada e deduzir em que etapa do processo seu comando está falhando.

[Voltar ao início](#)

## Confirmar se a AWS CLI está configurada

Vários erros podem ocorrer se os arquivos `config` e `credentials` ou o perfil ou usuário do IAM não estiverem configurados corretamente. Para obter mais informações sobre como resolver erros nos arquivos `config` e `credentials` ou no seu usuário ou perfis do IAM, consulte [the section called “Erros de acesso negado”](#) e [the section called “Credenciais inválidas e erros de chave”](#).

[Voltar ao início](#)

## Erros de comando não encontrado

Esse erro significa que o sistema operacional não pode encontrar o comando da AWS CLI. A instalação pode estar incompleta ou exigir atualização.

Causa possível: você está tentando usar um recurso mais recente da AWS CLI do que a versão instalada ou tem formatação incorreta

Exemplo de texto de erro:

```
$ aws s3 copy
usage: aws [options] <command> <subcommand> [<subcommand> ...] [parameters]
To see help text, you can run:

aws help
aws <command> help
```

```
aws <command> <subcommand> help
aws: error: argument subcommand: Invalid choice, valid choices are:
ls                               | website
cp                               | mv
....
```

Vários erros podem ocorrer se o comando estiver formatado incorretamente ou se você estiver usando uma versão anterior ao lançamento do recurso. Para obter mais informações sobre a solução de erros em torno desses dois problemas, consulte [the section called “Verificar a formatação do comando AWS CLI”](#) e [the section called “Confirme se você está executando uma versão recente da AWS CLI”](#).

### [Voltar ao início](#)

Causa possível: o terminal precisa ser reiniciado após a instalação

Exemplo de texto de erro:

```
$ aws --version
command not found: aws
```

Se o comando `aws` não puder ser encontrado após a primeira instalação ou atualização da AWS CLI, talvez seja necessário reiniciar o terminal para que ele reconheça todas as atualizações da PATH.

### [Voltar ao início](#)

Possível causa: AWS CLI não foi totalmente instalada

Exemplo de texto de erro:

```
$ aws --version
command not found: aws
```

Se o comando `aws` não puder ser encontrado após a primeira instalação ou atualização da AWS CLI, talvez a instalação não tenha sido totalmente concluída. Tente reinstalar seguindo os passos da sua plataforma em [Instalar a AWS CLI](#).

### [Voltar ao início](#)

## Possível causa: a AWS CLI não tem permissões (Linux)

Se não for possível encontrar o comando `aws` após a primeira instalação ou atualização da AWS CLI no Linux, talvez ela não tenha permissões execute na pasta em que está instalada. Execute o seguinte comando com o `PATH` para a instalação da AWS CLI para fornecer permissões [chmod](#) para a AWS CLI:

```
$ sudo chmod -R 755 /usr/local/aws-cli/
```

### [Voltar ao início](#)

Causa possível: o **PATH** do sistema operacional não foi atualizado durante a instalação.

Exemplo de texto de erro:

```
$ aws --version
command not found: aws
```

Talvez seja necessário adicionar o executável `aws` à variável de ambiente `PATH` do sistema operacional. Para adicionar a AWS CLI ao seu `PATH`, use as instruções a seguir para seu sistema operacional.

### Linux and macOS

1. Encontre o script de perfil do shell no diretório de usuário. Se não tiver certeza de qual shell você tem, execute `echo $SHELL`.

```
$ ls -a ~
.  ..  .bash_logout  .bash_profile  .bashrc  Desktop  Documents  Downloads
```

- Bash – `.bash_profile`, `.profile`, ou `.bash_login`
- Zsh – `.zshrc`
- Tcsh – `.tcshrc`, `.cshrc`, ou `.login`

2. Adicione um comando de exportação ao script de perfil. O comando a seguir adiciona seu compartimento local à variável `PATH` atual.

```
export PATH=/usr/local/bin:$PATH
```

3. Recarregue o perfil atualizado em sua sessão atual.

```
$ source ~/.bash_profile
```

## Windows

1. Em um prompt de comando do Windows, use o comando `where` com o parâmetro `/R` *path* para encontrar o local do arquivo `aws`. Os resultados retornam todas as pastas que contêm `aws`.

```
C:\> where /R c:\ aws
c:\Program Files\Amazon\AWSCLIV2\aws.exe
...
```

Por padrão, a AWS CLI versão 2 está localizada em:

```
c:\Program Files\Amazon\AWSCLIV2\aws.exe
```

2. Pressione a tecla Windows e digite **environment variables**.
3. Na lista de sugestões, escolha Edit environment variables for your account (Editar variáveis de ambiente para sua conta).
4. Selecione PATH e, em seguida, Edit (Editar).
5. Adicione o caminho encontrado no campo Variable value (Valor da variável). Por exemplo, **C:\Program Files\Amazon\AWSCLIV2\aws.exe**.
6. Escolha OK duas vezes para aplicar as novas configurações.
7. Feche todos os prompts de comando em execução e abra novamente a janela do prompt de comando.

[Voltar ao início](#)

## O comando “**aws --version**” retorna uma versão diferente da que você instalou

Seu terminal pode estar retornando um PATH diferente do AWS CLI que você espera.

Causa possível: o terminal precisa ser reiniciado após a instalação

Se o comando `aws` mostrar a versão errada, talvez seja necessário reiniciar o terminal para que ele reconheça todas as atualizações de PATH. Todos os terminais abertos precisam estar fechados, não apenas o terminal ativo.

[Voltar ao início](#)

Causa possível: o sistema precisa ser reiniciado após a instalação

Se o comando `aws` mostrar a versão errada e reiniciar o terminal não funcionar, talvez seja necessário reiniciar o sistema para que ele reconheça as atualizações de PATH.

[Voltar ao início](#)

Causa possível: você tem várias versões da AWS CLI

Se você atualizou a AWS CLI e usou um método de instalação diferente da instalação pré-existente, isso poderá fazer com que várias versões sejam instaladas. Por exemplo, se no Linux ou no macOS você usou `pip` para a instalação atual, mas tentou atualizar usando o arquivo de instalação `.pkg`, isso poderá causar alguns conflitos, especialmente com o PATH apontando para a versão antiga.

Para resolver isso, [desinstale todas as versões da AWS CLI](#) e execute uma instalação limpa.

Depois de desinstalar todas as versões, siga as instruções apropriadas do seu sistema operacional para instalar a versão desejada da [AWS CLI versão 1](#) ou da [AWS CLI versão 2](#).

 Note

Se isso estiver acontecendo depois que você instalou a AWS CLI versão 2 com uma instalação pré-existente da AWS CLI versão 1, siga as instruções de migração em [Instruções de instalação ao migrar da AWS CLI versão 1](#).

[Voltar ao início](#)

## O comando “`aws --version`” retorna uma versão após a desinstalação da AWS CLI

Isso geralmente ocorre quando ainda há uma AWS CLI instalada em algum lugar do sistema.

Causa possível: o terminal precisa ser reiniciado após a instalação

Se o comando `aws --version` mostrar a versão errada, talvez seja necessário reiniciar o terminal para que ele reconheça todas as atualizações.

### [Voltar ao início](#)

Causa possível: você tem várias versões da AWS CLI em seu sistema, ou não usou o mesmo método de desinstalação usado para instalar a AWS CLI original.

A AWS CLI talvez não seja desinstalada corretamente se você desinstalou a AWS CLI usando um método diferente do usado para instalá-la ou se você instalou várias versões. Por exemplo, se você usou `pip` para a instalação atual, deverá usar `pip` para desinstalá-la. Para resolver isso, desinstale a AWS CLI usando o mesmo método usado para instalá-la.

1. Siga as instruções apropriadas do seu sistema operacional e do método de instalação original para desinstalar a [AWS CLI versão 1](#) e a [AWS CLI versão 2](#).
2. Feche todos os terminais que você abriu.
3. Abra seu terminal preferido, insira o seguinte comando e confirme que nenhuma versão é retornada.

```
$ aws --version
command not found: aws
```

Se você ainda tiver uma versão listada na saída, a AWS CLI provavelmente foi instalada usando um método diferente, ou há várias versões. Se você não souber com qual método você instalou a AWS CLI, siga as instruções de cada método de desinstalação para a [AWS CLI versão 1](#) e a [AWS CLI versão 2](#) apropriado para o sistema operacional até que nenhuma saída de versão seja recebida.

#### Note

Se você usou um gerenciador de pacotes para instalar a AWS CLI (`pip`, `apt`, `brew` etc.), use o mesmo gerenciador de pacotes para desinstalá-la. Siga as instruções fornecidas pelo gerenciador de pacotes sobre como desinstalar todas as versões de um pacote.

### [Voltar ao início](#)

# A AWS CLI processou um comando com um nome de parâmetro incompleto

Possível causa: você usou uma abreviação reconhecida do parâmetro AWS CLI

Como a AWS CLI é criada usando Python, a AWS CLI usa a biblioteca `argparse` de Python, incluindo o argumento [allow\\_abbrev](#). As abreviações dos parâmetros são reconhecidas pela AWS CLI e processadas.

O exemplo de comando [create-change-set](#) a seguir altera o nome da pilha do CloudFormation. O parâmetro `--change-set-n` é reconhecido como uma abreviação de `--change-set-name` e a AWS CLI processa o comando.

```
$ aws cloudformation create-change-set --stack-name my-stack --change-set-n my-change-set
```

Quando a abreviação pode ser de vários comandos, o parâmetro não será reconhecido como uma abreviação.

O exemplo de comando [create-change-set](#) a seguir altera o nome da pilha do CloudFormation. O parâmetro `--change-set-` não é reconhecido como uma abreviação, pois há vários parâmetros dos quais ele pode ser uma abreviação, como `--change-set-name` e `--change-set-type`. Portanto, a AWS CLI não processa o comando.

```
$ aws cloudformation create-change-set --stack-name my-stack --change-set- my-change-set
```

## Warning

Não use abreviações de parâmetros propositadamente. Elas não são confiáveis e não são compatíveis com versões anteriores. Se algum novo parâmetro for adicionado a um comando que confunda as abreviações, os comandos serão interrompidos.

Além disso, se o parâmetro for um argumento de valor único, ele poderá causar um comportamento inesperado com os comandos. Se várias instâncias de um argumento de valor único forem transmitidas, somente a última instância será executada. No exemplo a seguir, o parâmetro `--filters` é um argumento de valor único. Os parâmetros `--filters` e `--filter` são especificados. O parâmetro `--filter` é uma abreviação



de `--filters`. Isso faz com que duas instâncias de `--filters` sejam aplicadas e somente o último argumento `--filter` se aplica.

```
$ aws ec2 describe-vpc-peering-connections \
  --filters Name=tag:TagName,Values=VpcPeeringConnection \
  --filter Name=status-code,Values=active
```

Confirme se você está usando parâmetros válidos antes de executar um comando para evitar comportamentos inesperados.

[Voltar ao início](#)

## Erros de acesso negado

Causa possível: o arquivo do programa AWS CLI não tem permissão de “execução”

No Linux ou no macOS, verifique se o programa `aws` tem permissões de execução para o usuário que está chamando. Normalmente, as permissões são definidas como 755.

Para adicionar permissão de execução ao usuário, execute o comando a seguir, substituindo `~/.local/bin/aws` pelo caminho para o programa no computador:

```
$ chmod +x ~/.local/bin/aws
```

[Voltar ao início](#)

Causa possível: sua identidade do IAM não tem permissão para executar a operação.

Exemplo de texto de erro:

```
$ aws s3 ls
An error occurred (AccessDenied) when calling the ListBuckets operation: Access
denied.
```

Quando você executa um comando da AWS CLI, as operações da AWS são realizadas em seu nome, usando credenciais que associam você a uma conta ou perfil do IAM. As políticas associadas devem conceder permissão para chamar as ações de API que correspondem aos comandos executados com a AWS CLI.

A maioria dos comandos chama uma única ação com um nome que corresponde ao nome do comando. No entanto, comandos personalizados como `aws s3 sync` chamam várias APIs. É possível ver quais APIs um comando chama usando a opção `--debug`.

Se você tiver certeza de que o usuário ou o perfil tem as permissões apropriadas atribuídas pela política, verifique se o comando da AWS CLI está usando as credenciais esperadas. Consulte a [próxima seção sobre credenciais](#) para verificar se as credenciais que a AWS CLI está usando são as esperadas.

Para obter informações sobre como atribuir permissões do IAM, consulte [Visão geral do gerenciamento de acesso: permissões e políticas](#) no Manual do usuário do IAM.

[Voltar ao início](#)

## Credenciais inválidas e erros de chave

Exemplo de texto de erro:

```
$ aws s3 ls
An error occurred (InvalidAccessKeyId) when calling the ListBuckets operation: The AWS
Access Key Id
you provided does not exist in our records.
```

```
$ aws s3 ls
An error occurred (InvalidClientTokenId) when calling the ListBuckets operation: The
security token
included in the request is invalid.
```

Causa possível: a AWS CLI está lendo credenciais de um local inesperado.

A AWS CLI pode estar lendo credenciais de um local diferente do que o esperado ou as informações de par de chaves estão incorretas. Execute a `aws configure list` para confirmar quais credenciais são usadas.

O exemplo a seguir mostra como verificar as credenciais usadas para o perfil padrão.

```
$ aws configure list
```

| Name | Value | Type | Location |
|------|-------|------|----------|
| ---- | ----- | ---- | -----    |

|            |           |                         |               |
|------------|-----------|-------------------------|---------------|
| profile    | <not set> | None                    | None          |
| access_key | *****XYVA | shared-credentials-file |               |
| secret_key | *****ZAGY | shared-credentials-file |               |
| region     | us-west-2 | config-file             | ~/.aws/config |

O exemplo a seguir mostra como verificar as credenciais de um perfil nomeado.

```
$ aws configure list --profile saanvi
```

| Name       | Value     | Type                    | Location      |
|------------|-----------|-------------------------|---------------|
| ----       | ----      | ----                    | -----         |
| profile    | saanvi    | manual                  | --profile     |
| access_key | *****     | shared-credentials-file |               |
| secret_key | *****     | shared-credentials-file |               |
| region     | us-west-2 | config-file             | ~/.aws/config |

Para confirmar os detalhes do par de chaves, verifique os arquivos config e credentials. Para obter mais informações sobre os arquivos config e credentials, consulte [the section called “Configurações do arquivo de configuração e credenciais”](#). Para obter mais informações sobre credenciais e autenticação, incluindo a precedência de credenciais, consulte [Autenticação e credenciais de acesso](#).

### [Voltar ao início](#)

Causa possível: o relógio do computador está fora de sincronia.

Se você estiver usando credenciais válidas, seu relógio poderá estar fora de sincronia. No Linux ou macOS, execute date para verificar a hora.

```
$ date
```

Se o relógio do sistema não estiver correto dentro de alguns minutos, use ntpd para sincronizar.

```
$ sudo service ntpd stop
$ sudo ntpdate time.nist.gov
$ sudo service ntpd start
$ ntpstat
```

No Windows, use as opções de data e hora no Painel de controle para configurar o relógio do sistema.

### [Voltar ao início](#)

# Assinatura não corresponde aos erros

Exemplo de texto de erro:

```
$ aws s3 ls
```

```
An error occurred (SignatureDoesNotMatch) when calling the ListBuckets operation: The request signature we calculated does not match the signature you provided. Check your key and signing method.
```

Quando a AWS CLI executa um comando, ela envia uma solicitação criptografada para os servidores da AWS a fim de executar as operações de serviço apropriadas da AWS. Suas credenciais (a chave de acesso e a chave secreta) estão envolvidas na criptografia e permitem à AWS autenticar a pessoa que está fazendo a solicitação. Há vários fatores que podem interferir na operação correta desse processo, conforme indicado a seguir.

Causa possível: o relógio está fora de sincronia com os servidores da AWS

Para ajudar a proteger contra [ataques de reprodução](#), a hora atual pode ser usada durante o processo de criptografia/descriptografia. Se a hora do cliente e a hora do servidor não coincidirem além do permitido, o processo poderá falhar e a solicitação será rejeitada. Isso também pode acontecer quando você executa um comando em uma máquina virtual cujo relógio está fora de sincronia com o relógio da máquina host. Uma causa possível é quando a máquina virtual hiberna e, depois de ativada, demora algum tempo para sincronizar o relógio com a máquina host.

No Linux ou macOS, execute `date` para verificar a hora.

```
$ date
```

Se o relógio do sistema não estiver correto dentro de alguns minutos, use `ntpd` para sincronizar.

```
$ sudo service ntpd stop
$ sudo ntpdate time.nist.gov
$ sudo service ntpd start
$ ntpstat
```

No Windows, use as opções de data e hora no Painel de controle para configurar o relógio do sistema.

[Voltar ao início](#)

Causa possível: o sistema operacional está processando incorretamente chaves da AWS que contêm determinados caracteres especiais

Se a chave da AWS incluir determinados caracteres especiais, como -, +, / ou %, algumas variantes do sistema operacional processarão a string incorretamente e farão com que a string de chave seja interpretada incorretamente.

Se você processar as chaves usando outras ferramentas ou scripts, como ferramentas que criam o arquivo de credenciais em uma nova instância como parte de sua criação, essas ferramentas e scripts poderão ter seu próprio tratamento de caracteres especiais que faça com que eles sejam transformados em algo não reconhecido pela AWS.

Sugerimos gerar novamente a chave secreta para obter uma que não inclua o caractere especial que está causando problemas.

[Voltar ao início](#)

## Erros de nenhum console do Windows encontrado

Exemplo de texto de erro:

```
$ aws s3 ls
No Windows console found. Are you running cmd.exe?
```

Ao executar um comando AWS CLI, você recebe uma mensagem de erro “Nenhum console do Windows encontrado. Are you running cmd.exe?” (Nenhum console do Windows encontrado. Você está executando o cmd.exe?”. Isso geralmente é um erro visto pela AWS CLI versão 1 se o Python `prompt_toolkit` que você instalou estiver desatualizado. Para resolver esse problema, instale uma versão recente do `prompt_toolkit` no [site do Python](#).

[Voltar ao início](#)

## Erros de certificado SSL

Possível causa: a AWS CLI não confia no certificado do proxy

Exemplo de texto de erro:

```
$ aws s3 ls
```

```
[SSL: CERTIFICATE_ VERIFY_FAILED] certificate verify failed
```

Quando você usa um comando AWS CLI, você recebe uma mensagem de erro de [SSL: CERTIFICATE\_ VERIFY\_FAILED] certificate verify failed. Isso ocorre porque a AWS CLI não confia no certificado do seu proxy devido a fatores como ele ser autoassinado, com sua empresa definida como a autoridade de certificação (CA). Isso impede que a AWS CLI encontre o certificado raiz de CA da sua empresa no registro de CA local.

Para corrigir isso, instrua a AWS CLI sobre onde encontrar o arquivo .pem da sua empresa usando a opção de arquivo de configuração [ca\\_bundle](#), a opção de linha de comando [--ca-bundle](#) ou a variável de ambiente [AWS\\_CA\\_BUNDLE](#).

[Voltar ao início](#)

Causa possível: sua configuração não está apontando para o local correto do certificado raiz da CA

Exemplo de texto de erro:

```
$ aws s3 ls
SSL validation failed for regionname [Errno 2] No such file or directory
```

Isso é causado devido ao local do arquivo do pacote da Autoridade de Certificação (CA) estar configurado incorretamente na AWS CLI. Para corrigir isso, confirme o local em que o arquivo .pem está localizado e atualize a configuração da AWS CLI usando o arquivo de configuração [ca\\_bundle](#), a opção de linha de comando da [--ca-bundle](#) ou a variável de ambiente [AWS\\_CA\\_BUNDLE](#).

[Voltar ao início](#)

## Erros JSON inválidos

Exemplo de texto de erro:

```
$ aws dynamodb update-table \
  --provisioned-throughput '{"ReadCapacityUnits":15,WriteCapacityUnits":10}' \
  --table-name MyDDBTable
Error parsing parameter '--provisioned-throughput': Invalid JSON: Expecting property
name enclosed in
double quotes: line 1 column 25 (char 24)
JSON received: {"ReadCapacityUnits":15,WriteCapacityUnits":10}
```

Ao usar um comando da AWS CLI, você recebe a mensagem de erro “Invalid JSON”. Esse erro geralmente aparece quando você insere um comando com um formato JSON esperado e a AWS CLI não consegue ler o JSON corretamente.

Possível causa: você não inseriu um JSON válido para a AWS CLI usar

Confirme se você inseriu um JSON válido para seu comando. Sugerimos usar um validador JSON para o JSON que você está tendo dificuldade de formatar.

Para uso mais avançado de JSON na linha de comando, considere usar um processador JSON de linha de comando, como jq, para criar strings JSON. Para obter mais informações sobre o jq, consulte o [repositório do jq](#) no GitHub.

### [Voltar ao início](#)

Possível causa: as regras de aspas do seu terminal estão impedindo que o JSON válido seja enviado à AWS CLI

Antes de a AWS CLI receber qualquer coisa de um comando, seu terminal processa o comando usando regras próprias de aspas e escape. Devido às regras de formatação de um terminal, parte do seu conteúdo JSON pode ser removido antes de o comando ser passado para a AWS CLI. Ao formular comandos, use as [regras de aspas do seu terminal](#).

Para solucionar problemas, use o echo para ver como o shell está lidando com seus parâmetros:

```
$ echo {"ReadCapacityUnits":15,"WriteCapacityUnits":10}
ReadCapacityUnits:15 WriteCapacityUnits:10
```

```
$ echo '{"ReadCapacityUnits":15,"WriteCapacityUnits":10}'
{"ReadCapacityUnits":15,"WriteCapacityUnits":10}
```

Modifique seu comando até que o JSON válido seja retornado.

Para uma solução de problemas mais detalhada, use o parâmetro --debug para visualizar os logs de depuração, pois eles exibirão exatamente o que foi passado para a AWS CLI:

```
$ aws dynamodb update-table \
  --provisioned-throughput '{"ReadCapacityUnits":15,WriteCapacityUnits":10}' \
  --table-name MyDDBTable \
  --debug
```

```
2022-07-19 22:25:07,741 - MainThread - awscli.clidriver - DEBUG - CLI version: aws-  
cli/1.18.147  
Python/2.7.18 Linux/5.4.196-119.356.amzn2int.x86_64 botocore/1.18.6  
2022-07-19 22:25:07,741 - MainThread - awscli.clidriver - DEBUG - Arguments entered  
to CLI:  
['dynamodb', 'update-table', '--provisioned-throughput',  
 '{"ReadCapacityUnits":15,WriteCapacityUnits":10}',  
 '--table-name', 'MyDDBTable', '--debug']
```

Use as regras de aspas do seu terminal para corrigir quaisquer problemas que a entrada JSON tenha ao ser enviada à AWS CLI. Para obter mais informações sobre regras de aspas, consulte [the section called “Aspas com strings”](#).

 Note

Se estiver enfrentando problemas para obter um JSON válido para a AWS CLI, recomendamos ignorar as regras de aspas do terminal referentes à entrada de dados JSON usando blobs para passar seus dados JSON diretamente à AWS CLI. Para obter mais informações sobre o Blobs, consulte [Blob](#).

[Voltar ao início](#)

## Recursos adicionais

Para obter ajuda adicional para problemas de AWS CLI, acesse a [Comunidade da AWS CLI](#) no GitHub ou a [Comunidade da AWS re:Post](#).

[Voltar ao início](#)



# Histórico do documento do guia do usuário da AWS CLI

A tabela a seguir descreve adições importantes feitas ao Manual do usuário da AWS Command Line Interface a partir de janeiro de 2019. Para receber notificações sobre atualizações dessa documentação, você pode se inscrever em o feed RSS.

| Alteração                                                                                             | Descrição                                                                                                                                                                                                                                                                                                            | Data                  |
|-------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| <a href="#">Informações atualizadas sobre credenciais e autenticação.</a>                             | Instruções e exemplos atualizados de credenciais e métodos de autenticação. Isso inclui a atualização de páginas de configuração relevantes. Para acomodar esse aumento na documentação, tópicos relevantes sobre credenciais foram movidos para a nova seção <a href="#">Autenticação e credenciais de acesso</a> . | 31 de março de 2023   |
| <a href="#">Os conteúdos referentes à AWS CLI V1 e V2 agora estão separados nos respectivos guias</a> | Para maior clareza e facilidade, o conteúdo da AWS CLI versão 1 e da AWS CLI versão 2 agora está separado nos próprios guias. Para a AWS CLI versão 2, consulte o <a href="#">Guia do usuário da AWS Command Line Interface</a> mais recente                                                                         | 2 de novembro de 2021 |
| <a href="#">Adição de informações sobre o alias AWS CLI.</a>                                          | Adição de informações sobre o alias AWS CLI. Os aliases são atalhos que você pode criar na AWS Command Line Interface (AWS CLI) para encurtar comandos ou scripts que utiliza com frequência.                                                                                                                        | 11 de março de 2021   |

|                                                                           |                                                                                                                                                                                                                                                                                                                                                                                               |                         |
|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|
| <a href="#">Informações atualizadas sobre saídas de filtros</a>           | Informações sobre filtros atualizadas e movidas para suas próprias páginas.                                                                                                                                                                                                                                                                                                                   | 1º de fevereiro de 2021 |
| <a href="#">Anúncio de descontinuação do Python 2.7, 3.4 e 3.5</a>        | O Python 2.7 foi descontinuado pela Python Software Foundation em 1º de janeiro de 2020. No futuro, os clientes que usam a AWS CLI versão 1 deverão fazer a transição para o Python 3 (no mínimo, Python 3.6). O suporte ao Python 2.7 foi descontinuado para novas versões da AWS CLI versão 1 a partir de 19/7/2021. As versões Python 3.4 e 3.5 foram descontinuadas a partir de 1/2/2021. | 29 de janeiro de 2021   |
| <a href="#">Adição de exemplo de script do Amazon S3</a>                  | Adição de um exemplo de script de ciclo de vida do Amazon S3.                                                                                                                                                                                                                                                                                                                                 | 15 de outubro de 2020   |
| <a href="#">Adição de exemplo de script do Amazon EC2</a>                 | Adição de um exemplo de script de tipo de instância do Amazon EC2.                                                                                                                                                                                                                                                                                                                            | 15 de outubro de 2020   |
| <a href="#">Adição de informações sobre novas tentativas</a>              | Adição de uma página sobre novas tentativas para recursos e comportamento de novas tentativas na AWS CLI.                                                                                                                                                                                                                                                                                     | 17 de setembro de 2020  |
| <a href="#">Página sobre paginação nos lados do servidor e do cliente</a> | Informações sobre paginação atualizadas e centralizadas em uma única página.                                                                                                                                                                                                                                                                                                                  | 17 de agosto de 2020    |

|                                                                                           |                                                                                                                                                                                                                    |                       |
|-------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| <a href="#">Página de comandos do s3 atualizada</a>                                       | Página de comandos de alto nível do s3 atualizada com novos exemplos e recursos.                                                                                                                                   | 30 de julho de 2020   |
| <a href="#">Atualização de informações sobre a instalação</a>                             | As informações de instalação, atualização e desinstalação para Linux, macOS e Windows foram atualizadas.                                                                                                           | 19 de maio de 2020    |
| <a href="#">Atualizado para remover o suporte ao Python 2.6 e 3.3 da AWS CLI versão 1</a> | Desde de 10 de janeiro de 2020, a AWS CLI versão 1 não oferece mais suporte ao uso das versões 2.6 ou 3.3 do Python. Atualize para uma versão mais recente do Python para usar a AWS CLI versão 1.17 ou posterior. | 10 de janeiro de 2020 |
| <a href="#">Nova seção MFA</a>                                                            | Adicionada uma nova seção que descreve como acessar a CLI usando a autenticação multifator e funções.                                                                                                              | 3 de maio de 2019     |
| <a href="#">Atualização da seção "Uso da CLI"</a>                                         | Principais melhorias e adições às instruções e aos procedimentos de uso.                                                                                                                                           | 7 de março de 2019    |
| <a href="#">Atualização da seção "Instalar a CLI"</a>                                     | Principais melhorias e adições às instruções e aos procedimentos de instalação da AWS CLI.                                                                                                                         | 7 de março de 2019    |
| <a href="#">Atualização da seção "Configurar a CLI"</a>                                   | Principais melhorias e adições às instruções e aos procedimentos de configuração da AWS CLI.                                                                                                                       | 7 de março de 2019    |